

Introducción a las Bases de Datos

Fundamentos de Organización de Datos

Práctica 1

Creación, consulta y mantenimiento de archivos secuenciales - Algorítmica Básica.

1. Realizar un algoritmo que cree un archivo de números enteros no ordenados y permita incorporar datos al archivo. Los números son ingresados desde teclado. La carga finaliza cuando se ingresa el número 30000, que no debe incorporarse al archivo. El nombre del archivo debe ser proporcionado por el usuario desde teclado.
2. Realizar un algoritmo, que utilizando el archivo de números enteros no ordenados creado en el ejercicio 1, informe por pantalla cantidad de números menores a 1500 y el promedio de los números ingresados. El nombre del archivo a procesar debe ser proporcionado por el usuario una única vez. Además, el algoritmo deberá listar el contenido del archivo en pantalla.
3. Realizar un programa que presente un menú con opciones para:
 - a. Crear un archivo de registros **no ordenados** de empleados y completarlo con datos ingresados desde teclado. De cada empleado se registra: número de empleado, apellido, nombre, edad y DNI. Algunos empleados se ingresan con DNI 00. La carga finaliza cuando se ingresa el String 'fin' como apellido.
 - b. Abrir el archivo anteriormente generado y
 - i. Listar en pantalla los datos de empleados que tengan un nombre o apellido determinado, el cual se proporciona desde el teclado.

- ii. Listar en pantalla los empleados de a uno por línea.
- iii. Listar en pantalla los empleados mayores de 70 años, próximos a jubilarse.

NOTA: El nombre del archivo a crear o utilizar debe ser proporcionado por el usuario.

4. Agregar al menú del programa del ejercicio 3, opciones para:

- a. Añadir uno o más empleados al final del archivo con sus datos ingresados por teclado. Tener en cuenta que no se debe agregar al archivo un empleado con un número de empleado ya registrado (control de unicidad).
- b. Modificar la edad de un empleado dado.
- c. Exportar el contenido del archivo a un archivo de texto llamado "todos_empleados.txt".
- d. Exportar a un archivo de texto llamado: "faltaDNIEmpleado.txt", los empleados que no tengan cargado el DNI (DNI en 00).

NOTA: Las búsquedas deben realizarse por número de empleado.

5. Realizar un programa para una tienda de celulares, que presente un menú con opciones para:

- a. Crear un archivo de registros no ordenados de celulares y cargarlo con datos ingresados desde un archivo de texto denominado "celulares.txt". Los registros correspondientes a los celulares deben contener: código de celular, nombre, descripción, marca, precio, stock mínimo y stock disponible.
- b. Listar en pantalla los datos de aquellos celulares que tengan un stock menor al stock mínimo.
- c. Listar en pantalla los celulares del archivo cuya descripción contenga una cadena de caracteres proporcionada por el usuario.
- d. Exportar el archivo creado en el inciso a) a un archivo de texto denominado "celulares.txt" con todos los celulares del mismo. El archivo de texto generado

podría ser utilizado en un futuro como archivo de carga (ver inciso a), por lo que debería respetar el formato dado para este tipo de archivos en la NOTA 2.

NOTA 1: El nombre del archivo binario de celulares debe ser proporcionado por el usuario.

NOTA 2: El archivo de carga debe editarse de manera que cada celular se especifique en tres líneas consecutivas. En la primera se especifica: código de celular, el precio y marca, en la segunda el stock disponible, stock mínimo y la descripción y en la tercera nombre en ese orden. Cada celular se carga leyendo tres líneas del archivo "celulares.txt".

6. Agregar al menú del programa del ejercicio 5, opciones para:

- a. Añadir uno o más celulares al final del archivo con sus datos ingresados por teclado.
- b. Modificar el stock de un celular dado.
- c. Exportar el contenido del archivo binario a un archivo de texto denominado: "SinStock.txt", con aquellos celulares que tengan stock 0.

NOTA: Las búsquedas deben realizarse por nombre de celular.

7. Realizar un programa que permita:

- a) Crear un archivo binario a partir de la información almacenada en un archivo de texto. El nombre del archivo de texto es: "novelas.txt". La información en el archivo de texto consiste en: código de novela, nombre, género y precio de diferentes novelas argentinas. Los datos de cada novela se almacenan en dos líneas en el archivo de texto. La primera línea contendrá la siguiente información: código novela, precio y género, y la segunda línea almacenará el nombre de la novela.

- b) Abrir el archivo binario y permitir la actualización del mismo. Se debe poder agregar una novela y modificar una existente. Las búsquedas se realizan por código de novela.

NOTA: El nombre del archivo binario es proporcionado por el usuario desde el teclado.

IMPORTANTE: Se recomienda implementar los ejercicios prácticos en Dev-Pascal. El ejecutable puede descargarse desde la plataforma Moodle.

Trabajo Práctico N° 1: **Creación, Consulta y Mantenimiento de Archivos** **Secuenciales - Algorítmica Básica.**

Ejercicio 1.

Realizar un algoritmo que cree un archivo de números enteros no ordenados y permita incorporar datos al archivo. Los números son ingresados desde teclado. La carga finaliza cuando se ingresa el número 30.000, que no debe incorporarse al archivo. El nombre del archivo debe ser proporcionado por el usuario desde teclado.

```
program TP1_E1;
{$codepage UTF8}
uses crt;
const
    num_salida=30000;
type
    t_archivo_enteros=file of int16;
procedure leer_numero(var num: int16);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        num:=num_salida
    else
        num:=random(high(int16));
    end;
end;
procedure cargar_archivo_enteros(var archivo_enteros: t_archivo_enteros);
var
    num: int16;
begin
    rewrite(archivo_enteros);
    textcolor(green); write('Los números ingresados son: ');
    leer_numero(num);
    while (num<>num_salida) do
    begin
        textcolor(yellow); write(num, ' ');
        write(archivo_enteros,num);
        leer_numero(num);
    end;
    close(archivo_enteros);
end;
var
    archivo_enteros: t_archivo_enteros;
begin
    randomize;
    assign(archivo_enteros,'E1_enteros');
    cargar_archivo_enteros(archivo_enteros);
end.
```

Ejercicio 2.

Realizar un algoritmo que, utilizando el archivo de números enteros no ordenados creado en el Ejercicio 1, informe por pantalla cantidad de números menores a 1.500 y el promedio de los números ingresados. El nombre del archivo a procesar debe ser proporcionado por el usuario una única vez. Además, el algoritmo deberá listar el contenido del archivo en pantalla.

```
program TP1_E2;
{$codepage UTF8}
uses crt;
const
    num_corte=1500;
type
    t_archivo_enteros=file of int16;
procedure procesar_archivo_enteros(var archivo_enteros: t_archivo_enteros; var nums_corte:
int16; var prom: real);
var
    num: int16;
    suma: real;
begin
    reset(archivo_enteros);
    suma:=0;
    textcolor(green); write('El contenido del archivo es: ');
    while (not eof(archivo_enteros)) do
        begin
            read(archivo_enteros,num);
            textcolor(yellow); write(num,' ');
            if (num<num_corte) then
                nums_corte:=nums_corte+1;
            suma:=suma+num;
        end;
    if (filesize(archivo_enteros)>0) then
        prom:=suma/filesize(archivo_enteros);
    writeln();
    close(archivo_enteros);
end;
var
    archivo_enteros: t_archivo_enteros;
    nums_corte: int16;
    prom: real;
begin
    nums_corte:=0; prom:=0;
    assign(archivo_enteros,'E1_enteros');
    procesar_archivo_enteros(archivo_enteros,nums_corte,prom);
    textcolor(green); write('La cantidad de números menores a '); textcolor(yellow);
write(nums_corte); textcolor(green); write(' es '); textcolor(red); writeln(nums_corte);
    textcolor(green); write('El promedio de los números ingresados es '); textcolor(red);
write(prom:0:2);
end.
```

Ejercicio 3.

Realizar un programa que presente un menú con opciones para:

(a) Crear un archivo de registros no ordenados de empleados y completarlo con datos ingresados desde teclado. De cada empleado, se registra: número de empleado, apellido, nombre, edad y DNI. Algunos empleados se ingresan con DNI 00. La carga finaliza cuando se ingresa el String "fin" como apellido.

(b) Abrir el archivo anteriormente generado y:

- (i) Listar en pantalla los datos de empleados que tengan un nombre o apellido determinado, el cual se proporciona desde el teclado.
- (ii) Listar en pantalla los empleados de a uno por línea.
- (iii) Listar en pantalla los empleados mayores de 70 años, próximos a jubilarse.

Nota: El nombre del archivo a crear o utilizar debe ser proporcionado por el usuario.

```

program TP1_E3;
{$codepage UTF8}
uses crt;
const
    apellido_salida='fin';
    edad_corte=70;
    opcion_salida=0;
type
    t_string10=string[10];
    t_registro_empleado=record
        numero: int16;
        apellido: t_string10;
        nombre: t_string10;
        edad: int8;
        dni: int32;
    end;
    t_archivo_empleados=file of t_registro_empleado;
function random_string(length: int8): t_string10;
var
    i: int8;
    string_aux: string;
begin
    string_aux:='';
    for i:= 1 to length do
        string_aux:=string_aux+chr(ord('A')+random(26));
    end;
    random_string:=string_aux;
end;
procedure leer_empleado(var registro_empleado: t_registro_empleado);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        registro_empleado.apellido:=apellido_salida
    else
        registro_empleado.apellido:=random_string(5+random(5));
        if (registro_empleado.apellido<>apellido_salida) then
            begin
                registro_empleado.numero:=1+random(1000);
                registro_empleado.nombre:=random_string(5+random(5));
                registro_empleado.edad:=18+random(high(int8)-18);
            end;
        end;
end;

```

```
    if (i<=10) then
        registro_empleado.dni:=0
    else
        registro_empleado.dni:=10000000+random(40000001);
    end;
end;
procedure cargar_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    rewrite(archivo_empleados);
    leer_empleado(registro_empleado);
    while (registro_empleado.apellido<>apellido_salida) do
        begin
            write(archivo_empleados,registro_empleado);
            leer_empleado(registro_empleado);
        end;
    close(archivo_empleados);
    textcolor(green); writeln('El archivo binario de empleados fue creado y cargado con éxito');
end;
procedure imprimir_registro_empleado(registro_empleado: t_registro_empleado);
begin
    textcolor(green); write('Número: '); textcolor(yellow); write(registro_empleado.numero);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_empleado.apellido);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_empleado.nombre);
    textcolor(green); write('; Edad: '); textcolor(yellow); write(registro_empleado.edad);
    textcolor(green); write('; DNI: '); textcolor(yellow); writeln(registro_empleado.dni);
end;
procedure imprimir1_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    texto: t_string10;
begin
    texto:=random_string(5+random(5));
    reset(archivo_empleados);
    textcolor(green); write('Los datos de los empleados con nombre o apellido ');
textcolor(yellow); write(texto); textcolor(green); writeln(' son: ');
    while (not eof(archivo_empleados)) do
        begin
            read(archivo_empleados,registro_empleado);
            if ((registro_empleado.nombre=texto) or (registro_empleado.apellido=texto)) then
                imprimir_registro_empleado(registro_empleado);
            end;
        close(archivo_empleados);
    end;
end;
procedure imprimir2_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_empleados);
    textcolor(green); writeln('Los empleados del archivo son: ');
    while (not eof(archivo_empleados)) do
        begin
            read(archivo_empleados,registro_empleado);
            imprimir_registro_empleado(registro_empleado);
        end;
    close(archivo_empleados);
end;
procedure imprimir3_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_empleados);
    textcolor(green); write('Los empleados mayores a '); textcolor(yellow); write(edad_corte);
textcolor(green); writeln(' años son: ');
```



```
while (not eof(archivo_empleados)) do
begin
    read(archivo_empleados,registro_empleado);
    if (registro_empleado.edad>edad_corte) then
        imprimir_registro_empleado(registro_empleado);
    end;
    close(archivo_empleados);
end;
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear un archivo de
registros no ordenados de empleados y completarlo con datos ingresados desde teclado');
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de empleados que tengan un nombre o apellido determinado');
    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Listar en pantalla los
empleados de a uno por línea');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Listar en pantalla los
empleados mayores a 70 años, próximos a jubilarse');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_empleados: t_archivo_empleados);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
    begin
        case opcion of
            1: cargar_archivo_empleados(archivo_empleados);
            2: imprimir1_archivo_empleados(archivo_empleados);
            3: imprimir2_archivo_empleados(archivo_empleados);
            4: imprimir3_archivo_empleados(archivo_empleados);
            else
                textcolor(green); writeln('La opción ingresada no corresponde a ninguna del menú de
opciones');
            end;
            writeln();
            leer_opcion(opcion);
        end;
    end;
end;
var
    archivo_empleados: t_archivo_empleados;
begin
    randomize;
    assign(archivo_empleados,'E3_empleados');
    menu_opciones(archivo_empleados);
end.
```

Ejercicio 4.

Agregar al menú del programa del Ejercicio 3 opciones para:

(a) *Añadir uno o más empleados al final del archivo con sus datos ingresados por teclado. Tener en cuenta que no se debe agregar al archivo un empleado con un número de empleado ya registrado (control de unicidad).*

(b) *Modificar la edad de un empleado dado.*

(c) *Exportar el contenido del archivo a un archivo de texto llamado “todos_empleados.txt”.*

(d) *Exportar a un archivo de texto llamado “faltaDNIEmpleado.txt” los empleados que no tengan cargado el DNI (DNI en 00).*

Nota: Las búsquedas deben realizarse por número de empleado.

```

program TP1_E4;
{$codepage UTF8}
uses crt;
const
    apellido_salida='fin';
    edad_corte=70;
    dni_corte=0;
    opcion_salida=0;
type
    t_string10=string[10];
    t_registro_empleado=record
        numero: int16;
        apellido: t_string10;
        nombre: t_string10;
        edad: int8;
        dni: int32;
    end;
    t_archivo_empleados=file of t_registro_empleado;
function random_string(length: int8): t_string10;
var
    i: int8;
    string_aux: string;
begin
    string_aux:='';
    for i:= 1 to length do
        string_aux:=string_aux+chr(ord('A')+random(26));
    random_string:=string_aux;
end;
procedure leer_empleado(var registro_empleado: t_registro_empleado);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        registro_empleado.apellido:=apellido_salida
    else
        registro_empleado.apellido:=random_string(5+random(5));
        if (registro_empleado.apellido<>apellido_salida) then
            begin
                registro_empleado.numero:=1+random(1000);
                registro_empleado.nombre:=random_string(5+random(5));
            end;
        end;
end;

```

```
registro_empleado.edad:=18+random(high(int8)-18);
if (i<=10) then
    registro_empleado.dni:=0
else
    registro_empleado.dni:=10000000+random(40000001);
end;
end;
procedure cargar_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    rewrite(archivo_empleados);
    leer_empleado(registro_empleado);
    while (registro_empleado.apellido<>apellido_salida) do
        begin
            write(archivo_empleados,registro_empleado);
            leer_empleado(registro_empleado);
        end;
    close(archivo_empleados);
    textcolor(green); writeln('El archivo binario de empleados fue creado y cargado con éxito');
end;
procedure imprimir_registro_empleado(registro_empleado: t_registro_empleado);
begin
    textcolor(green); write('Número: '); textcolor(yellow); write(registro_empleado.numero);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_empleado.apellido);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_empleado.nombre);
    textcolor(green); write('; Edad: '); textcolor(yellow); write(registro_empleado.edad);
    textcolor(green); write('; DNI: '); textcolor(yellow); writeln(registro_empleado.dni);
end;
procedure imprimir1_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    texto: t_string10;
begin
    texto:=random_string(5+random(5));
    reset(archivo_empleados);
    textcolor(green); write('Los datos de los empleados con nombre o apellido ');
textcolor(yellow); write(texto); textcolor(green); writeln(' son: ');
    while (not eof(archivo_empleados)) do
        begin
            read(archivo_empleados,registro_empleado);
            if ((registro_empleado.nombre=texto) or (registro_empleado.apellido=texto)) then
                imprimir_registro_empleado(registro_empleado);
            end;
        end;
    close(archivo_empleados);
end;
procedure imprimir2_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_empleados);
    textcolor(green); writeln('Los empleados del archivo son: ');
    while (not eof(archivo_empleados)) do
        begin
            read(archivo_empleados,registro_empleado);
            imprimir_registro_empleado(registro_empleado);
        end;
    close(archivo_empleados);
end;
procedure imprimir3_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_empleados);
```

```
    textcolor(green); write('Los empleados mayores a '); textcolor(yellow); write(edad_corte);
textcolor(green); writeln(' años son: ');
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados, registro_empleado);
        if (registro_empleado.edad > edad_corte) then
            imprimir_registro_empleado(registro_empleado);
        end;
    end;
    close(archivo_empleados);
end;
function control_unicidad(var archivo_empleados: t_archivo_empleados; numero: int16): boolean;
var
    registro_empleado: t_registro_empleado;
    ok: boolean;
begin
    ok := false;
    while ((not eof(archivo_empleados)) and (ok = false)) do
    begin
        read(archivo_empleados, registro_empleado);
        if (registro_empleado.numero = numero) then
            ok := true;
        end;
    end;
    control_unicidad := ok;
end;
procedure agregar_empleado(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    empleados: int16;
begin
    empleados := 0;
    reset(archivo_empleados);
    leer_empleado(registro_empleado);
    while (registro_empleado.apellido <> apellido_salida) do
    begin
        if (control_unicidad(archivo_empleados, registro_empleado.numero) = false) then
        begin
            seek(archivo_empleados, filesize(archivo_empleados));
            write(archivo_empleados, registro_empleado);
            empleados := empleados + 1;
        end;
        leer_empleado(registro_empleado);
    end;
    close(archivo_empleados);
    textcolor(green); write('Se han agregado '); textcolor(yellow); write(empleados);
textcolor(green); writeln(' empleados al final del archivo');
end;
procedure modificar_edad_empleado(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    numero: int16;
    ok: boolean;
begin
    numero := 1 + random(1000);
    ok := false;
    reset(archivo_empleados);
    while ((not eof(archivo_empleados)) and (ok = false)) do
    begin
        read(archivo_empleados, registro_empleado);
        if (registro_empleado.numero = numero) then
        begin
            registro_empleado.edad := 18 + random(high(int8) - 18);
            seek(archivo_empleados, filepos(archivo_empleados) - 1);
            write(archivo_empleados, registro_empleado);
            ok := true;
        end;
    end;
end;
```

```

close(archivo_empleados);
if (ok=true) then
begin
    textcolor(green); write('Se ha modificado la edad del empleado con número ');
textcolor(yellow); write(numero); textcolor(green); writeln(' en el archivo');
end
else
begin
    textcolor(green); write('No se ha encontrado el empleado con número '); textcolor(yellow);
write(numero); textcolor(green); writeln(' en el archivo');
end;
end;
procedure exportar_archivo_txt1(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    archivo_txt: text;
begin
    reset(archivo_empleados);
    assign(archivo_txt, 'E4_todos_empleados.txt'); rewrite(archivo_txt);
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados, registro_empleado);
        with registro_empleado do
            writeln(archivo_txt, 'Número: ', numero, '; Apellido: ', apellido, '; Nombre: ', nombre, ';
Edad: ', edad, '; DNI: ', dni);
        end;
        close(archivo_empleados);
        close(archivo_txt);
        textcolor(green); write('Se ha exportado el contenido del archivo al archivo de texto
llamado '); textcolor(yellow); writeln('E4_todos_empleados.txt');
    end;
end;
procedure exportar_archivo_txt2(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    archivo_txt: text;
begin
    reset(archivo_empleados);
    assign(archivo_txt, 'E4_faltaDNIEmpleado.txt'); rewrite(archivo_txt);
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados, registro_empleado);
        if (registro_empleado.dni=dni_corte) then
            with registro_empleado do
                writeln(archivo_txt, 'Número: ', numero, '; Apellido: ', apellido, '; Nombre: ', nombre, ';
Edad: ', edad, '; DNI: ', dni);
            end;
        close(archivo_empleados);
        close(archivo_txt);
        textcolor(green); write('Se ha exportado a un archivo de texto llamado ');
textcolor(yellow); write('E4_faltaDNIEmpleado.txt'); textcolor(green); writeln(' los empleados
que no tienen cargado el DNI (DNI en 00)');
    end;
end;
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear un archivo de
registros no ordenados de empleados y completarlo con datos ingresados desde teclado');
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de empleados que tengan un nombre o apellido determinado');
    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Listar en pantalla los
empleados de a uno por línea');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Listar en pantalla los
empleados mayores a 70 años, próximos a jubilarse');
    textcolor(yellow); write('OPCIÓN 5: '); textcolor(green); writeln('Añadir uno o más
empleados al final del archivo con sus datos ingresados por teclado');

```

```
    textcolor(yellow); write('OPCIÓN 6: '); textcolor(green); writeln('Modificar la edad de un
empleado dado');
    textcolor(yellow); write('OPCIÓN 7: '); textcolor(green); writeln('Exportar el contenido del
archivo a un archivo de texto llamado "todos_empleados.txt"');
    textcolor(yellow); write('OPCIÓN 8: '); textcolor(green); writeln('Exportar a un archivo de
texto llamado "faltaDNIEmpleado.txt" los empleados que no tengan cargado el DNI (DNI en 00)');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_empleados: t_archivo_empleados);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
        begin
            case opcion of
                1: cargar_archivo_empleados(archivo_empleados);
                2: imprimir1_archivo_empleados(archivo_empleados);
                3: imprimir2_archivo_empleados(archivo_empleados);
                4: imprimir3_archivo_empleados(archivo_empleados);
                5: agregar_empleado(archivo_empleados);
                6: modificar_edad_empleado(archivo_empleados);
                7: exportar_archivo_txt1(archivo_empleados);
                8: exportar_archivo_txt2(archivo_empleados);
            else
                textcolor(green); writeln('La opción ingresada no corresponde a ninguna del menú de
opciones');
            end;
            writeln();
            leer_opcion(opcion);
        end;
    end;
end;
var
    archivo_empleados: t_archivo_empleados;
begin
    randomize;
    assign(archivo_empleados, 'E4_empleados');
    menu_opciones(archivo_empleados);
end.
```

Ejercicio 5.

Realizar un programa para una tienda de celulares, que presente un menú con opciones para:

(a) *Crear un archivo de registros no ordenados de celulares y cargarlo con datos ingresados desde un archivo de texto denominado “celulares.txt”. Los registros correspondientes a los celulares deben contener: código de celular, nombre, descripción, marca, precio, stock mínimo y stock disponible.*

(b) *Listar en pantalla los datos de aquellos celulares que tengan un stock menor al stock mínimo.*

(c) *Listar en pantalla los celulares del archivo cuya descripción contenga una cadena de caracteres proporcionada por el usuario.*

(d) *Exportar el archivo creado en el inciso (a) a un archivo de texto denominado “celulares.txt” con todos los celulares del mismo. El archivo de texto generado podría ser utilizado en un futuro como archivo de carga (ver inciso (a)), por lo que debería respetar el formato dado para este tipo de archivos en la Nota 2.*

Nota 1: El nombre del archivo binario de celulares debe ser proporcionado por el usuario.

Nota 2: El archivo de carga debe editarse de manera que cada celular se especifique en tres líneas consecutivas. En la primera, se especifica: código de celular, precio y marca; en la segunda, stock disponible, stock mínimo y descripción; y, en la tercera, nombre en ese orden. Cada celular se carga leyendo tres líneas del archivo “celulares.txt”.

```
program TP1_E5;
{$codepage UTF8}
uses crt, sysutils;
const
    codigo_salida=0;
    opcion_salida=0;
type
    t_string20=string[20];
    t_registro_celular=record
        codigo: int16;
        nombre: t_string20;
        descripcion: t_string20;
        marca: t_string20;
        precio: real;
        stock_minimo: int16;
        stock_disponible: int16;
    end;
    t_archivo_celulares=file of t_registro_celular;
function random_string(length: int8): t_string20;
var
    i: int8;
    string_aux: string;
begin
    string_aux:='';
    for i:= 1 to length do
        string_aux:=string_aux+chr(ord('A')+random(26));
    end;
    random_string:=string_aux;
```

```

end;
procedure leer_celular(var registro_celular: t_registro_celular);
var
  vector_marcas: array[1..10] of t_string20=('Alcatel', 'Apple', 'Huawei', 'Lenovo', 'LG',
'Motorola', 'Nokia', 'Samsung', 'Sony', 'Xiaomi');
  vector_descripciones: array[1..5] of t_string20=('Gama baja', 'Gama media baja', 'Gama
media', 'Gama media alta', 'Gama alta');
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_celular.codigo:=codigo_salida
  else
    registro_celular.codigo:=1+random(1000);
    if (registro_celular.codigo<>codigo_salida) then
      begin
        registro_celular.nombre:=random_string(5+random(6));
        registro_celular.descripcion:=vector_descripciones[1+random(5)];
        registro_celular.marca:=vector_marcas[1+random(10)];
        registro_celular.precio:=1+random(100);
        registro_celular.stock_minimo:=1+random(10);
        registro_celular.stock_disponible:=random(101);
      end;
    end;
end;
procedure cargar_archivo_carga(var archivo_carga: text);
var
  registro_celular: t_registro_celular;
begin
  rewrite(archivo_carga);
  leer_celular(registro_celular);
  while (registro_celular.codigo<>codigo_salida) do
    begin
      with registro_celular do
        begin
          writeln(archivo_carga,codigo,' ',precio:0:2,' ',marca);
          writeln(archivo_carga,stock_disponible,' ',stock_minimo,' ',descripcion);
          writeln(archivo_carga,nombre);
        end;
      leer_celular(registro_celular);
    end;
  close(archivo_carga);
end;
procedure cargar_archivo_celulares(var archivo_celulares: t_archivo_celulares; var
archivo_carga: text);
var
  registro_celular: t_registro_celular;
begin
  rewrite(archivo_celulares);
  reset(archivo_carga);
  while (not eof(archivo_carga)) do
    with registro_celular do
      begin
        readln(archivo_carga,codigo,precio,marca); marca:=trim(marca);
        readln(archivo_carga,stock_disponible,stock_minimo,descripcion);
        descripcion:=trim(descripcion);
        readln(archivo_carga,nombre);
        write(archivo_celulares,registro_celular);
      end;
    close(archivo_celulares);
    close(archivo_carga);
    textcolor(green); writeln('El archivo binario de celulares fue creado y cargado con éxito');
  end;
end;
procedure imprimir_registro_celular(registro_celular: t_registro_celular);
begin
  textcolor(green); write('Código: '); textcolor(yellow); write(registro_celular.codigo);
  textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_celular.nombre);

```



```

    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_celular.descripcion);
    textcolor(green); write('; Marca: '); textcolor(yellow); write(registro_celular.marca);
    textcolor(green); write('; Precio: '); textcolor(yellow);
write(registro_celular.precio:0:2);
    textcolor(green); write('; Stock mínimo: '); textcolor(yellow);
write(registro_celular.stock_minimo);
    textcolor(green); write('; Stock disponible: '); textcolor(yellow);
writeln(registro_celular.stock_disponible);
end;
procedure imprimir1_archivo_celulares(var archivo_celulares: t_archivo_celulares);
var
    registro_celular: t_registro_celular;
begin
    reset(archivo_celulares);
    textcolor(green); writeln('Los datos de aquellos celulares que tienen un stock menor al
stock mínimo son: ');
    while (not eof(archivo_celulares)) do
        begin
            read(archivo_celulares,registro_celular);
            if (registro_celular.stock_disponible<registro_celular.stock_minimo) then
                imprimir_registro_celular(registro_celular);
            end;
        end;
    close(archivo_celulares);
end;
procedure imprimir2_archivo_celulares(var archivo_celulares: t_archivo_celulares);
var
    registro_celular: t_registro_celular;
    vector_descripciones: array[1..5] of t_string20=('Gama baja', 'Gama media baja', 'Gama
media', 'Gama media alta', 'Gama alta');
    descripcion: t_string20;
begin
    reset(archivo_celulares);
    descripcion:=vector_descripciones[1+random(5)];
    textcolor(green); write('Los celulares del archivo cuya descripción contiene la cadena de
caracteres '); textcolor(yellow); write(descripcion); textcolor(green); writeln(' son: ');
    while (not eof(archivo_celulares)) do
        begin
            read(archivo_celulares,registro_celular);
            if (registro_celular.descripcion=descripcion) then
                imprimir_registro_celular(registro_celular);
            end;
        end;
    close(archivo_celulares);
end;
procedure exportar_archivo_txt(var archivo_celulares: t_archivo_celulares);
var
    registro_celular: t_registro_celular;
    archivo_txt: text;
begin
    reset(archivo_celulares);
    assign(archivo_txt,'E5_celulares2.txt'); rewrite(archivo_txt);
    while (not eof(archivo_celulares)) do
        begin
            read(archivo_celulares,registro_celular);
            with registro_celular do
                begin
                    writeln(archivo_txt,codigo,' ',precio:0:2,' ',marca);
                    writeln(archivo_txt,stock_disponible,' ',stock_minimo,' ',descripcion);
                    writeln(archivo_txt,nombre);
                end;
            end;
        end;
    close(archivo_celulares);
    close(archivo_txt);
    textcolor(green); write('Se ha exportado el archivo creado en el inciso (a) a un archivo de
texto denominado '); textcolor(yellow); write('"celulares2.txt"); textcolor(green); writeln('
con todos los celulares del mismo');

```

```

end;
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear un archivo de
registros no ordenados de celulares y cargarlo con datos ingresados desde un archivo de texto
denominado "celulares1.txt"');
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de aquellos celulares que tengan un stock menor al stock mínimo');
    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Listar en pantalla los
celulares del archivo cuya descripción contenga una cadena de caracteres proporcionada por el
usuario');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Exportar el archivo
creado en el inciso (a) a un archivo de texto denominado "celulares2.txt" con todos los
celulares del mismo');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_celulares: t_archivo_celulares; var archivo_carga: text);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
        begin
            case opcion of
                1: cargar_archivo_celulares(archivo_celulares,archivo_carga);
                2: imprimir1_archivo_celulares(archivo_celulares);
                3: imprimir2_archivo_celulares(archivo_celulares);
                4: exportar_archivo_txt(archivo_celulares);
            else
                textcolor(green); writeln('La opción ingresada no corresponde a ninguna del menú de
opciones');
            end;
            writeln();
            leer_opcion(opcion);
        end;
    end;
end;
var
    archivo_celulares: t_archivo_celulares;
    archivo_carga: text;
begin
    randomize;
    assign(archivo_carga,'E5_celulares1.txt');
    assign(archivo_celulares,'E5_celulares2');
    cargar_archivo_carga(archivo_carga);
    menu_opciones(archivo_celulares,archivo_carga);
end.

```

Ejercicio 6.

Agregar al menú del programa del Ejercicio 5 opciones para:

- (a) *Añadir uno o más celulares al final del archivo con sus datos ingresados por teclado.*
- (b) *Modificar el stock de un celular dado.*
- (c) *Exportar el contenido del archivo binario a un archivo de texto denominado "SinStock.txt" con aquellos celulares que tengan stock 0.*

```

program TP1_E6;
{$codepage UTF8}
uses crt, sysutils;
const
    codigo_salida=0;
    stock_disponible_corte=0;
    opcion_salida=0;
type
    t_string20=string[20];
    t_registro_celular=record
        codigo: int16;
        nombre: t_string20;
        descripcion: t_string20;
        marca: t_string20;
        precio: real;
        stock_minimo: int16;
        stock_disponible: int16;
    end;
    t_archivo_celulares=file of t_registro_celular;
function random_string(length: int8): t_string20;
var
    i: int8;
    string_aux: string;
begin
    string_aux:='';
    for i:= 1 to length do
        string_aux:=string_aux+chr(ord('A')+random(26));
    random_string:=string_aux;
end;
procedure leer_celular(var registro_celular: t_registro_celular);
var
    vector_marcas: array[1..10] of t_string20=('Alcatel', 'Apple', 'Huawei', 'Lenovo', 'LG',
'Motorola', 'Nokia', 'Samsung', 'Sony', 'Xiaomi');
    vector_descripciones: array[1..5] of t_string20=('Gama baja', 'Gama media baja', 'Gama
media', 'Gama media alta', 'Gama alta');
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        registro_celular.codigo:=codigo_salida
    else
        registro_celular.codigo:=1+random(1000);
    if (registro_celular.codigo<>codigo_salida) then
        begin
            registro_celular.nombre:=random_string(5+random(6));
            registro_celular.descripcion:=vector_descripciones[1+random(5)];
            registro_celular.marca:=vector_marcas[1+random(10)];
            registro_celular.precio:=1+random(100);
            registro_celular.stock_minimo:=1+random(10);
            registro_celular.stock_disponible:=random(101);
        end;
    end;
end;

```

```

    end;
end;
procedure cargar_archivo_carga(var archivo_carga: text);
var
    registro_celular: t_registro_celular;
begin
    rewrite(archivo_carga);
    leer_celular(registro_celular);
    while (registro_celular.codigo<>codigo_salida) do
    begin
        with registro_celular do
        begin
            writeln(archivo_carga,codigo,' ',precio:0:2,' ',marca);
            writeln(archivo_carga,stock_disponible,' ',stock_minimo,' ',descripcion);
            writeln(archivo_carga,nombre);
        end;
        leer_celular(registro_celular);
    end;
    close(archivo_carga);
end;
procedure cargar_archivo_celulares(var archivo_celulares: t_archivo_celulares; var
archivo_carga: text);
var
    registro_celular: t_registro_celular;
begin
    rewrite(archivo_celulares);
    reset(archivo_carga);
    while (not eof(archivo_carga)) do
        with registro_celular do
        begin
            readln(archivo_carga,codigo,precio,marca); marca:=trim(marca);
            readln(archivo_carga,stock_disponible,stock_minimo,descripcion);
            descripcion:=trim(descripcion);
            readln(archivo_carga,nombre);
            write(archivo_celulares,registro_celular);
        end;
    end;
    close(archivo_celulares);
    close(archivo_carga);
    textcolor(green); writeln('El archivo binario de celulares fue creado y cargado con éxito');
end;
procedure imprimir_registro_celular(registro_celular: t_registro_celular);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_celular.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_celular.nombre);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_celular.descripcion);
    textcolor(green); write('; Marca: '); textcolor(yellow); write(registro_celular.marca);
    textcolor(green); write('; Precio: '); textcolor(yellow);
write(registro_celular.precio:0:2);
    textcolor(green); write('; Stock mínimo: '); textcolor(yellow);
write(registro_celular.stock_minimo);
    textcolor(green); write('; Stock disponible: '); textcolor(yellow);
writeln(registro_celular.stock_disponible);
end;
procedure imprimir1_archivo_celulares(var archivo_celulares: t_archivo_celulares);
var
    registro_celular: t_registro_celular;
begin
    reset(archivo_celulares);
    textcolor(green); writeln('Los datos de aquellos celulares que tienen un stock menor al
stock mínimo son: ');
    while (not eof(archivo_celulares)) do
    begin
        read(archivo_celulares,registro_celular);
        if (registro_celular.stock_disponible<registro_celular.stock_minimo) then
            imprimir_registro_celular(registro_celular);
        end;
    end;
end;

```

```

    end;
    close(archivo_celulares);
end;
procedure imprimir2_archivo_celulares(var archivo_celulares: t_archivo_celulares);
var
    registro_celular: t_registro_celular;
    vector_descripciones: array[1..5] of t_string20=('Gama baja', 'Gama media baja', 'Gama
media', 'Gama media alta', 'Gama alta');
    descripcion: t_string20;
begin
    reset(archivo_celulares);
    descripcion:=vector_descripciones[1+random(5)];
    textcolor(green); write('Los celulares del archivo cuya descripción contiene la cadena de
caracteres '); textcolor(yellow); write(descripcion); textcolor(green); writeln(' son: ');
    while (not eof(archivo_celulares)) do
    begin
        read(archivo_celulares,registro_celular);
        if (registro_celular.descripcion=descripcion) then
            imprimir_registro_celular(registro_celular);
        end;
    end;
    close(archivo_celulares);
end;
procedure exportar_archivo_txt1(var archivo_celulares: t_archivo_celulares);
var
    registro_celular: t_registro_celular;
    archivo_txt: text;
begin
    reset(archivo_celulares);
    assign(archivo_txt,'E6_celulares2.txt'); rewrite(archivo_txt);
    while (not eof(archivo_celulares)) do
    begin
        read(archivo_celulares,registro_celular);
        with registro_celular do
        begin
            writeln(archivo_txt,codigo,' ',precio:0:2,' ',marca);
            writeln(archivo_txt,stock_disponible,' ',stock_minimo,' ',descripcion);
            writeln(archivo_txt,nombre);
        end;
    end;
    close(archivo_celulares);
    close(archivo_txt);
    textcolor(green); write('Se ha exportado el archivo creado en el inciso (a) a un archivo de
texto denominado '); textcolor(yellow); write('"celulares2.txt"); textcolor(green); writeln('
con todos los celulares del mismo');
end;
function control_unicidad(var archivo_celulares: t_archivo_celulares; codigo: int16): boolean;
var
    registro_celular: t_registro_celular;
    ok: boolean;
begin
    ok:=false;
    while ((not eof(archivo_celulares)) and (ok=false)) do
    begin
        read(archivo_celulares,registro_celular);
        if (registro_celular.codigo=codigo) then
            ok:=true;
        end;
    end;
    control_unicidad:=ok;
end;
procedure agregar_celular(var archivo_celulares: t_archivo_celulares);
var
    registro_celular: t_registro_celular;
    celulares: int16;
begin
    celulares:=0;
    reset(archivo_celulares);

```

```

leer_celular(registro_celular);
while (registro_celular.codigo<>codigo_salida) do
begin
  if (control_unicidad(archivo_celulares,registro_celular.codigo)=false) then
  begin
    seek(archivo_celulares,filesize(archivo_celulares));
    write(archivo_celulares,registro_celular);
    celulares:=celulares+1;
  end;
  leer_celular(registro_celular);
end;
close(archivo_celulares);
textcolor(green); write('Se han agregado '); textcolor(yellow); write(celulares);
textcolor(green); writeln(' celulares al final del archivo');
end;
procedure modificar_stock_celular(var archivo_celulares: t_archivo_celulares);
var
  registro_celular: t_registro_celular;
  codigo: int16;
  ok: boolean;
begin
  codigo:=1+random(1000);
  ok:=false;
  reset(archivo_celulares);
  while ((not eof(archivo_celulares)) and (ok=false)) do
  begin
    read(archivo_celulares,registro_celular);
    if (registro_celular.codigo=codigo) then
    begin
      registro_celular.stock_disponible:=random(101);
      seek(archivo_celulares,filepos(archivo_celulares)-1);
      write(archivo_celulares,registro_celular);
      ok:=true;
    end;
  end;
  close(archivo_celulares);
  if (ok=true) then
  begin
    textcolor(green); write('Se ha modificado el stock del celular con código ');
    textcolor(yellow); write(codigo); textcolor(green); writeln(' en el archivo');
  end
  else
  begin
    textcolor(green); write('No se ha encontrado el celular con código '); textcolor(yellow);
    write(codigo); textcolor(green); writeln(' en el archivo');
  end;
end;
procedure exportar_archivo_txt2(var archivo_celulares: t_archivo_celulares);
var
  registro_celular: t_registro_celular;
  archivo_txt: text;
begin
  reset(archivo_celulares);
  assign(archivo_txt,'E6_SinStock.txt'); rewrite(archivo_txt);
  while (not eof(archivo_celulares)) do
  begin
    read(archivo_celulares,registro_celular);
    if (registro_celular.stock_disponible=stock_disponible_corte) then
    with registro_celular do
    begin
      writeln(archivo_txt,codigo,' ',precio:0:2,' ',marca);
      writeln(archivo_txt,stock_disponible,' ',stock_minimo,' ',descripcion);
      writeln(archivo_txt,nombre);
    end;
  end;
  close(archivo_celulares);
end;

```

```

close(archivo_txt);
textcolor(green); write('Se ha exportado el contenido del archivo binario a un archivo de
texto denominado '); textcolor(yellow); write('"SinStock.txt"'); textcolor(green); writeln('
con aquellos celulares que tienen stock 0');
end;
procedure leer_opcion(var opcion: int8);
begin
textcolor(red); writeln('MENÚ DE OPCIONES');
textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear un archivo de
registros no ordenados de celulares y cargarlo con datos ingresados desde un archivo de texto
denominado "celulares1.txt"');
textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de aquellos celulares que tengan un stock menor al stock mínimo');
textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Listar en pantalla los
celulares del archivo cuya descripción contenga una cadena de caracteres proporcionada por el
usuario');
textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Exportar el archivo
creado en el inciso (a) a un archivo de texto denominado "celulares2.txt" con todos los
celulares del mismo');
textcolor(yellow); write('OPCIÓN 5: '); textcolor(green); writeln('Añadir uno o más
celulares al final del archivo con sus datos ingresados por teclado');
textcolor(yellow); write('OPCIÓN 6: '); textcolor(green); writeln('Modificar el stock de un
celular dado');
textcolor(yellow); write('OPCIÓN 7: '); textcolor(green); writeln('Exportar el contenido del
archivo binario a un archivo de texto denominado "SinStock.txt" con aquellos celulares que
tengan stock 0');
textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
writeln();
end;
procedure menu_opciones(var archivo_celulares: t_archivo_celulares; var archivo_carga: text);
var
opcion: int8;
begin
leer_opcion(opcion);
while (opcion<>opcion_salida) do
begin
case opcion of
1: cargar_archivo_celulares(archivo_celulares,archivo_carga);
2: imprimir1_archivo_celulares(archivo_celulares);
3: imprimir2_archivo_celulares(archivo_celulares);
4: exportar_archivo_txt1(archivo_celulares);
5: agregar_celular(archivo_celulares);
6: modificar_stock_celular(archivo_celulares);
7: exportar_archivo_txt2(archivo_celulares);
else
textcolor(green); writeln('La opción ingresada no corresponde a ninguna del menú de
opciones');
end;
writeln();
leer_opcion(opcion);
end;
end;
var
archivo_celulares: t_archivo_celulares;
archivo_carga: text;
begin
randomize;
assign(archivo_carga,'E6_celulares1.txt');
assign(archivo_celulares,'E6_celulares2');
cargar_archivo_carga(archivo_carga);
menu_opciones(archivo_celulares,archivo_carga);
end.

```

Ejercicio 7.

Realizar un programa que permita:

(a) *Crear un archivo binario a partir de la información almacenada en un archivo de texto. El nombre del archivo de texto es: "novelas.txt". La información en el archivo de texto consiste en: código de novela, nombre, género y precio de diferentes novelas argentinas. Los datos de cada novela se almacenan en dos líneas en el archivo de texto. La primera línea contendrá la siguiente información: código novela, precio y género; y la segunda línea almacenará el nombre de la novela.*

(b) *Abrir el archivo binario y permitir la actualización del mismo. Se debe poder agregar una novela y modificar una existente. Las búsquedas se realizan por código de novela.*

Nota: El nombre del archivo binario es proporcionado por el usuario desde el teclado.

```

program TP1_E7;
{$codepage UTF8}
uses crt, sysutils;
const
  codigo_salida=0;
  opcion_salida=0;
type
  t_string20=string[20];
  t_registro_novela=record
    codigo: int16;
    nombre: t_string20;
    genero: t_string20;
    precio: real;
  end;
  t_archivo_novelas=file of t_registro_novela;
function random_string(length: int8): t_string20;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
    random_string:=string_aux;
  end;
procedure leer_novela(var registro_novela: t_registro_novela; ok: boolean);
var
  i: int8;
begin
  if (ok=true) then
  begin
    i:=random(100);
    if (i=0) then
      registro_novela.codigo:=codigo_salida
    else
      registro_novela.codigo:=1+random(1000);
    end;
    if (registro_novela.codigo<>codigo_salida) then
    begin
      registro_novela.nombre:=random_string(5+random(15));
      registro_novela.genero:=random_string(5+random(15));
      registro_novela.precio:=1+random(100);
    end;
  end;
end;

```



```

procedure cargar_archivo_carga(var archivo_carga: text);
var
    registro_novela: t_registro_novela;
begin
    rewrite(archivo_carga);
    leer_novela(registro_novela,true);
    while (registro_novela.codigo<>codigo_salida) do
    begin
        with registro_novela do
        begin
            writeln(archivo_carga,codigo,' ',precio:0:2,' ',genero);
            writeln(archivo_carga,nombre);
        end;
        leer_novela(registro_novela,true);
    end;
    close(archivo_carga);
end;
procedure cargar_archivo_novelas(var archivo_novelas: t_archivo_novelas; var archivo_carga:
text);
var
    registro_novela: t_registro_novela;
begin
    rewrite(archivo_novelas);
    reset(archivo_carga);
    while (not eof(archivo_carga)) do
    begin
        with registro_novela do
        begin
            readln(archivo_carga,codigo,precio,genero); genero:=trim(genero);
            readln(archivo_carga,nombre);
        end;
        write(archivo_novelas,registro_novela);
    end;
    close(archivo_novelas);
    close(archivo_carga);
    textcolor(green); writeln('El archivo binario de novelas fue creado y cargado con éxito');
end;
function control_unicidad(var archivo_novelas: t_archivo_novelas; codigo: int16): boolean;
var
    registro_novela: t_registro_novela;
    ok: boolean;
begin
    ok:=false;
    while ((not eof(archivo_novelas)) and (ok=false)) do
    begin
        read(archivo_novelas,registro_novela);
        if (registro_novela.codigo=codigo) then
            ok:=true;
        end;
        control_unicidad:=ok;
    end;
end;
procedure agregar_novela(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
    novelas: int16;
begin
    novelas:=0;
    reset(archivo_novelas);
    leer_novela(registro_novela,true);
    while (registro_novela.codigo<>codigo_salida) do
    begin
        if (control_unicidad(archivo_novelas,registro_novela.codigo)=false) then
        begin
            seek(archivo_novelas,filesize(archivo_novelas));
            write(archivo_novelas,registro_novela);
            novelas:=novelas+1;
        end;
    end;
end;

```

```

    end;
    leer_novela(registro_novela,true);
end;
close(archivo_novelas);
textcolor(green); write('Se han agregado '); textcolor(yellow); write(novelas);
textcolor(green); writeln(' novelas al final del archivo');
end;
procedure modificar_novela(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
    codigo: int16;
    ok: boolean;
begin
    codigo:=1+random(1000);
    ok:=false;
    reset(archivo_novelas);
    while ((not eof(archivo_novelas)) and (ok=false)) do
    begin
        read(archivo_novelas,registro_novela);
        if (registro_novela.codigo=codigo) then
        begin
            leer_novela(registro_novela,false);
            seek(archivo_novelas,filepos(archivo_novelas)-1);
            write(archivo_novelas,registro_novela);
            ok:=true;
        end;
    end;
    close(archivo_novelas);
    if (ok=true) then
    begin
        textcolor(green); write('Se ha modificado la novela con código '); textcolor(yellow);
        write(codigo); textcolor(green); writeln(' en el archivo');
    end
    else
    begin
        textcolor(green); write('No se ha encontrado la novela con código '); textcolor(yellow);
        write(codigo); textcolor(green); writeln(' en el archivo');
    end;
end;
procedure exportar_archivo_txt(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
    archivo_txt: text;
begin
    reset(archivo_novelas);
    assign(archivo_txt,'E7_novelas2.txt'); rewrite(archivo_txt);
    while (not eof(archivo_novelas)) do
    begin
        read(archivo_novelas,registro_novela);
        with registro_novela do
        begin
            writeln(archivo_txt,codigo,' ',precio:0:2,' ',genero);
            writeln(archivo_txt,nombre);
        end;
    end;
    close(archivo_novelas);
    close(archivo_txt);
    textcolor(green); write('Se ha exportado el archivo creado en el inciso (a) a un archivo de
    texto denominado '); textcolor(yellow); write('"novelas2.txt"'); textcolor(green); writeln('
    con todas las novelas del mismo');
end;
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear un archivo binario
    a partir de la información almacenada en un archivo de texto denominado "novelas.txt"');

```

```
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Añadir una o más novelas
al final del archivo con sus datos ingresados por teclado');
    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Modificar una novela
existente');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Exportar el archivo
creado en el inciso (a) a un archivo de texto denominado "novelas2.txt" con todas las novelas
del mismo');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_novelas: t_archivo_novelas; var archivo_carga: text);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
    begin
        case opcion of
            1: cargar_archivo_novelas(archivo_novelas,archivo_carga);
            2: agregar_novela(archivo_novelas);
            3: modificar_novela(archivo_novelas);
            4: exportar_archivo_txt(archivo_novelas);
            else
                textcolor(green); writeln('La opción ingresada no corresponde a ninguna del menú de
opciones');
            end;
            writeln();
            leer_opcion(opcion);
        end;
    end;
end;
var
    archivo_novelas: t_archivo_novelas;
    archivo_carga: text;
begin
    randomize;
    assign(archivo_carga,'E7_novelas1.txt');
    assign(archivo_novelas,'E7_novelas2');
    cargar_archivo_carga(archivo_carga);
    menu_opciones(archivo_novelas,archivo_carga);
end.
```

Introducción a las Bases de Datos

Fundamentos de Organización de Datos

Práctica 2

Archivos Secuenciales ordenados - Algorítmica Clásica

1. Una empresa posee un archivo con información de los ingresos percibidos por diferentes empleados en concepto de comisión, de cada uno de ellos se conoce: código de empleado, nombre y monto de la comisión. La información del archivo se encuentra ordenada por código de empleado y cada empleado puede aparecer más de una vez en el archivo de comisiones.

Realice un procedimiento que reciba el archivo anteriormente descrito y lo compacte. En consecuencia, deberá generar un nuevo archivo en el cual, cada empleado aparezca una única vez con el valor total de sus comisiones.

NOTA: No se conoce a priori la cantidad de empleados. Además, el archivo debe ser recorrido una única vez.

2. El encargado de ventas de un negocio de productos de limpieza desea administrar el stock de los productos que vende. Para ello, genera un archivo maestro donde figuran todos los productos que comercializa. De cada producto se maneja la siguiente información: código de producto, nombre comercial, precio de venta, stock actual y stock mínimo. Diariamente se genera un archivo detalle donde se registran todas las ventas de productos realizadas. De cada venta se registran: código de producto y cantidad de unidades vendidas. Se pide realizar un programa con opciones para:
 - a. Actualizar el archivo maestro con el archivo detalle, sabiendo que:
 - Ambos archivos están ordenados por código de producto.
 - Cada registro del maestro puede ser actualizado por 0, 1 ó más registros del archivo detalle.
 - El archivo detalle sólo contiene registros que están en el archivo maestro.
 - b. Listar en un archivo de texto llamado "*stock_minimo.txt*" aquellos productos cuyo stock actual esté por debajo del stock mínimo permitido.

3. A partir de información sobre la alfabetización en la Argentina, se necesita actualizar un archivo que contiene los siguientes datos: nombre de provincia, cantidad de personas alfabetizadas y total de encuestados. Se reciben dos archivos detalle provenientes de dos agencias de censo diferentes, dichos archivos contienen: nombre de la provincia, código de localidad, cantidad de alfabetizados y cantidad de encuestados. Se pide realizar los módulos necesarios para actualizar el archivo maestro a partir de los dos archivos detalle.

NOTA: Los archivos están ordenados por nombre de provincia y en los archivos detalle pueden venir 0, 1 ó más registros por cada provincia.

4. Se cuenta con un archivo de productos de una cadena de venta de alimentos congelados. De cada producto se almacena: código del producto, nombre, descripción, stock disponible, stock mínimo y precio del producto.

Se recibe diariamente un archivo detalle de cada una de las 30 sucursales de la cadena. Se debe realizar el procedimiento que recibe los 30 detalles y actualiza el stock del archivo maestro. La información que se recibe en los detalles es: código de producto y cantidad vendida. Además, se deberá informar en un archivo de texto: nombre de producto, descripción, stock disponible y precio de aquellos productos que tengan stock disponible por debajo del stock mínimo. Pensar alternativas sobre realizar el informe en el mismo procedimiento de actualización, o realizarlo en un procedimiento separado (analizar ventajas/desventajas en cada caso).

Nota: todos los archivos se encuentran ordenados por código de productos. En cada detalle puede venir 0 o N registros de un determinado producto.

5. Suponga que trabaja en una oficina donde está montada una LAN (red local). La misma fue construida sobre una topología de red que conecta 5 máquinas entre sí y todas las máquinas se conectan con un servidor central. Semanalmente cada máquina genera un archivo de logs informando las sesiones abiertas por cada usuario en cada terminal y por cuánto tiempo estuvo abierta. Cada archivo detalle contiene los siguientes campos: cod_usuario, fecha, tiempo_sesion. Debe realizar un procedimiento que reciba los archivos detalle y genere un archivo maestro con los siguientes datos: cod_usuario, fecha, tiempo_total_de_sesiones_abiertas.

Notas:

- Cada archivo detalle está ordenado por cod_usuario y fecha.
- Un usuario puede iniciar más de una sesión el mismo día en la misma máquina, o inclusive, en diferentes máquinas.
- El archivo maestro debe crearse en la siguiente ubicación física: /var/log.

6. Se desea modelar la información necesaria para un sistema de recuentos de casos de covid para el ministerio de salud de la provincia de buenos aires.

Diariamente se reciben archivos provenientes de los distintos municipios, la información contenida en los mismos es la siguiente: código de localidad, código cepa, cantidad de casos activos, cantidad de casos nuevos, cantidad de casos recuperados, cantidad de casos fallecidos.

El ministerio cuenta con un archivo maestro con la siguiente información: código localidad, nombre localidad, código cepa, nombre cepa, cantidad de casos activos, cantidad de casos nuevos, cantidad de recuperados y cantidad de fallecidos.

Se debe realizar el procedimiento que permita actualizar el maestro con los detalles recibidos, se reciben 10 detalles. **Todos los archivos están ordenados por código de localidad y código de cepa.**

Para la actualización se debe proceder de la siguiente manera:

1. Al número de fallecidos se le suman el valor de fallecidos recibido del detalle.
2. Idem anterior para los recuperados.
3. Los casos activos se actualizan con el valor recibido en el detalle.
4. Idem anterior para los casos nuevos hallados.

Realice las declaraciones necesarias, el programa principal y los procedimientos que requiera para la actualización solicitada e informe cantidad de localidades con más de 50 casos activos (las localidades pueden o no haber sido actualizadas).

7. Se dispone de un archivo maestro con información de los alumnos de la Facultad de Informática. Cada registro del archivo maestro contiene: código de alumno, apellido, nombre, cantidad de cursadas aprobadas y cantidad de materias con final aprobado. El archivo maestro está ordenado por código de alumno.

Además, se tienen dos archivos detalle con información sobre el desempeño académico de los alumnos: un archivo de cursadas y un archivo de exámenes finales. El archivo de cursadas contiene información sobre las materias cursadas por los alumnos. Cada registro incluye: código de alumno, código de materia, año de cursada y resultado (solo interesa si la cursada fue aprobada o desaprobada). Por su parte, el archivo de exámenes finales contiene información sobre los exámenes finales rendidos. Cada registro incluye: código de alumno, código de materia, fecha del examen y nota obtenida. **Ambos archivos detalle están ordenados por código de alumno y código de materia**, y pueden contener **0, 1 o más registros por alumno** en el archivo maestro. Un alumno podría cursar una materia muchas veces, así como también podría rendir el final de una materia en múltiples ocasiones.

Se debe desarrollar un programa que **actualice el archivo maestro**, ajustando la cantidad de cursadas aprobadas y la cantidad de materias con final aprobado, utilizando la información de los archivos detalle. Las reglas de actualización son las siguientes:

- Si un alumno aprueba una cursada, se incrementa en uno la cantidad de cursadas aprobadas.
- Si un alumno aprueba un examen final (nota ≥ 4), se incrementa en uno la cantidad de materias con final aprobado.

Notas:

- **Los archivos deben procesarse en un único recorrido.**
- No es necesario comprobar que no haya inconsistencias en la información de los archivos detalles. Esto es, no puede suceder que un alumno apruebe más de una vez la cursada de una misma materia (a lo sumo la aprueba una vez), algo similar ocurre con los exámenes finales.

8. Se quiere optimizar la gestión del consumo de yerba mate en distintas provincias de Argentina. Para ello, se cuenta con un archivo maestro que contiene la siguiente información: código de provincia, nombre de la provincia, cantidad de habitantes y cantidad total de kilos de yerba consumidos históricamente.

Cada mes, se reciben 16 archivos de relevamiento con información sobre el consumo de yerba en los distintos puntos del país. Cada archivo contiene: código de provincia y cantidad de kilos de yerba consumidos en ese relevamiento. Un archivo de relevamiento puede contener información de una o varias provincias, y una misma provincia puede aparecer cero, una o más veces en distintos archivos de relevamiento.

Tanto el archivo maestro como los archivos de relevamiento están ordenados por código de provincia.

Se desea realizar un programa que actualice el archivo maestro en base a la nueva información de consumo de yerba. Además, se debe informar en pantalla aquellas provincias (código y nombre) donde la cantidad total de yerba consumida supere los 10.000 kilos históricamente, junto con el promedio consumido de yerba por habitante. Es importante tener en cuenta tanto las provincias actualizadas como las que no fueron actualizadas.

Nota: cada archivo debe recorrerse una única vez.

9. Se cuenta con un archivo que posee información de las ventas que realiza una empresa a los diferentes clientes. Se necesita obtener un reporte con las ventas organizadas por cliente. Para ello, se deberá informar por pantalla: los datos personales del cliente, el total mensual (mes por mes cuánto compró) y finalmente el monto total comprado en el año por el cliente. Además, al finalizar el reporte, se debe informar el monto total de ventas obtenido por la empresa.

El formato del archivo maestro está dado por: cliente (cod cliente, nombre y apellido), año, mes, día y monto de la venta. El orden del archivo está dado por: cod cliente, año y mes.

Nota: tenga en cuenta que puede haber meses en los que los clientes no realizaron compras. No es necesario que informe tales meses en el reporte.

10. Se necesita contabilizar los votos de las diferentes mesas electorales registradas por provincia y localidad. Para ello, se posee un archivo con la siguiente información: código de provincia, código de localidad, número de mesa y cantidad de votos en dicha mesa. Presentar en pantalla un listado como se muestra a continuación:

Código de Provincia

Código de Localidad	Total de Votos
---------------------	----------------

.....	
-------	-------

.....	
-------	-------

Total de Votos Provincia: ____

Código de Provincia

Código de Localidad	Total de Votos
---------------------	----------------

.....	
-------	-------

Total de Votos Provincia: ____

.....

Total General de Votos: ____

NOTA: La información está ordenada por código de provincia y código de localidad.

11. Se tiene información en un archivo de las horas extras realizadas por los empleados de una empresa en un mes. Para cada empleado se tiene la siguiente información: departamento, división, número de empleado, categoría y cantidad de horas extras realizadas por el empleado. Se sabe que el archivo se encuentra ordenado por departamento, luego por división y, por último, por número de empleado. Presentar en pantalla un listado con el siguiente formato:

Departamento

División

Número de Empleado	Total de Hs.	Importe a cobrar
--------------------	--------------	------------------

.....		
-------	-------	-------

.....		
-------	-------	-------

Total de horas división: ____

Monto total por división: ____

División

.....

Total horas departamento: ____

Monto total departamento: ____

Para obtener el valor de la hora se debe cargar un arreglo desde un archivo de texto al iniciar el programa con el valor de la hora extra para cada categoría. La categoría varía de 1 a 15. En el archivo de texto debe haber una línea para cada categoría con el número de categoría y el valor de la hora, pero el arreglo debe ser de valores de horas, con la posición del valor coincidente con el número de categoría.

12. La empresa de software 'X' posee un servidor web donde se encuentra alojado el sitio web de la organización. En dicho servidor, se almacenan en un archivo todos los accesos que se realizan al sitio. La información que se almacena en el archivo es la siguiente: año, mes, día, idUsuario y tiempo de acceso al sitio de la organización. El archivo se encuentra ordenado por los siguientes criterios: año, mes, día e idUsuario.

Se debe realizar un procedimiento que genere un informe en pantalla, para ello se indicará el año calendario sobre el cual debe realizar el informe. El mismo debe respetar el formato mostrado a continuación:

Año : ---

Mes:-- 1

día:-- 1

idUsuario 1 Tiempo Total de acceso en el dia 1 mes 1

idUsuario N Tiempo total de acceso en el dia 1 mes 1

Tiempo total acceso dia 1 mes 1

día N

idUsuario 1 Tiempo Total de acceso en el dia N mes 1

idUsuario N Tiempo total de acceso en el dia N mes 1

Tiempo total acceso dia N mes 1

Total tiempo de acceso mes 1

Mes 12

día 1

idUsuario 1 Tiempo Total de acceso en el dia 1 mes 12

idUsuario N Tiempo total de acceso en el dia 1 mes 12

Tiempo total acceso dia 1 mes 12

```

día N
    idUsuario 1  Tiempo Total de acceso en el día N mes 12
    -----
    idUsuario N  Tiempo total de acceso en el día N mes 12
    Tiempo total acceso día N mes 12
    Total tiempo de acceso mes 12
    Total tiempo de acceso año

```

Se deberá tener en cuenta las siguientes aclaraciones:

- El año sobre el cual realizará el informe de accesos debe leerse desde el teclado.
- El año puede no existir en el archivo, en tal caso, debe informarse en pantalla “año no encontrado”.
- Debe definir las estructuras de datos necesarias.
- El recorrido del archivo debe realizarse una única vez procesando sólo la información necesaria.

13. Suponga que usted es administrador de un servidor de correo electrónico. En los logs del mismo (información guardada acerca de los movimientos que ocurren en el server) que se encuentra en la siguiente ruta: /var/log/logmail.dat se guarda la siguiente información: nro_usuario, nombreUsuario, nombre, apellido, cantidadMailEnviados. Diariamente el servidor de correo genera un archivo con la siguiente información: nro_usuario, cuentaDestino, cuerpoMensaje. Este archivo representa todos los correos enviados por los usuarios en un día determinado. Ambos archivos están ordenados por nro_usuario y se sabe que un usuario puede enviar cero, uno o más mails por día.

- Realice el procedimiento necesario para actualizar la información del log en un día particular. Defina las estructuras de datos que utilice su procedimiento.
- Genere un archivo de texto que contenga el siguiente informe dado un archivo detalle de un día determinado:

```
nro_usuarioX.....cantidadMensajesEnviados
```

```
.....
```

```
nro_usuarioX+n.....cantidadMensajesEnviados
```

Nota: tener en cuenta que en el listado deberán aparecer **todos** los usuarios que existen en el sistema. Considere la implementación de esta opción de las siguientes maneras:

- Como un procedimiento separado del punto a).
- En el mismo procedimiento de actualización del punto a). Qué cambios se requieren en el procedimiento del punto a) para realizar el informe en el mismo recorrido?

14. Una compañía aérea dispone de un archivo maestro donde guarda información sobre sus próximos vuelos. En dicho archivo se tiene almacenado el destino, fecha, hora de salida y la cantidad de asientos disponibles. La empresa recibe todos los días dos archivos detalles para actualizar el archivo maestro. En dichos archivos se tiene destino, fecha, hora de salida y cantidad de asientos comprados. Se sabe que los archivos están ordenados por destino más fecha y hora de salida, y que en los detalles pueden venir 0, 1 ó más registros por cada uno del maestro. Se pide realizar los módulos necesarios para:

- a. Actualizar el archivo maestro sabiendo que no se registró ninguna venta de pasaje sin asiento disponible.
- b. Generar una lista con aquellos vuelos (destino y fecha y hora de salida) que tengan menos de una cantidad específica de asientos disponibles. La misma debe ser ingresada por teclado.

NOTA: El archivo maestro y los archivos detalles sólo pueden recorrerse una vez.

15. Se desea modelar la información de una ONG dedicada a la asistencia de personas con carencias habitacionales. La ONG cuenta con un archivo maestro conteniendo información como se indica a continuación: Código pcia, nombre provincia, código de localidad, nombre de localidad, #viviendas sin luz, #viviendas sin gas, #viviendas de chapa, #viviendas sin agua, #viviendas sin sanitarios.

Mensualmente reciben detalles de las diferentes provincias indicando avances en las obras de ayuda en la edificación y equipamientos de viviendas en cada provincia. La información de los detalles es la siguiente: Código pcia, código localidad, #viviendas con luz, #viviendas construidas, #viviendas con agua, #viviendas con gas, #entrega sanitarios.

Se debe realizar el procedimiento que permita actualizar el maestro con los detalles recibidos, se reciben 10 detalles. **Todos los archivos están ordenados por código de provincia y código de localidad.**

Para la actualización del archivo maestro, se debe proceder de la siguiente manera:

- Al valor de viviendas sin luz se le resta el valor recibido en el detalle.
- Idem para viviendas sin agua, sin gas y sin sanitarios.
- A las viviendas de chapa se le resta el valor recibido de viviendas construidas

La misma combinación de provincia y localidad aparecen a lo sumo una única vez.

Realice las declaraciones necesarias, el programa principal y los procedimientos que requiera para la actualización solicitada e informe cantidad de localidades sin viviendas de chapa (las localidades pueden o no haber sido actualizadas).

16. La editorial X, autora de diversos semanarios, posee un archivo maestro con la información correspondiente a las diferentes emisiones de los mismos. De cada emisión se registra: fecha, código de semanario, nombre del semanario, descripción, precio, total de ejemplares y total de ejemplares vendidos.

Mensualmente se reciben 100 archivos detalles con las ventas de los semanarios en todo el país. La información que poseen los detalles es la siguiente: fecha, código de semanario y cantidad de ejemplares vendidos. Realice las declaraciones necesarias, la llamada al procedimiento y el procedimiento que recibe el archivo maestro y los 100 detalles y realice la actualización del archivo maestro en función de las ventas registradas. Además deberá informar fecha y semanario que tuvo más ventas y la misma información del semanario con menos ventas.

Nota: Todos los archivos están ordenados por fecha y código de semanario. No se realizan ventas de semanarios si no hay ejemplares para hacerlo

17. Una concesionaria de motos de la Ciudad de Chascomús, posee un archivo con información de las motos que posee a la venta. De cada moto se registra: código, nombre, descripción, modelo, marca y stock actual. Mensualmente se reciben 10 archivos detalles con información de las ventas de cada uno de los 10 empleados que trabajan. De cada archivo detalle se dispone de la siguiente información: código de moto, precio y fecha de la venta. Se debe realizar un proceso que actualice el stock del archivo maestro desde los archivos detalles. Además se debe informar cuál fue la moto más vendida.

NOTA: Todos los archivos están ordenados por código de la moto y el archivo maestro debe ser recorrido sólo una vez y en forma simultánea con los detalles.

18. Se cuenta con un archivo con información de los casos de COVID-19 registrados en los diferentes hospitales de la Provincia de Buenos Aires cada día. Dicho archivo contiene: código de localidad, nombre de localidad, código de municipio, nombre de municipio, código de hospital, nombre de hospital, fecha y cantidad de casos positivos detectados. El archivo está ordenado por localidad, luego por municipio y luego por hospital.

Escriba la definición de las estructuras de datos necesarias y un procedimiento que haga un listado con el siguiente formato:

Nombre: Localidad 1

Nombre: Municipio 1

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....

Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio 1

.....

Nombre Municipio N

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....
Nombre Hospital N.....Cantidad de casos Hospital N
Cantidad de casos Municipio N
Cantidad de casos Localidad 1

Nombre Localidad N

Nombre Municipio 1

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....

Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio 1

Nombre Municipio N

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....

Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio N

Cantidad de casos Localidad N

Cantidad de casos Totales en la Provincia

Además del informe en pantalla anterior, es necesario exportar a un archivo de texto la siguiente información: nombre de localidad, nombre de municipio y cantidad de casos del municipio, para aquellos municipios cuya cantidad de casos supere los 1500. El formato del archivo de texto deberá ser el adecuado para recuperar la información con la menor cantidad de lecturas posibles.

NOTA: El archivo debe recorrerse solo una vez.

19. A partir de un siniestro ocurrido se perdieron las actas de nacimiento y fallecimientos de toda la provincia de buenos aires de los últimos diez años. En pos de recuperar dicha información, se deberá procesar 2 archivos por cada una de las 50 delegaciones distribuidas en la provincia, un archivo de nacimientos y otro de fallecimientos y crear el archivo maestro reuniendo dicha información.

Los archivos detalles con nacimientos, contendrán la siguiente información: nro partida nacimiento, nombre, apellido, dirección detallada (calle, nro, piso, depto, ciudad), matrícula del médico, nombre y apellido de la madre, DNI madre, nombre y apellido del padre, DNI del padre.

En cambio, los 50 archivos de fallecimientos tendrán: nro partida nacimiento, DNI, nombre y apellido del fallecido, matrícula del médico que firma el deceso, fecha y hora del deceso y lugar.

Realizar un programa que cree el archivo maestro a partir de toda la información de los archivos detalles. Se debe almacenar en el maestro: nro partida nacimiento, nombre, apellido, dirección detallada (calle, nro, piso, depto, ciudad), matrícula del médico, nombre y apellido de la madre, DNI madre, nombre y apellido del padre, DNI del padre y si falleció,

además matrícula del médico que firma el deceso, fecha y hora del deceso y lugar. Se deberá, además, listar en un archivo de texto la información recolectada de cada persona.

Nota: Todos los archivos están ordenados por nro partida de nacimiento que es única. Tenga en cuenta que no necesariamente va a fallecer en el distrito donde nació la persona y además puede no haber fallecido.

IMPORTANTE: Se recomienda implementar los ejercicios prácticos en Dev-Pascal. El ejecutable puede descargarse desde la plataforma moodle.

Trabajo Práctico N° 2:

Archivos Secuenciales Ordenados - Algorítmica Clásica.

Ejercicio 1.

Una empresa posee un archivo con información de los ingresos percibidos por diferentes empleados en concepto de comisión. De cada uno de ellos, se conoce: código de empleado, nombre y monto de la comisión. La información del archivo se encuentra ordenada por código de empleado y cada empleado puede aparecer más de una vez en el archivo de comisiones.

Realizar un procedimiento que reciba el archivo anteriormente descrito y lo compacte. En consecuencia, deberá generar un nuevo archivo en el cual cada empleado aparezca una única vez con el valor total de sus comisiones.

Nota: No se conoce, a priori, la cantidad de empleados. Además, el archivo debe ser recorrido una única vez.

```
program TP2_E1;
{$codepage UTF8}
uses crt, sysutils;
const
    codigo_salida=999;
type
    t_string20=string[20];
    t_registro_empleado=record
        codigo: int16;
        nombre: t_string20;
        comision: real;
    end;
    t_archivo_empleados=file of t_registro_empleado;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_empleados; var
archivo_carga_detalle: text);
var
    registro_empleado: t_registro_empleado;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_empleado do
            begin
                readln(archivo_carga_detalle,codigo,comision,nombre); nombre:=trim(nombre);
                write(archivo_detalle,registro_empleado);
            end;
        end;
        close(archivo_detalle);
        close(archivo_carga_detalle);
    end;
procedure imprimir_registro_empleado(registro_empleado: t_registro_empleado);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_empleado.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_empleado.nombre);
    textcolor(green); write('; Comisión: $'); textcolor(yellow);
    writeln(registro_empleado.comision:0:2);
end;
procedure imprimir_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
```

```
reset(archivo_empleados);
while (not eof(archivo_empleados)) do
begin
    read(archivo_empleados,registro_empleado);
    imprimir_registro_empleado(registro_empleado);
end;
close(archivo_empleados);
end;
procedure leer_empleado(var archivo_detalle: t_archivo_empleados; var registro_empleado:
t_registro_empleado);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_empleado)
    else
        registro_empleado.codigo:=codigo_salida;
    end;
end;
procedure cargar_archivo_maestro(var archivo_maestro, archivo_detalle: t_archivo_empleados);
var
    registro_empleado_detalle, registro_empleado_maestro: t_registro_empleado;
    comision_total: real;
begin
    rewrite(archivo_maestro);
    reset(archivo_detalle);
    leer_empleado(archivo_detalle,registro_empleado_detalle);
    while (registro_empleado_detalle.codigo<>codigo_salida) do
    begin
        registro_empleado_maestro:=registro_empleado_detalle;
        comision_total:=0;
        while (registro_empleado_maestro.codigo=registro_empleado_detalle.codigo) do
        begin
            comision_total:=comision_total+registro_empleado_detalle.comision;
            leer_empleado(archivo_detalle,registro_empleado_detalle);
        end;
        registro_empleado_maestro.comision:=comision_total;
        write(archivo_maestro,registro_empleado_maestro);
    end;
    close(archivo_maestro);
    close(archivo_detalle);
end;
var
    archivo_detalle, archivo_maestro: t_archivo_empleados;
    archivo_carga_detalle: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE:'); writeln();
    assign(archivo_detalle,'E1_empleadosDetalle');
    assign(archivo_carga_detalle,'E1_empleadosDetalle.txt');
    cargar_archivo_detalle(archivo_detalle,archivo_carga_detalle);
    imprimir_archivo_empleados(archivo_detalle);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E1_empleadosMaestro');
    cargar_archivo_maestro(archivo_maestro,archivo_detalle);
    imprimir_archivo_empleados(archivo_maestro);
end.
```


Ejercicio 2.

El encargado de ventas de un negocio de productos de limpieza desea administrar el stock de los productos que vende. Para ello, genera un archivo maestro donde figuran todos los productos que comercializa. De cada producto, se maneja la siguiente información: código de producto, nombre comercial, precio de venta, stock actual y stock mínimo. Diariamente, se genera un archivo detalle donde se registran todas las ventas de productos realizadas. De cada venta, se registran: código de producto y cantidad de unidades vendidas. Se pide realizar un programa con opciones para:

(a) Actualizar el archivo maestro con el archivo detalle, sabiendo que:

- Ambos archivos están ordenados por código de producto.
- Cada registro del maestro puede ser actualizado por 0, 1 o más registros del archivo detalle.
- El archivo detalle sólo contiene registros que están en el archivo maestro.

(b) Listar en un archivo de texto llamado “stock_minimo.txt” aquellos productos cuyo stock actual esté por debajo del stock mínimo permitido.

```
program TP2_E2;
{$codepage UTF8}
uses crt, sysutils;
const
  codigo_salida=999;
  opcion_salida=0;
type
  t_string20=string[20];
  t_registro_producto=record
    codigo: int16;
    nombre: t_string20;
    precio: real;
    stock_actual: int16;
    stock_minimo: int16;
  end;
  t_registro_venta=record
    codigo: int16;
    cantidad_vendida: int16;
  end;
  t_archivo_maestro=file of t_registro_producto;
  t_archivo_detalle=file of t_registro_venta;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_producto: t_registro_producto;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_producto do
      begin
        readln(archivo_carga_maestro,codigo,precio,stock_actual,stock_minimo,nombre);
        nombre:=trim(nombre);
        write(archivo_maestro,registro_producto);
      end;
    end;
  close(archivo_maestro);
  close(archivo_carga_maestro);
  textcolor(green); writeln('El archivo binario maestro fue creado y cargado con éxito');
```

```
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,codigo,cantidad_vendida);
                write(archivo_detalle,registro_venta);
            end;
        end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
    textcolor(green); writeln('El archivo binario detalle fue creado y cargado con éxito');
end;
procedure imprimir_registro_producto(registro_producto: t_registro_producto);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_producto.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_producto.nombre);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
write(registro_producto.precio:0:2);
    textcolor(green); write('; Stock actual: '); textcolor(yellow);
write(registro_producto.stock_actual);
    textcolor(green); write('; Stock mínimo: '); textcolor(yellow);
writeln(registro_producto.stock_minimo);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
begin
    reset(archivo_maestro);
    textcolor(green); writeln('Los productos del archivo maestro son: ');
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_producto);
            imprimir_registro_producto(registro_producto);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Cantidad vendida: '); textcolor(yellow);
writeln(registro_venta.cantidad_vendida);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    textcolor(green); writeln('Las ventas del archivo detalle son: ');
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_detalle);
end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_venta)
```

```

else
    registro_venta.codigo:=codigo_salida;
end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle: t_archivo_detalle);
var
    registro_producto: t_registro_producto;
    registro_venta: t_registro_venta;
begin
    reset(archivo_maestro);
    reset(archivo_detalle);
    leer_venta(archivo_detalle,registro_venta);
    while (registro_venta.codigo<>codigo_salida) do
    begin
        read(archivo_maestro,registro_producto);
        while (registro_producto.codigo<>registro_venta.codigo) do
            read(archivo_maestro,registro_producto);
        while (registro_producto.codigo=registro_venta.codigo) do
        begin
            if (registro_venta.cantidad_vendida>=registro_producto.stock_actual) then
                registro_producto.stock_actual:=0
            else
                registro_producto.stock_actual:=registro_producto.stock_actual-
registro_venta.cantidad_vendida;
                leer_venta(archivo_detalle,registro_venta);
            end;
            seek(archivo_maestro,filepos(archivo_maestro)-1);
            write(archivo_maestro,registro_producto);
        end;
        close(archivo_maestro);
        close(archivo_detalle);
        textcolor(green); writeln('El archivo maestro fue actualizado con éxito');
    end;
procedure exportar_archivo_txt(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
    archivo_txt: text;
begin
    reset(archivo_maestro);
    assign(archivo_txt,'E2_stock_minimo.txt'); rewrite(archivo_txt);
    textcolor(green); writeln('Los productos cuyo stock actual está por debajo del stock mínimo
son: ');
    while (not eof(archivo_maestro)) do
    begin
        read(archivo_maestro,registro_producto);
        if (registro_producto.stock_actual<registro_producto.stock_minimo) then
        begin
            imprimir_registro_producto(registro_producto);
            with registro_producto do
                writeln(archivo_txt,codigo,' ',nombre,' ',precio:0:2,' ',stock_actual,'
',stock_minimo);
            end;
        end;
        close(archivo_maestro);
        close(archivo_txt);
        textcolor(green); writeln('El archivo de texto "stock_minimo.txt" fue creado y cargado con
éxito');
    end;
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear archivos de
registros ordenados de productos y cargarlos con datos ingresados desde archivos de texto');
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de los productos del archivo maestro');

```

```

    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Listar en pantalla los
datos de los productos del archivo detalle');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Actualizar el archivo
maestro con el archivo detalle');
    textcolor(yellow); write('OPCIÓN 5: '); textcolor(green); writeln('Listar en un archivo de
texto llamado "stock_minimo.txt" aquellos productos cuyo stock actual esté por debajo del
stock mínimo permitido');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_maestro: t_archivo_maestro; var archivo_detalle:
t_archivo_detalle; var archivo_carga_maestro, archivo_carga_detalle: text);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
    begin
        case opcion of
            1:
                begin
                    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
                    cargar_archivo_detalle(archivo_detalle,archivo_carga_detalle);
                end;
            2: imprimir_archivo_maestro(archivo_maestro);
            3: imprimir_archivo_detalle(archivo_detalle);
            4: actualizar_archivo_maestro(archivo_maestro,archivo_detalle);
            5: exportar_archivo_txt(archivo_maestro);
            else
                textcolor(green); writeln('La opción ingresada no corresponde a ninguna del menú de
opciones');
            end;
            writeln();
            leer_opcion(opcion);
        end;
    end;
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle: t_archivo_detalle;
    archivo_carga_maestro, archivo_carga_detalle: text;
begin
    assign(archivo_maestro,'E2_productosMaestro');
    assign(archivo_carga_maestro,'E2_productosMaestro.txt');
    assign(archivo_detalle,'E2_ventasDetalle');
    assign(archivo_carga_detalle,'E2_ventasDetalle.txt');
    menu_opciones(archivo_maestro,archivo_detalle,archivo_carga_maestro,archivo_carga_detalle);
end.

```

Ejercicio 3.

A partir de información sobre la alfabetización en la Argentina, se necesita actualizar un archivo que contiene los siguientes datos: nombre de provincia, cantidad de personas alfabetizadas y total de encuestados. Se reciben dos archivos detalle provenientes de dos agencias de censo diferentes. Dichos archivos contienen: nombre de la provincia, código de localidad, cantidad de alfabetizados y cantidad de encuestados. Se pide realizar los módulos necesarios para actualizar el archivo maestro a partir de los dos archivos detalle.

Nota: Los archivos están ordenados por nombre de provincia y, en los archivos detalle, pueden venir 0, 1 o más registros por cada provincia.

```
program TP2_E3;
{$codepage UTF8}
uses crt, sysutils;
const
    nombre_salida='ZZZ';
type
    t_string50=string[50];
    t_registro_provincia1=record
        nombre: t_string50;
        alfabetizados: int16;
        encuestados: int16;
    end;
    t_registro_provincia2=record
        nombre: t_string50;
        codigo_localidad: int16;
        alfabetizados: int16;
        encuestados: int16;
    end;
    t_archivo_maestro=file of t_registro_provincia1;
    t_archivo_detalle=file of t_registro_provincia2;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_provincia: t_registro_provincia1;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_provincia do
            begin
                readln(archivo_carga_maestro,alfabetizados,encuestados,nombre); nombre:=trim(nombre);
                write(archivo_maestro,registro_provincia);
            end;
        end;
        close(archivo_maestro);
        close(archivo_carga_maestro);
    end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_provincia: t_registro_provincia2;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_provincia do
            begin
                readln(archivo_carga_detalle,codigo_localidad,alfabetizados,encuestados,nombre);
                nombre:=trim(nombre);
```

```
        write(archivo_detalle,registro_provincia);
    end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_provincia1(registro_provincia: t_registro_provincia1);
begin
    textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_provincia.nombre);
    textcolor(green); write('; Alfabetizados: '); textcolor(yellow);
write(registro_provincia.alfabetizados);
    textcolor(green); write('; Encuestados: '); textcolor(yellow);
writeln(registro_provincia.encuestados);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_provincia: t_registro_provincia1;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_provincia);
            imprimir_registro_provincia1(registro_provincia);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_provincia2(registro_provincia: t_registro_provincia2);
begin
    textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_provincia.nombre);
    textcolor(green); write('; Código de localidad: '); textcolor(yellow);
write(registro_provincia.codigo_localidad);
    textcolor(green); write('; Alfabetizados: '); textcolor(yellow);
write(registro_provincia.alfabetizados);
    textcolor(green); write('; Encuestados: '); textcolor(yellow);
writeln(registro_provincia.encuestados);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_provincia: t_registro_provincia2;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_provincia);
            imprimir_registro_provincia2(registro_provincia);
        end;
    close(archivo_detalle);
end;
procedure leer_provincia(var archivo_detalle: t_archivo_detalle; var registro_provincia:
t_registro_provincia2);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_provincia)
    else
        registro_provincia.nombre:=nombre_salida;
    end;
end;
procedure minimo(var archivo_detalle1, archivo_detalle2: t_archivo_detalle; var
registro_provincia1, registro_provincia2, min: t_registro_provincia2);
begin
    if (registro_provincia1.nombre<=registro_provincia2.nombre) then
        begin
            min:=registro_provincia1;
            leer_provincia(archivo_detalle1,registro_provincia1);
        end
    else
        begin
            min:=registro_provincia2;
```

```

    leer_provincia(archivo_detalle2, registro_provincia2);
end;
end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle1, archivo_detalle2: t_archivo_detalle);
var
    registro_provincia: t_registro_provincia1;
    registro_provincia1, registro_provincia2, min: t_registro_provincia2;
begin
    reset(archivo_maestro);
    reset(archivo_detalle1); reset(archivo_detalle2);
    leer_provincia(archivo_detalle1, registro_provincia1);
    leer_provincia(archivo_detalle2, registro_provincia2);
    minimo(archivo_detalle1, archivo_detalle2, registro_provincia1, registro_provincia2, min);
    while (min.nombre <> nombre_salida) do
    begin
        read(archivo_maestro, registro_provincia);
        while (registro_provincia.nombre <> min.nombre) do
            read(archivo_maestro, registro_provincia);
        while (registro_provincia.nombre = min.nombre) do
        begin
            registro_provincia.alfabetizados := registro_provincia.alfabetizados + min.alfabetizados;
            registro_provincia.encuestados := registro_provincia.encuestados + min.encuestados;
            minimo(archivo_detalle1, archivo_detalle2, registro_provincia1, registro_provincia2, min);
        end;
        seek(archivo_maestro, filepos(archivo_maestro) - 1);
        write(archivo_maestro, registro_provincia);
    end;
    close(archivo_maestro);
    close(archivo_detalle1); close(archivo_detalle2);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle1, archivo_detalle2: t_archivo_detalle;
    archivo_carga_maestro, archivo_carga_detalle1, archivo_carga_detalle2: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro, 'E3_provinciasMaestro');
    assign(archivo_carga_maestro, 'E3_provinciasMaestro.txt');
    cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE 1:'); writeln();
    assign(archivo_detalle1, 'E3_provinciasDetalle1');
    assign(archivo_carga_detalle1, 'E3_provinciasDetalle1.txt');
    cargar_archivo_detalle(archivo_detalle1, archivo_carga_detalle1);
    imprimir_archivo_detalle(archivo_detalle1);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE 2:'); writeln();
    assign(archivo_detalle2, 'E3_provinciasDetalle2');
    assign(archivo_carga_detalle2, 'E3_provinciasDetalle2.txt');
    cargar_archivo_detalle(archivo_detalle2, archivo_carga_detalle2);
    imprimir_archivo_detalle(archivo_detalle2);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro, archivo_detalle1, archivo_detalle2);
    imprimir_archivo_maestro(archivo_maestro);
end.

```

Ejercicio 4.

Se cuenta con un archivo de productos de una cadena de venta de alimentos congelados. De cada producto, se almacena: código del producto, nombre, descripción, stock disponible, stock mínimo y precio del producto.

Se recibe, diariamente, un archivo detalle de cada una de las 30 sucursales de la cadena. Se debe realizar el procedimiento que recibe los 30 detalles y actualiza el stock del archivo maestro. La información que se recibe en los detalles es: código de producto y cantidad vendida. Además, se deberá informar en un archivo de texto: nombre de producto, descripción, stock disponible y precio de aquellos productos que tengan stock disponible por debajo del stock mínimo. Pensar alternativas sobre realizar el informe en el mismo procedimiento de actualización o realizarlo en un procedimiento separado (analizar ventajas/desventajas en cada caso).

Nota: Todos los archivos se encuentran ordenados por código de producto. En cada detalle, puede venir 0 o N registros de un determinado producto.

```
program TP2_E4;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=30;
  codigo_salida=999;
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_producto=record
    codigo: int16;
    nombre: t_string20;
    descripcion: t_string20;
    stock_disponible: int16;
    stock_minimo: int16;
    precio: real;
  end;
  t_registro_venta=record
    codigo: int16;
    cantidad_vendida: int16;
  end;
  t_archivo_maestro=file of t_registro_producto;
  t_archivo_detalle=file of t_registro_venta;
  t_vector_ventas=array[t_detalle] of t_registro_venta;
  t_vector_detalle=array[t_detalle] of t_archivo_detalle;
  t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_producto: t_registro_producto;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_producto do
      begin
        readln(archivo_carga_maestro,codigo,stock_disponible,stock_minimo,precio,nombre);
        nombre:=trim(nombre);
        readln(archivo_carga_maestro,descripcion);
        write(archivo_maestro,registro_producto);
      end;
    end;
  end;
```



```
close(archivo_maestro);
close(archivo_carga_maestro);
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,codigo,cantidad_vendida);
                write(archivo_detalle,registro_venta);
            end;
        end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_producto(registro_producto: t_registro_producto);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_producto.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_producto.nombre);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_producto.descripcion);
    textcolor(green); write('; Stock disponible: '); textcolor(yellow);
write(registro_producto.stock_disponible);
    textcolor(green); write('; Stock mínimo: '); textcolor(yellow);
write(registro_producto.stock_minimo);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
writeln(registro_producto.precio:0:2);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_producto);
            imprimir_registro_producto(registro_producto);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Cantidad vendida: '); textcolor(yellow);
writeln(registro_venta.cantidad_vendida);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_detalle);
end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_detalle)) then
```

```

    read(archivo_detalle,registro_venta)
  else
    registro_venta.codigo:=codigo_salida;
  end;
procedure minimo(var vector_detalle: t_vector_detalle; var vector_ventas: t_vector_ventas;
var min: t_registro_venta);
var
  i, pos: t_detalle;
begin
  min.codigo:=codigo_salida;
  for i:= 1 to detalles_total do
    if (vector_ventas[i].codigo<min.codigo) then
      begin
        min:=vector_ventas[i];
        pos:=i;
      end;
    if (min.codigo<codigo_salida) then
      leer_venta(vector_detalle[pos],vector_ventas[pos]);
    end;
  end;
procedure exportar_archivo_txt(var archivo_maestro: t_archivo_maestro);
var
  registro_producto: t_registro_producto;
  archivo_txt: text;
begin
  reset(archivo_maestro);
  assign(archivo_txt,'E4_stock_minimo.txt'); rewrite(archivo_txt);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro,registro_producto);
      if (registro_producto.stock_disponible<registro_producto.stock_minimo) then
        with registro_producto do
          writeln(archivo_txt,nombre,' ',descripcion,' ',stock_disponible,' ',precio:0:2);
        end;
      close(archivo_txt);
    end;
  end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
  registro_producto: t_registro_producto;
  min: t_registro_venta;
  vector_ventas: t_vector_ventas;
  i: t_detalle;
begin
  reset(archivo_maestro);
  for i:= 1 to detalles_total do
    begin
      reset(vector_detalle[i]);
      leer_venta(vector_detalle[i],vector_ventas[i]);
    end;
  minimo(vector_detalle,vector_ventas,min);
  while (min.codigo<>codigo_salida) do
    begin
      read(archivo_maestro,registro_producto);
      while (registro_producto.codigo<>min.codigo) do
        read(archivo_maestro,registro_producto);
      while (registro_producto.codigo=min.codigo) do
        begin
          if (min.cantidad_vendida>=registro_producto.stock_disponible) then
            registro_producto.stock_disponible:=0
          else
            registro_producto.stock_disponible:=registro_producto.stock_disponible-
min.cantidad_vendida;
            minimo(vector_detalle,vector_ventas,min);
          end;
        seek(archivo_maestro,filepos(archivo_maestro)-1);
        write(archivo_maestro,registro_producto);
      end;
    end;
  end;
end;

```

```
end;
exportar_archivo_txt(archivo_maestro);
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E4_productosMaestro');
    assign(archivo_carga_maestro,'E4_productosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
            assign(vector_detalle[i],'E4_ventasDetalle'+intToStr(i));
            assign(vector_carga_detalle[i],'E4_ventasDetalle'+intToStr(i)+'.txt');
            cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
            imprimir_archivo_detalle(vector_detalle[i]);
        end;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro,vector_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 5.

Suponer que se trabaja en una oficina donde está montada una LAN (red local). La misma fue construida sobre una topología de red que conecta 5 máquinas entre sí y todas las máquinas se conectan con un servidor central. Semanalmente, cada máquina genera un archivo de logs informando las sesiones abiertas por cada usuario en cada terminal y por cuánto tiempo estuvo abierta. Cada archivo detalle contiene los siguientes campos: *cod_usuario*, *fecha*, *tiempo_sesion*. Se debe realizar un procedimiento que reciba los archivos detalle y genere un archivo maestro con los siguientes datos: *cod_usuario*, *fecha*, *tiempo_total_de_sesiones_abiertas*.

Notas:

- Cada archivo detalle está ordenado por *cod_usuario* y *fecha*.
- Un usuario puede iniciar más de una sesión el mismo día en la misma máquina o, inclusive, en diferentes máquinas.
- El archivo maestro debe crearse en la siguiente ubicación física: */var/log*.

```
program TP2_E5;
{$codepage UTF8}
uses crt, sysutils;
const
    detalles_total=3; // detalles_total=5;
    codigo_salida=999;
type
    t_detalle=1..detalles_total;
    t_string20=string[20];
    t_registro_sesion=record
        codigo: int16;
        fecha: t_string20;
        tiempo: int16;
    end;
    t_archivo_sesiones=file of t_registro_sesion;
    t_vector_sesiones=array[t_detalle] of t_registro_sesion;
    t_vector_detalle=array[t_detalle] of t_archivo_sesiones;
    t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_sesiones; var
archivo_carga_detalle: text);
var
    registro_sesion: t_registro_sesion;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_sesion do
            begin
                readln(archivo_carga_detalle,codigo,tiempo,fecha); fecha:=trim(fecha);
                write(archivo_detalle,registro_sesion);
            end;
        close(archivo_detalle);
        close(archivo_carga_detalle);
    end;
procedure imprimir_registro_sesion(registro_sesion: t_registro_sesion);
begin
    textcolor(green); write('Código de usuario: '); textcolor(yellow);
write(registro_sesion.codigo);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_sesion.fecha);
    textcolor(green); write('; Tiempo: '); textcolor(yellow); writeln(registro_sesion.tiempo);
end;
```

```
procedure imprimir_archivo_sesiones(var archivo_sesiones: t_archivo_sesiones);
var
    registro_sesion: t_registro_sesion;
begin
    reset(archivo_sesiones);
    while (not eof(archivo_sesiones)) do
        begin
            read(archivo_sesiones, registro_sesion);
            imprimir_registro_sesion(registro_sesion);
        end;
    close(archivo_sesiones);
end;

procedure leer_sesion(var archivo_detalle: t_archivo_sesiones; var registro_sesion:
t_registro_sesion);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_sesion)
    else
        registro_sesion.codigo:=codigo_salida;
    end;
end;

procedure minimo(var vector_detalle: t_vector_detalle; var vector_sesiones:
t_vector_sesiones; var min: t_registro_sesion);
var
    i, pos: t_detalle;
begin
    min.codigo:=codigo_salida;
    for i:= 1 to detalles_total do
        if ((vector_sesiones[i].codigo<min.codigo) or ((vector_sesiones[i].codigo=min.codigo) and
(vector_sesiones[i].fecha<min.fecha))) then
            begin
                min:=vector_sesiones[i];
                pos:=i;
            end;
        if (min.codigo<codigo_salida) then
            leer_sesion(vector_detalle[pos], vector_sesiones[pos]);
        end;
    end;

procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_sesiones; var vector_detalle:
t_vector_detalle);
var
    registro_sesion, min: t_registro_sesion;
    vector_sesiones: t_vector_sesiones;
    i: t_detalle;
begin
    rewrite(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_sesion(vector_detalle[i], vector_sesiones[i]);
        end;
    minimo(vector_detalle, vector_sesiones, min);
    while (min.codigo<>codigo_salida) do
        begin
            registro_sesion.codigo:=min.codigo;
            while (registro_sesion.codigo=min.codigo) do
                begin
                    registro_sesion.fecha:=min.fecha;
                    registro_sesion.tiempo:=0;
                    while ((registro_sesion.codigo=min.codigo) and (registro_sesion.fecha=min.fecha)) do
                        begin
                            registro_sesion.tiempo:=registro_sesion.tiempo+min.tiempo;
                            minimo(vector_detalle, vector_sesiones, min);
                        end;
                    write(archivo_maestro, registro_sesion);
                end;
            end;
        end;
    close(archivo_maestro);
```

```
    for i:= 1 to detalles_total do
        close(vector_detalle[i]);
    end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_sesiones;
    i: t_detalle;
begin
    for i:= 1 to detalles_total do
        begin
            writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
            assign(vector_detalle[i], 'E5_sesionesDetalle'+inttoStr(i));
            assign(vector_carga_detalle[i], 'E5_sesionesDetalle'+inttoStr(i)+'.txt');
            cargar_archivo_detalle(vector_detalle[i], vector_carga_detalle[i]);
            imprimir_archivo_sesiones(vector_detalle[i]);
        end;
        writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
        assign(archivo_maestro, 'E5_sesionesMaestro');
        cargar_archivo_maestro(archivo_maestro, vector_detalle);
        imprimir_archivo_sesiones(archivo_maestro);
    end.
end.
```

Ejercicio 6.

Se desea modelar la información necesaria para un sistema de recuentos de casos de COVID para el Ministerio de Salud de la Provincia de Buenos Aires.

Diariamente, se reciben archivos provenientes de los distintos municipios. La información contenida en los mismos es la siguiente: código de localidad, código cepa, cantidad de casos activos, cantidad de casos nuevos, cantidad de casos recuperados, cantidad de casos fallecidos.

El ministerio cuenta con un archivo maestro con la siguiente información: código localidad, nombre localidad, código cepa, nombre cepa, cantidad de casos activos, cantidad de casos nuevos, cantidad de recuperados y cantidad de fallecidos.

Se debe realizar el procedimiento que permita actualizar el maestro con los detalles recibidos, se reciben 10 detalles. Todos los archivos están ordenados por código de localidad y código de cepa.

Para la actualización, se debe proceder de la siguiente manera:

- *Al número de fallecidos se le suman el valor de fallecidos recibido del detalle.*
- *Ídem anterior para los recuperados.*
- *Los casos activos se actualizan con el valor recibido en el detalle.*
- *Ídem anterior para los casos nuevos hallados.*

Realizar las declaraciones necesarias, el programa principal y los procedimientos que requiera para la actualización solicitada e informar cantidad de localidades con más de 50 casos activos (las localidades pueden o no haber sido actualizadas).

```
program TP2_E6;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=10;
  codigo_salida_detalle=999;
  codigo_salida_maestro=9999;
  casos_activos_corte=50;
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_localidad1=record
    codigo: int16;
    nombre: t_string20;
    codigo_cepa: int16;
    nombre_cepa: t_string20;
    casos_activos: int16;
    casos_nuevos: int16;
    casos_recuperados: int16;
    casos_fallecidos: int16;
  end;
  t_registro_localidad2=record
    codigo: int16;
    codigo_cepa: int16;
    casos_activos: int32;
    casos_nuevos: int16;
```

```

    casos_recuperados: int16;
    casos_fallecidos: int16;
end;
t_archivo_maestro=file of t_registro_localidad1;
t_archivo_detalle=file of t_registro_localidad2;
t_vector_localidades=array[t_detalle] of t_registro_localidad2;
t_vector_detalle=array[t_detalle] of t_archivo_detalle;
t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_localidad: t_registro_localidad1;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_localidad do
            begin
                readln(archivo_carga_maestro,codigo,codigo_cepa,casos_activos,casos_nuevos,casos_recuper
ados,casos_fallecidos,nombre_cepa); nombre_cepa:=trim(nombre_cepa);
                readln(archivo_carga_maestro,nombre);
                write(archivo_maestro,registro_localidad);
            end;
        end;
        close(archivo_maestro);
        close(archivo_carga_maestro);
    end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_localidad: t_registro_localidad2;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_localidad do
            begin
                readln(archivo_carga_detalle,codigo,codigo_cepa,casos_activos,casos_nuevos,casos_recuper
ados,casos_fallecidos);
                write(archivo_detalle,registro_localidad);
            end;
        end;
        close(archivo_detalle);
        close(archivo_carga_detalle);
    end;
procedure imprimir_registro_localidad1(registro_localidad: t_registro_localidad1);
begin
    textcolor(green); write('Código de localidad: '); textcolor(yellow);
write(registro_localidad.codigo);
    textcolor(green); write('; Nombre de localidad: '); textcolor(yellow);
write(registro_localidad.nombre);
    textcolor(green); write('; Código de cepa: '); textcolor(yellow);
write(registro_localidad.codigo_cepa);
    textcolor(green); write('; Nombre de cepa: '); textcolor(yellow);
write(registro_localidad.nombre_cepa);
    textcolor(green); write('; Casos activos: '); textcolor(yellow);
write(registro_localidad.casos_activos);
    textcolor(green); write('; Casos nuevos: '); textcolor(yellow);
write(registro_localidad.casos_nuevos);
    textcolor(green); write('; Casos recuperados: '); textcolor(yellow);
write(registro_localidad.casos_recuperados);
    textcolor(green); write('; Casos fallecidos: '); textcolor(yellow);
writeln(registro_localidad.casos_fallecidos);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_localidad: t_registro_localidad1;
begin

```



```

reset(archivo_maestro);
while (not eof(archivo_maestro)) do
begin
    read(archivo_maestro,registro_localidad);
    imprimir_registro_localidad1(registro_localidad);
end;
close(archivo_maestro);
end;
procedure imprimir_registro_localidad2(registro_localidad: t_registro_localidad2);
begin
    textcolor(green); write('Código de localidad: '); textcolor(yellow);
write(registro_localidad.codigo);
    textcolor(green); write('; Código de cepa: '); textcolor(yellow);
write(registro_localidad.codigo_cepa);
    textcolor(green); write('; Casos activos: '); textcolor(yellow);
write(registro_localidad.casos_activos);
    textcolor(green); write('; Casos nuevos: '); textcolor(yellow);
write(registro_localidad.casos_nuevos);
    textcolor(green); write('; Casos recuperados: '); textcolor(yellow);
write(registro_localidad.casos_recuperados);
    textcolor(green); write('; Casos fallecidos: '); textcolor(yellow);
writeln(registro_localidad.casos_fallecidos);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_localidad: t_registro_localidad2;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
    begin
        read(archivo_detalle,registro_localidad);
        imprimir_registro_localidad2(registro_localidad);
    end;
    close(archivo_detalle);
end;
procedure leer_localidad_detalle(var archivo_detalle: t_archivo_detalle; var
registro_localidad: t_registro_localidad2);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_localidad)
    else
        registro_localidad.codigo:=codigo_salida_detalle;
    end;
procedure minimo(var vector_detalle: t_vector_detalle; var vector_localidades:
t_vector_localidades; var min: t_registro_localidad2);
var
    i, pos: t_detalle;
begin
    min.codigo:=codigo_salida_detalle;
    for i:= 1 to detalles_total do
        if ((vector_localidades[i].codigo<min.codigo) or
((vector_localidades[i].codigo=min.codigo) and
(vector_localidades[i].codigo_cepa<min.codigo_cepa))) then
            begin
                min:=vector_localidades[i];
                pos:=i;
            end;
        if (min.codigo<codigo_salida_detalle) then
            leer_localidad_detalle(vector_detalle[pos],vector_localidades[pos]);
        end;
procedure actualizar_localidades_corte(casos_activos_localidad: int32; var localidades_corte:
int16);
begin
    if (casos_activos_localidad>casos_activos_corte) then
        localidades_corte:=localidades_corte+1;
end;

```

```

procedure leer_localidad_maestro(var archivo_maestro: t_archivo_maestro; var
registro_localidad: t_registro_localidad1);
begin
  if (not eof(archivo_maestro)) then
    read(archivo_maestro,registro_localidad)
  else
    registro_localidad.codigo:=codigo_salida_maestro;
  end;
procedure actualizar1_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
  registro_localidad, registro_localidad_actual: t_registro_localidad1;
  min: t_registro_localidad2;
  vector_localidades: t_vector_localidades;
  i: t_detalle;
  localidades_corte: int16;
  casos_activos_localidad: int32;
  ok: boolean;
begin
  localidades_corte:=0;
  reset(archivo_maestro);
  for i:= 1 to detalles_total do
    begin
      reset(vector_detalle[i]);
      leer_localidad_detalle(vector_detalle[i],vector_localidades[i]);
    end;
  minimo(vector_detalle,vector_localidades,min);
  while (min.codigo<>codigo_salida_detalle) do
    begin
      casos_activos_localidad:=0;
      leer_localidad_maestro(archivo_maestro,registro_localidad);
      while (registro_localidad.codigo<>min.codigo) do
        begin
          registro_localidad_actual:=registro_localidad;
          while (registro_localidad.codigo=registro_localidad_actual.codigo) do
            begin
              casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
              leer_localidad_maestro(archivo_maestro,registro_localidad);
            end;
          actualizar_localidades_corte(casos_activos_localidad,localidades_corte);
          casos_activos_localidad:=0;
        end;
      ok:=true;
      registro_localidad_actual:=registro_localidad;
      while (registro_localidad_actual.codigo=min.codigo) do
        begin
          while (registro_localidad.codigo_cepa<>min.codigo_cepa) do
            begin
              if (ok=true) then
                casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
                read(archivo_maestro,registro_localidad);
                ok:=true;
              end;
              while ((registro_localidad.codigo=min.codigo) and
(registro_localidad.codigo_cepa=min.codigo_cepa)) do
                begin
                  registro_localidad.casos_fallecidos:=registro_localidad.casos_fallecidos+min.casos_fal
lecidos;
                  registro_localidad.casos_recuperados:=registro_localidad.casos_recuperados+min.casos_r
ecuperados;
                  registro_localidad.casos_activos:=min.casos_activos;
                  registro_localidad.casos_nuevos:=min.casos_nuevos;
                  minimo(vector_detalle,vector_localidades,min);
                end;
              seek(archivo_maestro,filepos(archivo_maestro)-1);
              write(archivo_maestro,registro_localidad);
            end;
          end;
        end;
      end;
    end;
  end;
end;

```

```

    casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
    ok:=false;
    if (registro_localidad_actual.codigo<>min.codigo) then
    begin
        leer_localidad_maestro(archivo_maestro,registro_localidad);
        while (registro_localidad.codigo=registro_localidad_actual.codigo) do
        begin
            casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
            leer_localidad_maestro(archivo_maestro,registro_localidad);
        end;
        seek(archivo_maestro,filepos(archivo_maestro)-1);
    end;
    end;
    actualizar_localidades_corte(casos_activos_localidad,localidades_corte);
end;
casos_activos_localidad:=0;
leer_localidad_maestro(archivo_maestro,registro_localidad);
while (registro_localidad.codigo<>codigo_salida_maestro) do
begin
    registro_localidad_actual.codigo:=registro_localidad.codigo;
    while (registro_localidad.codigo=registro_localidad_actual.codigo) do
    begin
        casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
        leer_localidad_maestro(archivo_maestro,registro_localidad);
    end;
    actualizar_localidades_corte(casos_activos_localidad,localidades_corte);
    casos_activos_localidad:=0;
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    textcolor(green); write('La cantidad de localidades con más de '); textcolor(yellow);
write(casos_activos_corte); textcolor(green); write(' casos activos es '); textcolor(red);
writeln(localidades_corte);
    writeln();
end;
procedure actualizar2_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_localidad: t_registro_localidad1;
    min: t_registro_localidad2;
    vector_localidades: t_vector_localidades;
    i: t_detalle;
    localidad, localidades_corte: int16;
    casos_activos_localidad: int32;
    ok: boolean;
begin
    localidades_corte:=0;
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
    begin
        reset(vector_detalle[i]);
        leer_localidad_detalle(vector_detalle[i],vector_localidades[i]);
    end;
    minimo(vector_detalle,vector_localidades,min);
    leer_localidad_maestro(archivo_maestro,registro_localidad);
    while (registro_localidad.codigo<>codigo_salida_maestro) do
    begin
        localidad:=registro_localidad.codigo;
        casos_activos_localidad:=0;
        while (registro_localidad.codigo=localidad) do
        begin
            ok:=false;
            while ((registro_localidad.codigo=min.codigo) and
(registro_localidad.codigo_cepa=min.codigo_cepa)) do
                begin

```

```

        ok:=true;
        registro_localidad.casos_fallecidos:=registro_localidad.casos_fallecidos+min.casos_fal
lecidos;
        registro_localidad.casos_recuperados:=registro_localidad.casos_recuperados+min.casos_r
ecuperados;
        registro_localidad.casos_activos:=min.casos_activos;
        registro_localidad.casos_nuevos:=min.casos_nuevos;
        minimo(vector_detalle, vector_localidades, min);
    end;
    if (ok=true) then
    begin
        seek(archivo_maestro, filepos(archivo_maestro)-1);
        write(archivo_maestro, registro_localidad);
    end;
    casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
    leer_localidad_maestro(archivo_maestro, registro_localidad);
    end;
    actualizar_localidades_corte(casos_activos_localidad, localidades_corte);
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    textcolor(green); write('La cantidad de localidades con más de '); textcolor(yellow);
write(casos_activos_corte); textcolor(green); write(' casos activos es '); textcolor(red);
writeln(localidades_corte);
    writeln();
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro, 'E6_localidadesMaestro');
    assign(archivo_carga_maestro, 'E6_localidadesMaestro.txt');
    cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    for i:= 1 to detalles_total do
    begin
        writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ', i, ':'); writeln();
        assign(vector_detalle[i], 'E6_localidadesDetalle'+inttoStr(i));
        assign(vector_carga_detalle[i], 'E6_localidadesDetalle'+inttoStr(i)+'.txt');
        cargar_archivo_detalle(vector_detalle[i], vector_carga_detalle[i]);
        imprimir_archivo_detalle(vector_detalle[i]);
    end;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    //actualizar1_archivo_maestro(archivo_maestro, vector_detalle);
    actualizar2_archivo_maestro(archivo_maestro, vector_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.

```

Ejercicio 7.

Se dispone de un archivo maestro con información de los alumnos de la Facultad de Informática. Cada registro del archivo maestro contiene: código de alumno, apellido, nombre, cantidad de cursadas aprobadas y cantidad de materias con final aprobado. El archivo maestro está ordenado por código de alumno.

Además, se tienen dos archivos detalle con información sobre el desempeño académico de los alumnos: un archivo de cursadas y un archivo de exámenes finales. El archivo de cursadas contiene información sobre las materias cursadas por los alumnos. Cada registro incluye: código de alumno, código de materia, año de cursada y resultado (sólo interesa si la cursada fue aprobada o desaprobada). Por su parte, el archivo de exámenes finales contiene información sobre los exámenes finales rendidos. Cada registro incluye: código de alumno, código de materia, fecha del examen y nota obtenida. Ambos archivos detalle están ordenados por código de alumno y código de materia, y pueden contener 0, 1 o más registros por alumno en el archivo maestro. Un alumno podría cursar una materia muchas veces, así como también podría rendir el final de una materia en múltiples ocasiones.

Se debe desarrollar un programa que actualice el archivo maestro, ajustando la cantidad de cursadas aprobadas y la cantidad de materias con final aprobado, utilizando la información de los archivos detalle. Las reglas de actualización son las siguientes:

- *Si un alumno aprueba una cursada, se incrementa en uno la cantidad de cursadas aprobadas.*
- *Si un alumno aprueba un examen final (nota ≥ 4), se incrementa en uno la cantidad de materias con final aprobado.*

Notas:

- *Los archivos deben procesarse en un único recorrido.*
- *No es necesario comprobar que no haya inconsistencias en la información de los archivos detalles. Esto es, no puede suceder que un alumno apruebe más de una vez la cursada de una misma materia (a lo sumo, la aprueba una vez), algo similar ocurre con los exámenes finales.*

```
program TP2_E7;
{$codepage UTF8}
uses crt, sysutils;
const
  codigo_salida=999;
  resultado_corte='Aprobada'; nota_corte=4.0;
  anio_ini=2000; anio_fin=2025;
type
  t_string20=string[20];
  t_anio=anio_ini..anio_fin;
  t_registro_alumno=record
    codigo: int16;
    apellido: t_string20;
    nombre: t_string20;
    cursadas_aprobadas: int16;
    finales_aprobados: int16;
  end;
```

```

t_registro_cursada=record
  codigo: int16;
  codigo_materia: int16;
  anio: t_anio;
  resultado: t_string20;
end;
t_registro_final=record
  codigo: int16;
  codigo_materia: int16;
  fecha: t_string20;
  nota: real;
end;
t_registro_cursada_final=record
  codigo: int16;
  codigo_materia: int16;
  resultado: t_string20;
  nota: real;
end;
t_archivo_maestro=file of t_registro_alumno;
t_archivo_detalle1=file of t_registro_cursada;
t_archivo_detalle2=file of t_registro_final;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_alumno: t_registro_alumno;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_alumno do
      begin
        readln(archivo_carga_maestro,codigo,cursadas_aprobadas, finales_aprobados, apellido);
        apellido:=trim(apellido);
        readln(archivo_carga_maestro,nombre);
        write(archivo_maestro,registro_alumno);
      end;
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
  end;
procedure cargar_archivo_detalle1(var archivo_detalle1: t_archivo_detalle1; var
archivo_carga_detalle1: text);
var
  registro_cursada: t_registro_cursada;
begin
  rewrite(archivo_detalle1);
  reset(archivo_carga_detalle1);
  while (not eof(archivo_carga_detalle1)) do
    with registro_cursada do
      begin
        readln(archivo_carga_detalle1,codigo,codigo_materia,anio,resultado);
        resultado:=trim(resultado);
        write(archivo_detalle1,registro_cursada);
      end;
    end;
    close(archivo_detalle1);
    close(archivo_carga_detalle1);
  end;
procedure cargar_archivo_detalle2(var archivo_detalle2: t_archivo_detalle2; var
archivo_carga_detalle2: text);
var
  registro_final: t_registro_final;
begin
  rewrite(archivo_detalle2);
  reset(archivo_carga_detalle2);
  while (not eof(archivo_carga_detalle2)) do
    with registro_final do
      begin

```

```

        readln(archivo_carga_detalle2,codigo,codigo_materia,nota,fecha); fecha:=trim(fecha);
        write(archivo_detalle2,registro_final);
    end;
    close(archivo_detalle2);
    close(archivo_carga_detalle2);
end;
procedure imprimir_registro_alumno(registro_alumno: t_registro_alumno);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_alumno.codigo);
    textcolor(green); write('; Apellido: '); textcolor(yellow); write(registro_alumno.apellido);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_alumno.nombre);
    textcolor(green); write('; Cursadas aprobadas: '); textcolor(yellow);
write(registro_alumno.cursadas_aprobadas);
    textcolor(green); write('; Finales aprobados: '); textcolor(yellow);
writeln(registro_alumno.finales_aprobados);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_alumno: t_registro_alumno;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_alumno);
            imprimir_registro_alumno(registro_alumno);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_cursada(registro_cursada: t_registro_cursada);
begin
    textcolor(green); write('Código de alumno: '); textcolor(yellow);
write(registro_cursada.codigo);
    textcolor(green); write('; Código de materia: '); textcolor(yellow);
write(registro_cursada.codigo_materia);
    textcolor(green); write('; Año: '); textcolor(yellow); write(registro_cursada.anio);
    textcolor(green); write('; Resultado: '); textcolor(yellow);
writeln(registro_cursada.resultado);
end;
procedure imprimir_archivo_detalle1(var archivo_detalle1: t_archivo_detalle1);
var
    registro_cursada: t_registro_cursada;
begin
    reset(archivo_detalle1);
    while (not eof(archivo_detalle1)) do
        begin
            read(archivo_detalle1,registro_cursada);
            imprimir_registro_cursada(registro_cursada);
        end;
    close(archivo_detalle1);
end;
procedure imprimir_registro_final(registro_final: t_registro_final);
begin
    textcolor(green); write('Código de alumno: '); textcolor(yellow);
write(registro_final.codigo);
    textcolor(green); write('; Código de materia: '); textcolor(yellow);
write(registro_final.codigo_materia);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_final.fecha);
    textcolor(green); write('; Nota: '); textcolor(yellow); writeln(registro_final.nota:0:2);
end;
procedure imprimir_archivo_detalle2(var archivo_detalle2: t_archivo_detalle2);
var
    registro_final: t_registro_final;
begin
    reset(archivo_detalle2);
    while (not eof(archivo_detalle2)) do
        begin

```

```

    read(archivo_detalle2, registro_final);
    imprimir_registro_final(registro_final);
end;
close(archivo_detalle2);
end;
procedure leer_cursada(var archivo_detalle1: t_archivo_detalle1; var registro_cursada:
t_registro_cursada);
begin
    if (not eof(archivo_detalle1)) then
        read(archivo_detalle1, registro_cursada)
    else
        registro_cursada.codigo:=codigo_salida;
end;
procedure leer_final(var archivo_detalle2: t_archivo_detalle2; var registro_final:
t_registro_final);
begin
    if (not eof(archivo_detalle2)) then
        read(archivo_detalle2, registro_final)
    else
        registro_final.codigo:=codigo_salida;
end;
procedure minimo(var archivo_detalle1: t_archivo_detalle1; var archivo_detalle2:
t_archivo_detalle2; var registro_cursada: t_registro_cursada; var registro_final:
t_registro_final; var min: t_registro_cursada_final);
begin
    if (registro_cursada.codigo<=registro_final.codigo) then
        begin
            min.codigo:=registro_cursada.codigo;
            min.codigo_materia:=registro_cursada.codigo_materia;
            min.resultado:=registro_cursada.resultado;
            min.nota:=0;
            leer_cursada(archivo_detalle1, registro_cursada);
        end
    else
        begin
            min.codigo:=registro_final.codigo;
            min.codigo_materia:=registro_final.codigo_materia;
            min.resultado:='';
            min.nota:=registro_final.nota;
            leer_final(archivo_detalle2, registro_final);
        end;
end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle1: t_archivo_detalle1; var archivo_detalle2: t_archivo_detalle2);
var
    registro_alumno: t_registro_alumno;
    registro_cursada: t_registro_cursada;
    registro_final: t_registro_final;
    min: t_registro_cursada_final;
begin
    reset(archivo_maestro);
    reset(archivo_detalle1); reset(archivo_detalle2);
    leer_cursada(archivo_detalle1, registro_cursada);
    leer_final(archivo_detalle2, registro_final);
    minimo(archivo_detalle1, archivo_detalle2, registro_cursada, registro_final, min);
    while (min.codigo<>codigo_salida) do
        begin
            read(archivo_maestro, registro_alumno);
            while (registro_alumno.codigo<>min.codigo) do
                read(archivo_maestro, registro_alumno);
            while (registro_alumno.codigo=min.codigo) do
                begin
                    if (min.resultado=resultado_corte) then
                        registro_alumno.cursadas_aprobadas:=registro_alumno.cursadas_aprobadas+1;
                    if (min.nota>nota_corte) then
                        registro_alumno.finales_aprobados:=registro_alumno.finales_aprobados+1;

```



```
        minimo(archivo_detalle1,archivo_detalle2,registro_cursada,registro_final,min);
    end;
    seek(archivo_maestro,filepos(archivo_maestro)-1);
    write(archivo_maestro,registro_alumno);
end;
close(archivo_maestro);
close(archivo_detalle1); close(archivo_detalle2);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle1: t_archivo_detalle1;
    archivo_detalle2: t_archivo_detalle2;
    archivo_carga_maestro, archivo_carga_detalle1, archivo_carga_detalle2: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E7_alumnosMaestro');
assign(archivo_carga_maestro,'E7_alumnosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE 1:'); writeln();
    assign(archivo_detalle1,'E7_cursadasDetalle');
assign(archivo_carga_detalle1,'E7_cursadasDetalle.txt');
    cargar_archivo_detalle1(archivo_detalle1,archivo_carga_detalle1);
    imprimir_archivo_detalle1(archivo_detalle1);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE 2:'); writeln();
    assign(archivo_detalle2,'E7_finalesDetalle');
assign(archivo_carga_detalle2,'E7_finalesDetalle.txt');
    cargar_archivo_detalle2(archivo_detalle2,archivo_carga_detalle2);
    imprimir_archivo_detalle2(archivo_detalle2);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro,archivo_detalle1,archivo_detalle2);
    imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 8.

Se quiere optimizar la gestión del consumo de yerba mate en distintas provincias de Argentina. Para ello, se cuenta con un archivo maestro que contiene la siguiente información: código de provincia, nombre de la provincia, cantidad de habitantes y cantidad total de kilos de yerba consumidos históricamente.

Cada mes, se reciben 16 archivos de relevamiento con información sobre el consumo de yerba en los distintos puntos del país. Cada archivo contiene: código de provincia y cantidad de kilos de yerba consumidos en ese relevamiento. Un archivo de relevamiento puede contener información de una o varias provincias, y una misma provincia puede aparecer cero, una o más veces en distintos archivos de relevamiento.

Tanto el archivo maestro como los archivos de relevamiento están ordenados por código de provincia.

Se desea realizar un programa que actualice el archivo maestro en base a la nueva información de consumo de yerba. Además, se debe informar en pantalla aquellas provincias (código y nombre) donde la cantidad total de yerba consumida supere los 10.000 kilos históricamente, junto con el promedio consumido de yerba por habitante. Es importante tener en cuenta tanto las provincias actualizadas como las que no fueron actualizadas.

Nota: Cada archivo debe recorrerse una única vez.

```
program TP2_E8;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=16;
  codigo_salida=999;
  kilos_corte=10000;
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_provincia1=record
    codigo: int16;
    nombre: t_string20;
    habitantes: int16;
    kilos: int32;
  end;
  t_registro_provincia2=record
    codigo: int16;
    kilos: int32;
  end;
  t_archivo_maestro=file of t_registro_provincia1;
  t_archivo_detalle=file of t_registro_provincia2;
  t_vector_provincias=array[t_detalle] of t_registro_provincia2;
  t_vector_detalle=array[t_detalle] of t_archivo_detalle;
  t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_provincia: t_registro_provincia1;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
```

```
while (not eof(archivo_carga_maestro)) do
  with registro_provincia do
    begin
      readln(archivo_carga_maestro,codigo,habitantes,kilos,nombre); nombre:=trim(nombre);
      write(archivo_maestro,registro_provincia);
    end;
  close(archivo_maestro);
  close(archivo_carga_maestro);
end;

procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
  registro_provincia: t_registro_provincia2;
begin
  rewrite(archivo_detalle);
  reset(archivo_carga_detalle);
  while (not eof(archivo_carga_detalle)) do
    with registro_provincia do
      begin
        readln(archivo_carga_detalle,codigo,kilos);
        write(archivo_detalle,registro_provincia);
      end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
  end;
end;

procedure imprimir_registro_provincia1(registro_provincia: t_registro_provincia1);
begin
  textcolor(green); write('Código: '); textcolor(yellow); write(registro_provincia.codigo);
  textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_provincia.nombre);
  textcolor(green); write('; Habitantes: '); textcolor(yellow);
write(registro_provincia.habitantes);
  textcolor(green); write('; Kilos de yerba consumidos: '); textcolor(yellow);
writeln(registro_provincia.kilos);
end;

procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
  registro_provincia: t_registro_provincia1;
begin
  reset(archivo_maestro);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro,registro_provincia);
      imprimir_registro_provincia1(registro_provincia);
    end;
  close(archivo_maestro);
end;

procedure imprimir_registro_provincia2(registro_provincia: t_registro_provincia2);
begin
  textcolor(green); write('Código: '); textcolor(yellow); write(registro_provincia.codigo);
  textcolor(green); write('; Kilos de yerba consumidos: '); textcolor(yellow);
writeln(registro_provincia.kilos);
end;

procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
  registro_provincia: t_registro_provincia2;
begin
  reset(archivo_detalle);
  while (not eof(archivo_detalle)) do
    begin
      read(archivo_detalle,registro_provincia);
      imprimir_registro_provincia2(registro_provincia);
    end;
  close(archivo_detalle);
end;

procedure leer_provincia_detalle(var archivo_detalle: t_archivo_detalle; var
registro_provincia: t_registro_provincia2);
```

```

begin
  if (not eof(archivo_detalle)) then
    read(archivo_detalle, registro_provincia)
  else
    registro_provincia.codigo:=codigo_salida;
end;
procedure minimo(var vector_detalles: t_vector_detalles; var vector_provincias:
t_vector_provincias; var min: t_registro_provincia2);
var
  i, pos: t_detalle;
begin
  min.codigo:=codigo_salida;
  for i:= 1 to detalles_total do
    if (vector_provincias[i].codigo<min.codigo) then
      begin
        min:=vector_provincias[i];
        pos:=i;
      end;
    if (min.codigo<codigo_salida) then
      leer_provincia_detalle(vector_detalles[pos], vector_provincias[pos]);
end;
procedure imprimir_texto(registro_provincia: t_registro_provincia1);
begin
  if (registro_provincia.kilos>kilos_corte) then
    begin
      textcolor(green); write('El nombre y el código de esta provincia donde la cantidad total
de yerba consumida supera los '); textcolor(yellow); write(kilos_corte); textcolor(green);
write(' kilos son '); textcolor(red); write(registro_provincia.nombre); textcolor(green);
write(' y '); textcolor(red); write(registro_provincia.codigo); textcolor(green); write(',
respectivamente, mientras que el promedio consumido de yerba por habitante es ');
textcolor(red); writeln(registro_provincia.kilos/registro_provincia.habitantes:0:2);
    end;
end;
procedure actualizar1_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalles: t_vector_detalles);
var
  registro_provincia: t_registro_provincia1;
  min: t_registro_provincia2;
  vector_provincias: t_vector_provincias;
  i: t_detalle;
begin
  reset(archivo_maestro);
  for i:= 1 to detalles_total do
    begin
      reset(vector_detalles[i]);
      leer_provincia_detalle(vector_detalles[i], vector_provincias[i]);
    end;
  minimo(vector_detalles, vector_provincias, min);
  while (min.codigo<>codigo_salida) do
    begin
      read(archivo_maestro, registro_provincia);
      while (registro_provincia.codigo<>min.codigo) do
        begin
          imprimir_texto(registro_provincia);
          read(archivo_maestro, registro_provincia);
        end;
      while (registro_provincia.codigo=min.codigo) do
        begin
          registro_provincia.kilos:=registro_provincia.kilos+min.kilos;
          minimo(vector_detalles, vector_provincias, min);
        end;
      seek(archivo_maestro, filepos(archivo_maestro)-1);
      write(archivo_maestro, registro_provincia);
      imprimir_texto(registro_provincia);
    end;
  while (not eof(archivo_maestro)) do

```

```

begin
    read(archivo_maestro,registro_provincia);
    imprimir_texto(registro_provincia);
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
writeln();
end;
procedure leer_provincia_maestro(var archivo_maestro: t_archivo_maestro; var
registro_provincia: t_registro_provincia1);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro,registro_provincia)
    else
        registro_provincia.codigo:=codigo_salida;
end;
procedure actualizar2_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_provincia: t_registro_provincia1;
    min: t_registro_provincia2;
    vector_provincias: t_vector_provincias;
    i: t_detalle;
    ok: boolean;
begin
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_provincia_detalle(vector_detalle[i],vector_provincias[i]);
        end;
    minimo(vector_detalle,vector_provincias,min);
    leer_provincia_maestro(archivo_maestro,registro_provincia);
    while (registro_provincia.codigo<>codigo_salida) do
        begin
            ok:=false;
            while (registro_provincia.codigo=min.codigo) do
                begin
                    ok:=true;
                    registro_provincia.kilos:=registro_provincia.kilos+min.kilos;
                    minimo(vector_detalle,vector_provincias,min);
                end;
            if (ok=true) then
                begin
                    seek(archivo_maestro,filepos(archivo_maestro)-1);
                    write(archivo_maestro,registro_provincia);
                end;
            imprimir_texto(registro_provincia);
            leer_provincia_maestro(archivo_maestro,registro_provincia);
        end;
    close(archivo_maestro);
    for i:= 1 to detalles_total do
        close(vector_detalle[i]);
    writeln();
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E8_provinciasMaestro');
    assign(archivo_carga_maestro,'E8_provinciasMaestro.txt');

```

```
cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
imprimir_archivo_maestro(archivo_maestro);
for i:= 1 to detalles_total do
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
    assign(vector_detalle[i],'E8_provinciasDetalle'+intToStr(i));
    assign(vector_carga_detalle[i],'E8_provinciasDetalle'+intToStr(i)+'.txt');
    cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
    imprimir_archivo_detalle(vector_detalle[i]);
end;
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
//actualizar1_archivo_maestro(archivo_maestro,vector_detalle);
actualizar2_archivo_maestro(archivo_maestro,vector_detalle);
imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 9.

Se cuenta con un archivo que posee información de las ventas que realiza una empresa a los diferentes clientes. Se necesita obtener un reporte con las ventas organizadas por cliente. Para ello, se deberá informar por pantalla: los datos personales del cliente, el total mensual (mes por mes cuánto compró) y, finalmente, el monto total comprado en el año por el cliente. Además, al finalizar el reporte, se debe informar el monto total de ventas obtenido por la empresa.

El formato del archivo maestro está dado por: cliente (código cliente, nombre y apellido), año, mes, día y monto de la venta. El orden del archivo está dado por: código cliente, año y mes.

Nota: Tener en cuenta que puede haber meses en los que los clientes no realizaron compras. No es necesario informar tales meses en el reporte.

```
program TP2_E9;
{$codepage UTF8}
uses crt, sysutils;
const
  codigo_salida=999;
  dia_ini=1; dia_fin=31;
  mes_ini=1; mes_fin=12;
  anio_ini=2000; anio_fin=2025;
type
  t_string10=string[10];
  t_dia=dia_ini..dia_fin;
  t_mes=mes_ini..mes_fin;
  t_anio=anio_ini..anio_fin;
  t_registro_cliente=record
    codigo: int16;
    nombre: t_string10;
    apellido: t_string10;
  end;
  t_registro_fecha=record
    anio: t_anio;
    mes: t_mes;
    dia: t_dia;
  end;
  t_registro_venta=record
    cliente: t_registro_cliente;
    fecha: t_registro_fecha;
    monto: real;
  end;
  t_archivo_maestro=file of t_registro_venta;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_venta: t_registro_venta;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_venta do
      begin
        readln(archivo_carga_maestro,cliente.codigo,fecha.anio,fecha.mes,fecha.dia,monto,cliente
.nombre); cliente.nombre:=trim(cliente.nombre);
        readln(archivo_carga_maestro,cliente.apellido);
        write(archivo_maestro,registro_venta);
      end;
    end;
  end;
```

```

    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure imprimir_registro_cliente(registro_cliente: t_registro_cliente);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_cliente.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_cliente.nombre);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_cliente.apellido);
end;
procedure imprimir_registro_fecha(registro_fecha: t_registro_fecha);
begin
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_fecha.anio);
    textcolor(green); write('/'); textcolor(yellow); write(registro_fecha.mes);
    textcolor(green); write('/'); textcolor(yellow); write(registro_fecha.dia);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    imprimir_registro_cliente(registro_venta.cliente);
    imprimir_registro_fecha(registro_venta.fecha);
    textcolor(green); write('; Monto: $'); textcolor(yellow); writeln(registro_venta.monto:0:2);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro, registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_maestro);
end;
procedure leer_venta(var archivo_maestro: t_archivo_maestro; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro, registro_venta)
    else
        registro_venta.cliente.codigo:=codigo_salida;
end;
procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_venta: t_registro_venta;
    monto_total, monto_anio, monto_mes: real;
    codigo, anio, mes: int16;
begin
    monto_total:=0;
    reset(archivo_maestro);
    leer_venta(archivo_maestro, registro_venta);
    while (registro_venta.cliente.codigo<>codigo_salida) do
        begin
            codigo:=registro_venta.cliente.codigo;
            textcolor(green); write('CLIENTE: '); imprimir_registro_cliente(registro_venta.cliente);
writeln(); writeln();
            while (registro_venta.cliente.codigo=codigo) do
                begin
                    anio:=registro_venta.fecha.anio;
                    monto_anio:=0;
                    textcolor(green); write('AÑO '); textcolor(yellow); writeln(anio);
                    while ((registro_venta.cliente.codigo=codigo) and (registro_venta.fecha.anio=anio)) do
                        begin
                            mes:=registro_venta.fecha.mes;
                            monto_mes:=0;

```



```
    while ((registro_venta.cliente.codigo=codigo) and (registro_venta.fecha.anio=anio) and
(registro_venta.fecha.mes=mes)) do
    begin
        monto_mes:=monto_mes+registro_venta.monto;
        leer_venta(archivo_maestro,registro_venta);
    end;
    textcolor(green); write('Monto total del mes '); textcolor(yellow); write(mes);
textcolor(green); write(': $'); textcolor(red); writeln(monto_mes:0:2);
    monto_anio:=monto_anio+monto_mes;
    end;
    textcolor(green); write('Monto total del año '); textcolor(yellow); write(anio);
textcolor(green); write(': $'); textcolor(red); writeln(monto_anio:0:2); writeln();
    monto_total:=monto_total+monto_anio;
    end;
end;
textcolor(green); write('Monto total de ventas obtenido por la empresa: $'); textcolor(red);
writeln(monto_total:0:2);
close(archivo_maestro);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E9_ventasMaestro');
assign(archivo_carga_maestro,'E9_ventasMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
    procesar_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 10.

Se necesita contabilizar los votos de las diferentes mesas electorales registradas por provincia y localidad. Para ello, se posee un archivo con la siguiente información: código de provincia, código de localidad, número de mesa y cantidad de votos en dicha mesa. Presentar en pantalla un listado como se muestra a continuación:

Código de Provincia

Código de Localidad

Total de Votos

.....

.....

.....

.....

Total de Votos Provincia: ____

Código de Provincia

Código de Localidad

Total de Votos

.....

.....

Total de Votos Provincia: ____

.....

Total General de Votos: ____

Nota: La información está ordenada por código de provincia y código de localidad.

```
program TP2_E10;
{$codepage UTF8}
uses crt;
const
  provincia_salida=999;
type
  t_registro_mesa=record
    provincia: int16;
    localidad: int16;
    numero: int16;
    votos: int16;
  end;
  t_archivo_maestro=file of t_registro_mesa;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_mesa: t_registro_mesa;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_mesa do
      begin
        readln(archivo_carga_maestro,provincia,localidad,numero,votos);
```

```

        write(archivo_maestro, registro_mesa);
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure imprimir_registro_mesa(registro_mesa: t_registro_mesa);
begin
    textcolor(green); write('Código de provincia: '); textcolor(yellow);
write(registro_mesa.provincia);
    textcolor(green); write('; Código de localidad: '); textcolor(yellow);
write(registro_mesa.localidad);
    textcolor(green); write('Número de mesa: '); textcolor(yellow);
write(registro_mesa.numero);
    textcolor(green); write('; Votos: '); textcolor(yellow); writeln(registro_mesa.votos);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_mesa: t_registro_mesa;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro, registro_mesa);
            imprimir_registro_mesa(registro_mesa);
        end;
    close(archivo_maestro);
end;
procedure leer_mesa(var archivo_maestro: t_archivo_maestro; var registro_mesa:
t_registro_mesa);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro, registro_mesa)
    else
        registro_mesa.provincia:=provincia_salida;
end;
procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_mesa: t_registro_mesa;
    provincia, localidad, votos_total, votos_provincia, votos_localidad: int16;
begin
    votos_total:=0;
    reset(archivo_maestro);
    leer_mesa(archivo_maestro, registro_mesa);
    while (registro_mesa.provincia<>provincia_salida) do
        begin
            provincia:=registro_mesa.provincia;
            votos_provincia:=0;
            textcolor(green); write('Código de Provincia: '); textcolor(yellow); writeln(provincia);
            textcolor(green); writeln('Código de Localidad          Total de Votos');
            while (registro_mesa.provincia=provincia) do
                begin
                    localidad:=registro_mesa.localidad;
                    votos_localidad:=0;
                    while ((registro_mesa.provincia=provincia) and (registro_mesa.localidad=localidad)) do
                        begin
                            votos_localidad:=votos_localidad+registro_mesa.votos;
                            leer_mesa(archivo_maestro, registro_mesa);
                        end;
                    textcolor(yellow); write(localidad); textcolor(green);
write('          '); textcolor(red); writeln(votos_localidad);
                    votos_provincia:=votos_provincia+votos_localidad;
                end;
                textcolor(green); write('Total de Votos Provincia: '); textcolor(red);
writeln(votos_provincia); writeln();
                votos_total:=votos_total+votos_provincia;
            end;
        end;
    end;
end;

```

```
    textcolor(green); write('Total General de Votos: '); textcolor(red); writeln(votos_total);
    close(archivo_maestro);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro, 'E10_mesasMaestro');
    assign(archivo_carga_maestro, 'E10_mesasMaestro.txt');
    cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
    procesar_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 11.

Se tiene información en un archivo de las horas extras realizadas por los empleados de una empresa en un mes. Para cada empleado, se tiene la siguiente información: departamento, división, número de empleado, categoría y cantidad de horas extras realizadas por el empleado. Se sabe que el archivo se encuentra ordenado por departamento, luego por división y, por último, por número de empleado. Presentar en pantalla un listado con el siguiente formato:

Departamento

División

Número de Empleado Total de Hs. Importe a cobrar

-----	-----	-----
-----	-----	-----

Total de horas división: ____

Monto total por división: ____

División

Total horas departamento: ____

Monto total departamento: ____

Para obtener el valor de la hora, se debe cargar un arreglo desde un archivo de texto al iniciar el programa con el valor de la hora extra para cada categoría. La categoría varía de 1 a 15. En el archivo de texto, debe haber una línea para cada categoría con el número de categoría y el valor de la hora, pero el arreglo debe ser de valores de horas, con la posición del valor coincidente con el número de categoría.

```
program TP2_E11;
{$codepage UTF8}
uses crt;
const
    departamento_salida=999;
    categoria_ini=1; categoria_fin=15;
type
    t_categoria=categoria_ini..categoria_fin;
    t_registro_empleado=record
        departamento: int16;
        division: int16;
        empleado: int16;
        categoria: t_categoria;
        horas: int16;
    end;
    t_vector_horas=array[t_categoria] of real;
    t_archivo_maestro=file of t_registro_empleado;
```

```
procedure cargar_vector_horas(var vector_horas: t_vector_horas; var archivo_carga_vector:
text);
var
    categoria: t_categoria;
    valor_hora: real;
begin
    reset(archivo_carga_vector);
    while (not eof(archivo_carga_vector)) do
        begin
            readln(archivo_carga_vector,categoria,valor_hora);
            vector_horas[categoria]:=valor_hora;
        end;
    close(archivo_carga_vector);
end;
procedure imprimir_vector_horas(vector_horas: t_vector_horas);
var
    i: t_categoria;
begin
    for i:= categoria_ini to categoria_fin do
        begin
            textcolor(green); write('El valor de la hora de la categoría ',i,' es $');
            textcolor(yellow); writeln(vector_horas[i]:0:2);
        end;
    end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_empleado: t_registro_empleado;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        begin
            with registro_empleado do
                begin
                    readln(archivo_carga_maestro,departamento,division,empleado,categoria,horas);
                    write(archivo_maestro,registro_empleado);
                end;
            end;
        close(archivo_maestro);
        close(archivo_carga_maestro);
    end;
procedure imprimir_registro_empleado(registro_empleado: t_registro_empleado);
begin
    textcolor(green); write('Departamento: '); textcolor(yellow);
write(registro_empleado.departamento);
    textcolor(green); write('; División: '); textcolor(yellow);
write(registro_empleado.division);
    textcolor(green); write('; Número de empleado: '); textcolor(yellow);
write(registro_empleado.empleado);
    textcolor(green); write('; Categoría: '); textcolor(yellow);
write(registro_empleado.categoria);
    textcolor(green); write('; Horas extras: '); textcolor(yellow);
writeln(registro_empleado.horas);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_empleado);
            imprimir_registro_empleado(registro_empleado);
        end;
    close(archivo_maestro);
```

```

end;
procedure leer_empleado(var archivo_maestro: t_archivo_maestro; var registro_empleado:
t_registro_empleado);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro, registro_empleado)
    else
        registro_empleado.departamento:=departamento_salida;
    end;
procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro; vector_horas:
t_vector_horas);
var
    registro_empleado: t_registro_empleado;
    categoria: t_categoria;
    departamento, division, empleado, horas_departamento, horas_division, horas_empleado: int16;
    monto_departamento, monto_division, monto_empleado: real;
begin
    reset(archivo_maestro);
    leer_empleado(archivo_maestro, registro_empleado);
    while (registro_empleado.departamento<>departamento_salida) do
        begin
            departamento:=registro_empleado.departamento;
            horas_departamento:=0; monto_departamento:=0;
            textcolor(green); write('Departamento: '); textcolor(yellow); writeln(departamento);
            while (registro_empleado.departamento=departamento) do
                begin
                    division:=registro_empleado.division;
                    horas_division:=0; monto_division:=0;
                    textcolor(green); write('División: '); textcolor(yellow); writeln(division);
                    textcolor(green); writeln('Número de Empleado      Total de Hs.      Importe a
cobrar');
                    while ((registro_empleado.departamento=departamento) and
(registro_empleado.division=division)) do
                        begin
                            empleado:=registro_empleado.empleado; categoria:=registro_empleado.categoria;
                            horas_empleado:=0; monto_empleado:=0;
                            while ((registro_empleado.departamento=departamento) and
(registro_empleado.division=division) and (registro_empleado.empleado=empleado)) do
                                begin
                                    horas_empleado:=horas_empleado+registro_empleado.horas;
                                    leer_empleado(archivo_maestro, registro_empleado);
                                end;
                                    monto_empleado:=horas_empleado*vector_horas[categoria];
                                    textcolor(yellow); write(empleado); textcolor(green);
write('
'); textcolor(red); write(horas_empleado); textcolor(green);
write('
$'); textcolor(red); writeln(monto_empleado:0:2);
                                    horas_division:=horas_division+horas_empleado;
                                    monto_division:=monto_division+monto_empleado;
                                end;
                                    textcolor(green); write('Total horas división: '); textcolor(red);
writeln(horas_division);
                                    textcolor(green); write('Monto total división: $'); textcolor(red);
writeln(monto_division:0:2);
                                    horas_departamento:=horas_departamento+horas_division;
                                    monto_departamento:=monto_departamento+monto_division;
                                end;
                                    textcolor(green); write('Total horas departamento: '); textcolor(red);
writeln(horas_departamento);
                                    textcolor(green); write('Monto total departamento: $'); textcolor(red);
writeln(monto_departamento:0:2); writeln();
                                end;
                            end;
end;
var
    vector_horas: t_vector_horas;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_vector, archivo_carga_maestro: text;

```

```
begin
  writeln(); textcolor(red); writeln('IMPRESIÓN VECTOR HORAS:'); writeln();
  assign(archivo_carga_vector, 'E11_horasVector.txt');
  cargar_vector_horas(vector_horas, archivo_carga_vector);
  imprimir_vector_horas(vector_horas);
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
  assign(archivo_maestro, 'E11_empleadosMaestro');
  assign(archivo_carga_maestro, 'E11_empleadosMaestro.txt');
  cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
  imprimir_archivo_maestro(archivo_maestro);
  writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
  procesar_archivo_maestro(archivo_maestro, vector_horas);
end.
```


Ejercicio 12.

La empresa de software “X” posee un servidor web donde se encuentra alojado el sitio web de la organización. En dicho servidor, se almacenan, en un archivo, todos los accesos que se realizan al sitio. La información que se almacena en el archivo es la siguiente: año, mes, día, idUsuario y tiempo de acceso al sitio de la organización. El archivo se encuentra ordenado por los siguientes criterios: año, mes, día e idUsuario.

Se debe realizar un procedimiento que genere un informe en pantalla. Para ello, se indicará el año calendario sobre el cual debe realizar el informe. El mismo debe respetar el formato mostrado a continuación:

```
Año : ---
  Mes:-- 1
    día:-- 1
      idUsuario 1  Tiempo Total de acceso en el día 1 mes 1
      -----
      idUsuario N  Tiempo total de acceso en el día 1 mes 1
      Tiempo total acceso día 1 mes 1
    -----
  día N
    idUsuario 1  Tiempo Total de acceso en el día N mes 1
    -----
    idUsuario N  Tiempo total de acceso en el día N mes 1
    Tiempo total acceso día N mes 1
  Total tiempo de acceso mes 1
  -----
Mes 12
  día 1
    idUsuario 1  Tiempo Total de acceso en el día 1 mes 12
    -----
    idUsuario N  Tiempo total de acceso en el día 1 mes 12
    Tiempo total acceso día 1 mes 12
  -----
  día N
    idUsuario 1  Tiempo Total de acceso en el día N mes 12
    -----
    idUsuario N  Tiempo total de acceso en el día N mes 12
    Tiempo total acceso día N mes 12
  Total tiempo de acceso mes 12
Total tiempo de acceso año
```

Se deberá tener en cuenta las siguientes aclaraciones:

- El año sobre el cual realizará el informe de accesos debe leerse desde el teclado.

- El año puede no existir en el archivo, en tal caso debe informarse en pantalla “Año no encontrado”.
- Se debe definir las estructuras de datos necesarias.
- El recorrido del archivo debe realizarse una única vez, procesando sólo la información necesaria.

```

program TP2_E12;
{$codepage UTF8}
uses crt;
const
  anio_salida=999;
  dia_ini=1; dia_fin=31;
  mes_ini=1; mes_fin=12;
  anio_ini=2020; anio_fin=2025;
type
  t_dia=dia_ini..dia_fin;
  t_mes=mes_ini..mes_fin;
  t_anio=anio_ini..anio_fin;
  t_registro_fecha=record
    anio: int16;
    mes: t_mes;
    dia: t_dia;
  end;
  t_registro_usuario=record
    fecha: t_registro_fecha;
    usuario: int16;
    tiempo: real;
  end;
  t_archivo_maestro=file of t_registro_usuario;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_usuario: t_registro_usuario;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_usuario do
      begin
        readln(archivo_carga_maestro, fecha.anio, fecha.mes, fecha.dia, usuario, tiempo);
        write(archivo_maestro, registro_usuario);
      end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
  end;
procedure imprimir_registro_fecha(registro_fecha: t_registro_fecha);
begin
  textcolor(green); write('Fecha: '); textcolor(yellow); write(registro_fecha.anio);
  textcolor(green); write('/'); textcolor(yellow); write(registro_fecha.mes);
  textcolor(green); write('/'); textcolor(yellow); write(registro_fecha.dia);
end;
procedure imprimir_registro_usuario(registro_usuario: t_registro_usuario);
begin
  imprimir_registro_fecha(registro_usuario.fecha);
  textcolor(green); write('; ID usuario: '); textcolor(yellow);
write(registro_usuario.usuario);
  textcolor(green); write('; Tiempo de acceso: '); textcolor(yellow);
writeln(registro_usuario.tiempo:0:2);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
  registro_usuario: t_registro_usuario;
begin

```

```

reset(archivo_maestro);
while (not eof(archivo_maestro)) do
begin
    read(archivo_maestro,registro_usuario);
    imprimir_registro_usuario(registro_usuario);
end;
close(archivo_maestro);
end;
procedure leer_usuario(var archivo_maestro: t_archivo_maestro; var registro_usuario:
t_registro_usuario);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro,registro_usuario)
    else
        registro_usuario.fecha.anio:=anio_salida;
    end;
procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro; anio: t_anio);
var
    registro_usuario: t_registro_usuario;
    mes, dia, usuario: int16;
    tiempo_anio, tiempo_mes, tiempo_dia, tiempo_usuario: real;
begin
    reset(archivo_maestro);
    leer_usuario(archivo_maestro,registro_usuario);
    while ((registro_usuario.fecha.anio<>anio) and (registro_usuario.fecha.anio<>anio_salida))
do
    leer_usuario(archivo_maestro,registro_usuario);
    if (registro_usuario.fecha.anio<>anio_salida) then
    begin
        tiempo_anio:=0;
        textcolor(green); write('Año: '); textcolor(yellow); writeln(anio); writeln();
        while (registro_usuario.fecha.anio=anio) do
        begin
            mes:=registro_usuario.fecha.mes;
            tiempo_mes:=0;
            textcolor(green); write(' Mes: '); textcolor(yellow); writeln(mes);
            while ((registro_usuario.fecha.anio=anio) and (registro_usuario.fecha.mes=mes)) do
            begin
                dia:=registro_usuario.fecha.dia;
                tiempo_dia:=0;
                textcolor(green); write(' Día: '); textcolor(yellow); writeln(dia);
                textcolor(green); write(' idUsuario Tiempo Total de acceso en el día ');
                textcolor(yellow); write(dia); textcolor(green); write(' mes '); textcolor(yellow);
writeln(mes);
                while ((registro_usuario.fecha.anio=anio) and (registro_usuario.fecha.mes=mes) and
(registro_usuario.fecha.dia=dia)) do
                begin
                    usuario:=registro_usuario.usuario;
                    tiempo_usuario:=0;
                    while ((registro_usuario.fecha.anio=anio) and (registro_usuario.fecha.mes=mes) and
(registro_usuario.fecha.dia=dia) and (registro_usuario.usuario=usuario)) do
                    begin
                        tiempo_usuario:=tiempo_usuario+registro_usuario.tiempo;
                        leer_usuario(archivo_maestro,registro_usuario);
                    end;
                    textcolor(green); write(' '); textcolor(yellow); write(usuario);
                    textcolor(green); write(' '); textcolor(red); writeln(tiempo_usuario:0:2);
                    tiempo_dia:=tiempo_dia+tiempo_usuario;
                end;
                textcolor(green); write(' Tiempo total acceso día '); textcolor(yellow);
write(dia); textcolor(green); write(' mes '); textcolor(yellow); write(mes); textcolor(green);
write(': '); textcolor(red); writeln(tiempo_dia:0:2);
                tiempo_mes:=tiempo_mes+tiempo_dia;
            end;
            textcolor(green); write(' Tiempo total acceso mes '); textcolor(yellow); write(mes);
                    textcolor(green); write(': '); textcolor(red); writeln(tiempo_mes:0:2); writeln();

```

```
        tiempo_anio:=tiempo_anio+tiempo_mes;
    end;
    textcolor(green); write('Tiempo total acceso año '); textcolor(yellow); write(anio);
textcolor(green); write(': '); textcolor(red); writeln(tiempo_anio:0:2);
end
else
begin
    textcolor(green); write('Año '); textcolor(yellow); write(anio); textcolor(green);
writeln(' no encontrado');
end;
close(archivo_maestro);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    anio: t_anio;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E12_usuariosMaestro');
assign(archivo_carga_maestro,'E12_usuariosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
    anio:=anio_ini+random(anio_fin-anio_ini+1);
    procesar_archivo_maestro(archivo_maestro,anio);
end.
```

Ejercicio 13.

Suponer que se es administrador de un servidor de correo electrónico. En los logs del mismo (información guardada acerca de los movimientos que ocurren en el server) que se encuentran en la ruta `/var/log/logmail.dat`, se guarda la siguiente información: `nro_usuario`, `nombreUsuario`, `nombre`, `apellido`, `cantidadMailEnviados`. Diariamente, el servidor de correo genera un archivo con la siguiente información: `nro_usuario`, `cuentaDestino`, `cuerpoMensaje`. Este archivo representa todos los correos enviados por los usuarios en un día determinado. Ambos archivos están ordenados por `nro_usuario` y se sabe que un usuario puede enviar cero, uno o más mails por día.

(a) Realizar el procedimiento necesario para actualizar la información del log en un día particular. Definir las estructuras de datos que utilice el procedimiento.

(b) Generar un archivo de texto que contenga el siguiente informe dado un archivo detalle de un día determinado:

`nro_usuarioX.....cantidadMensajesEnviados`

.....

`nro_usuarioX+n.....cantidadMensajesEnviados`

Nota: Tener en cuenta que, en el listado, deberán aparecer todos los usuarios que existen en el sistema. Considerar la implementación de esta opción de las siguientes maneras:

- Como un procedimiento separado del inciso (a).
- En el mismo procedimiento de actualización del inciso (a). ¿Qué cambios se requieren en el procedimiento del inciso (a) para realizar el informe en el mismo recorrido?

```
program TP2_E13;
{$codepage UTF8}
uses crt, sysutils;
const
    usuario_salida=999;
type
    t_string20=string[20];
    t_registro_usuario1=record
        usuario: int16;
        nombre_usuario: t_string20;
        nombre: t_string20;
        apellido: t_string20;
        mails: int16;
    end;
    t_registro_usuario2=record
        usuario: int16;
        destino: t_string20;
        mensaje: t_string20;
    end;
    t_archivo_maestro=file of t_registro_usuario1;
    t_archivo_detalle=file of t_registro_usuario2;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
```

```

registro_usuario: t_registro_usuario1;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_usuario do
      begin
        readln(archivo_carga_maestro, usuario, mails, nombre_usuario);
nombre_usuario:=trim(nombre_usuario);
        readln(archivo_carga_maestro, nombre);
        readln(archivo_carga_maestro, apellido);
        write(archivo_maestro, registro_usuario);
      end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
  end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
  registro_usuario: t_registro_usuario2;
begin
  rewrite(archivo_detalle);
  reset(archivo_carga_detalle);
  while (not eof(archivo_carga_detalle)) do
    with registro_usuario do
      begin
        readln(archivo_carga_detalle, usuario, destino); destino:=trim(destino);
        readln(archivo_carga_detalle, mensaje);
        write(archivo_detalle, registro_usuario);
      end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
  end;
procedure imprimir_registro_usuario1(registro_usuario: t_registro_usuario1);
begin
  textcolor(green); write('Número de usuario: '); textcolor(yellow);
write(registro_usuario.usuario);
  textcolor(green); write('; Nombre de usuario: '); textcolor(yellow);
write(registro_usuario.nombre_usuario);
  textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_usuario.nombre);
  textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_usuario.apellido);
  textcolor(green); write('; Mails enviados: '); textcolor(yellow);
writeln(registro_usuario.mails);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
  registro_usuario: t_registro_usuario1;
begin
  reset(archivo_maestro);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro, registro_usuario);
      imprimir_registro_usuario1(registro_usuario);
    end;
  close(archivo_maestro);
end;
procedure imprimir_registro_usuario2(registro_usuario: t_registro_usuario2);
begin
  textcolor(green); write('Número de usuario: '); textcolor(yellow);
write(registro_usuario.usuario);
  textcolor(green); write('; Cuenta destino: '); textcolor(yellow);
write(registro_usuario.destino);
  textcolor(green); write('; Cuerpo del mensaje: '); textcolor(yellow);
writeln(registro_usuario.mensaje);
end;

```

```

procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_usuario: t_registro_usuario2;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle, registro_usuario);
            imprimir_registro_usuario2(registro_usuario);
        end;
    close(archivo_detalle);
end;

procedure leer_usuario(var archivo_detalle: t_archivo_detalle; var registro_usuario:
t_registro_usuario2);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_usuario)
    else
        registro_usuario.usuario:=usuario_salida;
    end;
end;

procedure imprimir_texto(registro_maestro: t_registro_usuario1);
begin
    textcolor(green); write('Número de Usuario: '); textcolor(yellow);
write(registro_maestro.usuario); textcolor(green); write(' ..... Cantidad de Mensajes
Enviados: '); textcolor(red); writeln(registro_maestro.mails);
end;

procedure exportar_archivo_txt(var archivo_maestro: t_archivo_maestro);
var
    registro_usuario: t_registro_usuario1;
    archivo_txt: text;
begin
    reset(archivo_maestro);
    assign(archivo_txt, 'E13_usuarios.txt'); rewrite(archivo_txt);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro, registro_usuario);
            writeln(archivo_txt, registro_usuario.usuario, ' ', registro_usuario.mails);
        end;
    close(archivo_txt);
end;

procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle: t_archivo_detalle);
var
    registro_maestro: t_registro_usuario1;
    registro_detalle: t_registro_usuario2;
begin
    reset(archivo_maestro);
    reset(archivo_detalle);
    leer_usuario(archivo_detalle, registro_detalle);
    while (registro_detalle.usuario<>usuario_salida) do
        begin
            read(archivo_maestro, registro_maestro);
            while (registro_maestro.usuario<>registro_detalle.usuario) do
                begin
                    imprimir_texto(registro_maestro);
                    read(archivo_maestro, registro_maestro);
                end;
            while (registro_maestro.usuario=registro_detalle.usuario) do
                begin
                    registro_maestro.mails:=registro_maestro.mails+1;
                    leer_usuario(archivo_detalle, registro_detalle);
                end;
            seek(archivo_maestro, filepos(archivo_maestro)-1);
            write(archivo_maestro, registro_maestro);
            imprimir_texto(registro_maestro);
        end;
    end;
end;

```

```
while (not eof(archivo_maestro)) do
begin
    read(archivo_maestro,registro_maestro);
    imprimir_texto(registro_maestro);
end;
exportar_archivo_txt(archivo_maestro);
close(archivo_maestro);
close(archivo_detalle);
writeln();
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle: t_archivo_detalle;
    archivo_carga_maestro, archivo_carga_detalle: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E13_usuariosMaestro');
    assign(archivo_carga_maestro,'E13_usuariosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE:'); writeln();
    assign(archivo_detalle,'E13_usuariosDetalle');
    assign(archivo_carga_detalle,'E13_usuariosDetalle.txt');
    cargar_archivo_detalle(archivo_detalle,archivo_carga_detalle);
    imprimir_archivo_detalle(archivo_detalle);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro,archivo_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.
```


Ejercicio 14.

Una compañía aérea dispone de un archivo maestro donde guarda información sobre sus próximos vuelos. En dicho archivo, se tiene almacenado el destino, fecha, hora de salida y la cantidad de asientos disponibles. La empresa recibe todos los días dos archivos detalles para actualizar el archivo maestro. En dichos archivos, se tiene destino, fecha, hora de salida y cantidad de asientos comprados. Se sabe que los archivos están ordenados por destino más fecha y hora de salida, y que, en los detalles, pueden venir 0, 1 o más registros por cada uno del maestro. Se pide realizar los módulos necesarios para:

- (a) Actualizar el archivo maestro sabiendo que no se registró ninguna venta de pasaje sin asiento disponible.
- (b) Generar una lista con aquellos vuelos (destino, fecha y hora de salida) que tengan menos de una cantidad específica de asientos disponibles. La misma debe ser ingresada por teclado.

Nota: El archivo maestro y los archivos detalles sólo pueden recorrerse una vez.

```
program TP2_E14;
{$codepage UTF8}
uses crt, sysutils;
const
    detalles_total=2;
    destino_salida='ZZZ';
type
    t_detalle=1..detalles_total;
    t_string20=string[20];
    t_registro_vuelo=record
        destino: t_string20;
        fecha: t_string20;
        hora: t_string20;
        asientos_disponibles: int16;
    end;
    t_registro_venta=record
        destino: t_string20;
        fecha: t_string20;
        hora: t_string20;
        asientos_vendidos: int16;
    end;
    t_archivo_maestro=file of t_registro_vuelo;
    t_archivo_detalle=file of t_registro_venta;
    t_vector_ventas=array[t_detalle] of t_registro_venta;
    t_vector_detalle=array[t_detalle] of t_archivo_detalle;
    t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_vuelo: t_registro_vuelo;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_vuelo do
            begin
                readln(archivo_carga_maestro,asientos_disponibles,destino); destino:=trim(destino);
                readln(archivo_carga_maestro,fecha);
```

```

        readln(archivo_carga_maestro,hora);
        write(archivo_maestro,registro_vuelo);
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,asientos_vendidos,destino); destino:=trim(destino);
                readln(archivo_carga_detalle,fecha);
                readln(archivo_carga_detalle,hora);
                write(archivo_detalle,registro_venta);
            end;
        end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_vuelo(registro_vuelo: t_registro_vuelo);
begin
    textcolor(green); write('Destino: '); textcolor(yellow); write(registro_vuelo.destino);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_vuelo.fecha);
    textcolor(green); write('; Hora: '); textcolor(yellow); write(registro_vuelo.hora);
    textcolor(green); write('; Asientos disponibles: '); textcolor(yellow);
writeln(registro_vuelo.asientos_disponibles);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_vuelo: t_registro_vuelo;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_vuelo);
            imprimir_registro_vuelo(registro_vuelo);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Destino: '); textcolor(yellow); write(registro_venta.destino);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_venta.fecha);
    textcolor(green); write('; Hora: '); textcolor(yellow); write(registro_venta.hora);
    textcolor(green); write('; Asientos vendidos: '); textcolor(yellow);
writeln(registro_venta.asientos_vendidos);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_detalle);
end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);

```

```

begin
  if (not eof(archivo_detalle)) then
    read(archivo_detalle, registro_venta)
  else
    registro_venta.destino:=destino_salida;
end;
procedure minimo(var vector_detalles: t_vector_detalles; var vector_ventas: t_vector_ventas;
var min: t_registro_venta);
var
  i, pos: t_detalle;
begin
  min.destino:=destino_salida;
  for i:= 1 to detalles_total do
    if ((vector_ventas[i].destino<min.destino) or ((vector_ventas[i].destino=min.destino) and
(vector_ventas[i].fecha<min.fecha)) or ((vector_ventas[i].destino=min.destino) and
(vector_ventas[i].fecha=min.fecha) and (vector_ventas[i].hora<min.hora))) then
      begin
        min:=vector_ventas[i];
        pos:=i;
      end;
    if (min.destino<destino_salida) then
      leer_venta(vector_detalles[pos], vector_ventas[pos]);
end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalles: t_vector_detalles; asientos: int16);
var
  registro_vuelo: t_registro_vuelo;
  min: t_registro_venta;
  vector_ventas: t_vector_ventas;
  i: t_detalle;
begin
  reset(archivo_maestro);
  for i:= 1 to detalles_total do
    begin
      reset(vector_detalles[i]);
      leer_venta(vector_detalles[i], vector_ventas[i]);
    end;
    minimo(vector_detalles, vector_ventas, min);
    while (min.destino<>destino_salida) do
      begin
        read(archivo_maestro, registro_vuelo);
        while (registro_vuelo.destino<>min.destino) do
          read(archivo_maestro, registro_vuelo);
        while (registro_vuelo.destino=min.destino) do
          begin
            while (registro_vuelo.fecha<>min.fecha) do
              read(archivo_maestro, registro_vuelo);
            while ((registro_vuelo.destino=min.destino) and (registro_vuelo.fecha=min.fecha)) do
              begin
                while (registro_vuelo.hora<>min.hora) do
                  read(archivo_maestro, registro_vuelo);
                while ((registro_vuelo.destino=min.destino) and (registro_vuelo.fecha=min.fecha) and
(registro_vuelo.hora=min.hora)) do
                  begin
                    registro_vuelo.asientos_disponibles:=registro_vuelo.asientos_disponibles-
min.asientos_vendidos;
                    minimo(vector_detalles, vector_ventas, min);
                  end;
                  seek(archivo_maestro, filepos(archivo_maestro)-1);
                  write(archivo_maestro, registro_vuelo);
                  if (registro_vuelo.asientos_disponibles<asientos) then
                    begin
                      textcolor(green); write('El vuelo con destino a '); textcolor(yellow);
write(registro_vuelo.destino); textcolor(green); write(' del '); textcolor(yellow);
write(registro_vuelo.fecha); textcolor(green); write(' a las '); textcolor(yellow);

```

```

write(registro_vuelo.hora); textcolor(green); write(' tiene menos de '); textcolor(yellow);
write(asientos); textcolor(green); writeln(' asientos disponibles');
    end;
    end;
    end;
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    writeln();
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
    asientos: int16;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E14_vuelosMaestro');
    assign(archivo_carga_maestro,'E14_vuelosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
            assign(vector_detalle[i],'E14_ventasDetalle'+inttoStr(i));
            assign(vector_carga_detalle[i],'E14_ventasDetalle'+inttoStr(i)+'.txt');
            cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
            imprimir_archivo_detalle(vector_detalle[i]);
        end;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    asientos:=10+random(291);
    actualizar_archivo_maestro(archivo_maestro,vector_detalle,asientos);
    imprimir_archivo_maestro(archivo_maestro);
end.

```

Ejercicio 15.

Se desea modelar la información de una ONG dedicada a la asistencia de personas con carencias habitacionales. La ONG cuenta con un archivo maestro conteniendo información como se indica a continuación: código provincia, nombre provincia, código de localidad, nombre de localidad, #viviendas sin luz, #viviendas sin gas, #viviendas de chapa, #viviendas sin agua, #viviendas sin sanitarios.

Mensualmente, reciben detalles de las diferentes provincias indicando avances en las obras de ayuda en la edificación y equipamientos de viviendas en cada provincia. La información de los detalles es la siguiente: código provincia, código localidad, #viviendas con luz, #viviendas construidas, #viviendas con agua, #viviendas con gas, #entrega sanitarios.

Se debe realizar el procedimiento que permita actualizar el maestro con los detalles recibidos, se reciben 10 detalles. Todos los archivos están ordenados por código de provincia y código de localidad.

Para la actualización del archivo maestro, se debe proceder de la siguiente manera:

- *Al valor de viviendas sin luz se le resta el valor recibido en el detalle.*
- *Ídem para viviendas sin agua, sin gas y sin sanitarios.*
- *A las viviendas de chapa se le resta el valor recibido de viviendas construidas.*

La misma combinación de provincia y localidad aparecen, a lo sumo, una única vez.

Realizar las declaraciones necesarias, el programa principal y los procedimientos que se requiera para la actualización solicitada e informar cantidad de localidades sin viviendas de chapa (las localidades pueden o no haber sido actualizadas).

```
program TP2_E15;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=10;
  codigo_salida=999;
  viviendas_de_chapa_corte=0;
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_provincia1=record
    codigo: int16;
    nombre: t_string20;
    codigo_localidad: int16;
    nombre_localidad: t_string20;
    viviendas_sin_luz: int16;
    viviendas_sin_gas: int16;
    viviendas_de_chapa: int16;
    viviendas_sin_agua: int16;
    viviendas_sin_sanitarios: int16;
  end;
  t_registro_provincia2=record
    codigo: int16;
    codigo_localidad: int16;
    viviendas_con_luz: int16;
```

```

viviendas_construidas: int16;
viviendas_con_agua: int16;
viviendas_con_gas: int16;
entrega_sanitarios: int16;
end;
t_archivo_maestro=file of t_registro_provincia1;
t_archivo_detalle=file of t_registro_provincia2;
t_vector_provincias=array[t_detalle] of t_registro_provincia2;
t_vector_detalle=array[t_detalle] of t_archivo_detalle;
t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
registro_provincia: t_registro_provincia1;
begin
rewrite(archivo_maestro);
reset(archivo_carga_maestro);
while (not eof(archivo_carga_maestro)) do
with registro_provincia do
begin
readln(archivo_carga_maestro,codigo,codigo_localidad,viviendas_sin_luz,viviendas_sin_gas
,viviendas_de_chapa,viviendas_sin_agua,viviendas_sin_sanitarios,nombre); nombre:=trim(nombre);
readln(archivo_carga_maestro,nombre_localidad);
nombre_localidad:=trim(nombre_localidad);
write(archivo_maestro,registro_provincia);
end;
close(archivo_maestro);
close(archivo_carga_maestro);
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
registro_provincia: t_registro_provincia2;
begin
rewrite(archivo_detalle);
reset(archivo_carga_detalle);
while (not eof(archivo_carga_detalle)) do
with registro_provincia do
begin
readln(archivo_carga_detalle,codigo,codigo_localidad,viviendas_con_luz,viviendas_constru
idas,viviendas_con_agua,viviendas_con_gas,entrega_sanitarios);
write(archivo_detalle,registro_provincia);
end;
close(archivo_detalle);
close(archivo_carga_detalle);
end;
procedure imprimir_registro_provincial(registro_provincia: t_registro_provincia1);
begin
textcolor(green); write('Código de provincia: '); textcolor(yellow);
write(registro_provincia.codigo);
textcolor(green); write('; Nombre de provincia: '); textcolor(yellow);
write(registro_provincia.nombre);
textcolor(green); write('; Código de localidad: '); textcolor(yellow);
write(registro_provincia.codigo_localidad);
textcolor(green); write('; Nombre de localidad: '); textcolor(yellow);
write(registro_provincia.nombre_localidad);
textcolor(green); write('; Viviendas sin luz: '); textcolor(yellow);
write(registro_provincia.viviendas_sin_luz);
textcolor(green); write('; Viviendas sin gas: '); textcolor(yellow);
write(registro_provincia.viviendas_sin_gas);
textcolor(green); write('; Viviendas de chapa: '); textcolor(yellow);
write(registro_provincia.viviendas_de_chapa);
textcolor(green); write('; Viviendas sin agua: '); textcolor(yellow);
write(registro_provincia.viviendas_sin_agua);
textcolor(green); write('; Viviendas sin sanitarios: '); textcolor(yellow);
writeln(registro_provincia.viviendas_sin_sanitarios);

```

```

end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_provincia: t_registro_provincia1;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro, registro_provincia);
            imprimir_registro_provincia1(registro_provincia);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_provincia2(registro_provincia: t_registro_provincia2);
begin
    textcolor(green); write('Código de provincia: '); textcolor(yellow);
write(registro_provincia.codigo);
    textcolor(green); write('; Código de localidad: '); textcolor(yellow);
write(registro_provincia.codigo_localidad);
    textcolor(green); write('; Viviendas con luz: '); textcolor(yellow);
write(registro_provincia.viviendas_con_luz);
    textcolor(green); write('; Viviendas construidas: '); textcolor(yellow);
write(registro_provincia.viviendas_construidas);
    textcolor(green); write('; Viviendas con agua: '); textcolor(yellow);
write(registro_provincia.viviendas_con_agua);
    textcolor(green); write('; Viviendas con gas: '); textcolor(yellow);
write(registro_provincia.viviendas_con_gas);
    textcolor(green); write('; Entrega sanitarios: '); textcolor(yellow);
writeln(registro_provincia.entrega_sanitarios);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_provincia: t_registro_provincia2;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle, registro_provincia);
            imprimir_registro_provincia2(registro_provincia);
        end;
    close(archivo_detalle);
end;
procedure leer_provincia_detalle(var archivo_detalle: t_archivo_detalle; var
registro_provincia: t_registro_provincia2);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_provincia)
    else
        registro_provincia.codigo:=codigo_salida;
end;
procedure minimo(var vector_detalle: t_vector_detalle; var vector_provincias:
t_vector_provincias; var min: t_registro_provincia2);
var
    i, pos: t_detalle;
begin
    min.codigo:=codigo_salida;
    for i:= 1 to detalles_total do
        if ((vector_provincias[i].codigo<min.codigo) or ((vector_provincias[i].codigo=min.codigo)
and (vector_provincias[i].codigo_localidad<min.codigo_localidad))) then
            begin
                min:=vector_provincias[i];
                pos:=i;
            end;
        if (min.codigo<codigo_salida) then
            leer_provincia_detalle(vector_detalle[pos], vector_provincias[pos]);
end;

```

```

procedure actualizar_localidades_corte(viviendas_de_chapa_localidad: int16; var
localidades_corte: int16);
begin
    if (viviendas_de_chapa_localidad=viviendas_de_chapa_corte) then
        localidades_corte:=localidades_corte+1;
    end;
procedure actualizar1_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_provincia: t_registro_provincia1;
    min: t_registro_provincia2;
    vector_provincias: t_vector_provincias;
    i: t_detalle;
    localidades_corte: int16;
    ok: boolean;
begin
    localidades_corte:=0;
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_provincia_detalle(vector_detalle[i],vector_provincias[i]);
        end;
        minimo(vector_detalle,vector_provincias,min);
        while (min.codigo<>codigo_salida) do
            begin
                read(archivo_maestro,registro_provincia);
                while (registro_provincia.codigo<>min.codigo) do
                    begin
                        actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte);
                        read(archivo_maestro,registro_provincia);
                    end;
                    ok:=true;
                    while (registro_provincia.codigo=min.codigo) do
                        begin
                            while (registro_provincia.codigo_localidad<>min.codigo_localidad) do
                                begin
                                    if (ok=true) then
                                        actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte
);
                                    read(archivo_maestro,registro_provincia);
                                    ok:=true;
                                end;
                                while ((registro_provincia.codigo=min.codigo) and
(registro_provincia.codigo_localidad=min.codigo_localidad)) do
                                    begin
                                        registro_provincia.viviendas_sin_luz:=registro_provincia.viviendas_sin_luz-
min.viviendas_con_luz;
                                        registro_provincia.viviendas_sin_agua:=registro_provincia.viviendas_sin_agua-
min.viviendas_con_agua;
                                        registro_provincia.viviendas_sin_gas:=registro_provincia.viviendas_sin_gas-
min.viviendas_con_gas;
                                        registro_provincia.viviendas_sin_sanitarios:=registro_provincia.viviendas_sin_sanitari
os-min.entrega_sanitarios;
                                        registro_provincia.viviendas_de_chapa:=registro_provincia.viviendas_de_chapa-
min.viviendas_construidas;
                                        minimo(vector_detalle,vector_provincias,min);
                                    end;
                                    seek(archivo_maestro,filepos(archivo_maestro)-1);
                                    write(archivo_maestro,registro_provincia);
                                    actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte);
                                    ok:=false;
                                end;
                            end;
                        while (not eof(archivo_maestro)) do
                            begin

```



```

    read(archivo_maestro,registro_provincia);
    actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte);
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    textcolor(green); write('La cantidad de localidades con '); textcolor(yellow);
write(viviendas_de_chapa_corte); textcolor(green); write(' viviendas de chapa es ');
textcolor(red); writeln(localidades_corte);
    writeln();
end;
procedure leer_provincia_maestro(var archivo_maestro: t_archivo_maestro; var
registro_provincia: t_registro_provincial);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro,registro_provincia)
    else
        registro_provincia.codigo:=codigo_salida;
end;
procedure actualizar2_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_provincia: t_registro_provincial;
    min: t_registro_provincia2;
    vector_provincias: t_vector_provincias;
    i: t_detalle;
    localidades_corte: int16;
    ok: boolean;
begin
    localidades_corte:=0;
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_provincia_detalle(vector_detalle[i],vector_provincias[i]);
        end;
    minimo(vector_detalle,vector_provincias,min);
    leer_provincia_maestro(archivo_maestro,registro_provincia);
    while (registro_provincia.codigo<>codigo_salida) do
        begin
            ok:=false;
            while ((registro_provincia.codigo=min.codigo) and
(registro_provincia.codigo_localidad=min.codigo_localidad)) do
                begin
                    ok:=true;
                    registro_provincia.viviendas_sin_luz:=registro_provincia.viviendas_sin_luz-
min.viviendas_con_luz;
                    registro_provincia.viviendas_sin_agua:=registro_provincia.viviendas_sin_agua-
min.viviendas_con_agua;
                    registro_provincia.viviendas_sin_gas:=registro_provincia.viviendas_sin_gas-
min.viviendas_con_gas;
                    registro_provincia.viviendas_sin_sanitarios:=registro_provincia.viviendas_sin_sanitarios-
min.entrega_sanitarios;
                    registro_provincia.viviendas_de_chapa:=registro_provincia.viviendas_de_chapa-
min.viviendas_construidas;
                    minimo(vector_detalle,vector_provincias,min);
                end;
            if (ok=true) then
                begin
                    seek(archivo_maestro,filepos(archivo_maestro)-1);
                    write(archivo_maestro,registro_provincia);
                end;
            actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte);
            leer_provincia_maestro(archivo_maestro,registro_provincia);
        end;
    close(archivo_maestro);
end;

```

```
for i:= 1 to detalles_total do
  close(vector_detalle[i]);
  textcolor(green); write('La cantidad de localidades con '); textcolor(yellow);
write(viviendas_de_chapa_corte); textcolor(green); write(' viviendas de chapa es ');
textcolor(red); writeln(localidades_corte);
  writeln();
end;
var
  vector_detalle: t_vector_detalle;
  vector_carga_detalle: t_vector_carga_detalle;
  archivo_maestro: t_archivo_maestro;
  archivo_carga_maestro: text;
  i: t_detalle;
begin
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
  assign(archivo_maestro,'E15_provinciasMaestro');
assign(archivo_carga_maestro,'E15_provinciasMaestro.txt');
  cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
  imprimir_archivo_maestro(archivo_maestro);
  for i:= 1 to detalles_total do
    begin
      writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
      assign(vector_detalle[i],'E15_provinciasDetalle'+intToStr(i));
assign(vector_carga_detalle[i],'E15_provinciasDetalle'+intToStr(i)+'.txt');
      cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
      imprimir_archivo_detalle(vector_detalle[i]);
    end;
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
  //actualizar1_archivo_maestro(archivo_maestro,vector_detalle);
  actualizar2_archivo_maestro(archivo_maestro,vector_detalle);
  imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 16.

La editorial X, autora de diversos semanarios, posee un archivo maestro con la información correspondiente a las diferentes emisiones de los mismos. De cada emisión, se registra: fecha, código de semanario, nombre del semanario, descripción, precio, total de ejemplares y total de ejemplares vendidos.

Mensualmente, se reciben 100 archivos detalles con las ventas de los semanarios en todo el país. La información que poseen los detalles es la siguiente: fecha, código de semanario y cantidad de ejemplares vendidos. Realizar las declaraciones necesarias, la llamada al procedimiento y el procedimiento que recibe el archivo maestro y los 100 detalles y realizar la actualización del archivo maestro en función de las ventas registradas. Además, se deberá informar fecha y semanario que tuvo más ventas y la misma información del semanario con menos ventas.

Nota: Todos los archivos están ordenados por fecha y código de semanario. No se realizan ventas de semanarios si no hay ejemplares para hacerlo.

```
program TP2_E16;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=100;
  fecha_salida='ZZZ';
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_semanario=record
    fecha: t_string20;
    codigo: int16;
    nombre: t_string20;
    descripcion: t_string20;
    precio: real;
    total_ejemplares: int16;
    ejemplares_vendidos: int16;
  end;
  t_registro_venta=record
    fecha: t_string20;
    codigo: int16;
    ejemplares_vendidos: int16;
  end;
  t_archivo_maestro=file of t_registro_semanario;
  t_archivo_detalle=file of t_registro_venta;
  t_vector_ventas=array[t_detalle] of t_registro_venta;
  t_vector_detalle=array[t_detalle] of t_archivo_detalle;
  t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_semanario: t_registro_semanario;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_semanario do
      begin
        readln(archivo_carga_maestro,codigo,precio,total_ejemplares,ejemplares_vendidos,fecha);
        fecha:=trim(fecha);
        readln(archivo_carga_maestro,nombre); nombre:=trim(nombre);
```

```

        readln(archivo_carga_maestro,descripcion); descripcion:=trim(descripcion);
        write(archivo_maestro,registro_semanario);
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,codigo,ejemplares_vendidos,fecha); fecha:=trim(fecha);
                write(archivo_detalle,registro_venta);
            end;
        end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_semanario(registro_semanario: t_registro_semanario);
begin
    textcolor(green); write('Fecha: '); textcolor(yellow); write(registro_semanario.fecha);
    textcolor(green); write('; Código: '); textcolor(yellow); write(registro_semanario.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_semanario.nombre);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
    write(registro_semanario.descripcion);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
    write(registro_semanario.precio:0:2);
    textcolor(green); write('; Total de ejemplares: '); textcolor(yellow);
    write(registro_semanario.total_ejemplares);
    textcolor(green); write('; Ejemplares vendidos: '); textcolor(yellow);
    writeln(registro_semanario.ejemplares_vendidos);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_semanario: t_registro_semanario;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_semanario);
            imprimir_registro_semanario(registro_semanario);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Fecha: '); textcolor(yellow); write(registro_venta.fecha);
    textcolor(green); write('; Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Ejemplares vendidos: '); textcolor(yellow);
    writeln(registro_venta.ejemplares_vendidos);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_detalle);
end;

```

```

end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_venta)
    else
        registro_venta.fecha:=fecha_salida;
    end;
procedure minimo(var vector_detalle: t_vector_detalle; var vector_ventas: t_vector_ventas;
var min: t_registro_venta);
var
    i, pos: t_detalle;
begin
    min.fecha:=fecha_salida;
    for i:= 1 to detalles_total do
        if ((vector_ventas[i].fecha<min.fecha) or ((vector_ventas[i].fecha=min.fecha) and
(vector_ventas[i].codigo<min.codigo))) then
            begin
                min:=vector_ventas[i];
                pos:=i;
            end;
        if (min.fecha<fecha_salida) then
            leer_venta(vector_detalle[pos],vector_ventas[pos]);
        end;
    end;
procedure actualizar_maximo_minimo(ventas: int16; fecha: t_string20; codigo: int16; var
ventas_max, ventas_min: int16; var fecha_max, fecha_min: t_string20; var codigo_max,
codigo_min: int16);
begin
    if (ventas>ventas_max) then
        begin
            ventas_max:=ventas;
            fecha_max:=fecha;
            codigo_max:=codigo;
        end;
    if (ventas<ventas_min) then
        begin
            ventas_min:=ventas;
            fecha_min:=fecha;
            codigo_min:=codigo;
        end;
    end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_semanario: t_registro_semanario;
    min: t_registro_venta;
    vector_ventas: t_vector_ventas;
    i: t_detalle;
    fecha_max, fecha_min: t_string20;
    ventas, ventas_max, ventas_min, codigo_max, codigo_min: int16;
begin
    ventas_max:=low(int16); fecha_max:=''; codigo_max:=0;
    ventas_min:=high(int16); fecha_min:=''; codigo_min:=0;
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_venta(vector_detalle[i],vector_ventas[i]);
        end;
    minimo(vector_detalle,vector_ventas,min);
    while (min.fecha<>fecha_salida) do
        begin
            read(archivo_maestro,registro_semanario);
            while (registro_semanario.fecha<>min.fecha) do
                read(archivo_maestro,registro_semanario);
            end;
        end;
    end;
end;

```

```

while (registro_semanario.fecha=min.fecha) do
begin
    ventas:=0;
    while (registro_semanario.codigo<>min.codigo) do
        read(archivo_maestro,registro_semanario);
        while ((registro_semanario.fecha=min.fecha) and (registro_semanario.codigo=min.codigo))
do
    begin
        if (registro_semanario.total_ejemplares>=min.ejemplares_vendidos) then
        begin
            registro_semanario.total_ejemplares:=registro_semanario.total_ejemplares-
min.ejemplares_vendidos;
            registro_semanario.ejemplares_vendidos:=registro_semanario.ejemplares_vendidos+min.e
jemplares_vendidos;
            ventas:=ventas+min.ejemplares_vendidos;
        end;
        minimo(vector_detalle,vector_ventas,min);
    end;
    seek(archivo_maestro,filepos(archivo_maestro)-1);
    write(archivo_maestro,registro_semanario);
    actualizar_maximo_minimo(ventas,registro_semanario.fecha,registro_semanario.codigo,venta
s_max,ventas_min,fecha_max,fecha_min,codigo_max,codigo_min);
    end;
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    textcolor(green); write('El semanario que tuvo más ventas fue el '); textcolor(red);
write(codigo_max); textcolor(green); write(' de la fecha '); textcolor(red); write(fecha_max);
textcolor(green); write(', con un total de '); textcolor(red); write(ventas_max);
textcolor(green); writeln(' ventas');
    textcolor(green); write('El semanario que tuvo menos ventas fue el '); textcolor(red);
write(codigo_min); textcolor(green); write(' de la fecha '); textcolor(red); write(fecha_min);
textcolor(green); write(', con un total de '); textcolor(red); write(ventas_min);
textcolor(green); writeln(' ventas');
    writeln();
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E16_semanariosMaestro');
    assign(archivo_carga_maestro,'E16_semanariosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,:'); writeln();
            assign(vector_detalle[i],'E16_ventasDetalle'+intToStr(i));
            assign(vector_carga_detalle[i],'E16_ventasDetalle'+intToStr(i)+'.txt');
            cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
            imprimir_archivo_detalle(vector_detalle[i]);
        end;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro,vector_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.

```

Ejercicio 17.

Una concesionaria de motos de la Ciudad de Chascomús posee un archivo con información de las motos que posee a la venta. De cada moto, se registra: código, nombre, descripción, modelo, marca y stock actual. Mensualmente, se reciben 10 archivos detalles con información de las ventas de cada uno de los 10 empleados que trabajan. De cada archivo detalle, se dispone de la siguiente información: código de moto, precio y fecha de la venta. Se debe realizar un proceso que actualice el stock del archivo maestro desde los archivos detalles. Además, se debe informar cuál fue la moto más vendida.

Nota: Todos los archivos están ordenados por código de la moto y el archivo maestro debe ser recorrido sólo una vez y en forma simultánea con los detalles.

```
program TP2_E17;
{$codepage UTF8}
uses crt, sysutils;
const
    detalles_total=3; // detalles_total=10;
    codigo_salida=999;
type
    t_detalle=1..detalles_total;
    t_string20=string[20];
    t_registro_moto=record
        codigo: int16;
        nombre: t_string20;
        descripcion: t_string20;
        modelo: t_string20;
        marca: t_string20;
        stock: int16;
    end;
    t_registro_venta=record
        codigo: int16;
        precio: real;
        fecha: t_string20;
    end;
    t_archivo_maestro=file of t_registro_moto;
    t_archivo_detalle=file of t_registro_venta;
    t_vector_ventas=array[t_detalle] of t_registro_venta;
    t_vector_detalle=array[t_detalle] of t_archivo_detalle;
    t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_moto: t_registro_moto;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_moto do
            begin
                readln(archivo_carga_maestro,codigo,stock,nombre); nombre:=trim(nombre);
                readln(archivo_carga_maestro,descripcion); descripcion:=trim(descripcion);
                readln(archivo_carga_maestro,modelo); modelo:=trim(modelo);
                readln(archivo_carga_maestro,marca); marca:=trim(marca);
                write(archivo_maestro,registro_moto);
            end;
        end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
```

```

procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,codigo,precio,fecha); fecha:=trim(fecha);
                write(archivo_detalle,registro_venta);
            end;
        close(archivo_detalle);
        close(archivo_carga_detalle);
    end;
procedure imprimir_registro_moto(registro_moto: t_registro_moto);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_moto.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_moto.nombre);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_moto.descripcion);
    textcolor(green); write('; Modelo: '); textcolor(yellow); write(registro_moto.modelo);
    textcolor(green); write('; Marca: '); textcolor(yellow); write(registro_moto.marca);
    textcolor(green); write('; Stock: '); textcolor(yellow); writeln(registro_moto.stock);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_moto: t_registro_moto;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_moto);
            imprimir_registro_moto(registro_moto);
        end;
        close(archivo_maestro);
    end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Precio: $'); textcolor(yellow); write(registro_venta.precio:0:2);
    textcolor(green); write('; Fecha: '); textcolor(yellow); writeln(registro_venta.fecha);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
        close(archivo_detalle);
    end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_venta)
    else
        registro_venta.codigo:=codigo_salida;
    end;
procedure minimo(var vector_detalle: t_vector_detalle; var vector_ventas: t_vector_ventas;
var min: t_registro_venta);

```



```

var
    i, pos: t_detalle;
begin
    min.codigo:=codigo_salida;
    for i:= 1 to detalles_total do
        if (vector_ventas[i].codigo<min.codigo) then
            begin
                min:=vector_ventas[i];
                pos:=i;
            end;
        if (min.codigo<codigo_salida) then
            leer_venta(vector_detalle[pos],vector_ventas[pos]);
        end;
    procedure actualizar_maximo(ventas: int16; registro_moto: t_registro_moto; var ventas_max:
    int16; var registro_moto_max: t_registro_moto);
    begin
        if (ventas>ventas_max) then
            begin
                ventas_max:=ventas;
                registro_moto_max:=registro_moto;
            end;
        end;
    procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
    vector_detalle: t_vector_detalle);
    var
        registro_moto, registro_moto_max: t_registro_moto;
        min: t_registro_venta;
        vector_ventas: t_vector_ventas;
        i: t_detalle;
        ventas, ventas_max: int16;
    begin
        ventas_max:=low(int16);
        reset(archivo_maestro);
        for i:= 1 to detalles_total do
            begin
                reset(vector_detalle[i]);
                leer_venta(vector_detalle[i],vector_ventas[i]);
            end;
        minimo(vector_detalle,vector_ventas,min);
        while (min.codigo<>codigo_salida) do
            begin
                ventas:=0;
                read(archivo_maestro,registro_moto);
                while (registro_moto.codigo<>min.codigo) do
                    read(archivo_maestro,registro_moto);
                while (registro_moto.codigo=min.codigo) do
                    begin
                        ventas:=ventas+1;
                        minimo(vector_detalle,vector_ventas,min);
                    end;
                registro_moto.stock:=registro_moto.stock-ventas;
                seek(archivo_maestro,filepos(archivo_maestro)-1);
                write(archivo_maestro,registro_moto);
                actualizar_maximo(ventas,registro_moto,ventas_max,registro_moto_max);
            end;
        close(archivo_maestro);
        for i:= 1 to detalles_total do
            close(vector_detalle[i]);
            textcolor(green); write('La moto más vendida fue la '); textcolor(red);
            write(registro_moto_max.marca); textcolor(green); write(' '); textcolor(red);
            write(registro_moto_max.modelo); textcolor(green); write(', con un total de ');
            textcolor(red); write(ventas_max); textcolor(green); writeln(' ventas');
            writeln();
        end;
    var
        vector_detalle: t_vector_detalle;

```

```
vector_carga_detalles: t_vector_carga_detalles;
archivo_maestro: t_archivo_maestro;
archivo_carga_maestro: text;
i: t_detalle;
begin
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
  assign(archivo_maestro, 'E17_motosMaestro');
assign(archivo_carga_maestro, 'E17_motosMaestro.txt');
  cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
  imprimir_archivo_maestro(archivo_maestro);
  for i:= 1 to detalles_total do
  begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ', i, ':'); writeln();
    assign(vector_detalles[i], 'E17_ventasDetalle'+intToStr(i));
assign(vector_carga_detalles[i], 'E17_ventasDetalle'+intToStr(i)+'.txt');
    cargar_archivo_detalle(vector_detalles[i], vector_carga_detalles[i]);
    imprimir_archivo_detalle(vector_detalles[i]);
  end;
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
  actualizar_archivo_maestro(archivo_maestro, vector_detalles);
  imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 18.

Se cuenta con un archivo con información de los casos de COVID-19 registrados en los diferentes hospitales de la Provincia de Buenos Aires cada día. Dicho archivo contiene: código de localidad, nombre de localidad, código de municipio, nombre de municipio, código de hospital, nombre de hospital, fecha y cantidad de casos positivos detectados. El archivo está ordenado por localidad, luego por municipio y luego por hospital.

Escribir la definición de las estructuras de datos necesarias y un procedimiento que haga un listado con el siguiente formato:

Nombre: Localidad 1

Nombre: Municipio 1

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....
Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio 1

.....
Nombre Municipio N

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....
Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio N

Cantidad de casos Localidad 1

Nombre Localidad N

Nombre Municipio 1

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....
Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio 1

.....
Nombre Municipio N

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....
Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio N

Cantidad de casos Localidad N

Cantidad de casos Totales en la Provincia

Además del informe en pantalla anterior, es necesario exportar a un archivo de texto la siguiente información: nombre de localidad, nombre de municipio y cantidad de casos del municipio para aquellos municipios cuya cantidad de casos supere los 1.500. El formato del archivo de texto deberá ser el adecuado para recuperar la información con la menor cantidad de lecturas posibles.

Nota: El archivo debe recorrerse sólo una vez.

```

program TP2_E18;
{$codepage UTF8}
uses crt, sysutils;
const
    codigo_salida=999;
    casos_municipio_corte=1500;
type
    t_string20=string[20];
    t_registro_hospital=record
        codigo_localidad: int16;
        nombre_localidad: t_string20;
        codigo_municipio: int16;
        nombre_municipio: t_string20;
        codigo_hospital: int16;
        nombre_hospital: t_string20;
        fecha: t_string20;
        casos_positivos: int16;
    end;
    t_archivo_maestro=file of t_registro_hospital;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_hospital: t_registro_hospital;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
    begin
        with registro_hospital do
        begin
            readln(archivo_carga_maestro,codigo_localidad,codigo_municipio,codigo_hospital,casos_pos
itivos,fecha); fecha:=trim(fecha);
            readln(archivo_carga_maestro,nombre_localidad);
            nombre_localidad:=trim(nombre_localidad);
            readln(archivo_carga_maestro,nombre_municipio);
            nombre_municipio:=trim(nombre_municipio);
            readln(archivo_carga_maestro,nombre_hospital); nombre_hospital:=trim(nombre_hospital);
            write(archivo_maestro,registro_hospital);
        end;
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure imprimir_registro_hospital(registro_hospital: t_registro_hospital);
begin
    textcolor(green); write('Código de localidad: '); textcolor(yellow);
write(registro_hospital.codigo_localidad);
    textcolor(green); write('; Nombre de localidad: '); textcolor(yellow);
write(registro_hospital.nombre_localidad);
    textcolor(green); write('; Código de municipio: '); textcolor(yellow);
write(registro_hospital.codigo_municipio);
    textcolor(green); write('; Nombre de municipio: '); textcolor(yellow);
write(registro_hospital.nombre_municipio);
    textcolor(green); write('; Código de hospital: '); textcolor(yellow);
write(registro_hospital.codigo_hospital);
    textcolor(green); write('; Nombre de hospital: '); textcolor(yellow);
write(registro_hospital.nombre_hospital);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_hospital.fecha);
    textcolor(green); write('; Casos positivos: '); textcolor(yellow);
writeln(registro_hospital.casos_positivos);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);

```

```

var
  registro_hospital: t_registro_hospital;
begin
  reset(archivo_maestro);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro, registro_hospital);
      imprimir_registro_hospital(registro_hospital);
    end;
  close(archivo_maestro);
end;

procedure leer_hospital(var archivo_maestro: t_archivo_maestro; var registro_hospital:
t_registro_hospital);
begin
  if (not eof(archivo_maestro)) then
    read(archivo_maestro, registro_hospital)
  else
    registro_hospital.codigo_localidad:=codigo_salida;
end;

procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
  registro_hospital: t_registro_hospital;
  archivo_txt: text;
  codigo_localidad, codigo_municipio, codigo_hospital, casos_total, casos_localidad,
casos_municipio, casos_hospital: int16;
  nombre_localidad, nombre_municipio, nombre_hospital: t_string20;
begin
  assign(archivo_txt, 'E18_municipiosCasosCorte.txt'); rewrite(archivo_txt);
  casos_total:=0;
  reset(archivo_maestro);
  leer_hospital(archivo_maestro, registro_hospital);
  while (registro_hospital.codigo_localidad<>codigo_salida) do
    begin
      codigo_localidad:=registro_hospital.codigo_localidad;
      nombre_localidad:=registro_hospital.nombre_localidad;
      casos_localidad:=0;
      textcolor(green); write('Localidad: '); textcolor(yellow); writeln(nombre_localidad);
      while (registro_hospital.codigo_localidad=codigo_localidad) do
        begin
          codigo_municipio:=registro_hospital.codigo_municipio;
          nombre_municipio:=registro_hospital.nombre_municipio;
          casos_municipio:=0;
          textcolor(green); write(' Municipio: '); textcolor(yellow); writeln(nombre_municipio);
          textcolor(green); writeln(' Hospital Cantidad de casos');
          while ((registro_hospital.codigo_localidad=codigo_localidad) and
(registro_hospital.codigo_municipio=codigo_municipio)) do
            begin
              codigo_hospital:=registro_hospital.codigo_hospital;
              nombre_hospital:=registro_hospital.nombre_hospital;
              casos_hospital:=0;
              while ((registro_hospital.codigo_localidad=codigo_localidad) and
(registro_hospital.codigo_municipio=codigo_municipio) and
(registro_hospital.codigo_hospital=codigo_hospital)) do
                begin
                  casos_hospital:=casos_hospital+registro_hospital.casos_positivos;
                  leer_hospital(archivo_maestro, registro_hospital);
                end;
                textcolor(green); write(' '); textcolor(yellow); write(nombre_hospital);
textcolor(green); write(' '); textcolor(red); writeln(casos_hospital);
casos_municipio:=casos_municipio+casos_hospital;
            end;
            textcolor(green); write(' Cantidad de casos Municipio: '); textcolor(red);
writeln(casos_municipio);
casos_localidad:=casos_localidad+casos_municipio;
            if (casos_municipio>casos_municipio_corte) then
              begin

```

```
        writeln(archivo_txt,casos_municipio,' ',nombre_localidad);
        writeln(archivo_txt,nombre_municipio);
    end;
end;
textcolor(green); write('Cantidad de casos Localidad: '); textcolor(red);
writeln(casos_localidad); writeln();
casos_total:=casos_total+casos_localidad;
end;
textcolor(green); write('Cantidad de casos Totales: '); textcolor(red);
writeln(casos_total);
close(archivo_maestro);
close(archivo_txt);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E18_hospitalesMaestro');
    assign(archivo_carga_maestro,'E18_hospitalesMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
    procesar_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 19.

A partir de un siniestro ocurrido, se perdieron las actas de nacimiento y fallecimientos de toda la Provincia de Buenos Aires de los últimos diez años. En pos de recuperar dicha información, se deberá procesar 2 archivos por cada una de las 50 delegaciones distribuidas en la provincia, un archivo de nacimientos y otro de fallecimientos, y crear el archivo maestro reuniendo dicha información.

Los archivos detalles con nacimientos contendrán la siguiente información: nro. partida nacimiento, nombre, apellido, dirección detallada (calle, nro., piso, depto., ciudad), matrícula del médico, nombre y apellido de la madre, DNI madre, nombre y apellido del padre, DNI del padre.

En cambio, los 50 archivos de fallecimientos tendrán: nro. partida nacimiento, DNI, nombre y apellido del fallecido, matrícula del médico que firma el deceso, fecha y hora del deceso y lugar.

Realizar un programa que cree el archivo maestro a partir de toda la información de los archivos detalles. Se debe almacenar en el maestro: nro. partida nacimiento, nombre, apellido, dirección detallada (calle, nro., piso, depto., ciudad), matrícula del médico, nombre y apellido de la madre, DNI madre, nombre y apellido del padre, DNI del padre y, si falleció, además, matrícula del médico que firma el deceso, fecha y hora del deceso y lugar. Se deberá, además, listar, en un archivo de texto, la información recolectada de cada persona.

Nota: Todos los archivos están ordenados por nro. partida de nacimiento, que es única. Tener en cuenta que no necesariamente va a fallecer en el distrito donde nació la persona y, además, puede no haber fallecido.

```
program TP2_E19;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_nacimientos_total=1; // detalles_nacimientos_total=50;
  detalles_fallecimientos_total=1; // detalles_fallecimientos_total=50;
  nro_partida_salida=999;
type
  t_detalle1=1..detalles_nacimientos_total;
  t_detalle2=1..detalles_fallecimientos_total;
  t_string10=string[10];
  t_registro_direccion=record
    calle: int16;
    nro: int16;
    piso: int16;
    depto: int16;
    ciudad: t_string10;
  end;
  t_registro_persona=record
    nombre: t_string10;
    apellido: t_string10;
    dni: int16;
  end;
  t_registro_deceso=record
    fecha: t_string10;
    hora: t_string10;
    lugar: t_string10;
```

```

end;
t_registro_nacimiento=record
  nro_partida: int16;
  nombre: t_string10;
  apellido: t_string10;
  direccion: t_registro_direccion;
  matricula_medico: int16;
  madre: t_registro_persona;
  padre: t_registro_persona;
end;
t_registro_fallecimiento=record
  nro_partida: int16;
  fallecido: t_registro_persona;
  matricula_medico: int16;
  deceso: t_registro_deceso;
end;
t_registro_acta=record
  nacimiento: t_registro_nacimiento;
  fallecio: boolean;
  matricula_medico: int16;
  deceso: t_registro_deceso;
end;
t_archivo_maestro=file of t_registro_acta;
t_archivo_detalle_nacimientos=file of t_registro_nacimiento;
t_archivo_detalle_fallecimientos=file of t_registro_fallecimiento;
t_vector_nacimientos=array[t_detalle1] of t_registro_nacimiento;
t_vector_fallecimientos=array[t_detalle2] of t_registro_fallecimiento;
t_vector_detalle_nacimientos=array[t_detalle1] of t_archivo_detalle_nacimientos;
t_vector_detalle_fallecimientos=array[t_detalle2] of t_archivo_detalle_fallecimientos;
t_vector_carga_detalle_nacimientos=array[t_detalle1] of text;
t_vector_carga_detalle_fallecimientos=array[t_detalle2] of text;
procedure cargar_archivo_detalle_nacimientos(var archivo_detalle:
t_archivo_detalle_nacimientos; var archivo_carga_detalle: text);
var
  registro_nacimiento: t_registro_nacimiento;
begin
  rewrite(archivo_detalle);
  reset(archivo_carga_detalle);
  while (not eof(archivo_carga_detalle)) do
    with registro_nacimiento do
      begin
        readln(archivo_carga_detalle,nro_partida,matricula_medico,madre.dni,padre.dni);
        readln(archivo_carga_detalle,nombre); nombre:=trim(nombre);
        readln(archivo_carga_detalle,apellido); apellido:=trim(apellido);
        readln(archivo_carga_detalle,direccion.calle,direccion.nro,direccion.piso,direccion.dept
o,direccion.ciudad); direccion.ciudad:=trim(direccion.ciudad);
        readln(archivo_carga_detalle,madre.nombre); madre.nombre:=trim(madre.nombre);
        readln(archivo_carga_detalle,madre.apellido); madre.apellido:=trim(madre.apellido);
        readln(archivo_carga_detalle,padre.nombre); padre.nombre:=trim(padre.nombre);
        readln(archivo_carga_detalle,padre.apellido); padre.apellido:=trim(padre.apellido);
        write(archivo_detalle,registro_nacimiento);
      end;
    end;
  close(archivo_detalle);
  close(archivo_carga_detalle);
end;
procedure cargar_archivo_detalle_fallecimientos(var archivo_detalle:
t_archivo_detalle_fallecimientos; var archivo_carga_detalle: text);
var
  registro_fallecimiento: t_registro_fallecimiento;
begin
  rewrite(archivo_detalle);
  reset(archivo_carga_detalle);
  while (not eof(archivo_carga_detalle)) do
    with registro_fallecimiento do
      begin
        readln(archivo_carga_detalle,nro_partida,matricula_medico,fallecido.dni);

```



```

        readln(archivo_carga_detalle,fallecido.nombre);
fallecido.nombre:=trim(fallecido.nombre);
        readln(archivo_carga_detalle,fallecido.apellido);
fallecido.apellido:=trim(fallecido.apellido);
        readln(archivo_carga_detalle,deceso.fecha); deceso.fecha:=trim(deceso.fecha);
        readln(archivo_carga_detalle,deceso.hora); deceso.hora:=trim(deceso.hora);
        readln(archivo_carga_detalle,deceso.lugar); deceso.lugar:=trim(deceso.lugar);
        write(archivo_detalle,registro_fallecimiento);
    end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_direccion(registro_direccion: t_registro_direccion);
begin
    textcolor(green); write('; Calle: '); textcolor(yellow); write(registro_direccion.calle);
    textcolor(green); write('; Nro.: '); textcolor(yellow); write(registro_direccion.nro);
    textcolor(green); write('; Piso: '); textcolor(yellow); write(registro_direccion.piso);
    textcolor(green); write('; Depto.: '); textcolor(yellow); write(registro_direccion.depto);
    textcolor(green); write('; Ciudad: '); textcolor(yellow); write(registro_direccion.ciudad);
end;
procedure imprimir_registro_persona(registro_persona: t_registro_persona; persona: string);
begin
    textcolor(green); write('; Nombre ',persona,': '); textcolor(yellow);
write(registro_persona.nombre);
    textcolor(green); write('; Apellido ',persona,': '); textcolor(yellow);
write(registro_persona.apellido);
    if (persona<>'padre') then
    begin
        textcolor(green); write('; DNI ',persona,': '); textcolor(yellow);
write(registro_persona.dni);
    end
    else
    begin
        textcolor(green); write('; DNI ',persona,': '); textcolor(yellow);
writeln(registro_persona.dni);
    end;
end;
procedure imprimir_registro_nacimiento(registro_nacimiento: t_registro_nacimiento);
begin
    textcolor(green); write('Nro. partida nacimiento: '); textcolor(yellow);
write(registro_nacimiento.nro_partida);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_nacimiento.nombre);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_nacimiento.apellido);
    imprimir_registro_direccion(registro_nacimiento.direccion);
    textcolor(green); write('; Matrícula médico: '); textcolor(yellow);
write(registro_nacimiento.matricula_medico);
    imprimir_registro_persona(registro_nacimiento.madre,'madre');
    imprimir_registro_persona(registro_nacimiento.padre,'padre');
end;
procedure imprimir_archivo_detalle_nacimientos(var archivo_detalle:
t_archivo_detalle_nacimientos);
var
    registro_nacimiento: t_registro_nacimiento;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
    begin
        read(archivo_detalle,registro_nacimiento);
        imprimir_registro_nacimiento(registro_nacimiento);
    end;
    close(archivo_detalle);
end;
procedure imprimir_registro_deceso(registro_deceso: t_registro_deceso);
begin

```

```

    textcolor(green); write('; Fecha deceso: '); textcolor(yellow);
write(registro_deceso.fecha);
    textcolor(green); write('; Hora deceso: '); textcolor(yellow); write(registro_deceso.hora);
    textcolor(green); write('; Lugar deceso: '); textcolor(yellow);
writeln(registro_deceso.lugar);
end;
procedure imprimir_registro_fallecimiento(registro_fallecimiento: t_registro_fallecimiento);
begin
    textcolor(green); write('Nro. partida nacimiento: '); textcolor(yellow);
write(registro_fallecimiento.nro_partida);
    imprimir_registro_persona(registro_fallecimiento.fallecido, 'fallecido');
    textcolor(green); write('; Matrícula médico deceso: '); textcolor(yellow);
write(registro_fallecimiento.matricula_medico);
    imprimir_registro_deceso(registro_fallecimiento.deceso);
end;
procedure imprimir_archivo_detalle_fallecimientos(var archivo_detalle:
t_archivo_detalle_fallecimientos);
var
    registro_fallecimiento: t_registro_fallecimiento;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle, registro_fallecimiento);
            imprimir_registro_fallecimiento(registro_fallecimiento);
        end;
        close(archivo_detalle);
    end;
procedure leer_nacimiento(var archivo_detalle: t_archivo_detalle_nacimientos; var
registro_nacimiento: t_registro_nacimiento);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_nacimiento)
    else
        registro_nacimiento.nro_partida:=nro_partida_salida;
    end;
procedure leer_fallecimiento(var archivo_detalle: t_archivo_detalle_fallecimientos; var
registro_fallecimiento: t_registro_fallecimiento);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_fallecimiento)
    else
        registro_fallecimiento.nro_partida:=nro_partida_salida;
    end;
procedure minimo_nacimiento(var vector_detalle_nacimientos: t_vector_detalle_nacimientos;
var vector_nacimientos: t_vector_nacimientos; var min_nacimiento: t_registro_nacimiento);
var
    i, pos: t_detalle1;
begin
    min_nacimiento.nro_partida:=nro_partida_salida;
    for i:= 1 to detalles_nacimientos_total do
        if (vector_nacimientos[i].nro_partida<min_nacimiento.nro_partida) then
            begin
                min_nacimiento:=vector_nacimientos[i];
                pos:=i;
            end;
        if (min_nacimiento.nro_partida<nro_partida_salida) then
            leer_nacimiento(vector_detalle_nacimientos[pos], vector_nacimientos[pos]);
        end;
procedure minimo_fallecimiento(var vector_detalle_fallecimientos:
t_vector_detalle_fallecimientos; var vector_fallecimientos: t_vector_fallecimientos; var
min_fallecimiento: t_registro_fallecimiento);
var
    i, pos: t_detalle2;
begin
    min_fallecimiento.nro_partida:=nro_partida_salida;

```

```

for i:= 1 to detalles_fallecimientos_total do
  if (vector_fallecimientos[i].nro_partida<min_fallecimiento.nro_partida) then
    begin
      min_fallecimiento:=vector_fallecimientos[i];
      pos:=i;
    end;
  if (min_fallecimiento.nro_partida<nro_partida_salida) then
    leer_fallecimiento(vector_detalle_fallecimientos[pos],vector_fallecimientos[pos]);
end;
procedure exportar_archivo_txt(var archivo_maestro: t_archivo_maestro);
var
  registro_acta: t_registro_acta;
  archivo_txt: text;
  i: int16;
begin
  i:=1;
  reset(archivo_maestro);
  assign(archivo_txt,'E19_actasMaestro.txt'); rewrite(archivo_txt);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro,registro_acta);
      with registro_acta do
        begin
          writeln(archivo_txt,'ACTA ',i);
          writeln(archivo_txt,'Nro. partida nacimiento: ',nacimient.nro_partida);
          writeln(archivo_txt,'Nombre: ',nacimient.nombre);
          writeln(archivo_txt,'Apellido: ',nacimient.apellido);
          writeln(archivo_txt,'Dirección: Calle ',nacimient.direccion.calle,', Nro.
',nacimient.direccion.nro,', Piso ',nacimient.direccion.piso,', Depto.
',nacimient.direccion.depto,', Ciudad ',nacimient.direccion.ciudad);
          writeln(archivo_txt,'Matrícula médico: ',nacimient.matricula_medico);
          writeln(archivo_txt,'Madre: Nombre ',nacimient.madre.nombre,', Apellido
',nacimient.madre.apellido,', DNI ',nacimient.madre.dni);
          writeln(archivo_txt,'Padre: Nombre ',nacimient.padre.nombre,', Apellido
',nacimient.padre.apellido,', DNI ',nacimient.padre.dni);
          if (fallecio=true) then
            begin
              writeln(archivo_txt,'Falleció: Sí');
              writeln(archivo_txt,'Matrícula médico deceso: ',matricula_medico);
              writeln(archivo_txt,'Deceso: Fecha ',deceso.fecha,', Hora ',deceso.hora,', Lugar
',deceso.lugar);
            end
          else
            begin
              writeln(archivo_txt,'Falleció: No');
            end;
          writeln(archivo_txt);
        end;
      i:=i+1;
    end;
  close(archivo_txt);
end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle_nacimientos: t_vector_detalle_nacimientos; var
vector_detalle_fallecimientos: t_vector_detalle_fallecimientos);
var
  registro_acta: t_registro_acta;
  min_nacimiento: t_registro_nacimiento;
  min_fallecimiento: t_registro_fallecimiento;
  vector_nacimientos: t_vector_nacimientos;
  vector_fallecimientos: t_vector_fallecimientos;
  i: t_detalle1;
  j: t_detalle2;
begin
  rewrite(archivo_maestro);
  for i:= 1 to detalles_nacimientos_total do

```

```

begin
    reset(vector_detalle_nacimientos[i]);
    leer_nacimiento(vector_detalle_nacimientos[i],vector_nacimientos[i]);
end;
for j:= 1 to detalles_fallecimientos_total do
begin
    reset(vector_detalle_fallecimientos[j]);
    leer_fallecimiento(vector_detalle_fallecimientos[j],vector_fallecimientos[j]);
end;
minimo_nacimiento(vector_detalle_nacimientos,vector_nacimientos,min_nacimiento);
minimo_fallecimiento(vector_detalle_fallecimientos,vector_fallecimientos,min_fallecimiento)
;
while (min_nacimiento.nro_partida<>nro_partida_salida) do
begin
    registro_acta.nacimiento:=min_nacimiento;
    if (min_nacimiento.nro_partida=min_fallecimiento.nro_partida) then
begin
        registro_acta.fallecio:=true;
        registro_acta.matricula_medico:=min_fallecimiento.matricula_medico;
        registro_acta.deceso:=min_fallecimiento.deceso;
        minimo_fallecimiento(vector_detalle_fallecimientos,vector_fallecimientos,min_fallecimie
nto);
    end
    else
        registro_acta.fallecio:=false;
        write(archivo_maestro,registro_acta);
        minimo_nacimiento(vector_detalle_nacimientos,vector_nacimientos,min_nacimiento);
    end;
    exportar_archivo_txt(archivo_maestro);
    close(archivo_maestro);
    for i:= 1 to detalles_nacimientos_total do
        close(vector_detalle_nacimientos[i]);
    for j:= 1 to detalles_fallecimientos_total do
        close(vector_detalle_fallecimientos[j]);
    end;
end;
procedure imprimir_registro_acta(registro_acta: t_registro_acta);
begin
    imprimir_registro_nacimiento(registro_acta.nacimiento);
    if (registro_acta.fallecio=true) then
begin
        textcolor(green); write('; Falleció: '); textcolor(yellow); write('Sí');
        textcolor(green); write('; Matrícula médico deceso: '); textcolor(yellow);
write(registro_acta.matricula_medico);
        imprimir_registro_deceso(registro_acta.deceso);
    end
    else
begin
        textcolor(green); write('; Falleció: '); textcolor(yellow); writeln('No');
    end;
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_acta: t_registro_acta;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
begin
        read(archivo_maestro,registro_acta);
        imprimir_registro_acta(registro_acta);
    end;
    close(archivo_maestro);
end;
var
    vector_detalle_nacimientos: t_vector_detalle_nacimientos;
    vector_detalle_fallecimientos: t_vector_detalle_fallecimientos;
    vector_carga_detalle_nacimientos: t_vector_carga_detalle_nacimientos;

```

```
vector_carga_detalle_fallecimientos: t_vector_carga_detalle_fallecimientos;
archivo_maestro: t_archivo_maestro;
i: t_detalle1;
j: t_detalle2;
begin
  for i:= 1 to detalles_nacimientos_total do
    begin
      writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE NACIMIENTOS ',i,':');
      writeln();
      assign(vector_detalle_nacimientos[i], 'E19_nacimientosDetalle'+intToStr(i));
      assign(vector_carga_detalle_nacimientos[i], 'E19_nacimientosDetalle'+intToStr(i)+'.txt');
      cargar_archivo_detalle_nacimientos(vector_detalle_nacimientos[i], vector_carga_detalle_nacimientos[i]);
      imprimir_archivo_detalle_nacimientos(vector_detalle_nacimientos[i]);
    end;
    for j:= 1 to detalles_fallecimientos_total do
      begin
        writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE FALLECIMIENTOS ',j,':');
        writeln();
        assign(vector_detalle_fallecimientos[j], 'E19_fallecimientosDetalle'+intToStr(j));
        assign(vector_carga_detalle_fallecimientos[j], 'E19_fallecimientosDetalle'+intToStr(j)+'.txt');
        ;
        cargar_archivo_detalle_fallecimientos(vector_detalle_fallecimientos[j], vector_carga_detalle_fallecimientos[j]);
        imprimir_archivo_detalle_fallecimientos(vector_detalle_fallecimientos[j]);
      end;
      writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
      assign(archivo_maestro, 'E19_actasMaestro');
      cargar_archivo_maestro(archivo_maestro, vector_detalle_nacimientos, vector_detalle_fallecimientos);
      imprimir_archivo_maestro(archivo_maestro);
    end.
```

Introducción a las Bases de Datos

Fundamentos de Organización de Datos

Práctica 3

Parte 1: Bajas en archivos

1. Modificar el ejercicio 4 de la práctica 1 (programa de gestión de empleados), agregándole una opción para realizar bajas copiando el último registro del archivo en la posición del registro a borrar y luego truncando el archivo en la posición del último registro de forma tal de evitar duplicados.
2. Definir un programa que genere un archivo con registros de longitud fija conteniendo información de asistentes a un congreso a partir de la información obtenida por teclado. Se deberá almacenar la siguiente información: nro de asistente, apellido y nombre, email, teléfono y D.N.I. Implementar un procedimiento que, a partir del archivo de datos generado, elimine de forma lógica todos los asistentes con nro de asistente inferior a 1000.

Para ello se podrá utilizar algún carácter especial situándolo delante de algún campo String a su elección. Ejemplo: '@Saldaño'.

3. Realizar un programa que genere un archivo de novelas filmadas durante el presente año. De cada novela se registra: código, género, nombre, duración, director y precio. El programa debe presentar un menú con las siguientes opciones:

- a. Crear el archivo y cargarlo a partir de datos ingresados por teclado. Se utiliza la técnica de lista invertida para recuperar espacio libre en el archivo. Para ello, durante la creación del archivo, en el primer registro del mismo se debe almacenar la cabecera de la lista. Es decir un registro ficticio, inicializando con el valor cero (0) el campo correspondiente al código de novela, el cual indica que no hay espacio libre dentro del archivo.
- b. Abrir el archivo existente y permitir su mantenimiento teniendo en cuenta el inciso a), se utiliza lista invertida para recuperación de espacio. En particular, para el campo de "enlace" de la lista (utilice el código de novela como enlace), se debe especificar los números de registro referenciados con signo negativo, . Una vez abierto el archivo, brindar operaciones para:
 - i. Dar de alta una novela leyendo la información desde teclado. Para esta operación, en caso de ser posible, deberá recuperarse el espacio libre. Es decir, si en el campo correspondiente al código de novela del registro cabecera hay un valor negativo, por ejemplo -5, se debe leer el registro en la posición 5, copiarlo en la posición 0 (actualizar la lista de espacio libre) y grabar el nuevo registro en la posición 5. Con el valor 0 (cero) en el registro cabecera se indica que no hay espacio libre.
 - ii. Modificar los datos de una novela leyendo la información desde teclado. El código de novela no puede ser modificado.
 - iii. Eliminar una novela cuyo código es ingresado por teclado. Por ejemplo, si se da de baja un registro en la posición 8, en el campo código de novela del registro cabecera deberá figurar -8, y en el registro en la posición 8 debe copiarse el antiguo registro cabecera.
- c. Listar en un archivo de texto todas las novelas, incluyendo las borradas, que representan la lista de espacio libre. El archivo debe llamarse "novelas.txt".

NOTA: Tanto en la creación como en la apertura el nombre del archivo debe ser proporcionado por el usuario.

4. Dada la siguiente estructura:

```
type

    reg_flor = record

        nombre: String[45];
        codigo: integer;
    end;

    tArchFlores = file of reg_flor;
```

Las bajas se realizan apilando registros borrados y las altas reutilizando registros borrados. El registro 0 se usa como cabecera de la pila de registros borrados: el número 0 en el campo código implica que no hay registros borrados y -N indica que el próximo registro a reutilizar es el N, siendo éste un número relativo de registro válido.

- a. Implemente el siguiente módulo:

{Abre el archivo y agrega una flor, recibida como parámetro manteniendo la política descrita anteriormente}

```
procedure agregarFlor (var a: tArchFlores ; nombre: string;
codigo:integer);
```

- b. Liste el contenido del archivo omitiendo las flores eliminadas. Modifique lo que considere necesario para obtener el listado.

5. Dada la estructura planteada en el ejercicio anterior, implemente el siguiente módulo:

{Abre el archivo y elimina la flor recibida como parámetro manteniendo la política descrita anteriormente}

```
procedure eliminarFlor (var a: tArchFlores; flor:reg_flor);
```

6. Una cadena de tiendas de indumentaria posee un archivo maestro no ordenado con la información correspondiente a las prendas que se encuentran a la venta. De cada prenda se registra: cod_prenda, descripción, colores, tipo_prenda, stock y precio_unitario. Ante un eventual cambio de temporada, se deben actualizar las prendas a la venta. Para ello reciben un archivo conteniendo: cod_prenda de las

prendas que quedarán obsoletas. Deberá implementar un procedimiento que reciba ambos archivos y realice la baja lógica de las prendas, para ello deberá modificar el stock de la prenda correspondiente a valor negativo.

Adicionalmente, deberá implementar otro procedimiento que se encargue de efectivizar las bajas lógicas que se realizaron sobre el archivo maestro con la información de las prendas a la venta. Para ello se deberá utilizar una estructura auxiliar (esto es, un archivo nuevo), en el cual se copien únicamente aquellas prendas que no están marcadas como borradas. Al finalizar este proceso de compactación del archivo, se deberá renombrar el archivo nuevo con el nombre del archivo maestro original.

7. Se cuenta con un archivo que almacena información sobre especies de aves en vía de extinción, para ello se almacena: código, nombre de la especie, familia de ave, descripción y zona geográfica. El archivo no está ordenado por ningún criterio. Realice un programa que permita borrar especies de aves extintas. Este programa debe disponer de dos procedimientos:
 - a. Un procedimiento que dada una especie de ave (su código) marque la misma como borrada (en caso de querer borrar múltiples especies de aves, se podría invocar este procedimiento repetidamente).
 - b. Un procedimiento que compacte el archivo, quitando definitivamente las especies de aves marcadas como borradas. Para quitar los registros se deberá copiar el último registro del archivo en la posición del registro a borrar y luego eliminar del archivo el último registro de forma tal de evitar registros duplicados.
 - i. Implemente una variante de este procedimiento de compactación del archivo (baja física) donde el archivo se trunque una sola vez.
8. Se cuenta con un archivo con información de las diferentes distribuciones de linux existentes. De cada distribución se conoce: nombre, año de lanzamiento, número de

versión del kernel, cantidad de desarrolladores y descripción. El nombre de las distribuciones no puede repetirse. Este archivo debe ser mantenido realizando bajas lógicas y utilizando la técnica de reutilización de espacio libre llamada **lista invertida**. Escriba la definición de las estructuras de datos necesarias y los siguientes procedimientos:

- a. **BuscarDistribucion:** módulo que recibe por parámetro el archivo, un nombre de distribución y devuelve la posición dentro del archivo donde se encuentra el registro correspondiente a la distribución dada (si existe) o devuelve -1 en caso de que no exista..
- b. **AltaDistribucion:** módulo que recibe como parámetro el archivo y el registro que contiene los datos de una nueva distribución, y se encarga de agregar la distribución al archivo reutilizando espacio disponible en caso de que exista. (El control de unicidad lo debe realizar utilizando el módulo anterior). En caso de que la distribución que se quiere agregar ya exista se debe informar "ya existe la distribución".
- c. **BajaDistribucion:** módulo que recibe como parámetro el archivo y el nombre de una distribución, y se encarga de dar de baja lógicamente la distribución dada. Para marcar una distribución como borrada se debe utilizar el campo cantidad de desarrolladores para mantener actualizada la lista invertida. Para verificar que la distribución a borrar exista debe utilizar el módulo BuscarDistribucion. En caso de no existir se debe informar "Distribución no existente".

Parte 2: Actualización maestro/detalle, reportes y merge con archivos no ordenados

Para los ejercicios de esta parte de la práctica, teniendo en cuenta que los archivos no están ordenados por ningún criterio, puede resultar necesario recorrer los archivos más de una vez. La idea es resolver los ejercicios sin ordenar los archivos dados, y comparar la eficiencia (en cuanto al número de lecturas/escrituras) de la solución brindada en esta práctica respecto a la solución para el mismo problema considerando los archivos ordenados.

1. El encargado de ventas de un negocio de productos de limpieza desea administrar el stock de los productos que vende. Para ello, genera un archivo maestro donde figuran todos los productos que comercializa. De cada producto se maneja la siguiente información: código de producto, nombre comercial, precio de venta, stock actual y stock mínimo. Diariamente se genera un archivo detalle donde se registran todas las ventas de productos realizadas. De cada venta se registran: código de producto y cantidad de unidades vendidas. Resuelve los siguientes puntos:
 - a. Se pide realizar un procedimiento que actualice el archivo maestro con el archivo detalle, teniendo en cuenta que:
 - i. Los archivos no están ordenados por ningún criterio.
 - ii. Cada registro del maestro puede ser actualizado por 0, 1 ó más registros del archivo detalle.
 - b. ¿Qué cambios realizaría en el procedimiento del punto anterior si se sabe que cada registro del archivo maestro puede ser actualizado por 0 o 1 registro del archivo detalle?
2. Se necesita contabilizar los votos de las diferentes mesas electorales registradas por localidad en la provincia de Buenos Aires. Para ello, se posee un archivo con la siguiente información: código de localidad, número de mesa y cantidad de votos en dicha mesa. Presentar en pantalla un listado como se muestra a continuación:

Código de Localidad	Total de Votos
.....	
.....	
Total General de Votos:	

NOTAS:

- La información en el archivo no está ordenada por ningún criterio.
- Trate de resolver el problema sin modificar el contenido del archivo dado.
- Puede utilizar una estructura auxiliar, como por ejemplo otro archivo, para llevar el control de las localidades que han sido procesadas.

3. Suponga que trabaja en una oficina donde está montada una LAN (red local). La misma fue construida sobre una topología de red que conecta 5 máquinas entre sí y todas las máquinas se conectan con un servidor central. Semanalmente cada máquina genera un archivo de logs informando las sesiones abiertas por cada usuario en cada terminal y por cuánto tiempo estuvo abierta. Cada archivo detalle contiene los siguientes campos: `cod_usuario`, `fecha`, `tiempo_sesion`. Debe realizar un procedimiento que reciba los archivos detalle y genere un archivo maestro con los siguientes datos: `cod_usuario`, `fecha`, `tiempo_total_de_sesiones_abiertas`.

Notas:

- Los archivos detalle no están ordenados por ningún criterio.
- Un usuario puede iniciar más de una sesión el mismo día en la misma máquina, o inclusive, en diferentes máquinas.

Trabajo Práctico N° 3: Bajas en Archivos y Actualización Maestro/Detalle, Reportes y Merge con Archivos no Ordenados.

PARTE I: Bajas en Archivos.

Ejercicio 1.

Modificar el Ejercicio 4 de la Práctica 1 (programa de gestión de empleados), agregándole una opción para realizar bajas copiando el último registro del archivo en la posición del registro a borrar y, luego, truncando el archivo en la posición del último registro de forma tal de evitar duplicados.

```
program TP3_E1;
{$codepage UTF8}
uses crt;
const
  apellido_salida='fin';
  edad_corte=70;
  dni_corte=0;
  opcion_salida=0;
type
  t_string10=string[10];
  t_registro_empleado=record
    numero: int16;
    apellido: t_string10;
    nombre: t_string10;
    edad: int8;
    dni: int32;
  end;
  t_archivo_empleados=file of t_registro_empleado;
function random_string(length: int8): t_string10;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_empleado(var registro_empleado: t_registro_empleado);
var
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_empleado.apellido:=apellido_salida
  else
    registro_empleado.apellido:=random_string(5+random(5));
  if (registro_empleado.apellido<>apellido_salida) then
    begin
      registro_empleado.numero:=1+random(1000);
      registro_empleado.nombre:=random_string(5+random(5));
      registro_empleado.edad:=18+random(high(int8)-18);
      if (i<=10) then
        registro_empleado.dni:=0
      else
        registro_empleado.dni:=10000000+random(40000001);
```

```

    end;
end;
procedure cargar_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    rewrite(archivo_empleados);
    leer_empleado(registro_empleado);
    while (registro_empleado.apellido <> apellido_salida) do
    begin
        write(archivo_empleados, registro_empleado);
        leer_empleado(registro_empleado);
    end;
    close(archivo_empleados);
    textcolor(green); writeln('El archivo binario de empleados fue creado y cargado con éxito');
end;
procedure imprimir_registro_empleado(registro_empleado: t_registro_empleado);
begin
    textcolor(green); write('Número: '); textcolor(yellow); write(registro_empleado.numero);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_empleado.apellido);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_empleado.nombre);
    textcolor(green); write('; Edad: '); textcolor(yellow); write(registro_empleado.edad);
    textcolor(green); write('; DNI: '); textcolor(yellow); writeln(registro_empleado.dni);
end;
procedure imprimir1_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    texto: t_string10;
begin
    texto:=random_string(5+random(5));
    reset(archivo_empleados);
    textcolor(green); write('Los datos de los empleados con nombre o apellido ');
textcolor(yellow); write(texto); textcolor(green); writeln(' son: ');
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados, registro_empleado);
        if ((registro_empleado.nombre=texto) or (registro_empleado.apellido=texto)) then
            imprimir_registro_empleado(registro_empleado);
        end;
        textcolor(green); write('Tamaño del archivo empleados: '); textcolor(yellow);
writeln(filesize(archivo_empleados));
        close(archivo_empleados);
    end;
end;
procedure imprimir2_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_empleados);
    textcolor(green); writeln('Los empleados del archivo son: ');
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados, registro_empleado);
        imprimir_registro_empleado(registro_empleado);
    end;
    close(archivo_empleados);
end;
procedure imprimir3_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_empleados);
    textcolor(green); write('Los empleados mayores a '); textcolor(yellow); write(edad_corte);
textcolor(green); writeln(' años son: ');
    while (not eof(archivo_empleados)) do
    begin

```

```
    read(archivo_empleados,registro_empleado);
    if (registro_empleado.edad>edad_corte) then
        imprimir_registro_empleado(registro_empleado);
    end;
    close(archivo_empleados);
end;
function control_unicidad(var archivo_empleados: t_archivo_empleados; numero: int16): boolean;
var
    registro_empleado: t_registro_empleado;
    ok: boolean;
begin
    ok:=false;
    while ((not eof(archivo_empleados)) and (ok=false)) do
        begin
            read(archivo_empleados,registro_empleado);
            if (registro_empleado.numero=numero) then
                ok:=true;
            end;
        end;
        control_unicidad:=ok;
    end;
end;
procedure agregar_empleado(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    empleados: int16;
begin
    empleados:=0;
    reset(archivo_empleados);
    leer_empleado(registro_empleado);
    while (registro_empleado.apellido<>apellido_salida) do
        begin
            if (control_unicidad(archivo_empleados,registro_empleado.numero)=false) then
                begin
                    seek(archivo_empleados,filesize(archivo_empleados));
                    write(archivo_empleados,registro_empleado);
                    empleados:=empleados+1;
                end;
            leer_empleado(registro_empleado);
        end;
    end;
    close(archivo_empleados);
    textcolor(green); write('Se han agregado '); textcolor(yellow); write(empleados);
textcolor(green); writeln(' empleados al final del archivo');
end;
procedure modificar_edad_empleado(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    numero: int16;
    ok: boolean;
begin
    numero:=1+random(1000);
    ok:=false;
    reset(archivo_empleados);
    while ((not eof(archivo_empleados)) and (ok=false)) do
        begin
            read(archivo_empleados,registro_empleado);
            if (registro_empleado.numero=numero) then
                begin
                    registro_empleado.edad:=18+random(high(int8)-18);
                    seek(archivo_empleados,filepos(archivo_empleados)-1);
                    write(archivo_empleados,registro_empleado);
                    ok:=true;
                end;
            end;
        end;
    end;
    close(archivo_empleados);
    if (ok=true) then
        begin
```

```

        textcolor(green); write('Se ha modificado la edad del empleado con número ');
textcolor(yellow); write(numero); textcolor(green); writeln(' en el archivo');
    end
    else
    begin
        textcolor(green); write('No se ha encontrado el empleado con número '); textcolor(yellow);
write(numero); textcolor(green); writeln(' en el archivo');
    end;
end;
procedure exportar_archivo_txt1(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    archivo_txt: text;
begin
    reset(archivo_empleados);
    assign(archivo_txt,'E1_todos_empleados.txt'); rewrite(archivo_txt);
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados,registro_empleado);
        with registro_empleado do
            writeln(archivo_txt,'Número: ',numero,'; Apellido: ',apellido,'; Nombre: ',nombre,';
Edad: ',edad,'; DNI: ',dni);
        end;
        close(archivo_empleados);
        close(archivo_txt);
        textcolor(green); write('Se ha exportado el contenido del archivo al archivo de texto
llamado '); textcolor(yellow); writeln('"todos_empleados.txt"');
    end;
end;
procedure exportar_archivo_txt2(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    archivo_txt: text;
begin
    reset(archivo_empleados);
    assign(archivo_txt,'E1_faltaDNIEmpleado.txt'); rewrite(archivo_txt);
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados,registro_empleado);
        if (registro_empleado.dni=dni_corte) then
            with registro_empleado do
                writeln(archivo_txt,'Número: ',numero,'; Apellido: ',apellido,'; Nombre: ',nombre,';
Edad: ',edad,'; DNI: ',dni);
            end;
        close(archivo_empleados);
        close(archivo_txt);
        textcolor(green); write('Se ha exportado a un archivo de texto llamado ');
textcolor(yellow); write('"faltaDNIEmpleado.txt"'); textcolor(green); writeln(' los empleados
que no tienen cargado el DNI (DNI en 00)');
    end;
end;
procedure eliminar_empleado(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado, registro_ultimo_empleado: t_registro_empleado;
    numero: int16;
begin
    numero:=1+random(1000);
    reset(archivo_empleados);
    seek(archivo_empleados,filesize(archivo_empleados)-1);
    read(archivo_empleados,registro_ultimo_empleado);
    seek(archivo_empleados,0);
    read(archivo_empleados,registro_empleado);
    while (not eof(archivo_empleados)) and (registro_empleado.numero<>numero) do
        read(archivo_empleados,registro_empleado);
    if (registro_empleado.numero=numero) then
    begin
        seek(archivo_empleados,filepos(archivo_empleados)-1);
        write(archivo_empleados,registro_ultimo_empleado);
    end;
end;

```



```

    seek(archivo_empleados, filesize(archivo_empleados)-1);
    truncate(archivo_empleados);
    textcolor(green); write('Se ha eliminado el empleado con número '); textcolor(yellow);
write(numero); textcolor(green); writeln(' en el archivo');
end
else
begin
    textcolor(green); write('No se ha encontrado el empleado con número '); textcolor(yellow);
write(numero); textcolor(green); writeln(' en el archivo');
end;
close(archivo_empleados);
end;
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear un archivo de
registros no ordenados de empleados y completarlo con datos ingresados desde teclado');
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de empleados que tengan un nombre o apellido determinado');
    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Listar en pantalla los
empleados de a uno por línea');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Listar en pantalla los
empleados mayores a 70 años, próximos a jubilarse');
    textcolor(yellow); write('OPCIÓN 5: '); textcolor(green); writeln('Añadir uno o más
empleados al final del archivo con sus datos ingresados por teclado');
    textcolor(yellow); write('OPCIÓN 6: '); textcolor(green); writeln('Modificar la edad de un
empleado dado');
    textcolor(yellow); write('OPCIÓN 7: '); textcolor(green); writeln('Exportar el contenido del
archivo a un archivo de texto llamado "todos_empleados.txt"');
    textcolor(yellow); write('OPCIÓN 8: '); textcolor(green); writeln('Exportar a un archivo de
texto llamado "faltaDNIEmpleado.txt" los empleados que no tengan cargado el DNI (DNI en 00)');
    textcolor(yellow); write('OPCIÓN 9: '); textcolor(green); writeln('Eliminar un empleado del
archivo, copiando el último registro en la posición del registro a borrar y truncando el
archivo');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_empleados: t_archivo_empleados);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
    begin
        case opcion of
            1: cargar_archivo_empleados(archivo_empleados);
            2: imprimir1_archivo_empleados(archivo_empleados);
            3: imprimir2_archivo_empleados(archivo_empleados);
            4: imprimir3_archivo_empleados(archivo_empleados);
            5: agregar_empleado(archivo_empleados);
            6: modificar_edad_empleado(archivo_empleados);
            7: exportar_archivo_txt1(archivo_empleados);
            8: exportar_archivo_txt2(archivo_empleados);
            9: eliminar_empleado(archivo_empleados);
        else
            textcolor(green); writeln('La opción ingresada no corresponde a ninguna de las
mostradas en el menú de opciones');
        end;
        writeln();
        leer_opcion(opcion);
    end;
end;
var
    archivo_empleados: t_archivo_empleados;

```

```
begin
  randomize;
  assign(archivo_empleados, 'E1_empleados');
  menu_opciones(archivo_empleados);
end.
```

Ejercicio 2.

Definir un programa que genere un archivo con registros de longitud fija conteniendo información de asistentes a un congreso a partir de la información obtenida por teclado. Se deberá almacenar la siguiente información: nro. de asistente, apellido y nombre, email, teléfono y DNI. Implementar un procedimiento que, a partir del archivo de datos generado, elimine, de forma lógica, todos los asistentes con nro. de asistente inferior a 1.000. Para ello, se podrá utilizar algún carácter especial situándolo delante de algún campo String a elección. Ejemplo: “@Saldaño”.

```

program TP3_E2;
{$codepage UTF8}
uses crt;
const
  numero_salida=0;
  numero_corte=1000;
type
  t_string20=string[20];
  t_registro_asistente=record
    numero: int16;
    apellido: t_string20;
    nombre: t_string20;
    email: t_string20;
    telefono: int32;
    dni: int32;
  end;
  t_archivo_asistentes=file of t_registro_asistente;
function random_string(length: int8): t_string20;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
    random_string:=string_aux;
  end;
procedure leer_asistente(var registro_asistente: t_registro_asistente);
var
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_asistente.numero:=numero_salida
  else
    registro_asistente.numero:=1+random(2000);
    if (registro_asistente.numero<>numero_salida) then
      begin
        registro_asistente.apellido:=random_string(5+random(5));
        registro_asistente.nombre:=random_string(5+random(5));
        registro_asistente.email:=random_string(5+random(6))+ '@gmail.com';
        registro_asistente.telefono:=1000000000+random(1000000001);
        registro_asistente.dni:=10000000+random(40000001);
      end;
  end;
procedure cargar_archivo_asistentes(var archivo_asistentes: t_archivo_asistentes);
var
  registro_asistente: t_registro_asistente;
begin
  rewrite(archivo_asistentes);
  leer_asistente(registro_asistente);
  while (registro_asistente.numero<>numero_salida) do

```

```
begin
    write(archivo_asistentes,registro_asistente);
    leer_asistente(registro_asistente);
end;
close(archivo_asistentes);
end;
procedure imprimir_registro_asistente(registro_asistente: t_registro_asistente);
begin
    textcolor(green); write('Número: '); textcolor(yellow); write(registro_asistente.numero);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_asistente.apellido);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_asistente.nombre);
    textcolor(green); write('; Email: '); textcolor(yellow); write(registro_asistente.email);
    textcolor(green); write('; Teléfono: '); textcolor(yellow);
write(registro_asistente.telefono);
    textcolor(green); write('; DNI: '); textcolor(yellow); writeln(registro_asistente.dni);
end;
procedure imprimir_archivo_asistentes(var archivo_asistentes: t_archivo_asistentes);
var
    registro_asistente: t_registro_asistente;
begin
    reset(archivo_asistentes);
    while (not eof(archivo_asistentes)) do
        begin
            read(archivo_asistentes,registro_asistente);
            imprimir_registro_asistente(registro_asistente);
        end;
        textcolor(green); write('Tamaño del archivo asistentes: '); textcolor(yellow);
writeln(filesize(archivo_asistentes));
        close(archivo_asistentes);
    end;
procedure eliminar_archivo_asistentes(var archivo_asistentes: t_archivo_asistentes);
var
    registro_asistente: t_registro_asistente;
begin
    reset(archivo_asistentes);
    while (not eof(archivo_asistentes)) do
        begin
            read(archivo_asistentes,registro_asistente);
            if (registro_asistente.numero<numero_corte) then
                begin
                    registro_asistente.apellido:='@'+registro_asistente.apellido;
                    seek(archivo_asistentes,filepos(archivo_asistentes)-1);
                    write(archivo_asistentes,registro_asistente);
                end;
            end;
        close(archivo_asistentes);
    end;
var
    archivo_asistentes: t_archivo_asistentes;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO ASISTENTES:'); writeln();
    assign(archivo_asistentes,'E2_asistentes');
    cargar_archivo_asistentes(archivo_asistentes);
    imprimir_archivo_asistentes(archivo_asistentes);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO ASISTENTES ACTUALIZADO:'); writeln();
    eliminar_archivo_asistentes(archivo_asistentes);
    imprimir_archivo_asistentes(archivo_asistentes);
end.
```

Ejercicio 3.

Realizar un programa que genere un archivo de novelas filmadas durante el presente año. De cada novela, se registra: código, género, nombre, duración, director y precio. El programa debe presentar un menú con las siguientes opciones:

(a) *Crear el archivo y cargarlo a partir de datos ingresados por teclado. Se utiliza la técnica de lista invertida para recuperar espacio libre en el archivo. Para ello, durante la creación del archivo, en el primer registro del mismo, se debe almacenar la cabecera de la lista. Es decir, un registro ficticio, inicializando con el valor cero (0) el campo correspondiente al código de novela, el cual indica que no hay espacio libre dentro del archivo.*

(b) *Abrir el archivo existente y permitir su mantenimiento teniendo en cuenta el inciso (a), se utiliza lista invertida para recuperación de espacio. En particular, para el campo de “enlace” de la lista (utilizar el código de novela como enlace), se debe especificar los números de registro referenciados con signo negativo. Una vez abierto el archivo, brindar operaciones para:*

(i) *Dar de alta una novela leyendo la información desde teclado. Para esta operación, en caso de ser posible, deberá recuperarse el espacio libre. Es decir, si, en el campo correspondiente al código de novela del registro cabecera, hay un valor negativo, por ejemplo -5, se debe leer el registro en la posición 5, copiarlo en la posición 0 (actualizar la lista de espacio libre) y grabar el nuevo registro en la posición 5. Con el valor 0 (cero) en el registro cabecera se indica que no hay espacio libre.*

(ii) *Modificar los datos de una novela leyendo la información desde teclado. El código de novela no puede ser modificado.*

(iii) *Eliminar una novela cuyo código es ingresado por teclado. Por ejemplo, si se da de baja un registro en la posición 8, en el campo código de novela del registro cabecera, deberá figurar -8 y, en el registro en la posición 8, debe copiarse el antiguo registro cabecera.*

(c) *Listar, en un archivo de texto, todas las novelas, incluyendo las borradas, que representan la lista de espacio libre. El archivo debe llamarse “novelas.txt”.*

Nota: Tanto en la creación como en la apertura, el nombre del archivo debe ser proporcionado por el usuario.

```
program TP3_E3;
{$codepage UTF8}
uses crt;
const
    codigo_salida=-1;
    codigo_cabecera=0;
    opcion_salida=0;
type
    t_string10=string[10];
    t_registro_novela=record
        codigo: int16;
        genero: t_string10;
        nombre: t_string10;
```

```

    duracion: int16;
    director: t_string10;
    precio: real;
end;
t_archivo_novelas=file of t_registro_novela;
function random_string(length: int8): t_string10;
var
    i: int8;
    string_aux: string;
begin
    string_aux:='';
    for i:= 1 to length do
        string_aux:=string_aux+chr(ord('A')+random(26));
    random_string:=string_aux;
end;
procedure leer_cabecera(var registro_novela: t_registro_novela);
begin
    registro_novela.codigo:=codigo_cabecera;
    registro_novela.genero:='Cabecera';
    registro_novela.nombre:='Cabecera';
    registro_novela.duracion:=codigo_cabecera;
    registro_novela.director:='Cabecera';
    registro_novela.precio:=codigo_cabecera;
end;
procedure leer_novela(var registro_novela: t_registro_novela);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        registro_novela.codigo:=codigo_salida
    else
        registro_novela.codigo:=1+random(1000);
        if (registro_novela.codigo<>codigo_salida) then
            begin
                registro_novela.genero:=random_string(5+random(5));
                registro_novela.nombre:=random_string(5+random(5));
                registro_novela.duracion:=30+random(121);
                registro_novela.director:=random_string(5+random(5));
                registro_novela.precio:=1+random(100);
            end;
        end;
end;
procedure cargar_archivo_novelas(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
begin
    rewrite(archivo_novelas);
    leer_cabecera(registro_novela);
    while (registro_novela.codigo<>codigo_salida) do
        begin
            write(archivo_novelas,registro_novela);
            leer_novela(registro_novela);
        end;
    close(archivo_novelas);
    textcolor(green); writeln('El archivo binario de novelas fue creado y cargado con éxito');
end;
procedure imprimir_registro_novela(registro_novela: t_registro_novela);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_novela.codigo);
    textcolor(green); write('; Género: '); textcolor(yellow); write(registro_novela.genero);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_novela.nombre);
    textcolor(green); write('; Duración: '); textcolor(yellow); write(registro_novela.duracion);
    textcolor(green); write('; Director: '); textcolor(yellow); write(registro_novela.director);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
    writeln(registro_novela.precio:0:2);
end;

```

```

procedure imprimir_archivo_novelas(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
begin
    reset(archivo_novelas);
    textcolor(green); writeln('Las novelas del archivo son: ');
    while (not eof(archivo_novelas)) do
    begin
        read(archivo_novelas, registro_novela);
        if (registro_novela.codigo > codigo_cabecera) then
            imprimir_registro_novela(registro_novela);
        end;
        textcolor(green); write('Tamaño del archivo novelas: '); textcolor(yellow);
        writeln(filesize(archivo_novelas));
        close(archivo_novelas);
    end;
procedure agregar_novela(var archivo_novelas: t_archivo_novelas);
var
    registro_novela, cabecera: t_registro_novela;
begin
    reset(archivo_novelas);
    read(archivo_novelas, cabecera);
    leer_novela(registro_novela);
    while (registro_novela.codigo = codigo_salida) do
        leer_novela(registro_novela);
    if (cabecera.codigo = codigo_cabecera) then
    begin
        seek(archivo_novelas, filesize(archivo_novelas));
        write(archivo_novelas, registro_novela);
    end
    else
    begin
        seek(archivo_novelas, cabecera.codigo * (-1));
        read(archivo_novelas, cabecera);
        seek(archivo_novelas, filepos(archivo_novelas) - 1);
        write(archivo_novelas, registro_novela);
        seek(archivo_novelas, 0);
        write(archivo_novelas, cabecera);
    end;
    close(archivo_novelas);
    textcolor(green); write('Se ha realizado el alta de la novela con código ');
    textcolor(yellow); write(registro_novela.codigo); textcolor(green); writeln(' en el archivo');
end;
procedure modificar_novela(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
    codigo: int16;
    ok: boolean;
begin
    ok := false;
    codigo := 1 + random(1000);
    reset(archivo_novelas);
    while ((not eof(archivo_novelas)) and (ok = false)) do
    begin
        read(archivo_novelas, registro_novela);
        if (registro_novela.codigo = codigo) then
        begin
            ok := true;
            leer_novela(registro_novela);
            while (registro_novela.codigo = codigo_salida) do
                leer_novela(registro_novela);
            registro_novela.codigo := codigo;
            seek(archivo_novelas, filepos(archivo_novelas) - 1);
            write(archivo_novelas, registro_novela);
        end;
    end;
end;

```

```

close(archivo_novelas);
if (ok=true) then
begin
    textcolor(green); write('Se ha realizado la modificación de la novela con código ');
textcolor(yellow); write(codigo); textcolor(green); writeln(' en el archivo');
end
else
begin
    textcolor(green); write('No se ha encontrado la novela con código '); textcolor(yellow);
write(codigo); textcolor(green); writeln(' en el archivo');
end;
end;
procedure eliminar_novela(var archivo_novelas: t_archivo_novelas);
var
    registro_novela, cabecera: t_registro_novela;
    codigo: int16;
    ok: boolean;
begin
    ok:=false;
    codigo:=1+random(1000);
    reset(archivo_novelas);
    read(archivo_novelas,cabecera);
    while ((not eof(archivo_novelas)) and (ok=false)) do
    begin
        read(archivo_novelas,registro_novela);
        if (registro_novela.codigo=codigo) then
        begin
            ok:=true;
            seek(archivo_novelas,filepos(archivo_novelas)-1);
            write(archivo_novelas,cabecera);
            cabecera.codigo:=(filepos(archivo_novelas)-1)*(-1);
            seek(archivo_novelas,0);
            write(archivo_novelas,cabecera);
        end;
    end;
    close(archivo_novelas);
    if (ok=true) then
    begin
        textcolor(green); write('Se ha realizado la baja de la novela con código ');
textcolor(yellow); write(codigo); textcolor(green); writeln(' en el archivo');
    end
    else
    begin
        textcolor(green); write('No se ha encontrado la novela con código '); textcolor(yellow);
write(codigo); textcolor(green); writeln(' en el archivo');
    end;
end;
procedure exportar_archivo_txt(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
    archivo_txt: text;
begin
    reset(archivo_novelas);
    assign(archivo_txt,'E3_novelas.txt'); rewrite(archivo_txt);
    while (not eof(archivo_novelas)) do
    begin
        read(archivo_novelas,registro_novela);
        with registro_novela do
            writeln(archivo_txt,'Código: ',codigo,'; Género: ',genero,'; Nombre: ',nombre,';
Duración: ',duracion,'; Director: ',director,'; Precio: $',precio:0:2);
        end;
    end;
    close(archivo_novelas);
    close(archivo_txt);
    textcolor(green); write('Se han exportado todas las novelas, incluyendo las borradas, a un
archivo de texto llamado '); textcolor(yellow); writeln('"E3_novelas.txt"');
end;

```



```
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear el archivo y
cargarlo a partir de datos ingresados por teclado');
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de las novelas del archivo, sin incluir las borradas');
    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Dar de alta una novela
leyendo la información desde teclado');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Modificar los datos de
una novela leyendo la información desde teclado');
    textcolor(yellow); write('OPCIÓN 5: '); textcolor(green); writeln('Eliminar una novela cuyo
código es ingresado por teclado');
    textcolor(yellow); write('OPCIÓN 6: '); textcolor(green); writeln('Exportar a un archivo de
texto llamado "novelas.txt" todas las novelas, incluyendo las borradas');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_novelas: t_archivo_novelas);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
    begin
        case opcion of
            1: cargar_archivo_novelas(archivo_novelas);
            2: imprimir_archivo_novelas(archivo_novelas);
            3: agregar_novela(archivo_novelas);
            4: modificar_novela(archivo_novelas);
            5: eliminar_novela(archivo_novelas);
            6: exportar_archivo_txt(archivo_novelas);
        else
            writeln('La opción ingresada no corresponde a ninguna del menú de opciones');
        end;
        writeln();
        leer_opcion(opcion);
    end;
end;
var
    archivo_novelas: t_archivo_novelas;
begin
    randomize;
    assign(archivo_novelas, 'E3_novelas');
    menu_opciones(archivo_novelas);
end.
```

Ejercicio 4.

Dada la siguiente estructura:

```
type
  reg_flor=record
    nombre: String[45];
    codigo: integer;
  end;
  tArchFlores=file of reg_flor;
```

Las bajas se realizan apilando registros borrados y las altas reutilizando registros borrados. El registro 0 se usa como cabecera de la pila de registros borrados: el número 0 en el campo código implica que no hay registros borrados y -N indica que el próximo registro a reutilizar es el N, siendo éste un número relativo de registro válido.

(a) Implementar el siguiente módulo:

{Abre el archivo y agrega una flor recibida como parámetro, manteniendo la política descrita anteriormente}

procedure agregarFlor(var a: tArchFlores; nombre: string; codigo: integer);

(b) Listar el contenido del archivo omitiendo las flores eliminadas. Modificar lo que se considere necesario para obtener el listado.

```
program TP3_E4;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
  codigo_cabecera=0;
type
  t_string45=string[45];
  t_registro_flor=record
    nombre: t_string45;
    codigo: int16;
  end;
  t_archivo_flores=file of t_registro_flor;
function random_string(length: int8): t_string45;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_flor(var registro_flor: t_registro_flor);
var
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_flor.codigo:=codigo_salida
  else
    registro_flor.codigo:=1+random(1000);
```

```
if (registro_flor.codigo<>codigo_salida) then
    registro_flor.nombre:=random_string(5+random(6));
end;
procedure cargar_archivo_flores(var archivo_flores: t_archivo_flores);
var
    registro_flor: t_registro_flor;
begin
    rewrite(archivo_flores);
    registro_flor.codigo:=codigo_cabecera;
    registro_flor.nombre:='Cabecera';
    while (registro_flor.codigo<>codigo_salida) do
        begin
            write(archivo_flores,registro_flor);
            leer_flor(registro_flor);
        end;
    close(archivo_flores);
end;
procedure imprimir_registro_flor(registro_flor: t_registro_flor);
begin
    textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_flor.nombre);
    textcolor(green); write('; Código: '); textcolor(yellow); writeln(registro_flor.codigo);
end;
procedure imprimir_archivo_flores(var archivo_flores: t_archivo_flores);
var
    registro_flor: t_registro_flor;
begin
    reset(archivo_flores);
    while (not eof(archivo_flores)) do
        begin
            read(archivo_flores,registro_flor);
            if (registro_flor.codigo>codigo_cabecera) then
                imprimir_registro_flor(registro_flor);
            end;
            textcolor(green); write('Tamaño del archivo flores: '); textcolor(yellow);
            writeln(filesize(archivo_flores));
            close(archivo_flores);
        end;
end;
procedure agregar_flor(var archivo_flores: t_archivo_flores; nombre: t_string45; codigo:
int16);
var
    registro_flor, cabecera: t_registro_flor;
begin
    reset(archivo_flores);
    read(archivo_flores,cabecera);
    registro_flor.nombre:=nombre;
    registro_flor.codigo:=codigo;
    if (cabecera.codigo=codigo_cabecera) then
        begin
            seek(archivo_flores,filesize(archivo_flores));
            write(archivo_flores,registro_flor);
        end
    else
        begin
            seek(archivo_flores,cabecera.codigo*(-1));
            read(archivo_flores,cabecera);
            seek(archivo_flores,filepos(archivo_flores)-1);
            write(archivo_flores,registro_flor);
            seek(archivo_flores,0);
            write(archivo_flores,cabecera);
        end;
    close(archivo_flores);
    textcolor(green); write('Se ha realizado el alta de la flor '); textcolor(yellow);
    write(nombre); textcolor(green); write(' con código '); textcolor(yellow); writeln(codigo);
    writeln();
end;
var
```

```
archivo_flores: t_archivo_flores;
codigo: int16;
nombre: t_string45;
begin
  randomize;
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES:'); writeln();
  assign(archivo_flores, 'E4_flores');
  cargar_archivo_flores(archivo_flores);
  imprimir_archivo_flores(archivo_flores);
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES AL AGREGAR FLOR:'); writeln();
  codigo:=1+random(1000);
  nombre:=random_string(5+random(6));
  agregar_flor(archivo_flores,nombre,codigo);
  imprimir_archivo_flores(archivo_flores);
end.
```

Ejercicio 5.

Dada la estructura planteada en el ejercicio anterior, implementar el siguiente módulo:

{Abre el archivo y elimina la flor recibida como parámetro, manteniendo la política descrita anteriormente}

procedure eliminarFlor(var a: tArchFlores; flor: reg_flor);

```

program TP3_E5;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
  codigo_cabecera=0;
type
  t_string45=string[45];
  t_registro_flor=record
    nombre: t_string45;
    codigo: int16;
  end;
  t_archivo_flores=file of t_registro_flor;
function random_string(length: int8): t_string45;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_flor(var registro_flor: t_registro_flor);
var
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_flor.codigo:=codigo_salida
  else
    registro_flor.codigo:=1+random(100);
  if (registro_flor.codigo<>codigo_salida) then
    registro_flor.nombre:=random_string(5+random(6));
end;
procedure cargar_archivo_flores(var archivo_flores: t_archivo_flores);
var
  registro_flor: t_registro_flor;
begin
  rewrite(archivo_flores);
  registro_flor.codigo:=codigo_cabecera;
  registro_flor.nombre:='Cabecera';
  while (registro_flor.codigo<>codigo_salida) do
  begin
    write(archivo_flores,registro_flor);
    leer_flor(registro_flor);
  end;
  close(archivo_flores);
end;
procedure imprimir_registro_flor(registro_flor: t_registro_flor);
begin
  textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_flor.nombre);
  textcolor(green); write('; Código: '); textcolor(yellow); writeln(registro_flor.codigo);
end;

```

```
procedure imprimir_archivo_flores(var archivo_flores: t_archivo_flores);
var
    registro_flor: t_registro_flor;
begin
    reset(archivo_flores);
    while (not eof(archivo_flores)) do
        begin
            read(archivo_flores,registro_flor);
            if (registro_flor.codigo>codigo_cabecera) then
                imprimir_registro_flor(registro_flor);
            end;
            textcolor(green); write('Tamaño del archivo flores: '); textcolor(yellow);
            writeln(filesize(archivo_flores));
            close(archivo_flores);
        end;
    procedure agregar_flor(var archivo_flores: t_archivo_flores; nombre: t_string45; codigo:
    int16);
    var
        registro_flor, cabecera: t_registro_flor;
    begin
        reset(archivo_flores);
        read(archivo_flores,cabecera);
        registro_flor.nombre:=nombre;
        registro_flor.codigo:=codigo;
        if (cabecera.codigo=codigo_cabecera) then
            begin
                seek(archivo_flores, filesize(archivo_flores));
                write(archivo_flores,registro_flor);
            end
        else
            begin
                seek(archivo_flores,cabecera.codigo*(-1));
                read(archivo_flores,cabecera);
                seek(archivo_flores,filepos(archivo_flores)-1);
                write(archivo_flores,registro_flor);
                seek(archivo_flores,0);
                write(archivo_flores,cabecera);
            end;
        close(archivo_flores);
        textcolor(green); write('Se ha realizado el alta de la flor '); textcolor(yellow);
        write(nombre); textcolor(green); write(' con código '); textcolor(yellow); writeln(codigo);
        writeln();
    end;
    procedure eliminar_flor(var archivo_flores: t_archivo_flores; flor: t_registro_flor);
    var
        registro_flor, cabecera: t_registro_flor;
        ok: boolean;
    begin
        ok:=false;
        reset(archivo_flores);
        read(archivo_flores,cabecera);
        while ((not eof(archivo_flores)) and (ok=false)) do
            begin
                read(archivo_flores,registro_flor);
                if (registro_flor.codigo=flor.codigo) then
                    begin
                        ok:=true;
                        seek(archivo_flores, filepos(archivo_flores)-1);
                        write(archivo_flores,cabecera);
                        cabecera.codigo:=(filepos(archivo_flores)-1)*(-1);
                        seek(archivo_flores,0);
                        write(archivo_flores,cabecera);
                    end;
                end;
            end;
        close(archivo_flores);
        if (ok=true) then
```

```
begin
    textcolor(green); write('Se ha realizado la baja de la flor '); textcolor(yellow);
write(flor.nombre); textcolor(green); write(' con código '); textcolor(yellow);
writeln(flor.codigo);
end
else
begin
    textcolor(green); write('No se ha encontrado la flor '); textcolor(yellow);
write(flor.nombre); textcolor(green); write(' con código '); textcolor(yellow);
writeln(flor.codigo);
end;
writeln();
end;
var
    archivo_flores: t_archivo_flores;
    registro_flor: t_registro_flor;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES:'); writeln();
    assign(archivo_flores,'E5_flores');
    cargar_archivo_flores(archivo_flores);
    imprimir_archivo_flores(archivo_flores);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES AL AGREGAR FLOR ANTES DE LA
BAJA:'); writeln();
    registro_flor.codigo:=1+random(100);
    registro_flor.nombre:=random_string(5+random(6));
    agregar_flor(archivo_flores,registro_flor.nombre,registro_flor.codigo);
    imprimir_archivo_flores(archivo_flores);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES AL ELIMINAR FLOR:'); writeln();
    registro_flor.codigo:=1+random(100);
    eliminar_flor(archivo_flores,registro_flor);
    imprimir_archivo_flores(archivo_flores);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES AL AGREGAR FLOR DESPUÉS DE LA
BAJA:'); writeln();
    registro_flor.codigo:=1+random(100);
    registro_flor.nombre:=random_string(5+random(6));
    agregar_flor(archivo_flores,registro_flor.nombre,registro_flor.codigo);
    imprimir_archivo_flores(archivo_flores);
end.
```

Ejercicio 6.

Una cadena de tiendas de indumentaria posee un archivo maestro no ordenado con la información correspondiente a las prendas que se encuentran a la venta. De cada prenda, se registra: `cod_prenda`, `descripción`, `colores`, `tipo_prenda`, `stock` y `precio_unitario`. Ante un eventual cambio de temporada, se deben actualizar las prendas a la venta. Para ello, reciben un archivo conteniendo: `cod_prenda` de las prendas que quedarán obsoletas. Se deberá implementar un procedimiento que reciba ambos archivos y realice la baja lógica de las prendas. Para ello, se deberá modificar el `stock` de la prenda correspondiente a valor negativo.

Adicionalmente, se deberá implementar otro procedimiento que se encargue de efectivizar las bajas lógicas que se realizaron sobre el archivo maestro con la información de las prendas a la venta. Para ello, se deberá utilizar una estructura auxiliar (esto es, un archivo nuevo), en el cual se copien, únicamente, aquellas prendas que no están marcadas como borradas. Al finalizar este proceso de compactación del archivo, se deberá renombrar el archivo nuevo con el nombre del archivo maestro original.

```

program TP3_E6;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
  stock_baja=-1;
type
  t_string20=string[20];
  t_registro_prenda=record
    codigo: int16;
    descripcion: t_string20;
    colores: t_string20;
    tipo: t_string20;
    stock: int16;
    precio: real;
  end;
  t_archivo_maestro=file of t_registro_prenda;
  t_archivo_detalle=file of int16;
function random_string(length: int8): t_string20;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_prenda(var registro_prenda: t_registro_prenda);
var
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_prenda.codigo:=codigo_salida
  else
    registro_prenda.codigo:=1+random(1000);
  if (registro_prenda.codigo<>codigo_salida) then
    begin
      registro_prenda.descripcion:=random_string(10+random(10));
    end;
  end;
end;

```



```

registro_prenda.colores:=random_string(5+random(6));
registro_prenda.tipo:=random_string(5+random(6));
registro_prenda.stock:=1+random(100);
registro_prenda.precio:=1+random(100);
end;
end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_prenda: t_registro_prenda;
begin
    rewrite(archivo_maestro);
    leer_prenda(registro_prenda);
    while (registro_prenda.codigo<>codigo_salida) do
    begin
        write(archivo_maestro,registro_prenda);
        leer_prenda(registro_prenda);
    end;
    close(archivo_maestro);
end;
procedure leer_codigo(var codigo: int16);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        codigo:=codigo_salida
    else
        codigo:=1+random(1000);
    end;
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    codigo: int16;
begin
    rewrite(archivo_detalle);
    leer_codigo(codigo);
    while (codigo<>codigo_salida) do
    begin
        write(archivo_detalle,codigo);
        leer_codigo(codigo);
    end;
    close(archivo_detalle);
end;
procedure imprimir_registro_prenda(registro_prenda: t_registro_prenda);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_prenda.codigo);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_prenda.descripcion);
    textcolor(green); write('; Colores: '); textcolor(yellow); write(registro_prenda.colores);
    textcolor(green); write('; Tipo: '); textcolor(yellow); write(registro_prenda.tipo);
    textcolor(green); write('; Stock: '); textcolor(yellow); write(registro_prenda.stock);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
writeln(registro_prenda.precio:0:2);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_prenda: t_registro_prenda;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
    begin
        read(archivo_maestro,registro_prenda);
        imprimir_registro_prenda(registro_prenda);
    end;
    textcolor(green); write('Tamaño del archivo maestro: '); textcolor(yellow);
writeln(filesize(archivo_maestro));
close(archivo_maestro);

```

```

end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
  codigo: int16;
begin
  reset(archivo_detalle);
  textcolor(green); write('Códigos: ');
  while (not eof(archivo_detalle)) do
    begin
      read(archivo_detalle,codigo);
      textcolor(yellow); write(codigo,' ');
    end;
  close(archivo_detalle);
  writeln();
end;
procedure bajas_logicas(var archivo_maestro: t_archivo_maestro; var archivo_detalle:
t_archivo_detalle);
var
  registro_prenda: t_registro_prenda;
  codigo: int16;
begin
  reset(archivo_maestro);
  reset(archivo_detalle);
  while (not eof(archivo_detalle)) do
    begin
      read(archivo_detalle,codigo);
      seek(archivo_maestro,0);
      read(archivo_maestro,registro_prenda);
      while ((not eof(archivo_maestro)) and (registro_prenda.codigo<>codigo)) do
        read(archivo_maestro,registro_prenda);
      if (registro_prenda.codigo=codigo) then
        begin
          seek(archivo_maestro,filepos(archivo_maestro)-1);
          registro_prenda.stock:=stock_baja;
          write(archivo_maestro,registro_prenda);
        end;
      end;
    close(archivo_maestro);
    close(archivo_detalle);
  end;
procedure bajas_fisicas(var archivo_maestro: t_archivo_maestro);
var
  registro_prenda: t_registro_prenda;
  archivo_maestro_aux: t_archivo_maestro;
begin
  reset(archivo_maestro);
  assign(archivo_maestro_aux,'E6_prendasMaestroAux'); rewrite(archivo_maestro_aux);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro,registro_prenda);
      if (registro_prenda.stock>0) then
        write(archivo_maestro_aux,registro_prenda);
      end;
    close(archivo_maestro);
    close(archivo_maestro_aux);
    erase(archivo_maestro);
    rename(archivo_maestro_aux,'E6_prendasMaestro');
  end;
var
  archivo_maestro: t_archivo_maestro;
  archivo_detalle: t_archivo_detalle;
begin
  randomize;
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
  assign(archivo_maestro,'E6_prendasMaestro');
  cargar_archivo_maestro(archivo_maestro);

```

```
imprimir_archivo_maestro(archivo_maestro);  
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE:'); writeln();  
assign(archivo_detalle,'E6_bajasDetalle');  
cargar_archivo_detalle(archivo_detalle);  
imprimir_archivo_detalle(archivo_detalle);  
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO CON BAJAS LÓGICAS:');  
writeln();  
bajas_logicas(archivo_maestro,archivo_detalle);  
imprimir_archivo_maestro(archivo_maestro);  
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO CON BAJAS FÍSICAS:');  
writeln();  
bajas_fisicas(archivo_maestro);  
imprimir_archivo_maestro(archivo_maestro);  
end.
```

Ejercicio 7.

Se cuenta con un archivo que almacena información sobre especies de aves en vía de extinción. Para ello, se almacena: código, nombre de la especie, familia de ave, descripción y zona geográfica. El archivo no está ordenado por ningún criterio. Realizar un programa que permita borrar especies de aves extintas. Este programa debe disponer de dos procedimientos:

(a) Un procedimiento que, dada una especie de ave (su código), marque la misma como borrada (en caso de querer borrar múltiples especies de aves, se podría invocar este procedimiento repetidamente).

(b) Un procedimiento que compacte el archivo, quitando, definitivamente, las especies de aves marcadas como borradas. Para quitar los registros, se deberá copiar el último registro del archivo en la posición del registro a borrar y, luego, eliminar del archivo el último registro de forma tal de evitar registros duplicados.

(i) Implementar una variante de este procedimiento de compactación del archivo (baja física) donde el archivo se trunque una sola vez.

```
program TP3_E7;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
  codigo_baja=-1;
type
  t_string20=string[20];
  t_registro_ave=record
    codigo: int16;
    nombre: t_string20;
    familia: t_string20;
    descripcion: t_string20;
    zona: t_string20;
  end;
  t_archivo_aves=file of t_registro_ave;
function random_string(length: int8): t_string20;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_ave(var registro_ave: t_registro_ave);
var
  i: int8;
begin
  i:=random(10);
  if (i=0) then
    registro_ave.codigo:=codigo_salida
  else
    registro_ave.codigo:=1+random(1000);
  if (registro_ave.codigo<>codigo_salida) then
    begin
      registro_ave.nombre:=random_string(5+random(6));
```

```
registro_ave.familia:=random_string(5+random(6));
registro_ave.descripcion:=random_string(10+random(10));
registro_ave.zona:=random_string(5+random(6));
end;
end;
procedure cargar_archivo_aves(var archivo_aves: t_archivo_aves);
var
    registro_ave: t_registro_ave;
begin
    rewrite(archivo_aves);
    leer_ave(registro_ave);
    while (registro_ave.codigo<>codigo_salida) do
    begin
        write(archivo_aves,registro_ave);
        leer_ave(registro_ave);
    end;
    close(archivo_aves);
end;
procedure imprimir_registro_ave(registro_ave: t_registro_ave);
begin
    textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_ave.nombre);
    textcolor(green); write('; Código: '); textcolor(yellow); write(registro_ave.codigo);
    textcolor(green); write('; Familia: '); textcolor(yellow); write(registro_ave.familia);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_ave.descripcion);
    textcolor(green); write('; Zona geográfica: '); textcolor(yellow);
writeln(registro_ave.zona);
end;
procedure imprimir_archivo_aves(var archivo_aves: t_archivo_aves);
var
    registro_ave: t_registro_ave;
begin
    reset(archivo_aves);
    while (not eof(archivo_aves)) do
    begin
        read(archivo_aves,registro_ave);
        imprimir_registro_ave(registro_ave);
    end;
    textcolor(green); write('Tamaño del archivo aves: '); textcolor(yellow);
writeln(filesize(archivo_aves));
    close(archivo_aves);
end;
procedure leer_codigo(var codigo: int16);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        codigo:=codigo_salida
    else
        codigo:=1+random(1000);
end;
procedure baja_logica(var archivo_aves: t_archivo_aves; codigo: int16);
var
    registro_ave: t_registro_ave;
    ok: boolean;
begin
    ok:=false;
    reset(archivo_aves);
    while (not eof(archivo_aves)) and (ok=false) do
    begin
        read(archivo_aves,registro_ave);
        if (registro_ave.codigo=codigo) then
        begin
            ok:=true;
            registro_ave.codigo:=codigo_baja;
```

```

        seek(archivo_aves,filepos(archivo_aves)-1);
        write(archivo_aves,registro_ave);
    end;
end;
close(archivo_aves);
if (ok=true) then
begin
    textcolor(green); write('Se ha realizado la baja de la especie de ave con código ');
textcolor(yellow); write(codigo); textcolor(green); writeln(' en el archivo');
end
else
begin
    textcolor(green); write('No se ha encontrado la especie de ave con código ');
textcolor(yellow); write(codigo); textcolor(green); writeln(' en el archivo');
end;
end;
procedure bajas_logicas(var archivo_aves: t_archivo_aves);
var
    codigo: int16;
begin
    leer_codigo(codigo);
    while (codigo<>codigo_salida) do
    begin
        baja_logica(archivo_aves,codigo);
        leer_codigo(codigo);
    end;
    writeln();
end;
procedure bajas_fisicas1(var archivo_aves: t_archivo_aves);
var
    registro_ave, registro_ultima_ave: t_registro_ave;
    pos: int16;
begin
    reset(archivo_aves);
    while (not eof(archivo_aves)) do
    begin
        read(archivo_aves,registro_ave);
        if (registro_ave.codigo=codigo_baja) then
        begin
            pos:=filepos(archivo_aves)-1;
            seek(archivo_aves,filesize(archivo_aves)-1);
            read(archivo_aves,registro_ultima_ave);
            seek(archivo_aves,pos);
            write(archivo_aves,registro_ultima_ave);
            seek(archivo_aves,filesize(archivo_aves)-1);
            truncate(archivo_aves);
            seek(archivo_aves,pos);
        end;
    end;
    close(archivo_aves);
end;
procedure bajas_fisicas2(var archivo_aves: t_archivo_aves);
var
    registro_ave, registro_ultima_ave: t_registro_ave;
    pos_act, pos_ult: int16;
begin
    reset(archivo_aves);
    if (not eof(archivo_aves)) then
    begin
        pos_act:=0; pos_ult:=filesize(archivo_aves)-1;
        while (pos_act<=pos_ult) do
        begin
            seek(archivo_aves,pos_act);
            read(archivo_aves,registro_ave);
            if (registro_ave.codigo=codigo_baja) then
            begin

```

```
        seek(archivo_aves,pos_ult);
        read(archivo_aves,registro_ultima_ave);
        seek(archivo_aves,pos_act);
        write(archivo_aves,registro_ultima_ave);
        pos_ult:=pos_ult-1;
    end
    else
        pos_act:=pos_act+1;
    end;
    seek(archivo_aves,pos_ult+1);
    truncate(archivo_aves);
end;
close(archivo_aves);
end;
var
    archivo_aves: t_archivo_aves;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO AVES:'); writeln();
    assign(archivo_aves,'E7_aves');
    cargar_archivo_aves(archivo_aves);
    imprimir_archivo_aves(archivo_aves);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO AVES CON BAJAS LÓGICAS:'); writeln();
    bajas_logicas(archivo_aves);
    imprimir_archivo_aves(archivo_aves);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO AVES CON BAJAS FÍSICAS:'); writeln();
    bajas_fisicas1(archivo_aves);
    imprimir_archivo_aves(archivo_aves);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO AVES CON BAJAS FÍSICAS:'); writeln();
    bajas_fisicas2(archivo_aves);
    imprimir_archivo_aves(archivo_aves);
end.
```

Ejercicio 8.

Se cuenta con un archivo con información de las diferentes distribuciones de Linux existentes. De cada distribución, se conoce: nombre, año de lanzamiento, número de versión del kernel, cantidad de desarrolladores y descripción. El nombre de las distribuciones no puede repetirse. Este archivo debe ser mantenido realizando bajas lógicas y utilizando la técnica de reutilización de espacio libre llamada lista invertida. Escribir la definición de las estructuras de datos necesarias y los siguientes procedimientos:

(a) *BuscarDistribucion*: Módulo que recibe por parámetro el archivo, un nombre de distribución y devuelve la posición dentro del archivo donde se encuentra el registro correspondiente a la distribución dada, si existe, o devuelve -1, en caso de que no exista.

(b) *AltaDistribucion*: Módulo que recibe como parámetro el archivo y el registro que contiene los datos de una nueva distribución, y se encarga de agregar la distribución al archivo reutilizando espacio disponible en caso de que exista. (El control de unicidad se debe realizar utilizando el módulo anterior). En caso de que la distribución que se quiere agregar ya exista, se debe informar “Ya existe la distribución”.

(c) *BajaDistribucion*: Módulo que recibe como parámetro el archivo y el nombre de una distribución, y se encarga de dar de baja lógicamente la distribución dada. Para marcar una distribución como borrada, se debe utilizar el campo cantidad de desarrolladores para mantener actualizada la lista invertida. Para verificar que la distribución a borrar exista, se debe utilizar el módulo *BuscarDistribucion*. En caso de no existir, se debe informar “Distribución no existente”.

```
program TP3_E8;
{$codepage UTF8}
uses crt;
const
  nombre_salida='ZZZ';
  desarrolladores_cabecera=0;
type
  t_string20=string[20];
  t_registro_distribucion=record
    nombre: t_string20;
    anio: int16;
    kernel: int16;
    desarrolladores: int16;
    descripcion: t_string20;
  end;
  t_archivo_distribuciones=file of t_registro_distribucion;
function random_string(length: int8): t_string20;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_cabecera(var registro_distribucion: t_registro_distribucion);
begin
  registro_distribucion.nombre:='Cabecera';
```



```

registro_distribucion.anio:=desarrolladores_cabecera;
registro_distribucion.kernel:=desarrolladores_cabecera;
registro_distribucion.desarrolladores:=desarrolladores_cabecera;
registro_distribucion.descripcion:='Cabecera';
end;
procedure leer_distribucion(var registro_distribucion: t_registro_distribucion);
var
  i: int8;
begin
  i:=random(10);
  if (i=0) then
    registro_distribucion.nombre:=nombre_salida
  else
    registro_distribucion.nombre:=random_string(5+random(6));
    if (registro_distribucion.nombre<>nombre_salida) then
      begin
        registro_distribucion.anio:=2000+random(26);
        registro_distribucion.kernel:=1+random(10);
        registro_distribucion.desarrolladores:=1+random(100);
        registro_distribucion.descripcion:=random_string(10+random(10));
      end;
    end;
end;
procedure cargar_archivo_distribuciones(var archivo_distribuciones: t_archivo_distribuciones);
var
  registro_distribucion: t_registro_distribucion;
begin
  rewrite(archivo_distribuciones);
  leer_cabecera(registro_distribucion);
  while (registro_distribucion.nombre<>nombre_salida) do
    begin
      write(archivo_distribuciones,registro_distribucion);
      leer_distribucion(registro_distribucion);
    end;
  close(archivo_distribuciones);
end;
procedure imprimir_registro_distribucion(registro_distribucion: t_registro_distribucion);
begin
  textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_distribucion.nombre);
  textcolor(green); write('; Año: '); textcolor(yellow); write(registro_distribucion.anio);
  textcolor(green); write('; Kernel: '); textcolor(yellow);
write(registro_distribucion.kernel);
  textcolor(green); write('; Desarrolladores: '); textcolor(yellow);
write(registro_distribucion.desarrolladores);
  textcolor(green); write('; Descripción: '); textcolor(yellow);
writeln(registro_distribucion.descripcion);
end;
procedure imprimir_archivo_distribuciones(var archivo_distribuciones:
t_archivo_distribuciones);
var
  registro_distribucion: t_registro_distribucion;
begin
  reset(archivo_distribuciones);
  while (not eof(archivo_distribuciones)) do
    begin
      read(archivo_distribuciones,registro_distribucion);
      imprimir_registro_distribucion(registro_distribucion);
    end;
    textcolor(green); write('Tamaño del archivo distribuciones: '); textcolor(yellow);
writeln(filesize(archivo_distribuciones));
  close(archivo_distribuciones);
end;
procedure buscar_distribucion(var archivo_distribuciones: t_archivo_distribuciones; nombre:
t_string20; var pos: int16);
var
  registro_distribucion: t_registro_distribucion;
  ok: boolean;

```

```

begin
    ok:=false;
    reset(archivo_distribuciones);
    while (not eof(archivo_distribuciones)) and (ok=false) do
        begin
            read(archivo_distribuciones,registro_distribucion);
            if (registro_distribucion.nombre=nombre) then
                ok:=true;
            end;
            if (ok=true) then
                pos:=filepos(archivo_distribuciones)-1
            else
                pos:=-1;
            close(archivo_distribuciones);
        end;
    procedure alta_distribucion(var archivo_distribuciones: t_archivo_distribuciones;
        registro_distribucion: t_registro_distribucion);
    var
        cabecera: t_registro_distribucion;
        pos: int16;
    begin
        buscar_distribucion(archivo_distribuciones,registro_distribucion.nombre,pos);
        if (pos=-1) then
            begin
                reset(archivo_distribuciones);
                read(archivo_distribuciones,cabecera);
                if (cabecera.desarrolladores=desarrolladores_cabecera) then
                    begin
                        seek(archivo_distribuciones,filesize(archivo_distribuciones));
                        write(archivo_distribuciones,registro_distribucion);
                    end
                else
                    begin
                        seek(archivo_distribuciones,cabecera.desarrolladores*(-1));
                        read(archivo_distribuciones,cabecera);
                        seek(archivo_distribuciones,filepos(archivo_distribuciones)-1);
                        write(archivo_distribuciones,registro_distribucion);
                        seek(archivo_distribuciones,0);
                        write(archivo_distribuciones,cabecera);
                    end;
                close(archivo_distribuciones);
                textcolor(green); write('Se ha realizado el alta de la distribución '); textcolor(yellow);
                write(registro_distribucion.nombre); textcolor(green); writeln(' en el archivo');
            end
        else
            begin
                textcolor(green); write('Ya existe la distribución '); textcolor(yellow);
                write(registro_distribucion.nombre); textcolor(green); writeln(' en el archivo');
            end;
            writeln();
        end;
    procedure baja_distribucion(var archivo_distribuciones: t_archivo_distribuciones; nombre:
        t_string20);
    var
        registro_distribucion, cabecera: t_registro_distribucion;
        pos: int16;
    begin
        buscar_distribucion(archivo_distribuciones,nombre,pos);
        if (pos<>-1) then
            begin
                reset(archivo_distribuciones);
                read(archivo_distribuciones,cabecera);
                read(archivo_distribuciones,registro_distribucion);
                while (registro_distribucion.nombre<>nombre) do
                    read(archivo_distribuciones,registro_distribucion);
                seek(archivo_distribuciones,filepos(archivo_distribuciones)-1);
            end;
        end;
    end;

```

```

write(archivo_distribuciones,cabecera);
cabecera.desarrolladores:=(filepos(archivo_distribuciones)-1)*(-1);
seek(archivo_distribuciones,0);
write(archivo_distribuciones,cabecera);
close(archivo_distribuciones);
textcolor(green); write('Se ha realizado la baja de la distribución '); textcolor(yellow);
write(nombre); textcolor(green); writeln(' en el archivo');
end
else
begin
textcolor(green); write('No se ha encontrado la distribución '); textcolor(yellow);
write(nombre); textcolor(green); writeln(' en el archivo');
end;
writeln();
end;
var
archivo_distribuciones: t_archivo_distribuciones;
registro_distribucion: t_registro_distribucion;
pos: int16;
begin
randomize;
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DISTRIBUCIONES:'); writeln();
assign(archivo_distribuciones,'E8_distribuciones');
cargar_archivo_distribuciones(archivo_distribuciones);
imprimir_archivo_distribuciones(archivo_distribuciones);
writeln(); textcolor(red); writeln('MÓDULO BuscarDistribucion:'); writeln();
registro_distribucion.nombre:=random_string(5+random(6));
buscar_distribucion(archivo_distribuciones,registro_distribucion.nombre,pos);
if (pos<>-1) then
begin
textcolor(green); write('Se encontró la distribución '); textcolor(yellow);
write(registro_distribucion.nombre); textcolor(green); write(' en la posición ');
textcolor(red); write(pos); textcolor(green); writeln(' del archivo');
end
else
begin
textcolor(green); write('No se encontró la distribución '); textcolor(yellow);
write(registro_distribucion.nombre); textcolor(green); writeln(' en el archivo');
end;
writeln(); textcolor(red); writeln('MÓDULO AltaDistribucion:'); writeln();
leer_distribucion(registro_distribucion);
while (registro_distribucion.nombre=nombre_salida) do
leer_distribucion(registro_distribucion);
alta_distribucion(archivo_distribuciones,registro_distribucion);
imprimir_archivo_distribuciones(archivo_distribuciones);
writeln(); textcolor(red); writeln('MÓDULO BajaDistribucion:'); writeln();
registro_distribucion.nombre:=random_string(5+random(6));
baja_distribucion(archivo_distribuciones,registro_distribucion.nombre);
imprimir_archivo_distribuciones(archivo_distribuciones);
end.

```

PARTE II: Actualización Maestro/Detalle, Reportes y Merge con Archivos no Ordenados.

Para los ejercicios de esta parte de la práctica, teniendo en cuenta que los archivos no están ordenados por ningún criterio, puede resultar necesario recorrer los archivos más de una vez. La idea es resolver los ejercicios sin ordenar los archivos dados y comparar la eficiencia (en cuanto al número de lecturas/escrituras) de la solución brindada en esta práctica respecto a la solución para el mismo problema considerando los archivos ordenados.

Ejercicio 9.

El encargado de ventas de un negocio de productos de limpieza desea administrar el stock de los productos que vende. Para ello, genera un archivo maestro donde figuran todos los productos que comercializa. De cada producto, se maneja la siguiente información: código de producto, nombre comercial, precio de venta, stock actual y stock mínimo. Diariamente, se genera un archivo detalle donde se registran todas las ventas de productos realizadas. De cada venta, se registran: código de producto y cantidad de unidades vendidas. Resolver los siguientes puntos:

(a) Se pide realizar un procedimiento que actualice el archivo maestro con el archivo detalle, teniendo en cuenta que:

- Los archivos no están ordenados por ningún criterio.
- Cada registro del maestro puede ser actualizado por 0, 1 o más registros del archivo detalle.

(b) ¿Qué cambios se realizarían en el procedimiento del inciso anterior si se sabe que cada registro del archivo maestro puede ser actualizado por 0 o 1 registro del archivo detalle?

```
program TP3_E9;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
type
  t_string10=string[10];
  t_registro_producto=record
    codigo: int16;
    nombre: t_string10;
    precio: real;
    stock_actual: int16;
    stock_minimo: int16;
  end;
  t_registro_venta=record
    codigo: int16;
    cantidad_vendida: int16;
  end;
  t_archivo_maestro=file of t_registro_producto;
  t_archivo_detalle=file of t_registro_venta;
function random_string(length: int8): t_string10;
var
  i: int8;
```

```
    string_aux: string;
begin
    string_aux:='';
    for i:= 1 to length do
        string_aux:=string_aux+chr(ord('A')+random(26));
        random_string:=string_aux;
    end;
procedure leer_producto(var registro_producto: t_registro_producto);
var
    i: int8;
begin
    i:=random(10);
    if (i=0) then
        registro_producto.codigo:=codigo_salida
    else
        registro_producto.codigo:=1+random(1000);
        if (registro_producto.codigo<>codigo_salida) then
            begin
                registro_producto.nombre:=random_string(5+random(5));
                registro_producto.precio:=1+random(100);
                registro_producto.stock_actual:=10+random(91);
                registro_producto.stock_minimo:=1+random(50);
            end;
        end;
end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
begin
    rewrite(archivo_maestro);
    leer_producto(registro_producto);
    while (registro_producto.codigo<>codigo_salida) do
        begin
            write(archivo_maestro,registro_producto);
            leer_producto(registro_producto);
        end;
    close(archivo_maestro);
end;
procedure leer_venta(var registro_venta: t_registro_venta);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        registro_venta.codigo:=codigo_salida
    else
        registro_venta.codigo:=1+random(1000);
        if (registro_venta.codigo<>codigo_salida) then
            registro_venta.cantidad_vendida:=1+random(10);
        end;
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    leer_venta(registro_venta);
    while (registro_venta.codigo<>codigo_salida) do
        begin
            write(archivo_detalle,registro_venta);
            leer_venta(registro_venta);
        end;
    close(archivo_detalle);
end;
procedure imprimir_registro_producto(registro_producto: t_registro_producto);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_producto.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_producto.nombre);
```

```
    textcolor(green); write('; Precio: $'); textcolor(yellow);
write(registro_producto.precio:0:2);
    textcolor(green); write('; Stock actual: '); textcolor(yellow);
write(registro_producto.stock_actual);
    textcolor(green); write('; Stock mínimo: '); textcolor(yellow);
writeln(registro_producto.stock_minimo);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_producto);
            imprimir_registro_producto(registro_producto);
        end;
        textcolor(green); write('Tamaño del archivo maestro: '); textcolor(yellow);
writeln(filesize(archivo_maestro));
        close(archivo_maestro);
    end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Cantidad vendida: '); textcolor(yellow);
writeln(registro_venta.cantidad_vendida);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
        close(archivo_detalle);
    end;
procedure actualizar1_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle: t_archivo_detalle);
var
    registro_producto: t_registro_producto;
    registro_venta: t_registro_venta;
    ventas: int16;
begin
    reset(archivo_maestro);
    reset(archivo_detalle);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_producto);
            ventas:=0;
            seek(archivo_detalle,0);
            while (not eof(archivo_detalle)) do
                begin
                    read(archivo_detalle,registro_venta);
                    if (registro_venta.codigo=registro_producto.codigo) then
                        ventas:=ventas+registro_venta.cantidad_vendida;
                    end;
                if (ventas>0) then
                    begin
                        registro_producto.stock_actual:=registro_producto.stock_actual-ventas;
                        seek(archivo_maestro,filepos(archivo_maestro)-1);
                        write(archivo_maestro,registro_producto);
                    end;
                end;
            end;
        end;
```

```
close(archivo_maestro);
close(archivo_detalle);
end;
procedure actualizar2_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle: t_archivo_detalle);
var
    registro_producto: t_registro_producto;
    registro_venta: t_registro_venta;
begin
    reset(archivo_maestro);
    reset(archivo_detalle);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_producto);
            seek(archivo_detalle,0);
            read(archivo_detalle,registro_venta);
            while ((not eof(archivo_detalle)) and (registro_venta.codigo<>registro_producto.codigo))
do
                read(archivo_detalle,registro_venta);
            if (registro_venta.codigo=registro_producto.codigo) then
                begin
                    registro_producto.stock_actual:=registro_producto.stock_actual-
registro_venta.cantidad_vendida;
                    seek(archivo_maestro,filepos(archivo_maestro)-1);
                    write(archivo_maestro,registro_producto);
                end;
            end;
        end;
        close(archivo_maestro);
        close(archivo_detalle);
    end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle: t_archivo_detalle;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E9_productosMaestro');
    cargar_archivo_maestro(archivo_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE:'); writeln();
    assign(archivo_detalle,'E9_productosDetalle');
    cargar_archivo_detalle(archivo_detalle);
    imprimir_archivo_detalle(archivo_detalle);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO 1:'); writeln();
    actualizar1_archivo_maestro(archivo_maestro,archivo_detalle);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO 2:'); writeln();
    actualizar2_archivo_maestro(archivo_maestro,archivo_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 10.

Se necesita contabilizar los votos de las diferentes mesas electorales registradas por localidad en la Provincia de Buenos Aires. Para ello, se posee un archivo con la siguiente información: código de localidad, número de mesa y cantidad de votos en dicha mesa. Presentar en pantalla un listado como se muestra a continuación:

Código de Localidad	Total de Votos
.....	
.....	
Total General de Votos:	

Notas:

- La información en el archivo no está ordenada por ningún criterio.
- Tratar de resolver el problema sin modificar el contenido del archivo dado.
- Se puede utilizar una estructura auxiliar, como por ejemplo otro archivo, para llevar el control de las localidades que han sido procesadas.

```

program TP3_E10;
{$codepage UTF8}
uses crt;
const
    localidad_salida=-1;
type
    t_registro_mesa=record
        localidad: int16;
        numero: int16;
        votos: int32;
    end;
    t_archivo_mesas=file of t_registro_mesa;
procedure leer_mesa(var registro_mesa: t_registro_mesa);
var
    i: int8;
begin
    i:=random(10);
    if (i=0) then
        registro_mesa.localidad:=localidad_salida
    else
        registro_mesa.localidad:=1+random(100);
        if (registro_mesa.localidad<>localidad_salida) then
            begin
                registro_mesa.numero:=1+random(100);
                registro_mesa.votos:=10+random(991);
            end;
    end;
end;
procedure cargar_archivo_mesas(var archivo_mesas: t_archivo_mesas);
var
    registro_mesa: t_registro_mesa;
begin
    rewrite(archivo_mesas);
    leer_mesa(registro_mesa);
    while (registro_mesa.localidad<>localidad_salida) do

```



```
begin
    write(archivo_mesas, registro_mesa);
    leer_mesa(registro_mesa);
end;
close(archivo_mesas);
end;
procedure imprimir_registro_mesa(registro_mesa: t_registro_mesa);
begin
    textcolor(green); write('Código de localidad: '); textcolor(yellow);
write(registro_mesa.localidad);
    textcolor(green); write('; Número de mesa: '); textcolor(yellow);
write(registro_mesa.numero);
    textcolor(green); write('; Votos: '); textcolor(yellow); writeln(registro_mesa.votos);
end;
procedure imprimir_archivo_mesas(var archivo_mesas: t_archivo_mesas);
var
    registro_mesa: t_registro_mesa;
begin
    reset(archivo_mesas);
    while (not eof(archivo_mesas)) do
        begin
            read(archivo_mesas, registro_mesa);
            imprimir_registro_mesa(registro_mesa);
        end;
        textcolor(green); write('Tamaño del archivo mesas: '); textcolor(yellow);
writeln(filesize(archivo_mesas));
        close(archivo_mesas);
    end;
procedure cargar_archivo_mesas_aux(var archivo_mesas: t_archivo_mesas; var archivo_mesas_aux:
t_archivo_mesas);
var
    registro_mesa, registro_mesa_aux: t_registro_mesa;
    ok: boolean;
begin
    reset(archivo_mesas);
    rewrite(archivo_mesas_aux);
    while (not eof(archivo_mesas)) do
        begin
            ok:=false;
            read(archivo_mesas, registro_mesa);
            seek(archivo_mesas_aux, 0);
            while (not eof(archivo_mesas_aux)) and (ok=false) do
                begin
                    read(archivo_mesas_aux, registro_mesa_aux);
                    if (registro_mesa_aux.localidad=registro_mesa.localidad) then
                        ok:=true;
                    end;
                    if (ok=true) then
                        begin
                            registro_mesa_aux.votos:=registro_mesa_aux.votos+registro_mesa.votos;
                            seek(archivo_mesas_aux, filepos(archivo_mesas_aux)-1);
                            write(archivo_mesas_aux, registro_mesa_aux);
                        end
                    else
                        write(archivo_mesas_aux, registro_mesa);
                    end;
                end;
            close(archivo_mesas);
            close(archivo_mesas_aux);
        end;
procedure imprimir_archivo_mesas_aux(var archivo_mesas: t_archivo_mesas);
var
    registro_mesa: t_registro_mesa;
    votos_total: int32;
begin
    votos_total:=0;
    reset(archivo_mesas);
```

```
textcolor(green); writeln('Código de Localidad      Total de Votos');
while (not eof(archivo_mesas)) do
begin
    read(archivo_mesas,registro_mesa);
    textcolor(yellow); write(registro_mesa.localidad); textcolor(green);
write('      '); textcolor(red); writeln(registro_mesa.votos);
    votos_total:=votos_total+registro_mesa.votos;
end;
textcolor(green); write('Total General de Votos:      '); textcolor(red);
writeln(votos_total);
close(archivo_mesas);
end;
var
    archivo_mesas, archivo_mesas_aux: t_archivo_mesas;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MESAS:'); writeln();
    assign(archivo_mesas,'E10_mesas');
    cargar_archivo_mesas(archivo_mesas);
    imprimir_archivo_mesas(archivo_mesas);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MESAS AUXILIAR:'); writeln();
    assign(archivo_mesas_aux,'E10_mesas_aux');
    cargar_archivo_mesas_aux(archivo_mesas,archivo_mesas_aux);
    imprimir_archivo_mesas(archivo_mesas_aux);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MESAS AUXILIAR:'); writeln();
    imprimir_archivo_mesas_aux(archivo_mesas_aux);
end.
```

Ejercicio 11.

Suponer que se trabaja en una oficina donde está montada una LAN (red local). La misma fue construida sobre una topología de red que conecta 5 máquinas entre sí y todas las máquinas se conectan con un servidor central. Semanalmente, cada máquina genera un archivo de logs informando las sesiones abiertas por cada usuario en cada terminal y por cuánto tiempo estuvo abierta. Cada archivo detalle contiene los siguientes campos: *cod_usuario*, *fecha*, *tiempo_sesion*. Se debe realizar un procedimiento que reciba los archivos detalle y genere un archivo maestro con los siguientes datos: *cod_usuario*, *fecha*, *tiempo_total_de_sesiones_abiertas*.

Notas:

- Los archivos detalle no están ordenados por ningún criterio.
- Un usuario puede iniciar más de una sesión el mismo día en la misma máquina o, inclusive, en diferentes máquinas.

```
program TP3_E11;
{$codepage UTF8}
uses crt, sysutils;
const
    detalles_total=5;
    codigo_salida=-1;
type
    t_detalle=1..detalles_total;
    t_string20=string[20];
    t_registro_sesion=record
        codigo: int16;
        fecha: t_string20;
        tiempo: int16;
    end;
    t_archivo_sesiones=file of t_registro_sesion;
    t_vector_detalle=array[t_detalle] of t_archivo_sesiones;
procedure leer_sesion(var registro_sesion: t_registro_sesion);
var
    i: int8;
begin
    i:=random(10);
    if (i=0) then
        registro_sesion.codigo:=codigo_salida
    else
        registro_sesion.codigo:=1+random(10);
        if (registro_sesion.codigo<>codigo_salida) then
            begin
                registro_sesion.fecha:='2025-01-'+inttoStr(1+random(30));
                registro_sesion.tiempo:=1+random(100);
            end;
    end;
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_sesiones);
var
    registro_sesion: t_registro_sesion;
begin
    rewrite(archivo_detalle);
    leer_sesion(registro_sesion);
    while (registro_sesion.codigo<>codigo_salida) do
        begin
            write(archivo_detalle,registro_sesion);
            leer_sesion(registro_sesion);
        end;
end;
```

```

    close(archivo_detalle);
end;
procedure imprimir_registro_sesion(registro_sesion: t_registro_sesion);
begin
    textcolor(green); write('Código de usuario: '); textcolor(yellow);
write(registro_sesion.codigo);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_sesion.fecha);
    textcolor(green); write('; Tiempo: '); textcolor(yellow); writeln(registro_sesion.tiempo);
end;
procedure imprimir_archivo_sesiones(var archivo_sesiones: t_archivo_sesiones);
var
    registro_sesion: t_registro_sesion;
begin
    reset(archivo_sesiones);
    while (not eof(archivo_sesiones)) do
        begin
            read(archivo_sesiones,registro_sesion);
            imprimir_registro_sesion(registro_sesion);
        end;
        textcolor(green); write('Tamaño del archivo sesiones: '); textcolor(yellow);
writeln(filesize(archivo_sesiones));
        close(archivo_sesiones);
    end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_sesiones; var vector_detalles:
t_vector_detalles);
var
    registro_sesion_detalle, registro_sesion_maestro: t_registro_sesion;
    i: t_detalle;
    ok: boolean;
begin
    rewrite(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalles[i]);
            while (not eof(vector_detalles[i])) do
                begin
                    ok:=false;
                    read(vector_detalles[i],registro_sesion_detalle);
                    seek(archivo_maestro,0);
                    while (not eof(archivo_maestro)) and (ok=false) do
                        begin
                            read(archivo_maestro,registro_sesion_maestro);
                            if ((registro_sesion_maestro.codigo=registro_sesion_detalle.codigo) and
(registro_sesion_maestro.fecha=registro_sesion_detalle.fecha)) then
                                ok:=true;
                            end;
                            if (ok=true) then
                                begin
                                    registro_sesion_maestro.tiempo:=registro_sesion_maestro.tiempo+registro_sesion_detalle
.tiempo;
                                    seek(archivo_maestro,filepos(archivo_maestro)-1);
                                    write(archivo_maestro,registro_sesion_maestro);
                                end
                            else
                                write(archivo_maestro,registro_sesion_detalle);
                            end;
                            close(vector_detalles[i]);
                        end;
                    close(archivo_maestro);
                end;
            end;
        var
            vector_detalles: t_vector_detalles;
            archivo_maestro: t_archivo_sesiones;
            i: t_detalle;
        begin
            randomize;

```

```
for i:= 1 to detalles_total do
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
    assign(vector_detalle[i],'E11_sesionesDetalle'+inttoStr(i));
    cargar_archivo_detalle(vector_detalle[i]);
    imprimir_archivo_sesiones(vector_detalle[i]);
end;
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
assign(archivo_maestro,'E11_sesionesMaestro');
cargar_archivo_maestro(archivo_maestro,vector_detalle);
imprimir_archivo_sesiones(archivo_maestro);
end.
```

Introducción a las Bases de Datos

Fundamentos de Organización de Datos

Práctica 4

Políticas para la resolución de underflow:

Política izquierda: se intenta redistribuir con el hermano adyacente izquierdo, si no es posible, se fusiona con hermano adyacente izquierdo.

Política derecha: se intenta redistribuir con el hermano adyacente derecho, si no es posible, se fusiona con hermano adyacente derecho.

Política izquierda o derecha: se intenta redistribuir con el hermano adyacente izquierdo, si no es posible, se intenta con el hermano adyacente derecho, si tampoco es posible, se fusiona con hermano adyacente izquierdo.

Política derecha o izquierda: se intenta redistribuir con el hermano adyacente derecho, si no es posible, se intenta con el hermano adyacente izquierdo, si tampoco es posible, se fusiona con hermano adyacente derecho.

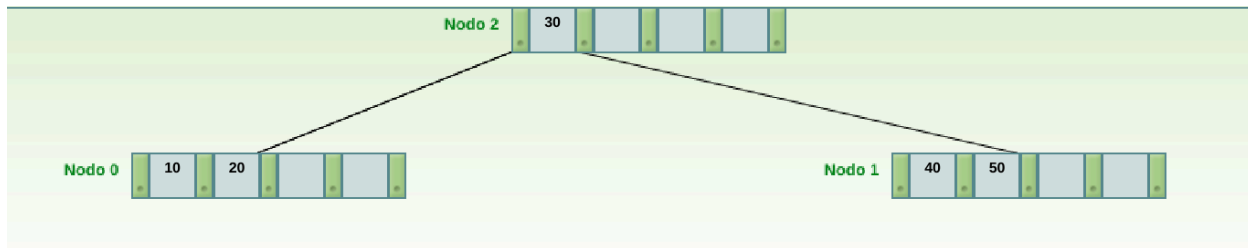
Casos especiales: en cualquier política si se tratase de un nodo hoja de un extremo del árbol debe intentarse redistribuir con el hermano adyacente que el mismo posea.

Aclaración:

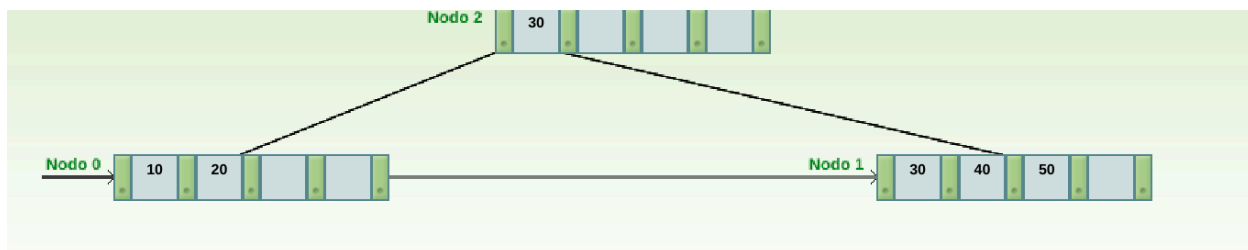
- En caso de underflow lo primero que se intenta **SIEMPRE** es redistribuir y el hermano adyacente se encuentra en condiciones de ceder elementos si al hacerlo no se produce underflow en el.
- En caso de overflow **SIEMPRE** se genera un nuevo nodo. Las claves se distribuyen equitativamente entre el nodo desbordado y el nuevo.

En el caso de órdenes impares se debe promocionar la clave o la copia (en árbol B+) que se encuentra en la posición del medio.

Ejemplo árbol B, orden 5

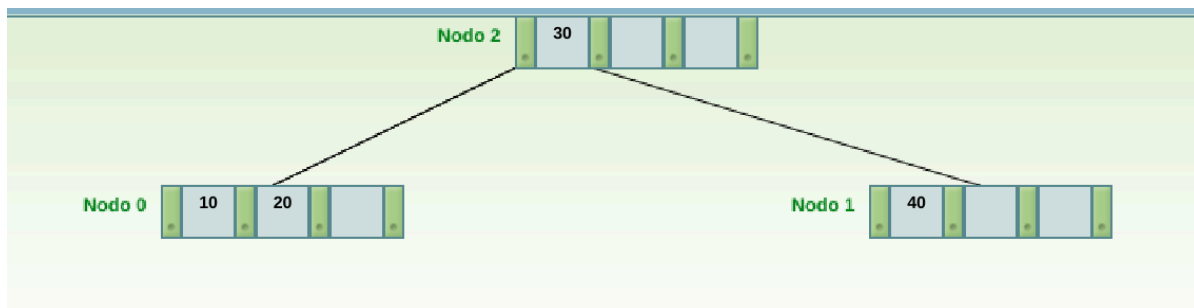


Ejemplo árbol B+, orden 5

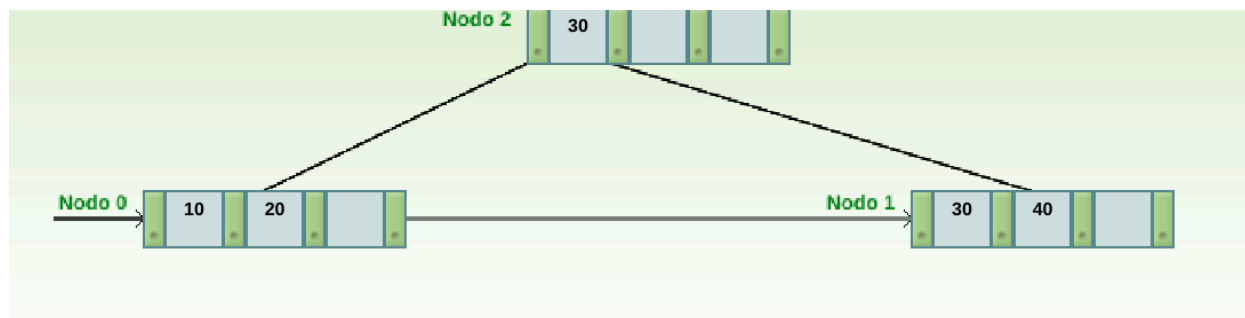


En el caso de órdenes pares se elige la menor de las claves mayores o su copia (en árbol B+) para promocionar.

Ejemplo árbol B, orden 4:



Ejemplo árbol B+, orden 4:



Parte 1: Archivos de datos, índices y árboles B

1. Considere que desea almacenar en un archivo la información correspondiente a los alumnos de la Facultad de Informática de la UNLP. De los mismos deberá guardarse nombre y apellido, DNI, legajo y año de ingreso. Suponga que dicho archivo se organiza como un árbol B de orden M.
 - a. Defina en Pascal las estructuras de datos necesarias para organizar el archivo de alumnos como un árbol B de orden M.
 - b. Suponga que la estructura de datos que representa una persona (registro de persona) ocupa 64 bytes, que cada nodo del árbol B tiene un tamaño de 512 bytes y que los números enteros ocupan 4 bytes, ¿cuántos registros de persona entrarían en un nodo del árbol B? ¿Cuál sería el orden del árbol B en este caso (el valor de M)? Para resolver este inciso, puede utilizar la fórmula $N = (M-1) * A + M * B + C$, donde N es el tamaño del nodo (en bytes), A es el tamaño de un registro (en bytes), B es el tamaño de cada enlace a un hijo y C es el tamaño que ocupa el campo referido a la cantidad de claves. El objetivo es reemplazar estas variables con los valores dados y obtener el valor de M (M debe ser un número entero, ignorar la parte decimal).
 - c. ¿Qué impacto tiene sobre el valor de M organizar el archivo con toda la información de los alumnos como un árbol B?
 - d. ¿Qué dato seleccionaría como clave de identificación para organizar los elementos (alumnos) en el árbol B? ¿Hay más de una opción?
 - e. Describa el proceso de búsqueda de un alumno por el criterio de ordenamiento especificado en el punto previo. ¿Cuántas lecturas de nodos se necesitan para encontrar un alumno por su clave de identificación en el peor y en el mejor de los casos? ¿Cuáles serían estos casos?
 - f. ¿Qué ocurre si desea buscar un alumno por un criterio diferente? ¿Cuántas lecturas serían necesarias en el peor de los casos?

2. Una mejora respecto a la solución propuesta en el ejercicio 1 sería mantener por un lado el archivo que contiene la información de los alumnos de la Facultad de Informática (archivo de datos no ordenado) y por otro lado mantener un índice al archivo de datos que se estructura como un árbol B que ofrece acceso indizado por DNI de los alumnos.
- Defina en Pascal las estructuras de datos correspondientes para el archivo de alumnos y su índice.
 - Suponga que cada nodo del árbol B cuenta con un tamaño de 512 bytes. ¿Cuál sería el orden del árbol B (valor de M) que se emplea como índice? Asuma que los números enteros ocupan 4 bytes. Para este inciso puede emplear una fórmula similar al punto 1b, pero considere además que en cada nodo se deben almacenar los $M-1$ enlaces a los registros correspondientes en el archivo de datos.
 - ¿Qué implica que el orden del árbol B sea mayor que en el caso del ejercicio 1?
 - Describa con sus palabras el proceso para buscar el alumno con el DNI 12345678 usando el índice definido en este punto.
 - ¿Qué ocurre si desea buscar un alumno por su número de legajo? ¿Tiene sentido usar el índice que organiza el acceso al archivo de alumnos por DNI? ¿Cómo haría para brindar acceso indizado al archivo de alumnos por número de legajo?
 - Suponga que desea buscar los alumnas que tienen DNI en el rango entre 40000000 y 45000000. ¿Qué problemas tiene este tipo de búsquedas con apoyo de un árbol B que solo provee acceso indizado por DNI al archivo de alumnos?
3. Los árboles B+ representan una mejora sobre los árboles B dado que conservan la propiedad de acceso indexado a los registros del archivo de datos por alguna clave, pero permiten además un recorrido secuencial rápido. Al igual que en el ejercicio 2, considere que por un lado se tiene el archivo que contiene la información de los alumnos de la Facultad de Informática (archivo de datos no ordenado) y por otro lado se tiene un índice al archivo de datos, pero en este caso el índice se estructura como un árbol B+ que ofrece acceso indizado por DNI al archivo de alumnos. Resuelva los siguientes incisos:
- ¿Cómo se organizan los elementos (claves) de un árbol B+? ¿Qué elementos se encuentran en los nodos internos y que elementos se encuentran en los nodos hojas?
 - ¿Qué característica distintiva presentan los nodos hojas de un árbol B+? ¿Por qué?
 - Defina en Pascal las estructuras de datos correspondientes para el archivo de alumnos y su índice (árbol B+). Por simplicidad, suponga que todos los nodos del árbol B+ (nodos internos y nodos hojas) tienen el mismo tamaño

- d. Describa, con sus palabras, el proceso de búsqueda de un alumno con un DNI específico haciendo uso de la estructura auxiliar (índice) que se organiza como un árbol B+. ¿Qué diferencia encuentra respecto a la búsqueda en un índice estructurado como un árbol B?
- e. Explique con sus palabras el proceso de búsqueda de los alumnos que tienen DNI en el rango entre 40000000 y 45000000, apoyando la búsqueda en un índice organizado como un árbol B+. ¿Qué ventajas encuentra respecto a este tipo de búsquedas en un árbol B?

4. Dado el siguiente algoritmo de búsqueda en un árbol B:

```

procedure buscar(NRR, clave, NRR_encontrado, pos_encontrada, resultado)
var clave_encontrada: boolean;
begin
  if (nodo = null)
    resultado := false; {clave no encontrada}
  else
    posicionarYLeerNodo(A, nodo, NRR);
    claveEncontrada(A, nodo, clave, pos, clave_encontrada);
    if (clave_encontrada) then begin
      NRR_encontrado := NRR; { NRR actual }
      pos_encontrada := pos; { posicion dentro del array }
      resultado := true;
    end
  else
    buscar(nodo.hijos[pos], clave, NRR_encontrado, pos_encontrada, resultado)
end;
```

Asuma que el archivo se encuentra abierto y que, para la primera llamada, el parámetro NRR contiene la posición de la raíz del árbol. Responda detalladamente:

- a. PosicionarYLeerNodo(): Indique qué hace y la forma en que deben ser enviados los parámetros (valor o referencia). Implemente este módulo en Pascal.
- b. claveEncontrada(): Indique qué hace y la forma en que deben ser enviados los parámetros (valor o referencia). ¿Cómo lo implementaría?
- c. ¿Existe algún error en este código? En caso afirmativo, modifique lo que considere necesario.
- d. ¿Qué cambios son necesarios en el procedimiento de búsqueda implementado sobre un árbol B para que funcione en un árbol B+?

5. Defina los siguientes conceptos:

- Overflow
- Underflow
- Redistribución
- Fusión o concatenación

En los dos últimos casos, ¿cuándo se aplica cada uno?

6. Suponga que se tiene un archivo que contiene información de los empleados de una empresa. De cada empleado se mantiene la siguiente información: DNI, legajo, nombre completo y salario. Considere que se mantiene además un índice, organizado como árbol B de orden 4, que provee acceso indizado a los empleados por su DNI. Grafique como queda el archivo de empleados (archivo de datos) y el archivo índice (árbol B) tras la inserción de los siguientes registros:

DNI	Legajo	Nombre y apellido	Salario
35.000.000	100	Juan Perez	\$250000
40.000.000	101	Lucio Redivo	\$400000
32.000.000	102	Nicolás Lapro	\$720000
28.000.000	103	Luis Scola	\$2000000
26.000.000	104	Andres Nocioni	\$1500000
37.000.000	105	Facundo Campazzo	\$1200000
25.000.000	106	Emanuel Ginobili	\$5000000
23.000.000	107	Pepe Sanchez	\$1000000
21.000.000	108	Alejandro Montecchia	\$800000
36.000.000	109	Marcos Delia	\$300000
45.000.000	110	Leandro Bolmaro	\$600000

Notas:

- Grafique los estados de ambos archivos (datos e índice) cuando ocurren cambios relevantes en el índice como la creación de un nuevo nodo.
- Además de graficar los archivos con sus respectivas estructuras internas, resulta útil que grafique la vista del archivo índice como un árbol B.

Parte 2: Operaciones en árboles B y B+

Para los siguientes ejercicios debe:

- **Indicar los nodos leídos y escritos en cada operación.**
- **Todas las operaciones deben estar claramente justificadas, enunciando las mismas indefectiblemente tal cual se presenta en la materia.**

- Los números de nodo deben asignarse en forma coherente con el crecimiento del archivo. La reutilización de nodos libres se debe efectuar con política LIFO (último en entrar, primero en salir).
- Para los siguientes ejercicios sólo interesa graficar los estados del árbol B que representa el índice (no es necesario dibujar la estructura interna de los archivos como si se solicitó en el ejercicio 6).

7. Dado el siguiente árbol B de orden 5, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +320, -390, -400, -533. Justificar detalladamente cada operación indicando lecturas y escrituras en orden de ocurrencia. Para la resolución de underflow debe utilizar **política a izquierda**. Graficar cada operación por separado.

2: 0 (220) 1 (390) 4 (455) 5 (541) 3

0: (10)(150) 1: (225)(241)(331)(360) 4: (400)(407) 5: (508)(533) 3: (690)(823)

8. Dado el siguiente árbol B de orden 4, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +5, +9, +80, +51, -50, -92.

Política de resolución de underflows: derecha.

2: 0 (66) 1

0: (22)(32)(50)

1: (77) (79) (92)

9. Dado el siguiente árbol B de orden 6, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +15, +71, +3, +48, -56, -71.

Política de resolución de underflows: derecha o izquierda.

0: (34) (56) (78) (100) (176)

10. Dado el siguiente árbol B de orden 5, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +450, -485, -511, -614.

Política de resolución de underflows: derecha.

2: 0 (315) 1 (485) 4 (547) 5 (639) 3

0: (148)(223) 1: (333)(390)(442)(454) 4: (508)(511) 5: (614)(633) 3: (789)(915)

11. Dado un árbol B de orden 5 y con política izquierda, para cada operación dada:

- Dibuje el árbol resultante
- Explique las decisiones tomadas
- Escriba las lecturas y escrituras

Operaciones: -76, -400, +900, +12

nodo 2: 3 i 0(76)4(300)1(600)3

nodo 0: 4 h(21)(45)(46)(70)

nodo 4: 2 h(100)(200)

nodo 1: 1 h(400)

nodo 3: 2 h(700)(800)

12. Dada las siguientes operaciones, mostrar la construcción paso a paso de un árbol B de orden 4: +50, +70, +40, +15, +90, +120, +115, +45, +30, +100, +112, +77, -45, -40, -50, -90, -100.

Política de resolución de underflows: izquierda o derecha.

13. Dadas las siguientes operaciones, mostrar la construcción paso a paso de un árbol B de orden 5:

Política de resolución de underflows: izquierda.

+80, +50, +70, +120, +23, +52, +59, +65, +30, +40, +45, +31, +34, +38, +60, +63, +64, -23, -30, -31, -40, -45, -38.

14. Dado el siguiente árbol B de orden 6, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +300, +577, -586, -570, -380, -460.

Política de resolución de underflows: izquierda o derecha.

2: 0 (216) 1 (460) 4 (570) 5 (689) 3 (777) 6

0: (100)(159)(171)

5: (586)(599)(615)(623)(680)

1: (222)(256)(358)(380)(423)

3: (703)(725)

4: (505)(522)

6: (789)(915)(1000)

15. Dada las siguientes operaciones, mostrar cómo se construye el árbol B de orden 4: +65, +89, +23, +45, +20, +96, +10, +55, -23, +110, +50, -10, +25, -50, -45, +120, +130, +70, +75, +73, +100, -120, -110.

Política de resolución de underflows: derecha.

16. Dado el siguiente árbol B + de orden 4 y con política de resolución de underflows a derecha, realice las siguientes operaciones indicando lecturas y escrituras en el orden de ocurrencia. Además, debe describir detalladamente lo que sucede en cada operación. +80, -400.

4: 0 (340) 1 (400) 2 (500) 3

0: (11)(50)(77) 1: (340)(350)(360) 2: (402)(410)(420) 3: (520)(530) -1

17. Dado el siguiente árbol B+ de orden 4, mostrar como quedaría el mismo luego de realizar las siguientes operaciones: +120, +110, +52, +70, +15, -45, -52, +22, +19, -66, -22, -19, -23, -89.

Política de resolución de underflows: derecha.

2: 0 (66) 1

0: (23)(45) 1

1: (66) (67) (89)

18. Dada las siguientes operaciones, mostrar la construcción paso a paso de un árbol B+ de orden 4:

+67, +56, +96, +10, +28, +95, +16, +46, +23, +36, +120, +130, +60, +57, -96, -67, -95, -60, -120, -57, -56.

Política de resolución de underflows: derecha o izquierda.

19. Dada las siguientes operaciones, mostrar la construcción paso a paso de un árbol B+ de orden 6:

+52, +23, +10, +99, +63, +74, +19, +85, +14, +73, +5, +7, +41, +100, +130, +44, -63, -73, +15, +16, -74, -52.

Política de resolución de underflows: izquierda.

20. Dado un árbol B+ de orden 4 y con política izquierda o derecha, para cada operación dada:

- Dibuje el árbol resultante.
- Explique brevemente las decisiones tomadas.
- Escriba las lecturas y escrituras,

Operaciones: +4, +44, -94, -104

nodo 7: 1 i 2(69)6
 nodo 2: 2 i 0(30)1(51)3
 nodo 6: 1 i 4(94)5
 nodo 0: 3 h(5)(10)(20)->1
 nodo 1: 2 h(30)(40)->3
 nodo 3: 2 h(51)(60)->4
 nodo 4: 2 h(69)(80)->5
 nodo 5: 1 h(104)->-1

21. Dado el árbol B+ que se detalla más abajo, con orden 6, es decir, capacidad de 5 claves como máximo. Muestre los estados sucesivos al realizar la siguiente secuencia de operaciones: +159, -5 y -190, además indicar nodos leídos y escritos en el orden de ocurrencia. Política de resolución underflow derecha.

Nodo 2: 5, i, 0(10)1(60)3(115)4(145)5(179)6

Nodo 0: 2, h, (1)(5) -> 1

Nodo 1: 2, h, (34)(44) -> 3

Nodo 3: 2, h, (60)(113) -> 4

Nodo 4: 4, h, (120)(125)(131)(139) -> 5

Nodo 5: 5, h, (145)(153)(158)(160)(177) -> 6

Nodo 6: 2, h, (179)(190) -> -1

22. Dado un árbol B de orden 5 y con política izquierda o derecha, para cada operación dada:

- Dibuje el árbol resultante
- Explique detalladamente las decisiones tomadas
- Escriba las lecturas y escrituras

Operaciones: +165, +260, +800, -110,

Árbol: Nodo 8: 1 i 2 (150) 7

Nodo 2: 1 i 0 (120) 3

Nodo 7: 2 i 4 (210)6(300)1

Nodo 0: 2 h (30)(110)

Nodo 3: 1 h (130)

Nodo 4: 4 h (160)(170)(180)(200)

Nodo 6: 4 h (220)(230)(240)(250)

Nodo 1: 4 h (400)(500)(600)(700)

23. Dado un árbol B+ de orden 5 y con política izquierda o derecha, para cada operación dada:

- Dibuje el árbol resultante
- Explique detalladamente las decisiones tomadas
- Escriba las lecturas y escrituras

Operaciones: +250, -300, -40

Árbol: nodo 8: 1 i 2(70)7

nodo2: 1 i 0(50)4

nodo7: 4 i 5(90)6(120)3(210)9(300)1

nodo 0: 1 h(40)->4

nodo 4: 1 h(50)->5

nodo 5: 2 h(70)(80)->6

nodo 6: 2 h(90)(100)->3

nodo 3: 2 h(120)(200)->9

nodo 9: 4 h(210)(220)(230)(240)->1

nodo 1: 2 h(400)(500)-> -1

Trabajo Práctico N° 4: Árboles B.

Políticas para la resolución de underflow:

- Política izquierda: Se intenta redistribuir con el hermano adyacente izquierdo; si no es posible, se fusiona con hermano adyacente izquierdo.
- Política derecha: Se intenta redistribuir con el hermano adyacente derecho; si no es posible, se fusiona con hermano adyacente derecho.
- Política izquierda o derecha: Se intenta redistribuir con el hermano adyacente izquierdo; si no es posible, se intenta con el hermano adyacente derecho; si tampoco es posible, se fusiona con hermano adyacente izquierdo.
- Política derecha o izquierda: Se intenta redistribuir con el hermano adyacente derecho; si no es posible, se intenta con el hermano adyacente izquierdo; si tampoco es posible, se fusiona con hermano adyacente derecho.

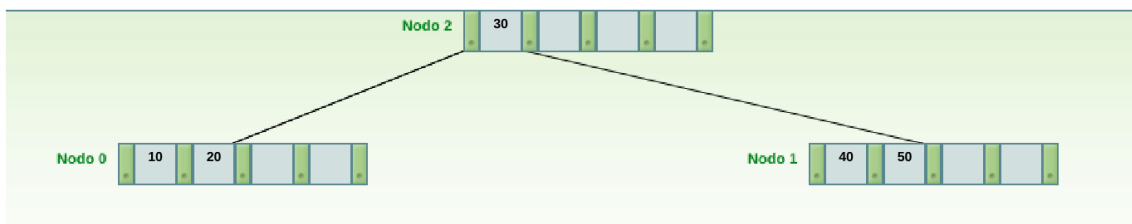
Casos especiales: En cualquier política, si se tratase de un nodo hoja de un extremo del árbol, debe intentarse redistribuir con el hermano adyacente que el mismo posea.

Aclaración:

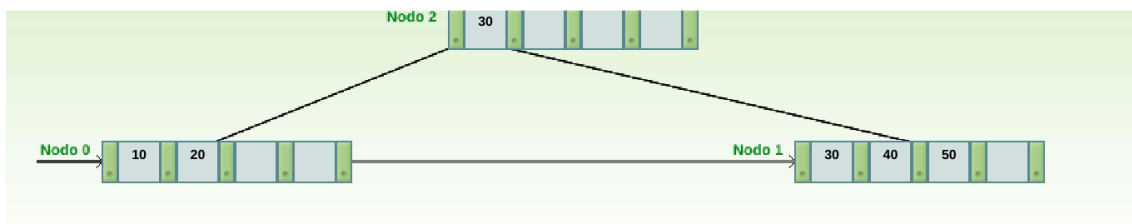
- En caso de underflow, lo primero que se intenta SIEMPRE es redistribuir y el hermano adyacente se encuentra en condiciones de ceder elementos si, al hacerlo, no se produce underflow en él.
- En caso de overflow, SIEMPRE se genera un nuevo nodo. Las claves se distribuyen, equitativamente, entre el nodo desbordado y el nuevo.

En el caso de órdenes impares, se debe promocionar la clave o la copia (en árbol B+) que se encuentra en la posición del medio.

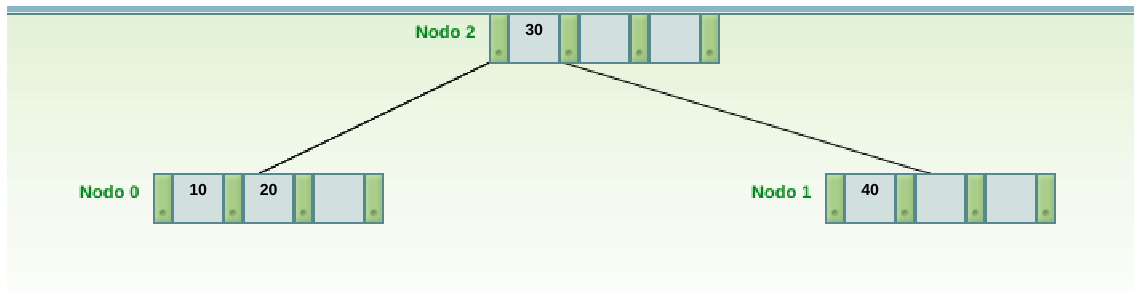
Ejemplo árbol B, orden 5:



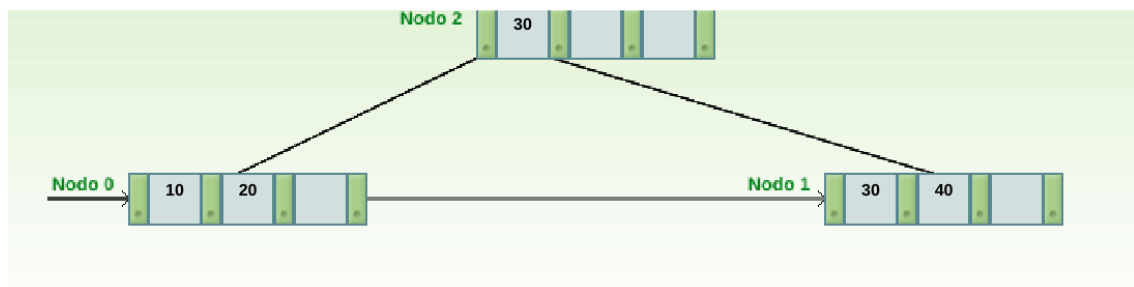
Ejemplo árbol B+, orden 5:



Ejemplo árbol B, orden 4:



Ejemplo árbol B+, orden 4:



PARTE I: Archivos de Datos, Índices y Árboles B.**Ejercicio 1.**

Considerar que se desea almacenar, en un archivo, la información correspondiente a los alumnos de la Facultad de Informática de la UNLP. De los mismos, se deberá guardar nombre y apellido, DNI, legajo y año de ingreso. Suponer que dicho archivo se organiza como un árbol B de orden M.

(a) Definir, en Pascal, las estructuras de datos necesarias para organizar el archivo de alumnos como un árbol B de orden M.

```

program TP4_E1;
const
  M=10;
type
  t_registro_alumno=record
    nombre: string;
    apellido: string;
    dni: int32;
    legajo: int16;
    anio_ingreso: int16;
  end;
  t_registro_nodo=record
    cantidad_claves: int16;
    claves: array[0..M-1] of t_registro_alumno;
    hijos: array[0..M] of int16;
  end;
  t_arbol_datos=file of t_registro_nodo;
var
  archivo_datos: t_archivo_datos;
begin
end.

```

(b) Suponer que la estructura de datos que representa una persona (registro de persona) ocupa 64 bytes, que cada nodo del árbol B tiene un tamaño de 512 bytes y que los números enteros ocupan 4 bytes, ¿cuántos registros de persona entrarían en un nodo del árbol B? ¿Cuál sería el orden del árbol B en este caso (el valor de M)? Para resolver este inciso, se puede utilizar la fórmula $N = (M - 1) * A + M * B + C$, donde N es el tamaño del nodo (en bytes), A es el tamaño de un registro (en bytes), B es el tamaño de cada enlace a un hijo y C es el tamaño que ocupa el campo referido a la cantidad de claves. El objetivo es reemplazar estas variables con los valores dados y obtener el valor de M (M debe ser un número entero, ignorar la parte decimal).

$$N = (M - 1) * A + M * B + C$$

$$N = AM - A + BM + C$$

$$512 = 64M - 64 + 4M + 4$$

$$512 = 68M - 60$$

$$68M = 512 + 60$$

$$68M = 572$$

$$M = \frac{572}{68}$$

$M \cong 8,41$.

Por lo tanto, entrarían 7 registros de persona en un nodo del árbol B, ya que el orden del árbol B en este caso (el valor de M) sería 8.

(c) ¿Qué impacto tiene sobre el valor de M organizar el archivo con toda la información de los alumnos como un árbol B?

El impacto que tiene sobre el valor de M organizar el archivo con toda la información de los alumnos como un árbol B es directo:

- Cuanto más grande sea el registro de alumno, menor será el valor de M, lo cual disminuye la eficiencia del árbol B, ya que se incrementa su profundidad y, con ello, el costo en tiempo y recursos para buscar o modificar datos.
- Cuanto más chico sea el registro de alumno, mayor será el valor de M, lo cual aumenta la eficiencia del árbol B, ya que se reduce su profundidad y, con ello, el costo en tiempo y recursos para buscar o modificar datos.

(d) ¿Qué dato se seleccionaría como clave de identificación para organizar los elementos (alumnos) en el árbol B? ¿Hay más de una opción?

Los datos que se seleccionaría como clave de identificación para organizar los elementos (alumnos) en el árbol B son el DNI o el legajo, ya que ambos son únicos por alumno.

Sí, hay más de una opción, pero las claves numéricas y únicas son las más eficientes para un árbol B.

(e) Describir el proceso de búsqueda de un alumno por el criterio de ordenamiento especificado en el inciso previo. ¿Cuántas lecturas de nodos se necesitan para encontrar un alumno por su clave de identificación en el peor y en el mejor de los casos? ¿Cuáles serían estos casos?

El proceso de búsqueda de un alumno por el criterio de ordenamiento especificado en el inciso previo es:

- Se comienza en la raíz del árbol B.
- Se comparan las claves almacenadas en ese nodo con la clave buscada (DNI).
- Si la clave está en ese nodo, la búsqueda termina.
- Si la clave no está en ese nodo:
 - Se determina en qué subárbol (hijo) continuar según el rango de claves.
 - Se lee el nodo hijo correspondiente.
- Se repite el proceso hasta:
 - Encontrar la clave (caso exitoso).
 - O llegar a un nodo hoja sin encontrarla (caso fallido).

Para encontrar un alumno por su clave de identificación, se necesitan:

- En el peor de los casos, h lecturas, siendo h la altura del árbol. Este caso ocurre cuando el alumno buscado no está en el árbol o está en un nodo hoja (nodo del último nivel) del árbol.
- En el mejor de los casos, una única lectura. Este caso ocurre cuando el alumno buscado está en el nodo raíz.

(f) *¿Qué ocurre si se desea buscar un alumno por un criterio diferente? ¿Cuántas lecturas serían necesarias en el peor de los casos?*

Lo que ocurre si se desea buscar un alumno por un criterio diferente al criterio de ordenamiento es que no se puede aprovechar la estructura jerárquica del árbol, por lo que se debe realizar una búsqueda secuencial (lineal), es decir, recorrer todos los registros almacenados, sin ayuda de la estructura del árbol.

En el peor de los casos, serían necesarias n lecturas, siendo n la cantidad total de nodos del árbol.

Ejercicio 2.

Una mejora respecto a la solución propuesta en el Ejercicio 1 sería mantener, por un lado, el archivo que contiene la información de los alumnos de la Facultad de Informática (archivo de datos no ordenado) y, por otro lado, mantener un índice al archivo de datos que se estructura como un árbol B que ofrece acceso indizado por DNI de los alumnos.

(a) Definir, en Pascal, las estructuras de datos correspondientes para el archivo de alumnos y su índice.

```
program TP4_E2;
const
  M=10;
type
  t_registro_alumno=record
    nombre: string;
    apellido: string;
    dni: int32;
    legajo: int16;
    anio_ingreso: int16;
  end;
  t_registro_nodo=record
    cantidad_claves: int16;
    claves: array[0..M-1] of int32;
    enlaces: array[0..M-1] of int16;
    hijos: array[0..M] of int16;
  end;
  t_archivo_datos=file of t_registro_alumno;
  t_archivo_indice=file of t_registro_nodo;
var
  archivo_datos: t_archivo_datos;
  archivo_indice: t_archivo_indice;
begin
end.
```

(b) Suponer que cada nodo del árbol B cuenta con un tamaño de 512 bytes. ¿Cuál sería el orden del árbol B (valor de M) que se emplea como índice? Asumir que los números enteros ocupan 4 bytes. Para este inciso, se puede emplear una fórmula similar al inciso (b) del Ejercicio 1, pero considerar, además, que, en cada nodo, se deben almacenar los M - 1 enlaces a los registros correspondientes en el archivo de datos.

$$N = (M - 1) * A + (M - 1) * A + M * B + C$$

$$N = 2 * (M - 1) * A + BM + C$$

$$N = 2AM - 2A + BM + C$$

$$512 = 2 * 4M - 2 * 4 + 4M + 4$$

$$512 = 8M - 8 + 4M + 4$$

$$512 = 12M - 4$$

$$12M = 512 + 4$$

$$12M = 516$$

$$M = \frac{516}{12}$$

$$M = 43.$$

Por lo tanto, el orden del árbol B (valor de M) que se emplea como índice sería 43.

(c) *¿Qué implica que el orden del árbol B sea mayor que en el caso del Ejercicio 1?*

Lo que implica que el orden del árbol B sea mayor que en el caso del Ejercicio 1 es que aumenta la eficiencia del árbol B, ya que se reduce su profundidad y, con ello, el costo en tiempo y recursos para buscar o modificar datos.

(d) *Describir el proceso para buscar el alumno con el DNI 12.345.678 usando el índice definido en este ejercicio.*

El proceso para buscar el alumno con el DNI 12.345.678 usando el índice definido en este ejercicio es:

- Se comienza en la raíz del árbol B.
- Se comparan las claves almacenadas en ese nodo con la clave buscada (DNI 12.345.678).
- Si la clave está en ese nodo, la búsqueda termina y:
 - Se recupera el puntero al registro correspondiente en el archivo de datos.
 - Usando el puntero recuperado, se lee, de manera directa, el registro de alumno en el archivo de datos.
- Si la clave no está en ese nodo:
 - Se determina en qué subárbol (hijo) continuar según el rango de claves.
 - Se lee el nodo hijo correspondiente.
- Se repite el proceso hasta:
 - Encontrar la clave (caso exitoso).
 - O llegar a un nodo hoja sin encontrarla (caso fallido).

(e) *¿Qué ocurre si desea buscar un alumno por su número de legajo? ¿Tiene sentido usar el índice que organiza el acceso al archivo de alumnos por DNI? ¿Cómo se haría para brindar acceso indizado al archivo de alumnos por número de legajo?*

Lo que ocurre si se desea buscar un alumno por un criterio diferente al criterio de ordenamiento (por ejemplo, por su número de legajo) es que no se puede aprovechar la estructura jerárquica del árbol, por lo que se debe realizar una búsqueda secuencial (lineal), es decir, recorrer todos los registros almacenados, sin ayuda de la estructura del árbol.

No, no tiene sentido usar el índice que organiza el acceso al archivo de alumnos por DNI, ya que este índice sólo ordena y permite búsquedas rápidas por DNI y, por lo tanto, no sirve para buscar por número de legajo.

Para brindar acceso indizado al archivo de alumnos por número de legajo, se debería crear un segundo índice al archivo de datos estructurado como un árbol B que ofrezca acceso indizado, ahora, por número de legajo de los alumnos.

(f) Suponer que se desea buscar los alumnos que tienen DNI en el rango entre 40.000.000 y 45.000.000. ¿Qué problemas tiene este tipo de búsquedas con apoyo de un árbol B que sólo provee acceso indizado por DNI al archivo de alumnos?

Los problemas que tiene este tipo de búsqueda con apoyo de un árbol B que sólo provee acceso indizado por DNI al archivo de alumnos son que no se puede hacer un recorrido secuencial entre nodos hoja fácilmente y, por lo tanto, la búsqueda por rango requiere búsquedas independientes por cada valor dentro del rango, lo cual puede volverse muy ineficiente si hay muchos valores en el rango.

Ejercicio 3.

Los árboles B^+ representan una mejora sobre los árboles B dado que conservan la propiedad de acceso indexado a los registros del archivo de datos por alguna clave, pero permiten, además, un recorrido secuencial rápido. Al igual que en el Ejercicio 2, considerar que, por un lado, se tiene el archivo que contiene la información de los alumnos de la Facultad de Informática (archivo de datos no ordenado) y, por otro lado, se tiene un índice al archivo de datos, pero, en este caso, el índice se estructura como un árbol B^+ que ofrece acceso indizado por DNI al archivo de alumnos. Resolver los siguientes incisos:

(a) ¿Cómo se organizan los elementos (claves) de un árbol B^+ ? ¿Qué elementos se encuentran en los nodos internos y que elementos se encuentran en los nodos hojas?

Los elementos (claves) de un árbol B^+ se organizan de forma jerárquica, con dos tipos principales de nodos:

- Los elementos que se encuentran en los nodos internos son las claves y los punteros a los hijos.
- Los elementos que se encuentran en los nodos hoja son las claves con los punteros a los registros correspondientes y el puntero al siguiente nodo hoja.

(b) ¿Qué característica distintiva presentan los nodos hojas de un árbol B^+ ? ¿Por qué?

La característica distintiva que presentan los nodos hojas de un árbol B^+ es que contienen todas las claves del árbol y están enlazados secuencialmente, lo cual hace a este tipo de árbol ideal para búsquedas exactas y por rango.

(c) Definir, en Pascal, las estructuras de datos correspondientes para el archivo de alumnos y su índice (árbol B^+). Por simplicidad, suponer que todos los nodos del árbol B^+ (nodos internos y nodos hojas) tienen el mismo tamaño.

```
program TP4_E3;
const
  M=10;
type
  t_registro_alumno=record
    nombre: string;
    apellido: string;
    dni: int32;
    legajo: int16;
    anio_ingreso: int16;
  end;
  t_lista=^t_registro_nodo;
  t_registro_nodo=record
    es_hoja: boolean;
    cantidad_claves: int16;
    claves: array[0..M-1] of int32;
    enlaces: array[0..M-1] of int16; // si es_hoja=true
```

```

hijos: array[0..M] of int16; // si es_hoja=false
sig: t_lista; // sig: int16; // si es_hoja=true
end;
t_archivo_datos=file of t_registro_alumno;
t_archivo_indice=file of t_registro_nodo;
var
  archivo_datos: t_archivo_datos;
  archivo_indice: t_archivo_indice;
begin
end.

```

(d) Describir el proceso de búsqueda de un alumno con un DNI específico haciendo uso de la estructura auxiliar (índice) que se organiza como un árbol B+. ¿Qué diferencia se encuentra respecto a la búsqueda en un índice estructurado como un árbol B?

El proceso de búsqueda de un alumno con un DNI específico haciendo uso de la estructura auxiliar (índice) que se organiza como un árbol B+ es:

- Se comienza en la raíz del árbol B+.
- Se comparan las claves almacenadas en ese nodo con la clave buscada (DNI específico).
- Se elige el puntero al hijo adecuado según el rango en el que cae el DNI buscado.
- Se repite este proceso hasta llegar a un nodo hoja.
- Si la clave está en el nodo hoja (caso exitoso), la búsqueda termina y:
 - Se recupera el puntero al registro correspondiente en el archivo de datos.
 - Usando el puntero recuperado, se lee, de manera directa, el registro de alumno en el archivo de datos.
- Si la clave no está en el nodo hoja (caso fallido), la búsqueda también termina.

(e) Explicar el proceso de búsqueda de los alumnos que tienen DNI en el rango entre 40.000.000 y 45.000.000, apoyando la búsqueda en un índice organizado como un árbol B+. ¿Qué ventajas se encuentran respecto a este tipo de búsquedas en un árbol B?

El proceso de búsqueda de los alumnos que tienen DNI en el rango entre 40.000.000 y 45.000.000, apoyando la búsqueda en un índice organizado como un árbol B+ es:

- Se comienza en la raíz del árbol B+.
- Se comparan las claves almacenadas en ese nodo con la clave mínima del rango (DNI 40.000.000).
- Se elige el puntero al hijo adecuado según el rango en el que cae el DNI mínimo.
- Se repite este proceso hasta llegar a un nodo hoja.
- Se busca la primera clave mayor o igual al DNI mínimo y menor o igual al DNI máximo (clave máxima del rango, es decir, 45.000.000).
- Si se encuentra una clave en este rango en el nodo hoja:
 - Se recupera el puntero al registro correspondiente en el archivo de datos.
 - Se siguen leyendo claves y recuperando los punteros hasta que el DNI sea mayor al DNI máximo, que es cuando la búsqueda termina.

- Usando los punteros recuperados, se leen, de manera directa, los registros de alumno en el archivo de datos.
- Si no se encuentra una clave en este rango en el nodo hoja, la búsqueda termina.

Las ventajas que se encuentran respecto a este tipo de búsquedas en un árbol B son que se puede hacer un recorrido secuencial entre nodos hoja fácilmente y, por lo tanto, la búsqueda por rango no requiere búsquedas independientes por cada valor dentro del rango, lo cual es muy eficiente si hay muchos valores en el rango.

Ejercicio 4.

Dado el siguiente algoritmo de búsqueda en un árbol B:

```

procedure buscar(NRR, clave, NRR_encontrado, pos_encontrada, resultado);
var
  clave_encontrada: boolean;
begin
  if (nodo=null)
    resultado:=false
  else
    begin
      posicionarYLeerNodo(A,nodo,NRR);
      claveEncontrada(A,nodo,clave,pos,clave_encontrada);
      if (clave_encontrada) then
        begin
          NRR_encontrado:=NRR;
          pos_encontrada:=pos;
          resultado:=true;
        end
      else
        buscar(nodo.hijos[pos],clave,NRR_encontrado,pos_encontrada,resultado)
      end;
    end;
end;

```

Asumir que el archivo se encuentra abierto y que, para la primera llamada, el parámetro NRR contiene la posición de la raíz del árbol. Responder detalladamente:

(a) *PosicionarYLeerNodo(): Indicar qué hace y la forma en que deben ser enviados los parámetros (valor o referencia). Implementar este módulo en Pascal.*

PosicionarYLeerNodo() es un procedimiento que permite acceder, directamente, a un nodo del árbol B en el archivo, usando su número lógico (NRR), y traerlo a memoria para ser procesado.

Las variables *A* y *nodo* deben ser pasadas por referencia, mientras que *NRR* por valor.

```

procedure PosicionarYLeerNodo(NRR: int16; var A: t_archivo_datos; var nodo: t_registro_nodo);
begin
  seek(A,NRR);
  read(A,nodo);
end;

```

(b) *claveEncontrada(): Indicar qué hace y la forma en que deben ser enviados los parámetros (valor o referencia). ¿Cómo se implementaría?*

claveEncontrada() es un procedimiento que compara la clave buscada con las claves del nodo, devuelve si fue encontrada y en qué posición, o a qué nodo hijo debe seguir la búsqueda si no está.

Las variables *pos* y *clave_encontrada* deben ser pasadas por referencia, mientras que *nodo* y *clave* por valor.

```

procedure claveEncontrada(nodo: t_registro_nodo; clave: int16; var pos: int16; var
clave_encontrada: boolean);
var
  i: int16;
begin
  i:=1;
  while ((i<=nodo.cantClaves) and (clave>nodo.claves[i].id)) do
    i:=i+1;
  if ((i<=nodo.cantClaves) and (clave=nodo.claves[i].id)) then
  begin
    clave_encontrada:=true;
    pos:=i;
  end
  else
  begin
    clave_encontrada:=false;
    pos:=i;
  end;
end;
end;

```

(c) ¿Existe algún error en este código? En caso afirmativo, modificar lo que se considere necesario.

Sí, existen errores en este código.

```

procedure buscar(NRR, clave: int16; var A: t_archivo_datos; var NRR_encontrado,
pos_encontrada: int16; var resultado: boolean);
var
  nodo: t_registro_nodo;
  pos: int16;
  clave_encontrada: boolean;
begin
  if (NRR=-1) then
    resultado:=false
  else
  begin
    posicionarYLeerNodo(NRR,A,nodo);
    claveEncontrada(nodo,clave,pos,clave_encontrada);
    if (clave_encontrada) then
    begin
      NRR_encontrado:=NRR;
      pos_encontrada:=pos;
      resultado:=true;
    end
    else
    begin
      buscar(nodo.hijos[pos],clave,A,NRR_encontrado,pos_encontrada,resultado)
    end;
  end;
end;
end;

```

(d) ¿Qué cambios son necesarios en el procedimiento de búsqueda implementado sobre un árbol B para que funcione en un árbol B+?

El cambio necesario en el procedimiento de búsqueda implementado sobre un árbol B para que funcione en un árbol B+ es que debería detectar si el nodo es hoja, ya que debería

terminar de buscar sólo al llegar a un nodo hoja porque sólo en estos es que están los punteros a los datos reales.

Ejercicio 5.

Definir los siguientes conceptos:

- *Overflow.*
- *Underflow.*
- *Redistribución.*
- *Fusión o concatenación.*

En los dos últimos casos, ¿cuándo se aplica cada uno?

Overflow: Se produce cuando se quiere agregar una clave a un nodo que ya tiene la cantidad máxima de claves permitidas.

Underflow: Se produce cuando se quiere eliminar una clave de un nodo que ya tiene la cantidad mínima de claves permitidas.

Redistribución: Se aplica cuando un nodo tiene *underflow* y es posible trasladar una llave de un nodo hermano adyacente a este nodo para que el *underflow* deje de ocurrir.

Fusión o concatenación: Se aplica cuando un nodo tiene *underflow* y un nodo adyacente hermano está al mínimo y no se puede redistribuir, por lo que se fusiona con un nodo adyacente disminuyendo la cantidad de nodos y, en algunos casos, la altura del árbol.

Ejercicio 6.

Suponer que se tiene un archivo que contiene información de los empleados de una empresa. De cada empleado, se mantiene la siguiente información: DNI, legajo, nombre completo y salario. Considerar que se mantiene, además, un índice, organizado como árbol B de orden 4, que provee acceso indizado a los empleados por su DNI. Graficar cómo queda el archivo de empleados (archivo de datos) y el archivo índice (árbol B) tras la inserción de los siguientes registros:

DNI	Legajo	Nombre y apellido	Salario
35.000.000	100	Juan Pérez	\$250.000
40.000.000	101	Lucio Redivo	\$400.000
32.000.000	102	Nicolás Lapro	\$720.000
28.000.000	103	Luis Scola	\$2.000.000
26.000.000	104	Andrés Nocioni	\$1.500.000
37.000.000	105	Facundo Campazzo	\$1.200.000
25.000.000	106	Emanuel Ginobili	\$5.000.000
23.000.000	107	Pepe Sánchez	\$1.000.000
21.000.000	108	Alejandro Montecchia	\$800.000
36.000.000	109	Marcos Delia	\$300.000
45.000.000	110	Leandro Bolmaro	\$600.000

Notas:

- Graficar los estados de ambos archivos (datos e índice) cuando ocurren cambios relevantes en el índice como la creación de un nuevo nodo.
- Además de graficar los archivos con sus respectivas estructuras internas, resulta útil que se grafique la vista del archivo índice como un árbol B.

Inserción de las claves 35.000.000, 40.000.000 y 32.000.000:

Archivo de datos: [35, 40, 32].

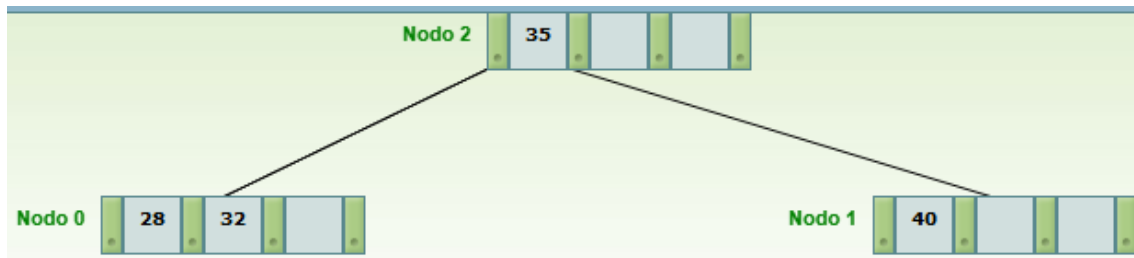
Archivo índice:



Inserción de la clave 28.000.000:

Archivo de datos: [35, 40, 32, 28].

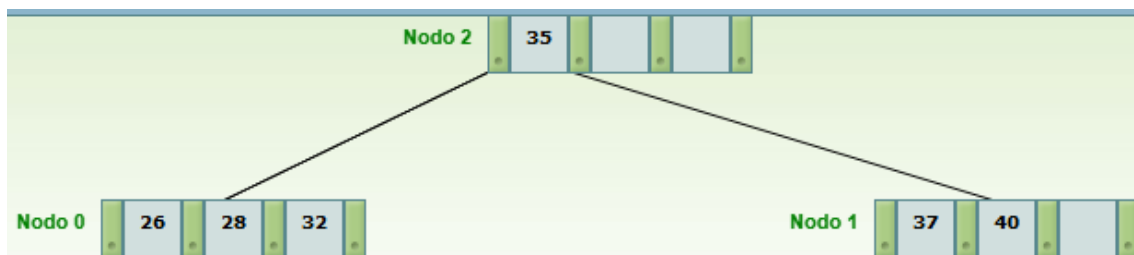
Archivo índice:



Inserción de las claves 26.000.000 y 37.000.000:

Archivo de datos: [35, 40, 32, 28, 26, 37].

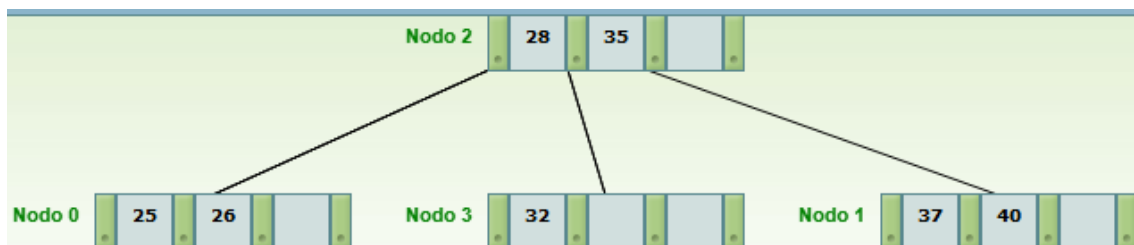
Archivo índice:



Inserción de la clave 25.000.000:

Archivo de datos: [35, 40, 32, 28, 26, 37, 25].

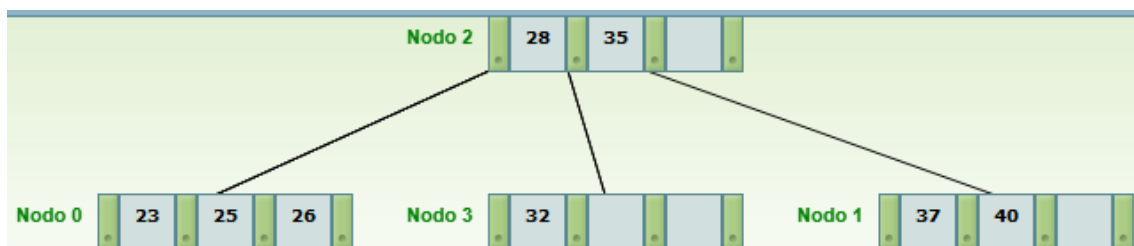
Archivo índice:



Inserción de la clave 23.000.000:

Archivo de datos: [35, 40, 32, 28, 26, 37, 25, 23].

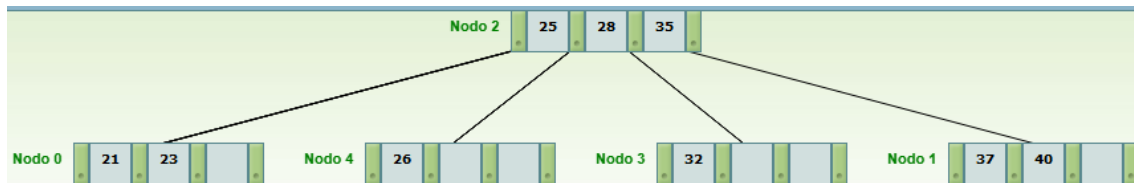
Archivo índice:



Inserción de la clave 21.000.000:

Archivo de datos: [35, 40, 32, 28, 26, 37, 25, 23, 21].

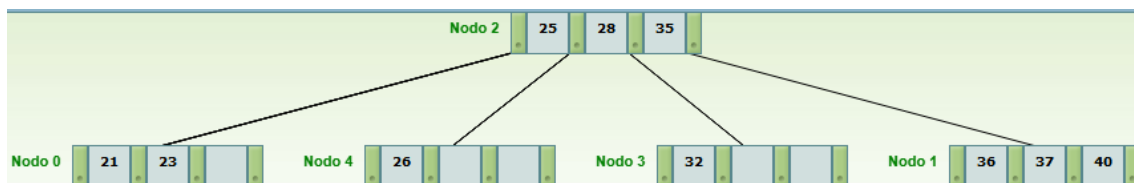
Archivo índice:



Inserción de la clave 36.000.000:

Archivo de datos: [35, 40, 32, 28, 26, 37, 25, 23, 21, 36].

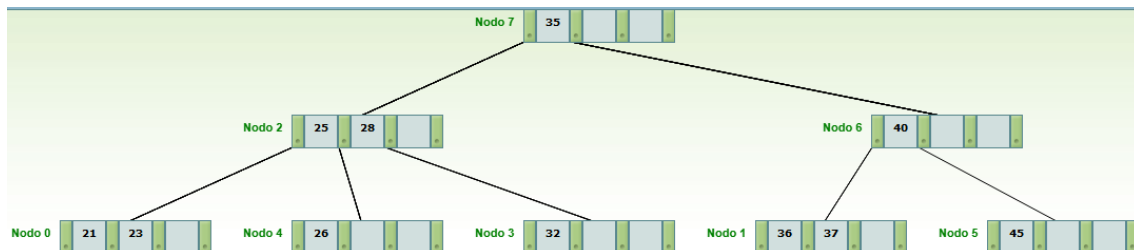
Archivo índice:



Inserción de la clave 45.000.000:

Archivo de datos: [35, 40, 32, 28, 26, 37, 25, 23, 21, 36, 45].

Archivo índice:



PARTE II: Operaciones en Árboles B y B+.

Para los siguientes ejercicios, se debe:

- Indicar los nodos leídos y escritos en cada operación.
- Todas las operaciones deben estar, claramente, justificadas, enunciando las mismas, indefectiblemente, tal cual se presenta en la materia.
- Los números de nodo deben asignarse en forma coherente con el crecimiento del archivo. La reutilización de nodos libres se debe efectuar con política LIFO (último en entrar, primero en salir).
- Para los siguientes ejercicios, sólo interesa graficar los estados del árbol B que representa el índice (no es necesario dibujar la estructura interna de los archivos como si se solicitó en el Ejercicio 6).

Ejercicio 7.

Dado el siguiente árbol B de orden 5, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +320, -390, -400, -533. Justificar, detalladamente, cada operación indicando lecturas y escrituras en orden de ocurrencia. Para la resolución de underflow, se debe utilizar política izquierda. Graficar cada operación por separado.

2: 0 (220) 1 (390) 4 (455) 5 (541) 3

0: (10)(150) 1: (225)(241)(331)(360) 4: (400)(407) 5: (508)(533) 3: (690)(823)

Operación +320:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de la clave 320.
- Se produce *overflow* en el nodo hoja 2, división del mismo y promoción de la clave 390.
- Operaciones: L2, L1, E1, E6, E2, E7, E8.

8: 2 (390) 7

2: 0 (220) 1 (320) 6 7: 4 (455) 5 (541) 3

0: (10)(150) 1: (225)(241) 6: (331)(360) 4: (400)(407) 5: (508)(533) 3: (690)(823)

Operación -390:

- Se reemplaza la clave 390 por la menor clave del subárbol derecho (400).
- No se produce *underflow* en el nodo hoja 4.
- Operaciones: L8, L7, L4, E4, E8.

8: 2 (400) 7

2: 0 (220) 1 (320) 6 7: 4 (455) 5 (541) 3

0: (10)(150) 1: (225)(241) 6: (331)(360) 4: (407) 5: (508)(533) 3: (690)(823)

Operación -400:

- Se reemplaza la clave 400 por la menor clave del subárbol derecho (407).
- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente derecho, el nodo hoja 5, ya que se trata de un nodo hoja de un extremo y no tiene hermano adyacente izquierdo.
- Operaciones: L8, L7, L4, L5, E4, E5, E7, E8.

8: 2 (407) 7

2: 0 (220) 1 (320) 6 **7:** 4 (508) 5 (541) 3

0: (10)(150) **1:** (225)(241) **6:** (331)(360) **4:** (455) **5:** (533) **3:** (690)(823)

Operación -533:

- Se produce *underflow* en el nodo hoja 5. Hay que redistribuir siguiendo la política izquierda, pero el hermano adyacente izquierdo ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente izquierdo, el nodo hoja 4.
- Operaciones: L8, L7, L5, L4, E4, E7.

8: 2 (407) 7

2: 0 (220) 1 (320) 6 **7:** 4 (541) 3

0: (10)(150) **1:** (225)(241) **6:** (331)(360) **4:** (455)(508) **3:** (690)(823)

Ejercicio 8.

Dado el siguiente árbol B de orden 4, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +5, +9, +80, +51, -50, -92. Política de resolución de underflows: derecha.

2: 0 (66) 1

0: (22)(32)(50) 1: (77)(79)(92)

Operación +5:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de la clave 32.
- Operaciones: L2, L0, E0, E3, E2.

2: 0 (32) 3 (66) 1

0: (5)(22) 3: (50) 1: (77)(79)(92)

Operación +9:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (32) 3 (66) 1

0: (5)(9)(22) 3: (50) 1: (77)(79)(92)

Operación +80:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de la clave 80.
- Operaciones: L2, L1, E1, E4, E2.

2: 0 (32) 3 (66) 1 (80) 4

0: (5)(9)(22) 3: (50) 1: (77)(79) 4: (92)

Operación +51:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (32) 3 (66) 1 (80) 4

0: (5)(9)(22) 3: (50)(51) 1: (77)(79) 4: (92)

Operación -50:

- No se produce *underflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (32) 3 (66) 1 (80) 4

0: (5)(9)(22) **3:** (51) **1:** (77)(79) **4:** (92)

Operación -92:

- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 1, ya que se trata de un nodo hoja de un extremo y no tiene hermano adyacente derecho.
- Operaciones: L2, L4, L1, E1, E4, E2.

2: 0 (32) 3 (66) 1 (79) 4

0: (5)(9)(22) **3:** (51) **1:** (77) **4:** (80)

Ejercicio 9.

Dado el siguiente árbol B de orden 6, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +15, +71, +3, +48, -56, -71. Política de resolución de underflows: derecha o izquierda.

0: (34)(56)(78)(100)(176)

Operación +15:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de la clave 78.
- Operaciones: L0, E0, E1, E2.

2: 0 (78) 1

0: (15)(34)(56) 1: (100)(176)

Operación +71:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (78) 1

0: (15)(34)(56)(71) 1: (100)(176)

Operación +3:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (78) 1

0: (3)(15)(34)(56)(71) 1: (100)(176)

Operación +48:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de la clave 48.
- Operaciones: L2, L0, E0, E3, E2.

2: 0 (48) 3 (78) 1

0: (3)(15)(34) 3: (56)(71) 1: (100)(176)

Operación -56:

- Se produce *underflow* en el nodo hoja 3.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 0, ya que el hermano adyacente derecho, el nodo hoja 1, ya contiene la cantidad mínima de claves permitidas (2).

- Operaciones: L2, L3, L1, L0, E0, E3, E2.

2: 0 (34) 3 (78) 1

0: (3)(15) **3:** (48)(71) **1:** (100)(176)

Operación -71:

- Se produce *underflow* en el nodo hoja 3. Hay que redistribuir siguiendo la política derecha o izquierda, pero ambos nodos hoja (1 y 0) ya contienen la cantidad mínima de claves permitidas (2).
- Se fusiona con el hermano adyacente derecho, el nodo hoja 1.
- Operaciones: L2, L3, L1, L0, E3, E2.

2: 0 (34) 3

0: (3)(15) **3:** (48)(78)(100)(176)

Ejercicio 10.

Dado el siguiente árbol B de orden 5, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +450, -485, -511, -614. Política de resolución de underflows: derecha.

2: 0 (315) 1 (485) 4 (547) 5 (639) 3

0: (148)(223) 1: (333)(390)(442)(454) 4: (508)(511) 5: (614)(633) 3: (789)(915)

Operación +450:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de la clave 442.
- Se produce *overflow* en el nodo hoja 2, división del mismo y promoción de la clave 485.
- Operaciones: L2, L1, E1, E6, E2, E7, E8.

8: 2 (485) 7

2: 0 (315) 1 (442) 6 7: 4 (547) 5 (639) 3

0: (148)(223) 1: (333)(390) 6: (450)(454) 4: (508)(511) 5: (614)(633) 3: (789)(915)

Operación -485:

- Se reemplaza la clave 485 por la menor clave del subárbol derecho (508).
- No se produce *underflow* en el nodo hoja 4.
- Operaciones: L8, L7, L4, E4, E8.

8: 2 (508) 7

2: 0 (315) 1 (442) 6 7: 4 (547) 5 (639) 3

0: (148)(223) 1: (333)(390) 6: (450)(454) 4: (511) 5: (614)(633) 3: (789)(915)

Operación -511:

- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente derecho, el nodo hoja 5.
- Operaciones: L8, L7, L4, L5, E4, E5, E7.

8: 2 (508) 7

2: 0 (315) 1 (442) 6 7: 4 (614) 5 (639) 3

0: (148)(223) 1: (333)(390) 6: (450)(454) 4: (547) 5: (633) 3: (789)(915)

Operación -614:

- Se reemplaza la clave 614 por la menor clave del subárbol derecho (633).
- Se produce *underflow* en el nodo hoja 5.
- Se redistribuye con el hermano adyacente derecho, el nodo hoja 3.
- Operaciones: L8, L7, L5, L3, E5, E3, E7.

8: 2 (508) 7

2: 0 (315) 1 (442) 6 **7:** 4 (633) 5 (789) 3

0: (148)(223) **1:** (333)(390) **6:** (450)(454) **4:** (547) **5:** (639) **3:** (915)

Ejercicio 11.

Dado un árbol *B* de orden 5 y con política izquierda, para cada operación dada:

- Dibujar el árbol resultante.
- Explicar las decisiones tomadas.
- Escribir las lecturas y escrituras.

Operaciones: -76, -400, +900, +12.

Nodo 2: 3 i 0(76)4(300)1(600)3

Nodo 0: 4 h(21)(45)(46)(70)

Nodo 4: 2 h(100)(200)

Nodo 1: 1 h(400)

Nodo 3: 2 h(700)(800)

2: 0 (76) 4 (300) 1 (600) 3

0: (21)(45)(46)(70) 4: (100)(200) 1: (400) 3: (700)(800)

Operación -76:

- Se reemplaza la clave -76 por la menor clave del subárbol derecho (100).
- No se produce *underflow* en el nodo hoja 4.
- Operaciones: L2, L4, E4, E2.

2: 0 (100) 4 (300) 1 (600) 3

0: (21)(45)(46)(70) 4: (200) 1: (400) 3: (700)(800)

Operación -400:

- Se produce *underflow* en el nodo hoja 1. Hay que redistribuir siguiendo la política izquierdo, pero el hermano adyacente izquierdo, el nodo hoja 4, ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente izquierdo, el nodo hoja 4.
- Operaciones: L2, L1, L4, E4, E2.

2: 0 (100) 4 (600) 3

0: (21)(45)(46)(70) 4: (200)(300) 3: (700)(800)

Operación +900:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (100) 4 (600) 3

0: (21)(45)(46)(70) 4: (200)(300) 3: (700)(800)(900)

Operación +12:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de la clave 45.
- Operaciones: L2, L0, E0, E5, E2.

2: 0 (45) 5 (100) 4 (600) 3

0: (12)(21) **5:** (46)(70) **4:** (200)(300) **3:** (700)(800)(900)

Ejercicio 12.

Dadas las siguientes operaciones, mostrar la construcción, paso a paso, de un árbol B de orden 4: +50, +70, +40, +15, +90, +120, +115, +45, +30, +100, +112, +77, -45, -40, -50, -90, -100. Política de resolución de underflows: izquierda o derecha.

Operación +50:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (50)

Operación +70:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (50)(70)

Operación +40:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (40)(50)(70)

Operación +15:

- Se produce *overflow* en el nodo 0, división del mismo y promoción de la clave 50.
- Operaciones: L0, E0, E1, E2.

2: 0 (50) 1

0: (15)(40) **1:** (70)

Operación +90:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (50) 1

0: (15)(40) **1:** (70)(90)

Operación +120:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (50) 1

0: (15)(40) 1: (70)(90)(120)

Operación +115:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de la clave 115.
- Operaciones: L2, L1, E1, E3, E2.

2: 0 (50) 1 (115) 3

0: (15)(40) 1: (70)(90) 3: (120)

Operación +45:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (50) 1 (115) 3

0: (15)(40)(45) 1: (70)(90) 3: (120)

Operación +30:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de la clave 40.
- Operaciones: L2, L0, E0, E4, E2.

2: 0 (40) 4 (50) 1 (115) 3

0: (15)(30) 4: (45) 1: (70)(90) 3: (120)

Operación +100:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (40) 4 (50) 1 (115) 3

0: (15)(30) 4: (45) 1: (70)(90)(100) 3: (120)

Operación +112:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de la clave 100.
- Se produce *overflow* en el nodo 2, división del mismo y promoción de la clave 100.
- Operaciones: L2, L1, E1, E5, E2, E6, E7.

7: 2 (100) 6

2: 0 (40) 4 (50) 1 6: 5 (115) 3

0: (15)(30) 4: (45) 1: (70)(90) 5: (112) 3: (120)

Operación +77:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L7, L2, L1, E1.

7: 2 (100) 6

2: 0 (40) 4 (50) 1 6: 5 (115) 3

0: (15)(30) 4: (45) 1: (70)(77)(90) 5: (112) 3: (120)

Operación -45:

- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 0.
- Operaciones: L7, L2, L4, L0, E0, E4, E2.

7: 2 (100) 6

2: 0 (30) 4 (50) 1 6: 5 (115) 3

0: (15) 4: (40) 1: (70)(77)(90) 5: (112) 3: (120)

Operación -40:

- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente derecho, el nodo hoja 1, ya que el hermano adyacente izquierdo, el nodo hoja 0, ya contiene la cantidad mínima de claves permitidas (1).
- Operaciones: L7, L2, L4, L0, L1, E4, E1, E2.

7: 2 (100) 6

2: 0 (30) 4 (70) 1 6: 5 (115) 3

0: (15) 4: (50) 1: (77)(90) 5: (112) 3: (120)

Operación -50:

- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente derecho, el nodo hoja 1, ya que el hermano adyacente izquierdo, el nodo hoja 0, ya contiene la cantidad mínima de claves permitidas (1).
- Operaciones: L7, L2, L4, L0, L1, E4, E1, E2.

7: 2 (100) 6

2: 0 (30) 4 (77) 1 6: 5 (115) 3

0: (15) 4: (70) 1: (90) 5: (112) 3: (120)

Operación -90:

- Se produce *underflow* en el nodo hoja 1. Hay que redistribuir siguiendo la política izquierda o derecha, pero el hermano adyacente izquierdo, el nodo hoja 4, ya contiene la cantidad mínima de claves permitidas (1) y no tiene hermano adyacente derecho.
- Se fusiona con el hermano adyacente izquierdo, el nodo hoja 4.
- Operaciones: L7, L2, L1, L4, E4, E2.

7: 2 (100) 6

2: 0 (30) 4 6: 5 (115) 3

0: (15) 4: (70)(77) 5: (112) 3: (120)

Operación -100:

- Se reemplaza la clave 100 por la menor clave del subárbol derecho (112).
- Se produce *underflow* en el nodo hoja 5. Hay que redistribuir siguiendo la política izquierda o derecha, pero no tiene hermano adyacente izquierdo y el hermano adyacente derecho, el nodo hoja 3, ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente derecho, el nodo hoja 3, ya que no tiene hermano adyacente izquierdo.
- Se produce *underflow* en el nodo 6. Hay que redistribuir siguiendo la política izquierda o derecha, pero el hermano adyacente izquierdo, el nodo hoja 3, ya contiene la cantidad mínima de claves permitidas (1) y no tiene hermano adyacente derecho.
- Se fusiona con el hermano adyacente izquierdo, el nodo 2.
- Operaciones: L7, L6, L5, L3, L2, E5, E2.

2: 0 (30) 4 (112) 5

0: (15) 4: (70)(77) 5: (115)(120)

Ejercicio 13.

Dadas las siguientes operaciones, mostrar la construcción, paso a paso, de un árbol B de orden 5: +80, +50, +70, +120, +23, +52, +59, +65, +30, +40, +45, +31, +34, +38, +60, +63, +64, -23, -30, -31, -40, -45, -38. Política de resolución de underflows: izquierda.

Operación +80:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (80)

Operación +50:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (50)(80)

Operación +70:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (50)(70)(80)

Operación +120:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (50)(70)(80)(120)

Operación +23:

- Se produce *overflow* en el nodo 0, división del mismo y promoción de la clave 70.
- Operaciones: L0, E0, E1, E2.

2: 0 (70) 1

0: (23)(50) **1:** (80)(120)

Operación +52:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (70) 1

0: (23)(50)(52) 1: (80)(120)

Operación +59:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (70) 1

0: (23)(50)(52)(59) 1: (80)(120)

Operación +65:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de la clave 52.
- Operaciones: L2, L0, E0, E3, E2.

2: 0 (52) 3 (70) 1

0: (23)(50) 3: (59)(65) 1: (80)(120)

Operación +30:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (52) 3 (70) 1

0: (23)(30)(50) 3: (59)(65) 1: (80)(120)

Operación +40:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (52) 3 (70) 1

0: (23)(30)(40)(50) 3: (59)(65) 1: (80)(120)

Operación +45:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de la clave 40.
- Operaciones: L2, L0, E0, E4, E2.

2: 0 (40) 4 (52) 3 (70) 1

0: (23)(30) 4: (45)(50) 3: (59)(65) 1: (80)(120)

Operación +31:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (40) 4 (52) 3 (70) 1

0: (23)(30)(31) 4: (45)(50) 3: (59)(65) 1: (80)(120)

Operación +34:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (40) 4 (52) 3 (70) 1

0: (23)(30)(31)(34) 4: (45)(50) 3: (59)(65) 1: (80)(120)

Operación +38:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de la clave 31.
- Operaciones: L2, L0, E0, E5, E2.

2: 0 (31) 5 (40) 4 (52) 3 (70) 1

0: (23)(30) 5: (34)(38) 4: (45)(50) 3: (59)(65) 1: (80)(120)

Operación +60:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (31) 5 (40) 4 (52) 3 (70) 1

0: (23)(30) 5: (34)(38) 4: (45)(50) 3: (59)(60)(65) 1: (80)(120)

Operación +63:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (31) 5 (40) 4 (52) 3 (70) 1

0: (23)(30) 5: (34)(38) 4: (45)(50) 3: (59)(60)(63)(65) 1: (80)(120)

Operación +64:

- Se produce *overflow* en el nodo hoja 3, división del mismo y promoción de la clave 63.
- Se produce *overflow* en el nodo 2, división del mismo y promoción de la clave 52.
- Operaciones: L2, L3, E3, E6, E2, E7, E8.

8: 2 (52) 7

2: 0 (31) 5 (40) 4 7: 3 (63) 6 (70) 1

0: (23)(30) 5: (34)(38) 4: (45)(50) 3: (59)(60) 6: (64)(65) 1: (80)(120)

Operación -23:

- No se produce *underflow* en el nodo hoja 0.
- Operaciones: L8, L2, L0, E0.

8: 2 (52) 7

2: 0 (31) 5 (40) 4 7: 3 (63) 6 (70) 1

0: (30) 5: (34)(38) 4: (45)(50) 3: (59)(60) 6: (64)(65) 1: (80)(120)

Operación -30:

- Se produce *underflow* en el nodo hoja 0.
- Se redistribuye con el hermano adyacente derecho, el nodo hoja 5, ya que se trata de un nodo hoja de un extremo y no tiene hermano adyacente izquierdo.
- Operaciones: L8, L2, L0, E0, E5, E2.

8: 2 (52) 7

2: 0 (34) 5 (40) 4 7: 3 (63) 6 (70) 1

0: (31) 5: (38) 4: (45)(50) 3: (59)(60) 6: (64)(65) 1: (80)(120)

Operación -31:

- Se produce *underflow* en el nodo hoja 0. Hay que redistribuir siguiendo la política izquierda, pero no tiene hermano adyacente izquierdo y el hermano adyacente derecho, el nodo hoja 5, ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente derecho, el nodo hoja 5, ya que no tiene hermano adyacente izquierdo.
- Operaciones: L8, L2, L0, L5, E0, E2.

8: 2 (52) 7

2: 0 (40) 4 7: 3 (63) 6 (70) 1

0: (34)(38) 4: (45)(50) 3: (59)(60) 6: (64)(65) 1: (80)(120)

Operación -40:

- Se reemplaza la clave 40 por la menor clave del subárbol derecho (45).
- No se produce *underflow* en el nodo hoja 4.
- Operaciones: L8, L2, L4, E4, E2.

8: 2 (52) 7

2: 0 (45) 4 7: 3 (63) 6 (70) 1

0: (34)(38) 4: (50) 3: (59)(60) 6: (64)(65) 1: (80)(120)

Operación -45:

- Se reemplaza la clave 45 por la menor clave del subárbol derecho (50).
- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 0.
- Operaciones: L8, L2, L4, L0, E0, E4, E2.

8: 2 (52) 7

2: 0 (38) **4** **7:** 3 (63) **6** (70) **1**

0: (34) **4:** (50) **3:** (59)(60) **6:** (64)(65) **1:** (80)(120)

Operación -38:

- Se reemplaza la clave 38 por la menor clave del subárbol derecho (50).
- Se produce *underflow* en el nodo hoja 4. Hay que redistribuir siguiendo la política izquierda, pero el hermano adyacente izquierdo, el nodo hoja 0, ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente izquierdo, el nodo hoja 0.
- Se produce *underflow* en el nodo 2.
- Se redistribuye con el hermano adyacente derecho, el nodo 7, ya que no tiene hermano adyacente izquierdo.
- Operaciones: L8, L2, L4, L0, L7, E0, E2, E7, E8.

8: 2 (63) **7**

2: 0 (52) **3** **7:** 6 (70) **1**

0: (34)(50) **3:** (59)(60) **6:** (64)(65) **1:** (80)(120)

Ejercicio 14.

Dado el siguiente árbol B de orden 6, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +300, +577, -586, -570, -380, -460. Política de resolución de underflows: izquierda o derecha.

2: 0 (216) 1 (460) 4 (570) 5 (689) 3 (777) 6
 0: (100)(159)(171) 1: (222)(256)(358)(380)(423) 4: (505)(522)
 5: (586)(599)(615)(623)(680) 3: (703)(725) 6: (789)(915)(1000)

Operación +300:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de la clave 358.
- Se produce *overflow* en el nodo hoja 2, división del mismo y promoción de la clave 570.
- Operaciones: L2, L1, E1, E7, E2, E8, E9.

9: 2 (570) 8
 2: 0 (216) 1 (358) 7 (460) 4 8: 5 (689) 3 (777) 6
 0: (100)(159)(171) 1: (222)(256)(300) 7: (380)(423) 4: (505)(522)
 5: (586)(599)(615)(623)(680) 3: (703)(725) 6: (789)(915)(1000)

Operación +577:

- No se produce *overflow* en el nodo hoja 4.
- Operaciones: L9, L8, L4, E4.

9: 2 (570) 8
 2: 0 (216) 1 (358) 7 (460) 4 8: 5 (689) 3 (777) 6
 0: (100)(159)(171) 1: (222)(256)(300) 7: (380)(423) 4: (505)(522)(577)
 5: (586)(599)(615)(623)(680) 3: (703)(725) 6: (789)(915)(1000)

Operación -586:

- No se produce *underflow* en el nodo hoja 5.
- Operaciones: L9, L8, L5, E5.

9: 2 (570) 8
 2: 0 (216) 1 (358) 7 (460) 4 8: 5 (689) 3 (777) 6
 0: (100)(159)(171) 1: (222)(256)(300) 7: (380)(423) 4: (505)(522)(577)
 5: (599)(615)(623)(680) 3: (703)(725) 6: (789)(915)(1000)

Operación -570:

- Se reemplaza la clave 570 por la menor clave del subárbol derecho (599).
- No se produce *underflow* en el nodo hoja 5.
- Operaciones: L9, L2, L4, E4, E9.

9: 2 (599) 8

2: 0 (216) 1 (358) 7 (460) 4 8: 5 (689) 3 (777) 6

0: (100)(159)(171) 1: (222)(256)(300) 7: (380)(423) 4: (505)(522)(577)

5: (615)(623)(680) 3: (703)(725) 6: (789)(915)(1000)

Operación -380:

- Se produce *underflow* en el nodo hoja 7.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 1.
- Operaciones: L9, L2, L7, L1, E1, E7, E2.

9: 2 (599) 8

2: 0 (216) 1 (300) 7 (460) 4 8: 5 (689) 3 (777) 6

0: (100)(159)(171) 1: (222)(256) 7: (358)(423) 4: (505)(522)(577)

5: (615)(623)(680) 3: (703)(725) 6: (789)(915)(1000)

Operación -460:

- Se reemplaza la clave 460 por la menor clave del subárbol derecho (505).
- No se produce *underflow* en el nodo hoja 4.
- Operaciones: L9, L2, L4, E4, E2.

9: 2 (599) 8

2: 0 (216) 1 (300) 7 (505) 4 8: 5 (689) 3 (777) 6

0: (100)(159)(171) 1: (222)(256) 7: (358)(423) 4: (522)(577)

5: (615)(623)(680) 3: (703)(725) 6: (789)(915)(1000)

Ejercicio 15.

Dada las siguientes operaciones, mostrar cómo se construye el árbol B de orden 4: +65, +89, +23, +45, +20, +96, +10, +55, -23, +110, +50, -10, +25, -50, -45, +120, +130, +70, +75, +73, +100, -120, -110. Política de resolución de underflows: derecha.

Operación +65:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (65)

Operación +89:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (65)(89)

Operación +23:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (23)(65)(89)

Operación +45:

- Se produce *overflow* en el nodo 0, división del mismo y promoción de la clave 65.
- Operaciones: L0, E0, E1, E2.

2: 0 (65) 1

0: (23)(45) **1:** (89)

Operación +20:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (65) 1

0: (20)(23)(45) **1:** (89)

Operación +96:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (65) 1

0: (20)(23)(45) 1: (89)(96)

Operación +10:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de la clave 23.
- Operaciones: L2, L0, E0, E3, E2.

2: 0 (23) 3 (65) 1

0: (10)(20) 3: (45) 1: (89)(96)

Operación +55:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (23) 3 (65) 1

0: (10)(20) 3: (45)(55) 1: (89)(96)

Operación -23:

- Se reemplaza la clave 23 por la menor clave del subárbol derecho (45).
- No se produce *underflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3, E2.

2: 0 (45) 3 (65) 1

0: (10)(20) 3: (55) 1: (89)(96)

Operación +110:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (45) 3 (65) 1

0: (10)(20) 3: (55) 1: (89)(96)(110)

Operación +50:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (45) 3 (65) 1

0: (10)(20) 3: (50)(55) 1: (89)(96)(110)

Operación -10:

- No se produce *underflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (45) 3 (65) 1

0: (20) 3: (50)(55) 1: (89)(96)(110)

Operación +25:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (45) 3 (65) 1

0: (20)(25) 3: (50)(55) 1: (89)(96)(110)

Operación -50:

- No se produce *underflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (45) 3 (65) 1

0: (20)(25) 3: (55) 1: (89)(96)(110)

Operación -45:

- Se reemplaza la clave 45 por la menor clave del subárbol derecho (55).
- Se produce *underflow* en el nodo hoja 3.
- Se redistribuye con el hermano adyacente derecho, el nodo hoja 1.
- Operaciones: L2, L3, L1, E3, E1, E2.

2: 0 (55) 3 (89) 1

0: (20)(25) 3: (65) 1: (96)(110)

Operación +120:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (55) 3 (89) 1

0: (20)(25) 3: (65) 1: (96)(110)(120)

Operación +130:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de la clave 120.
- Operaciones: L2, L1, E1, E4, E2.

2: 0 (55) 3 (89) 1 (120) 4

0: (20)(25) 3: (65) 1: (96)(110) 4: (130)

Operación +70:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (55) 3 (89) 1 (120) 4

0: (20)(25) 3: (65)(70) 1: (96)(110) 4: (130)

Operación +75:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (55) 3 (89) 1 (120) 4

0: (20)(25) 3: (65)(70)(75) 1: (96)(110) 4: (130)

Operación +73:

- Se produce *overflow* en el nodo hoja 3, división del mismo y promoción de la clave 73.
- Se produce *overflow* en el nodo 2, división del mismo y promoción de la clave 89.
- Operaciones: L2, L3, E3, E5, E2, E6, E7.

7: 2 (89) 6

2: 0 (55) 3 (73) 5 6: 1 (120) 4

0: (20)(25) 3: (65)(70) 5: (75) 1: (96)(110) 4: (130)

Operación +100:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L7, L6, L1, E1.

7: 2 (89) 6

2: 0 (55) 3 (73) 5 6: 1 (120) 4

0: (20)(25) 3: (65)(70) 5: (75) 1: (96)(100)(110) 4: (130)

Operación -120:

- Se reemplaza la clave 120 por la menor clave del subárbol derecho (130).
- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 1, ya que se trata de un nodo hoja de un extremo y no tiene hermano adyacente derecho.
- Operaciones: L7, L6, L4, L1, E1, E4, E6.

7: 2 (89) 6

2: 0 (55) 3 (73) 5 6: 1 (110) 4

0: (20)(25) 3: (65)(70) 5: (75) 1: (96)(100) 4: (130)

Operación -110:

- Se reemplaza la clave 110 por la menor clave del subárbol derecho (130).

- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 1, ya que se trata de un nodo hoja de un extremo y no tiene hermano adyacente derecho.
- Operaciones: L7, L6, L4, L1, E1, E4, E6.

7: 2 (89) 6

2: 0 (55) 3 (73) 5 6: 1 (100) 4

0: (20)(25) 3: (65)(70) 5: (75) 1: (96) 4: (130)

Ejercicio 16.

Dado el siguiente árbol B+ de orden 4 y con política de resolución de underflows a derecha, realizar las siguientes operaciones: +80, -400. Indicando lecturas y escrituras en el orden de ocurrencia. Además, se debe describir, detalladamente, lo que sucede en cada operación.

4: 0 (340) 1 (400) 2 (500) 3

0: (11)(50)(77) 1 1: (340)(350)(360) 2 2: (402)(410)(420) 3 3: (520)(530) -1

Operación +80:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de una copia de la clave 77.
- Se produce *overflow* en el nodo 4, división del mismo y promoción del separador 400.
- Operaciones: L4, L0, E0, E5, E4, E6, E7.

7: 4 (400) 6

4: 0 (77) 5 (340) 1 **6**: 2 (500) 3

0: (11)(50) **5**: (77)(80) 1 **1**: (340)(350)(360) 2 **2**: (402)(410)(420) 3 **3**: (520)(530) -1

Operación -400:

- La clave 400 no se encuentra en ningún nodo hoja.
- Operaciones: L7, L6, L2.

7: 4 (400) 6

4: 0 (77) 5 (340) 1 **6**: 2 (500) 3

0: (11)(50) **5**: (77)(80) 1 **1**: (340)(350)(360) 2 **2**: (402)(410)(420) 3 **3**: (520)(530) -1

Ejercicio 17.

Dado el siguiente árbol B+ de orden 4, mostrar cómo quedaría el mismo luego de realizar las siguientes operaciones: +120, +110, +52, +70, +15, -45, -52, +22, +19, -66, -22, -19, -23, -89. Política de resolución de underflows: derecha.

2: 0 (66) 1

0: (23)(45) 1 1: (66)(67)(89) -1

Operación +120:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de una copia de la clave 89.
- Operaciones: L2, L1, E1, E3, E2.

2: 0 (66) 1 (89) 3

0: (23)(45) 1 1: (66)(67) 3 3: (89)(120) -1

Operación +110:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (66) 1 (89) 3

0: (23)(45) 1 1: (66)(67) 3 3: (89)(110)(120) -1

Operación +52:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (66) 1 (89) 3

0: (23)(45)(52) 1 1: (66)(67) 3 3: (89)(110)(120) -1

Operación +70:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (66) 1 (89) 3

0: (23)(45)(52) 1 1: (66)(67)(70) 3 3: (89)(110)(120) -1

Operación +15:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de una copia de la clave 45.
- Operaciones: L2, L0, E0, E4, E2.

2: 0 (45) 4 (66) 1 (89) 3

0: (15)(23) 4 4: (45)(52) 1 1: (66)(67)(70) 3 3: (89)(110)(120) -1

Operación -45:

- No se produce *underflow* en el nodo hoja 4.
- Operaciones: L2, L4, E4.

2: 0 (45) 4 (66) 1 (89) 3

0: (15)(23) 4 4: (52) 1 1: (66)(67)(70) 3 3: (89)(110)(120) -1

Operación -52:

- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente derecho, el nodo hoja 1.
- Operaciones: L2, L4, L1, E4, E1, E2.

2: 0 (45) 4 (67) 1 (89) 3

0: (15)(23) 4 4: (66) 1 1: (67)(70) 3 3: (89)(110)(120) -1

Operación +22:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (45) 4 (67) 1 (89) 3

0: (15)(22)(23) 4 4: (66) 1 1: (67)(70) 3 3: (89)(110)(120) -1

Operación +19:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de una copia de la clave 22.
- Se produce *overflow* en el nodo 2, división del mismo y promoción de la clave 67.
- Operaciones: L2, L0, E0, E5, E2, E6, E7.

7: 2 (67) 6

2: 0 (22) 5 (45) 4 6: 1 (89) 3

0: (15)(19) 5 5: (22)(23) 4 4: (66) 1 1: (67)(70) 3 3: (89)(110)(120) -1

Operación -66:

- Se produce *underflow* en el nodo hoja 4.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 5, ya que se trata de un nodo hoja de un extremo y no tiene hermano adyacente derecho.
- Operaciones: L7, L2, L4, E5, E4, E2.

7: 2 (67) 6

2: 0 (22) 5 (23) 4 6: 1 (89) 3

0: (15)(19) 5 5: (22) 4 4: (23) 1 1: (67)(70) 3 3: (89)(110)(120) -1

Operación -22:

- Se produce *underflow* en el nodo hoja 5. Hay que redistribuir siguiendo la política derecha, pero el hermano adyacente derecho ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente derecho, el nodo hoja 5.
- Operaciones: L7, L2, L5, L4, E5, E2.

7: 2 (67) 6

2: 0 (22) 5 6: 1 (89) 3

0: (15)(19) 5 5: (23) 1 1: (67)(70) 3 3: (89)(110)(120) -1

Operación -19:

- No se produce *underflow* en el nodo hoja 0.
- Operaciones: L7, L2, L0, E0.

7: 2 (67) 6

2: 0 (22) 5 6: 1 (89) 3

0: (15) 5 5: (23) 1 1: (67)(70) 3 3: (89)(110)(120) -1

Operación -23:

- Se produce *underflow* en el nodo hoja 5. Hay que redistribuir siguiendo la política derecha, pero no tiene hermano adyacente derecho y el hermano adyacente izquierdo, el nodo hoja 0, ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente izquierdo, el nodo hoja 0, ya que no tiene hermano adyacente derecho.
- Se produce *underflow* en el nodo 2. Hay que redistribuir siguiendo la política derecha, pero el hermano adyacente derecho, el nodo 6, ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente derecho, el nodo 6.
- Operaciones: L7, L2, L5, L0, E0, E2.

2: 0 (67) 1 (89) 3

0: (15) 1 1: (67)(70) 3 3: (89)(110)(120) -1

Operación -89:

- No se produce *underflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (67) 1 (89) 3

0: (15) 1 1: (67)(70) 3 3: (110)(120) -1

Ejercicio 18.

Dada las siguientes operaciones, mostrar la construcción, paso a paso, de un árbol B+ de orden 4: +67, +56, +96, +10, +28, +95, +16, +46, +23, +36, +120, +130, +60, +57, -96, -67, -95, -60, -120, -57, -56. Política de resolución de underflows: derecha o izquierda.

Operación +67:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (67) -1

Operación +56:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (56)(67) -1

Operación +96:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (56)(67)(96) -1

Operación +10:

- Se produce *overflow* en el nodo 0, división del mismo y promoción de una copia de la clave 67.
- Operaciones: L0, E0, E1, E2.

2: 0 (67) 1

0: (10)(56) 1 **1:** (67)(96) -1

Operación +28:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (67) 1

0: (10)(28)(56) 1 **1:** (67)(96) -1

Operación +95:

- No se produce *overflow* en el nodo hoja 1.

- Operaciones: L2, L1, E1.

2: 0 (67) 1

0: (10)(28)(56) 1 1: (67)(95)(96) -1

Operación +16:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de una copia de la clave 28.
- Operaciones: L2, L0, E0, E3, E2.

2: 0 (28) 3 (67) 1

0: (10)(16) 3 3: (28)(56) 1 1: (67)(95)(96) -1

Operación +46:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (28) 3 (67) 1

0: (10)(16) 3 3: (28)(46)(56) 1 1: (67)(95)(96) -1

Operación +23:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (28) 3 (67) 1

0: (10)(16)(23) 3 3: (28)(46)(56) 1 1: (67)(95)(96) -1

Operación +36:

- Se produce *overflow* en el nodo hoja 3, división del mismo y promoción de una copia de la clave 46.
- Operaciones: L2, L3, E3, E4, E2.

2: 0 (28) 3 (46) 4 (67) 1

0: (10)(16)(23) 3 3: (28)(36) 4 4: (46)(56) 1 1: (67)(95)(96) -1

Operación +120:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de una copia de la clave 96.
- Se produce *overflow* en el nodo 2, división del mismo y promoción del separador 67.
- Operaciones: L2, L1, E1, E5, E2, E6, E7.

7: 2 (67) 6

2: 0 (28) 3 (46) 4 6: 1 (96) 5

0: (10)(16)(23) 3 **3:** (28)(36) 4 **4:** (46)(56) 1 **1:** (67)(95) 5 **5:** (96)(120) -1

Operación +130:

- No se produce *overflow* en el nodo hoja 5.
- Operaciones: L7, L6, L5, E5.

7: 2 (67) 6

2: 0 (28) 3 (46) 4 **6:** 1 (96) 5

0: (10)(16)(23) 3 **3:** (28)(36) 4 **4:** (46)(56) 1 **1:** (67)(95) 5 **5:** (96)(120)(130) -1

Operación +60:

- No se produce *overflow* en el nodo hoja 4.
- Operaciones: L7, L2, L4, E4.

7: 2 (67) 6

2: 0 (28) 3 (46) 4 **6:** 1 (96) 5

0: (10)(16)(23) 3 **3:** (28)(36) 4 **4:** (46)(56)(60) 1 **1:** (67)(95) 5 **5:** (96)(120)(130) -1

Operación +57:

- Se produce *overflow* en el nodo 4, división del mismo y promoción de una copia de la clave 57.
- Operaciones: L7, L2, L4, E4, E8, E2.

7: 2 (67) 6

2: 0 (28) 3 (46) 4 (57) 8 **6:** 1 (96) 5

0: (10)(16)(23) 3 **3:** (28)(36) 4 **4:** (46)(56) 8 **8:** (57)(60) 1 **1:** (67)(95) 5 **5:** (96)(120)(130) -1

Operación -96:

- No se produce *underflow* en el nodo hoja 5.
- Operaciones: L7, L6, L5, E5.

7: 2 (67) 6

2: 0 (28) 3 (46) 4 (57) 8 **6:** 1 (96) 5

0: (10)(16)(23) 3 **3:** (28)(36) 4 **4:** (46)(56) 8 **8:** (57)(60) 1 **1:** (67)(95) 5 **5:** (120)(130) -1

Operación -67:

- No se produce *underflow* en el nodo hoja 1.
- Operaciones: L7, L6, L1, E1.

7: 2 (67) 6

2: 0 (28) 3 (46) 4 (57) 8 **6:** 1 (96) 5

0: (10)(16)(23) 3 **3:** (28)(36) 4 **4:** (46)(56) 8 **8:** (57)(60) 1 **1:** (95) 5 **5:** (120)(130) -1

Operación -95:

- Se produce *underflow* en el nodo hoja 1.
- Se redistribuye con el hermano adyacente derecho, el nodo hoja 5.
- Operaciones: L7, L6, L1, E1.

7: 2 (67) 6

2: 0 (28) 3 (46) 4 (57) 8 6: 1 (130) 5

0: (10)(16)(23) 3 3: (28)(36) 4 4: (46)(56) 8 8: (57)(60) 1 1: (120) 5 5: (130) -1

Operación -60:

- No se produce *underflow* en el nodo hoja 8.
- Operaciones: L7, L2, L8, E8.

7: 2 (67) 6

2: 0 (28) 3 (46) 4 (57) 8 6: 1 (130) 5

0: (10)(16)(23) 3 3: (28)(36) 4 4: (46)(56) 8 8: (57) 1 1: (120) 5 5: (130) -1

Operación -120:

- Se produce *underflow* en el nodo hoja 1. Hay que redistribuir siguiendo la política derecha o izquierda, pero el hermano adyacente derecho, el nodo hoja 5, ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente derecho, el nodo hoja 5.
- Se produce *underflow* en el nodo 6.
- Se redistribuye con el hermano adyacente izquierdo, el nodo 2, ya que no tiene hermano adyacente derecho.
- Operaciones: L7, L6, L1, L5, E1, E2, E6, E7.

7: 2 (57) 6

2: 0 (28) 3 (46) 4 6: 8 (67) 1

0: (10)(16)(23) 3 3: (28)(36) 4 4: (46)(56) 8 8: (57) 1 1: (130) -1

Operación -57:

- Se produce *underflow* en el nodo hoja 8. Hay que redistribuir siguiendo la política derecha o izquierda, pero el hermano adyacente derecho, el nodo hoja 1, ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona el hermano adyacente derecho, el nodo hoja 1.
- Se produce *underflow* en el nodo 6.
- Se redistribuye con el hermano adyacente izquierdo, el nodo 2, ya que no tiene hermano adyacente derecho.
- Operaciones: L7, L6, L8, L1, E8, E2, E6, E7.

7: 2 (46) 6

2: 0 (28) 3 6: 4 (57) 8

0: (10)(16)(23) 3 3: (28)(36) 4 4: (46)(56) 8 8: (130) -1

Operación -56:

- No se produce *underflow* en el nodo hoja 4.
- Operaciones: L7, L6, L4, E4.

7: 2 (46) 6

2: 0 (28) 3 6: 4 (57) 8

0: (10)(16)(23) 3 3: (28)(36) 4 4: (46) 8 8: (130) -1

Ejercicio 19.

Dada las siguientes operaciones, mostrar la construcción, paso a paso, de un árbol B+ de orden 6: +52, +23, +10, +99, +63, +74, +19, +85, +14, +73, +5, +7, +41, +100, +130, +44, -63, -73, +15, +16, -74, -52. Política de resolución de underflows: izquierda.

Operación +52:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (52) -1

Operación +23:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (23)(52) -1

Operación +10:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (10)(23)(52) -1

Operación +99:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (10)(23)(52)(99) -1

Operación +63:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L0, E0.

0: (10)(23)(52)(63)(99) -1

Operación +74:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de una copia de la clave 63.
- Operaciones: L0, E0, E1, E2.

2: 0 (63) 1

0: (10)(23)(52) 1 1: (63)(74)(99) -1

Operación +19:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (63) 1

0: (10)(19)(23)(52) 1 1: (63)(74)(99) -1

Operación +85:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (63) 1

0: (10)(19)(23)(52) 1 1: (63)(74)(85)(99) -1

Operación +14:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (63) 1

0: (10)(14)(19)(23)(52) 1 1: (63)(74)(85)(99) -1

Operación +73:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (63) 1

0: (10)(14)(19)(23)(52) 1 1: (63)(73)(74)(85)(99) -1

Operación +5:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de una copia de la clave 19.
- Operaciones: L2, L0, E0, E3, E2.

2: 0 (19) 3 (63) 1

0: (5)(10)(14) 3 3: (19)(23)(52) 1 1: (63)(73)(74)(85)(99) -1

Operación +7:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (19) 3 (63) 1

0: (5)(7)(10)(14) 3 3: (19)(23)(52) 1 1: (63)(73)(74)(85)(99) -1

Operación +41:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (19) 3 (63) 1

0: (5)(7)(10)(14) 3 3: (19)(23)(41)(52) 1 1: (63)(73)(74)(85)(99) -1

Operación +100:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de una copia de la clave 85.
- Operaciones: L2, L1, E1, E4, E2.

2: 0 (19) 3 (63) 1 (85) 4

0: (5)(7)(10)(14) 3 3: (19)(23)(41)(52) 1 1: (63)(73)(74) 4 4: (85)(99)(100) -1

Operación +130:

- No se produce *overflow* en el nodo hoja 4.
- Operaciones: L2, L4, E4.

2: 0 (19) 3 (63) 1 (85) 4

0: (5)(7)(10)(14) 3 3: (19)(23)(41)(52) 1 1: (63)(73)(74) 4 4: (85)(99)(100)(130) -1

Operación +44:

- No se produce *overflow* en el nodo hoja 3.
- Operaciones: L2, L3, E3.

2: 0 (19) 3 (63) 1 (85) 4

0: (5)(7)(10)(14) 3 3: (19)(23)(41)(44)(52) 1 1: (63)(73)(74) 4 4: (85)(99)(100)(130) -1

Operación -63:

- No se produce *underflow* en el nodo hoja 1.
- Operaciones: L2, L1, E1.

2: 0 (19) 3 (63) 1 (85) 4

0: (5)(7)(10)(14) 3 3: (19)(23)(41)(44)(52) 1 1: (73)(74) 4 4: (85)(99)(100)(130) -1

Operación -73:

- Se produce *underflow* en el nodo hoja 1.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 3.
- Operaciones: L2, L1, E3, E1, E2.

2: 0 (19) 3 (52) 1 (85) 4

0: (5)(7)(10)(14) 3 3: (19)(23)(41)(44) 1 1: (52)(74) 4 4: (85)(99)(100)(130) -1

Operación +15:

- No se produce *overflow* en el nodo hoja 0.
- Operaciones: L2, L0, E0.

2: 0 (19) 3 (52) 1 (85) 4

0: (5)(7)(10)(14)(15) 3 3: (19)(23)(41)(44) 1 1: (52)(74) 4 4: (85)(99)(100)(130) -1

Operación +16:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de una copia de la clave 14.
- Operaciones: L2, L0, E0, E5, E2.

2: 0 (14) 5 (19) 3 (52) 1 (85) 4

0: (5)(7)(10) 5 5: (14)(15)(16) 3 3: (19)(23)(41)(44) 1 1: (52)(74) 4 4: (85)(99)(100)(130) -1

Operación -74:

- Se produce *underflow* en el nodo hoja 1.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 3.
- Operaciones: L2, L1, L3, E3, E1, E2.

2: 0 (14) 5 (19) 3 (44) 1 (85) 4

0: (5)(7)(10) 5 5: (14)(15)(16) 3 3: (19)(23)(41) 1 1: (44)(52) 4 4: (85)(99)(100)(130) -1

Operación -52:

- Se produce *underflow* en el nodo hoja 1.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 3.
- Operaciones: L2, L1, L3, E3, E1, E2.

2: 0 (14) 5 (19) 3 (41) 1 (85) 4

0: (5)(7)(10) 5 5: (14)(15)(16) 3 3: (19)(23) 1 1: (41)(44) 4 4: (85)(99)(100)(130) -1

Ejercicio 20.

Dado un árbol B+ de orden 4 y con política izquierda o derecha, para cada operación dada:

- Dibujar el árbol resultante.
- Explicar, brevemente, las decisiones tomadas.
- Escribir las lecturas y escrituras.

Operaciones: +4, +44, -94, -104.

Nodo 7: 1 i 2(69)6

Nodo 2: 2 i 0(30)1(51)3

Nodo 6: 1 i 4(94)5

Nodo 0: 3 h(5)(10)(20)->1

Nodo 1: 2 h(30)(40)->3

Nodo 3: 2 h(51)(60)->4

Nodo 4: 2 h(69)(80)->5

Nodo 5: 1 h(104)->-1

7: 2 (69) 6

2: 0 (30) 1 (51) 3 6: 4 (94) 5

0: (5)(10)(20) 1 1: (30)(40) 3 3: (51)(60) 4 4: (69)(80) 5 5: (104) -1

Operación +4:

- Se produce *overflow* en el nodo hoja 0, división del mismo y promoción de una copia de la clave 10.
- Operaciones: L7, L2, L0, E0, E8, E2.

7: 2 (69) 6

2: 0 (10) 8 (30) 1 (51) 3 6: 4 (94) 5

0: (4)(5) 8 8: (10)(20) 1 1: (30)(40) 3 3: (51)(60) 4 4: (69)(80) 5 5: (104) -1

Operación +44:

- No se produce *overflow* en el nodo hoja 1.
- Operaciones: L7, L2, L1, E1.

7: 2 (69) 6

2: 0 (10) 8 (30) 1 (51) 3 6: 4 (94) 5

0: (4)(5) 8 8: (10)(20) 1 1: (30)(40)(44) 3 3: (51)(60) 4 4: (69)(80) 5 5: (104) -1

Operación -94:

- La clave 94 no se encuentra en ningún nodo hoja.
- Operaciones: L7, L6, L5.

7: 2 (69) 6

2: 0 (10) 8 (30) 1 (51) 3 6: 4 (94) 5

0: (4)(5) 8 8: (10)(20) 1 1: (30)(40)(44) 3 3: (51)(60) 4 4: (69)(80) 5 5: (104) -1

Operación -104:

- Se produce *underflow* en el nodo hoja 5.
- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 4.
- Operaciones: L7, L6, L5, L4, E4, E5, E6.

7: 2 (69) 6

2: 0 (10) 8 (30) 1 (51) 3 6: 4 (80) 5

0: (4)(5) 8 8: (10)(20) 1 1: (30)(40)(44) 3 3: (51)(60) 4 4: (69) 5 5: (80) -1

Ejercicio 21.

Dado el árbol B+ que se detalla más abajo, con orden 6, es decir, capacidad de 5 claves como máximo, mostrar los estados sucesivos al realizar la siguiente secuencia de operaciones: +159, -5, -190. Además, indicar nodos leídos y escritos en el orden de ocurrencia. Política de resolución underflow: derecha.

Nodo 2: 5, i, 0(10)1(60)3(115)4(145)5(179)6

Nodo 0: 2, h, (1)(5) -> 1

Nodo 1: 2, h, (34)(44) -> 3

Nodo 3: 2, h, (60)(113) -> 4

Nodo 4: 4, h, (120)(125)(131)(139) -> 5

Nodo 5: 5, h, (145)(153)(158)(160)(177) -> 6

Nodo 6: 2, h, (179)(190) -> -1

2: 0 (10) 1 (60) 3 (115) 4 (145) 5 (179) 6

0: (1)(5) 1 1: (34)(44) 3 3: (60)(113) 4 4: (120)(125)(131)(139) 5 5: (145)(153)(158)(160)(177) 6 6: (179)(190) -1

Operación +159:

- Se produce *overflow* en el nodo hoja 5, división del mismo y promoción de una copia de la clave 159.
- Se produce *overflow* en el nodo 2, división del mismo y promoción del separador 145.
- Operaciones: L2, L5, E5, E7, E2, E8, E9.

9: 2 (145) 8

2: 0 (10) 1 (60) 3 (115) 4 8: 5 (159) 7 (179) 6

0: (1)(5) 1 1: (34)(44) 3 3: (60)(113) 4 4: (120)(125)(131)(139) 5 5: (145)(153)(158) 7 7: (159)(160)(177) 6 6: (179)(190) -1

Operación -5:

- Se produce *underflow* en el nodo hoja 0. Hay que redistribuir siguiendo la política derecha, pero el hermano adyacente derecho, el nodo hoja 1, ya contiene la cantidad mínima de claves permitidas (2).
- Se fusiona con el hermano adyacente derecho, el nodo hoja 1.
- Operaciones: L9, L2, L0, L1, E0, E2.

9: 2 (145) 8

2: 0 (60) 3 (115) 4 8: 5 (159) 7 (179) 6

0: (1)(34)(44) 3 3: (60)(113) 4 4: (120)(125)(131)(139) 5 5: (145)(153)(158) 7 7: (159)(160)(177) 6 6: (179)(190) -1

Operación -190:

- Se produce *underflow* en el nodo hoja 6.

- Se redistribuye con el hermano adyacente izquierdo, el nodo hoja 7, ya que se trata de un nodo hoja de un extremo y no tiene hermano adyacente derecho.
- Operaciones: L9, L8, L6, L7, E7, E6, E8.

9: 2 (145) 7

2: 0 (60) 3 (115) 4 **8:** 5 (159) 7 (177) 6

0: (1)(34)(44) 3 **3:** (60)(113) 4 **4:** (120)(125)(131)(139) 5 **5:** (145)(153)(158) 7 **7:** (159)(160) 6 **6:** (177)(179) -1

Ejercicio 22.

Dado un árbol *B* de orden 5 y con política izquierda o derecha, para cada operación dada:

- Dibujar el árbol resultante.
- Explicar, detalladamente, las decisiones tomadas.
- Escribir las lecturas y escrituras.

Operaciones: +165, +260, +800, -110.

Árbol:

Nodo 8: 1 i 2 (150) 7

Nodo 2: 1 i 0 (120) 3

Nodo 7: 2 i 4 (210)6(300)1

Nodo 0: 2 h (30)(110)

Nodo 3: 1 h (130)

Nodo 4: 4 h (160)(170)(180)(200)

Nodo 6: 4 h (220)(230)(240)(250)

Nodo 1: 4 h (400)(500)(600)(700)

8: 2 (150) 7

2: 0 (120) 3 7: 4 (210) 6 (300) 1

0: (30)(110) 3: (130) 4: (160)(170)(180)(200) 6: (220)(230)(240)(250) 1: (400)(500)(600)(700)

Operación +165:

- Se produce *overflow* en el nodo hoja 4, división del mismo y promoción de la clave 170.
- Operaciones: L8, L7, L4, E4, E9, E7.

8: 2 (150) 7

2: 0 (120) 3 7: 4 (170) 9 (210) 6 (300) 1

0: (30)(110) 3: (130) 4: (160)(165) 9: (180)(200) 6: (220)(230)(240)(250) 1: (400)(500)(600)(700)

Operación +260:

- Se produce *overflow* en el nodo hoja 6, división del mismo y promoción de la clave 240.
- Operaciones: L8, L7, L6, E6, E10, E7.

8: 2 (150) 7

2: 0 (120) 3 7: 4 (170) 9 (210) 6 (240) 10 (300) 1

0: (30)(110) 3: (130) 4: (160)(165) 9: (180)(200) 6: (220)(230) 10: (250)(260) 1: (400)(500)(600)(700)

Operación +800:

- Se produce *overflow* en el nodo hoja 1, división del mismo y promoción de la clave 600.
- Se produce *overflow* en el nodo 7, división del mismo y promoción de la clave 240.
- Operaciones: L8, L7, L1, E1, E11, E7, E12, E8.

8: 2 (150) 7 (240) 12

2: 0 (120) 3 **7:** 4 (170) 9 (210) 6 **12:** 10 (300) 1 (600) 11

0: (30)(110) **3:** (130) **4:** (160)(165) **9:** (180)(200) **6:** (220)(230) **10:** (250)(260) **1:** (400)(500) **11:** (700)(800)

Operación -110:

- No se produce *underflow* en el nodo hoja 0.
- Operaciones: L8, L2, L0, E0.

8: 2 (150) 7 (240) 12

2: 0 (120) 3 **7:** 4 (170) 9 (210) 6 **12:** 10 (300) 1 (600) 11

0: (30) **3:** (130) **4:** (160)(165) **9:** (180)(200) **6:** (220)(230) **10:** (250)(260) **1:** (400)(500) **11:** (700)(800)

Ejercicio 23.

Dado un árbol B+ de orden 5 y con política izquierda o derecha, para cada operación dada:

- Dibujar el árbol resultante.
- Explicar, detalladamente, las decisiones tomadas.
- Escribir las lecturas y escrituras.

Operaciones: +250, -300, -40.

Árbol:

Nodo 8: 1 i 2(70)7

Nodo 2: 1 i 0(50)4

Nodo 7: 4 i 5(90)6(120)3(210)9(300)1

Nodo 0: 1 h(40)->4

Nodo 4: 1 h(50)->5

Nodo 5: 2 h(70)(80)->6

Nodo 6: 2 h(90)(100)->3

Nodo 3: 2 h(120)(200)->9

Nodo 9: 4 h(210)(220)(230)(240)->1

Nodo 1: 2 h(400)(500)->-1

8: 2 (70) 7

2: 0 (50) 4 7: 5 (90) 6 (120) 3 (210) 9 (300) 1

0: (40) 4 4: (50) 5 5: (70)(80) 6 6: (90)(100) 3 3: (120)(200) 9 9: (210)(220)(230)(240)

1 1: (400)(500) -1

Operación +250:

- Se produce *overflow* en el nodo hoja 9, división del mismo y promoción de una copia de la clave 230.
- Se produce *overflow* en el nodo 7, división del mismo y promoción del separador 210.
- Operaciones: L8, L7, L9, E9, E10, E7, E11, E8.

8: 2 (70) 7 (210) 11

2: 0 (50) 4 7: 5 (90) 6 (120) 3 11: 9 (230) 10 (300) 1

0: (40) 4 4: (50) 5 5: (70)(80) 6 6: (90)(100) 3 3: (120)(200) 9 9: (210)(220) 10 10: (230)(240)(250) 1 1: (400)(500) -1

Operación -300:

- La clave 300 no se encuentra en ningún nodo hoja.
- Operaciones: L8, L11, L1.

8: 2 (70) 7 (210) 11

2: 0 (50) 4 7: 5 (90) 6 (120) 3 11: 9 (230) 10 (300) 1

0: (40) 4 **4:** (50) 5 **5:** (70)(80) 6 **6:** (90)(100) 3 **3:** (120)(200) 9 **9:** (210)(220) 10 **10:** (230)(240)(250) 1 **1:** (400)(500) -1

Operación -40:

- Se produce *underflow* en el nodo hoja 0. Hay que redistribuir siguiendo la política izquierda o derecha, pero no tiene hermano adyacente izquierdo y el hermano adyacente derecho, el nodo hoja 4, ya contiene la cantidad mínima de claves permitidas (1).
- Se fusiona con el hermano adyacente derecho, el nodo hoja 4, ya que no tiene hermano adyacente izquierdo.
- Se produce *underflow* en el nodo 2.
- Se redistribuye con el hermano adyacente derecho, el nodo 7, ya que no tiene hermano adyacente izquierdo.
- Operaciones: L8, L2, L0, L4, L7, E0, E2, E7, E8.

8: 2 (90) 7 (210) 11

2: 0 (70) 5 **7:** 6 (120) 3 **11:** 9 (230) 10 (300) 1

0: (50) 5 **5:** (70)(80) 6 **6:** (90)(100) 3 **3:** (120)(200) 9 **9:** (210)(220) 10 **10:** (230)(240)(250) 1 **1:** (400)(500) -1

Introducción a las Bases de Datos

Fundamentos de Organización de Datos

Práctica 5

Hashing (Dispersión)

Parte 1: Preguntas conceptuales

1. Defina el concepto de hashing (o dispersión). ¿Cómo se relaciona este concepto con archivos?
2. Explique el concepto de *función de dispersión*. Enumere al menos tres funciones de dispersión y explique brevemente cómo funciona cada una.
3. Explique los conceptos de *sinónimo*, *colisión* y *desborde (overflow)*. ¿Qué condición es necesaria en el archivo directo para que pueda ocurrir una colisión y no un desborde?
4. ¿Qué alternativas existen para reducir el número de colisiones (y por ende de desbordes) en un archivo organizado mediante la técnica de hashing?
5. Explique brevemente qué es la *densidad de empaquetamiento*. ¿Cuáles son las consecuencias de tener una menor densidad de empaquetamiento en un archivo directo?
6. Explique brevemente cómo funcionan las siguientes técnicas de resolución de desbordes que se pueden utilizar en hashing estático.
 - Saturación progresiva
 - Saturación progresiva encadenada
 - Saturación progresiva encadenada con área de desborde separada
 - Dispersión doble

Parte 2: Dispersión extensible

Nota: (+) indica una operación de inserción, (-) indica una operación de baja.

7. Para las siguientes claves, realice el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos registros. El número natural indica el orden de llegada de las operaciones. **Se debe mostrar el estado del archivo para cada operación. Justifique brevemente ante colisión y desborde los pasos que realiza.**

1	+ Darin	00111111	2	+ Alterio	11110100
3	+ Sbaraglia	10100101	4	+ De la Serna	01010111
5	+ Altavista	01101011	6	+ Grandinetti	10101010
7	- Altavista	01101011	8	- Sbaraglia	10100101

8. Realice el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada registro tiene capacidad para dos claves. El número natural indica el orden de llegada de las operaciones. **Se debe mostrar el estado del archivo para cada operación. Justifique brevemente ante colisión y desborde los pasos que realiza.**

1	+ Buenos Aires	1001	2	+ San Juan	0100
3	+ Entre Ríos	1110	4	+ Corrientes	0010
5	+ San Luis	0101	6	+ Tucumán	0111
7	+ Río Negro	0011	8	+ Jujuy	1111
9	+ Salta	1010	10	- Río Negro	...0011

9. Para las siguientes claves, realice el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos registros. El número natural indica el orden de llegada de las operaciones. **Se debe mostrar el estado del archivo para cada operación. Justifique brevemente ante colisión y desborde los pasos que realiza.**

1	+ Tristana	11110010	2	+ Jarvan IV	00111010
3	+ Teemo	01010100	4	+ Annie	10100101
5	+ Ryze	10101110	6	+ Morgana	01101011
7	+ Garen	11001011	8	- Teemo	01010100

10. Para las siguientes claves, realice el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos registros. El número natural indica el orden de llegada de las operaciones. **Se debe mostrar el estado del archivo para cada operación. Justifique brevemente ante colisión y desborde los pasos que realiza.**

1	+ Guillermo.B	01100011	2	+ Gomez	00000001
3	+ Gustavo.B	01010110	4	+ Sosa	11110100
5	+ Enria	00110101	6	+ Guli	00101000
7	- Gustavo.B	01010110	8	- Sosa	11110100

11. Para las siguientes claves, realice el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos registros. El número natural indica el orden de llegada de las operaciones. **Se debe mostrar el estado del archivo para cada operación. Justifique brevemente ante colisión y desborde los pasos que realiza.**

1	+ Mansilla	01100010	2	+ Cetré	10001000
3	+ Ascacibar	01010111	4	+ Carrillo	11110101
5	+ Manyoma	00110100	6	+ Méndez	00101001
7	+ Alario	11000101	8	- Mansilla	01100010

12. Realice el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos claves. El número natural indica el orden de llegada de las operaciones. **Deberá explicar los pasos que realiza en cada operación y dibujar los estados sucesivos correspondientes (inclusive el estado inicial).**

1	+ Aconcagua	10100111	2	+ Kilimanjaro	10101010
3	+ Mont Blanc	00111110	4	+ Cervino	01101111
5	+ Etna	00110101	6	+ Chañi	11110000
7	+ Cho Oyu	01011101	8	+ Vinicunca	01011011
9	- Chañi	11110000	10	- Cervino	01101111

Trabajo Práctico N° 5: ***Hashing.***

PARTE I: Preguntas Conceptuales.

Ejercicio 1.

Definir el concepto de hashing (o dispersión). ¿Cómo se relaciona este concepto con archivos?

Hashing (o dispersión) es una técnica utilizada para transformar una clave (como una palabra, un número o un conjunto de datos) en una dirección o índice dentro de una estructura de datos, generalmente una tabla o un archivo. Esta transformación se realiza mediante una función de dispersión (función *hash*), que toma la clave como entrada y produce un número entero, la dirección *hash* o el código *hash*, que se utiliza como índice para acceder, rápidamente, a los datos.

Hashing (o dispersión) es un método de asignación de claves a posiciones en una tabla o archivo mediante una función de dispersión (función *hash*), con el objetivo de facilitar el almacenamiento y recuperación eficiente de información.

- Técnica para generar una dirección base única para una llave dada. La dispersión se usa cuando se requiere acceso rápido a una llave.
- Técnica que convierte la llave del registro en un número aleatorio, el que sirve, después, para determinar dónde se almacena el registro.
- Técnica de almacenamiento y recuperación que usa una función de *hash* para mapear registros en dirección de almacenamiento.

Ejercicio 2.

Explicar el concepto de función de dispersión. Enumerar, al menos, tres funciones de dispersión y explicar, brevemente, cómo funciona cada una.

Una función de dispersión (función *hash*) es un procedimiento que toma una clave (como una palabra, un número o un conjunto de datos) como entrada y la convierte en un número entero, la dirección *hash* o el código *hash*, que se utiliza como índice en una tabla o en un archivo para acceder, rápidamente, a los datos.

El objetivo principal es asignar claves a posiciones de manera uniforme para facilitar el acceso rápido a los datos, minimizando las colisiones (casos en los que diferentes claves generan el mismo índice).

A continuación, se enumeran tres funciones de dispersión:

1. Método de la división: Toma la clave k (por ejemplo, un número entero) y la divide por un número m (preferentemente primo), y se usa el resto como la dirección *hash*.
 $\text{hash}(k) = k \bmod m$.
2. Método de la multiplicación: Toma la clave k (por ejemplo, un número entero), la multiplica por un número fraccionario ($A = 0,618$), toma la parte decimal, y la multiplica por un número m para obtener un índice.
 $\text{hash}(k) = \text{floor}(m * ((k * A) \bmod 1))$.
3. Método de la suma de caracteres (para cadenas): Suma los valores ASCII (o numéricos) de todos los caracteres de la clave y, luego, aplica una operación como $\bmod m$.

Ejercicio 3.

Explicar los conceptos de sinónimo, colisión y desborde (overflow). ¿Qué condición es necesaria en el archivo directo para que pueda ocurrir una colisión y no un desborde?

Sinónimo: Es una clave diferente que, al ser procesada por la función de dispersión, produce la misma dirección *hash* que otra clave. Es decir, dos claves distintas generan el mismo índice.

Colisión: Situación en la que un registro es asignado a una dirección que está utilizada por otro registro. Se produce cuando dos o más claves se asignan a la misma posición en la tabla o en el archivo.

Desborde (overflow): Situación en la que un registro es asignado a una dirección que está utilizada por otro registro y no queda espacio para este nuevo. Se produce cuando no hay espacio disponible en la posición calculada por la función de dispersión ni en las posiciones alternativas previstas para manejar colisiones.

La condición que es necesaria en el archivo directo para que pueda ocurrir una colisión y no un desborde es que exista, al menos, una posición disponible donde se pueda reubicar el sinónimo.

Ejercicio 4.

¿Qué alternativas existen para reducir el número de colisiones (y, por ende, de desbordes) en un archivo organizado mediante la técnica de hashing?

Las alternativas que existen para reducir el número de colisiones (y, por ende, de desbordes) en un archivo organizado mediante la técnica de *hashing* son:

- Algoritmos de dispersión sin colisiones o que éstas nunca produzcan *overflow* (perfectos e imposibles de conseguir).
- Buscar métodos que distribuyan los registros de la forma más aleatoria posible.
- Distribuir pocos registros en muchas direcciones.
- Colocar más de un registro por dirección.

Ejercicio 5.

Explicar, brevemente, qué es la densidad de empaquetamiento. ¿Cuáles son las consecuencias de tener una menor densidad de empaquetamiento en un archivo directo?

La densidad de empaquetamiento es la proporción de espacio del archivo asignado que, en realidad, almacena registros. Las consecuencias de tener una menor densidad de empaquetamiento en un archivo directo es menos *overflow* y más desperdicio de espacio.

Ejercicio 6.

Explicar, brevemente, cómo funcionan las siguientes técnicas de resolución de desbordes que se pueden utilizar en hashing estático: saturación progresiva, saturación encadenada, saturación progresiva encadenada con área de desborde separada y dispersión doble.

Saturación progresiva: Cuando se completa el nodo, se busca el próximo hasta encontrar uno libre.

Saturación encadenada: Es similar a la saturación progresiva, pero los registros de saturación se encadenan y no ocupan, necesariamente, posiciones contiguas.

Saturación progresiva encadenada con área de desborde separada: No utiliza nodos de direcciones para los *overflows*, éstos van a nodos especiales.

Dispersión doble: Las técnicas de saturación tienden a agrupar en zonas contiguas y generan búsquedas largas cuando la densidad tiende a uno. La solución de esta técnica de resolución de colisiones es almacenar los registros de *overflow* en zonas no relacionadas, aplicándoles una segunda función *hash* a la llave para producir un número entero, el cual se suma a la dirección original tantas veces como sea necesario hasta encontrar una dirección con espacio.

PARTE II: Dispersión Extensible.**Ejercicio 7.**

Para las siguientes claves, realizar el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos registros. El número natural indica el orden de llegada de las operaciones. Se debe mostrar el estado del archivo para cada operación. Justificar, brevemente, ante colisión y desborde los pasos que se realizan.

1	+ Darín	00111111	2	+ Alterio	11110100
3	+ Sbaraglia	10100101	4	+ De la Serna	01010111
5	+ Altavista	01101011	6	+ Grandinetti	10101010
7	- Altavista	01101011	8	- Sbaraglia	10100101

Estado inicial de la tabla de dispersión y del archivo:

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0		

Inserción de la clave “Darín”:

- No se produce colisión ni *overflow* en el bloque 0.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Darín (00111111)	

Inserción de la clave “Alterio”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Darín (00111111)	Alterio (11110100)

Inserción de la clave “Sbaraglia”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 1.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 1	
Sufijo	#Bloque
(0)	1
(1)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Darín (00111111)	Sbaraglia (10100101)
1	1	Alterio (11110100)	

Inserción de la clave “De la Serna”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 2.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	1
(01)	2
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Darín (00111111)	De la Serna (01010111)
1	1	Alterio (11110100)	
2	2	Sbaraglia (10100101)	

Inserción de la clave “Altavista”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 3.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	1
(001)	2
(010)	1
(011)	3
(100)	1
(101)	2
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	3	Darín (00111111)	De la Serna (01010111)
1	1	Alterio (11110100)	
2	2	Sbaraglia (10100101)	
3	3	Altavista (01110111)	

Inserción de la clave “Grandinetti”:

- Se produce colisión en el bloque 1, pero no se produce *overflow*.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	1
(001)	2
(010)	1
(011)	3
(100)	1
(101)	2
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	3	Darín (00111111)	De la Serna (01010111)
1	1	Alterio (11110100)	Grandinetti (10101010)
2	2	Sbaraglia (10100101)	
3	3	Altavista (01101011)	

Baja de la clave “Altavista”:

- Se borra la clave “Altavista”.
- Se libera el bloque 3, ya que se puede fusionar con el bloque 0.
- Se reducen en uno los bits del bloque 0.
- Se reduce a la mitad la cantidad de celdas de la tabla de dispersión y disminuyen en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	1
(01)	2
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Darín (00111111)	De la Serna (01010111)
1	1	Alterio (11110100)	Grandinetti (10101010)
2	2	Sbaraglia (10100101)	
3	3	Altavista (01101011)	

Baja de la clave “Sbaraglia”:

- Se borra la clave “Sbaraglia”.
- Se libera el bloque 2, ya que se puede fusionar con el bloque 0.
- Se reducen en uno los bits del bloque 0.
- Se reduce a la mitad la cantidad de celdas de la tabla de dispersión y disminuyen en uno los bits de dispersión.

Bits de dispersión: 1	
Sufijo	#Bloque
(0)	1
(1)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Darín (00111111)	De la Serna (01010111)
1	1	Alterio (11110100)	Grandinetti (10101010)
2	2	Sbaraglia (10100101)	
3	3	Altavista (01101011)	

Ejercicio 8.

Realizar el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos claves. El número natural indica el orden de llegada de las operaciones. Se debe mostrar el estado del archivo para cada operación. Justificar, brevemente, ante colisión y desborde los pasos que se realizan.

1	+ Buenos Aires	...1001	2	+ San Juan	...0100
3	+ Entre Ríos	...1110	4	+ Corrientes	...0010
5	+ San Luis	...0101	6	+ Tucumán	...0111
7	+ Río Negro	...0011	8	+ Jujuy	...1111
9	+ Salta	...1010	10	- Río Negro	...0011

Estado inicial de la tabla de dispersión y del archivo:

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0		

Inserción de la clave “Buenos Aires”:

- No se produce colisión ni *overflow* en el bloque 0.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Buenos Aires (...1001)	

Inserción de la clave “San Juan”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Buenos Aires (...1001)	San Juan (...0100)

Inserción de la clave “Entre Ríos”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 1.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 1	
Sufijo	#Bloque
(0)	1
(1)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Buenos Aires (...1001)	
1	1	San Juan (...0100)	Entre Ríos (...1110)

Inserción de la clave “Corrientes”:

- Se produce colisión y *overflow* en el bloque 1.
- Se genera el bloque 2.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	2
(01)	0
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Buenos Aires (...1001)	
1	2	Entre Ríos (...1110)	Corrientes (...0010)
2	2	San Juan (...0100)	

Inserción de la clave “San Luis”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	2
(01)	0
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Buenos Aires (...1001)	San Luis (...0101)
1	2	Entre Ríos (...1110)	Corrientes (...0010)
2	2	San Juan (...0100)	

Inserción de la clave “Tucumán”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 3.
- No se duplica la cantidad de celdas de la tabla de dispersión y no aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	2
(01)	3
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Tucumán (...0111)	
1	2	Entre Ríos (...1110)	Corrientes (...0010)
2	2	San Juan (...0100)	
3	2	Buenos Aires (...1001)	San Luis (...0101)

Inserción de la clave “Río Negro”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	2
(01)	3
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Tucumán (...0111)	Río Negro (...0011)
1	2	Entre Ríos (...1110)	Corrientes (...0010)
2	2	San Juan (...0100)	
3	2	Buenos Aires (...1001)	San Luis (...0101)

Inserción de la clave “Jujuy”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 4.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	3
(010)	1
(011)	4
(100)	2
(101)	3
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	3	Tucumán (...0111)	Jujuy (...1111)
1	2	Entre Ríos (...1110)	Corrientes (...0010)
2	2	San Juan (...0100)	
3	2	Buenos Aires (...1001)	San Luis (...0101)
4	3	Río Negro (...0011)	

Inserción de la clave “Salta”:

- Se produce colisión y *overflow* en el bloque 1.
- Se genera el bloque 5.
- No se duplica la cantidad de celdas de la tabla de dispersión y no aumentan en uno los bits de dispersión.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	3
(010)	5
(011)	4
(100)	2
(101)	3
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	3	Tucumán (...0111)	Jujuy (...1111)
1	3	Entre Ríos (...1110)	
2	2	San Juan (...0100)	
3	2	Buenos Aires (...1001)	San Luis (...0101)
4	3	Río Negro (...0011)	
5	3	Corrientes (...0010)	Salta (...1010)

Baja de la clave “Río Negro”:

- Se borra la clave “Río Negro”.
- Se libera el bloque 4, ya que se puede fusionar con el bloque 0.
- Se reducen en uno los bits del bloque 0.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	3
(010)	5
(011)	0
(100)	2
(101)	3
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Tucumán (...0111)	Jujuy (...1111)
1	3	Entre Ríos (...1110)	
2	2	San Juan (...0100)	
3	2	Buenos Aires (...1001)	San Luis (...0101)
4	3	Río Negro (...0011)	
5	3	Corrientes (...0010)	Salta (...1010)

Ejercicio 9.

Para las siguientes claves, realizar el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos registros. El número natural indica el orden de llegada de las operaciones. Se debe mostrar el estado del archivo para cada operación. Justificar, brevemente, ante colisión y desborde los pasos que se realizan.

1	+ Tristana	11110010	2	+ Jarvan IV	00111010
3	+ Teemo	01010100	4	+ Annie	10100101
5	+ Ryze	10101110	6	+ Morgana	01101011
7	+ Garen	11001011	8	- Teemo	01010100

Estado inicial de la tabla de dispersión y del archivo:

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0		

Inserción de la clave “Tristana”:

- No se produce colisión ni *overflow* en el bloque 0.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Tristana (11110010)	

Inserción de la clave “Jarvan IV”:

- Se produce colisión en el bloque, pero no se produce *overflow*.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Tristana (11110010)	Jarvan IV (00111010)

Inserción de la clave “Teemo”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 1.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.
- No son suficientes los bits de dispersión para redistribuir las claves.
- Se produce colisión y *overflow* en el bloque 1.
- Se genera el bloque 2.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	2
(01)	0
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	1		
1	2	Tristana (11110010)	Jarvan IV (00111010)
2	2	Teemo (01010100)	

Inserción de la clave “Annie”:

- No se produce colisión ni *overflow* en el bloque 0.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	2
(01)	0
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Annie (10100101)	
1	2	Tristana (11110010)	Jarvan IV (00111010)
2	2	Teemo (01010100)	

Inserción de la clave “Ryze”:

- Se produce colisión y *overflow* en el bloque 1.
- Se genera el bloque 3.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	0
(010)	3
(011)	0
(100)	2
(101)	0
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Annie (10100101)	
1	3	Ryze (10101110)	
2	2	Teemo (01010100)	
3	3	Tristana (11110010)	Jarvan IV (00111010)

Inserción de la clave “Morgana”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	0
(010)	3
(011)	0
(100)	2
(101)	0
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Annie (10100101)	Morgana (01101011)
1	3	Ryze (10101110)	
2	2	Teemo (01010100)	
3	3	Tristana (11110010)	Jarvan IV (00111010)

Inserción de la clave “Garen”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 4.
- No se duplica la cantidad de celdas de la tabla de dispersión y no aumentan los bits de dispersión.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	4
(010)	3
(011)	0
(100)	2
(101)	4
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Morgana (01101011)	Garen (11001011)
1	3	Ryze (10101110)	
2	2	Teemo (01010100)	
3	3	Tristana (11110010)	Jarvan IV (00111010)
4	2	Annie (10100101)	

Baja de la clave “Teemo”:

- Se borra la clave “Teemo”.
- No se libera el bloque 2, ya que no se puede fusionar.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	4
(010)	3
(011)	0
(100)	2
(101)	4
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Morgana (01101011)	Garen (11001011)
1	3	Ryze (10101110)	
2	2	Teemo (01010100)	
3	3	Tristana (11110010)	Jarvan IV (00111010)
4	2	Annie (10100101)	

Ejercicio 10.

Para las siguientes claves, realizar el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos registros. El número natural indica el orden de llegada de las operaciones. Se debe mostrar el estado del archivo para cada operación. Justificar, brevemente, ante colisión y desborde los pasos que se realizan.

1	+ Guillermo.B	01100011	2	+ Gómez	00000001
3	+ Gustavo.B	01010110	4	+ Sosa	11110100
5	+ Enría	00110101	6	+ Guli	00101000
7	- Gustavo.B	01010110	8	- Sosa	11110100

Estado inicial de la tabla de dispersión y del archivo:

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0		

Inserción de la clave “Guillermo.B”:

- No se produce colisión ni *overflow* en el bloque 0.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Guillermo.B (01100011)	

Inserción de la clave “Gómez”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Guillermo.B (01100011)	Gómez (00000001)

Inserción de la clave “Gustavo.B”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 1.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 1	
Sufijo	#Bloque
(0)	1
(1)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Guillermo.B (01100011)	Gómez (00000001)
1	1	Gustavo.B (01010110)	

Inserción de la clave “Sosa”:

- Se produce colisión en el bloque 1, pero no se produce *overflow*.

Bits de dispersión: 1	
Sufijo	#Bloque
(0)	1
(1)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Guillermo.B (01100011)	Gómez (00000001)
1	1	Gustavo.B (01010110)	Sosa (11110100)

Inserción de la clave “Enria”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 2.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	1
(01)	2
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Guillermo.B (01100011)	
1	1	Gustavo.B (01010110)	Sosa (11110100)
2	2	Gómez (00000001)	Enria (00110101)

Inserción de la clave “Guli”:

- Se produce colisión y *overflow* en el bloque 1.
- Se genera el bloque 3.
- No se duplica la cantidad de celdas de la tabla de dispersión y no aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	3
(01)	2
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Guillermo.B (01100011)	
1	2	Gustavo.B (01010110)	
2	2	Gómez (00000001)	Enria (00110101)
3	2	Sosa (11110100)	Guli (00101000)

Baja de la clave “Gustavo.B”:

- Se borra la clave “Gustavo.B”.
- Se libera el bloque 1, ya que se puede fusionar con el bloque 3.
- Se reducen en uno los bits del bloque 3.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	3
(01)	2
(10)	3
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Guillermo.B (01100011)	
1	2	Gustavo.B (01010110)	
2	2	Gómez (00000001)	Enria (00110101)
3	1	Sosa (11110100)	Guli (00101000)

Baja de la clave “Sosa”:

- Se borra la clave “Sosa”.
- No se libera el bloque 3, ya que sigue ocupado por la clave “Guli”.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	3
(01)	2
(10)	3
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Guillermo.B (01100011)	
1	2	Gustavo.B (01010110)	
2	2	Gómez (00000001)	Enria (00110101)
3	1	Sosa (11110100)	Guli (00101000)

Ejercicio 11.

Para las siguientes claves, realizar el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos registros. El número natural indica el orden de llegada de las operaciones. Se debe mostrar el estado del archivo para cada operación. Justificar, brevemente, ante colisión y desborde los pasos que se realizan.

1	+ Mansilla	01100010	2	+ Cetré	10001000
3	+ Ascacibar	01010111	4	+ Carrillo	11110101
5	+ Manyoma	00110100	6	+ Méndez	00101001
7	+ Alario	11000101	8	- Mansilla	01100010

Estado inicial de la tabla de dispersión y del archivo:

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0		

Inserción de la clave “Mansilla”:

- No se produce colisión ni *overflow* en el bloque 0.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Mansilla (01100010)	

Inserción de la clave “Cetré”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Mansilla (01100010)	Cetré (10001000)

Inserción de la clave “Ascacibar”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 1.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 1	
Sufijo	#Bloque
(0)	1
(1)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Ascacibar (01010111)	
1	1	Mansilla (01100010)	Cetré (10001000)

Inserción de la clave “Carrillo”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 1	
Sufijo	#Bloque
(0)	1
(1)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Ascacibar (01010111)	Carrillo (11110101)
1	1	Mansilla (01100010)	Cetré (10001000)

Inserción de la clave “Manyoma”:

- Se produce colisión y *overflow* en el bloque 1.
- Se genera el bloque 2.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	2
(01)	0
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Ascacibar (01010111)	Carrillo (11110101)
1	2	Mansilla (01100010)	
2	2	Cetré (10001000)	Manyoma (00110100)

Inserción de la clave “Méndez”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 3.
- No se duplica la cantidad de celdas de la tabla de dispersión y no aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	2
(01)	3
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Ascacibar (01010111)	
1	2	Mansilla (01100010)	
2	2	Cetré (10001000)	Manyoma (00110100)
3	2	Carrillo (11110101)	Méndez (00101001)

Inserción de la clave “Alario”:

- Se produce colisión y *overflow* en el bloque 3.
- Se genera el bloque 4.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	4
(010)	1
(011)	0
(100)	2
(101)	3
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Ascacibar (01010111)	
1	2	Mansilla (01100010)	
2	2	Cetré (10001000)	Manyoma (00110100)
3	3	Carrillo (11110101)	Alario (11000101)
4	3	Méndez (00101001)	

Baja de la clave “Mansilla”:

- Se borra la clave “Mansilla”.
- Se libera el bloque 1, ya que se puede fusionar con el bloque 2.
- Se reducen en uno los bits del bloque 2.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	4
(010)	2
(011)	0
(100)	2
(101)	3
(110)	2
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Ascacibar (01010111)	
1	2	Mansilla (01100010)	
2	1	Cetré (10001000)	Manyoma (00110100)
3	3	Carrillo (11110101)	Alario (11000101)
4	3	Méndez (00101001)	

Ejercicio 12.

Realizar el proceso de dispersión mediante el método de hashing extensible, sabiendo que cada nodo tiene capacidad para dos claves. El número natural indica el orden de llegada de las operaciones. Se deberán explicar los pasos que se realizan en cada operación y dibujar los estados sucesivos correspondientes (inclusive el estado inicial).

1	+ Aconcagua	10100111	2	+ Kilimanjaro	10101010
3	+ Mont Blanc	00111110	4	+ Cervino	01101111
5	+ Etna	00110101	6	+ Chañi	11110000
7	+ Cho Oyu	01011101	8	+ Vinicunca	01011011
9	- Chañi	11110000	10	- Cervino	01101111

Estado inicial de la tabla de dispersión y del archivo:

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0		

Inserción de la clave “Aconcagua”:

- No se produce colisión ni *overflow* en el bloque 0.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Aconcagua (10100111)	

Inserción de la clave “Kilimanjaro”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

#Bloque	Bits	Clave R1	Clave R2
0	0	Aconcagua (10100111)	Kilimanjaro (10101010)

Inserción de la clave “Mont Blanc”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 1.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 1	
Sufijo	#Bloque
(0)	1
(1)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Aconcagua (10100111)	
1	1	Kilimanjaro (10101010)	Mont Blanc (00111110)

Inserción de la clave “Cervino”:

- Se produce colisión en el bloque 0, pero no se produce *overflow*.

Bits de dispersión: 1	
Sufijo	#Bloque
(0)	1
(1)	0

#Bloque	Bits	Clave R1	Clave R2
0	1	Aconcagua (10100111)	Cervino (01101111)
1	1	Kilimanjaro (10101010)	Mont Blanc (00111110)

Inserción de la clave “Etna”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 2.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	1
(01)	2
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Aconcagua (10100111)	Cervino (01101111)
1	1	Kilimanjaro (10101010)	Mont Blanc (00111110)
2	2	Etna (00110101)	

Inserción de la clave “Chañi”:

- Se produce colisión y *overflow* en el bloque 1.
- Se genera el bloque 3.
- No se duplica la cantidad de celdas de la tabla de dispersión y no aumentan en uno los bits de dispersión.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	3
(01)	2
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Aconcagua (10100111)	Cervino (01101111)
1	2	Kilimanjaro (10101010)	Mont Blanc (00111110)
2	2	Etna (00110101)	
3	2	Chañi (11110000)	

Inserción de la clave “Cho Oyu”:

- Se produce colisión en el bloque 2, pero no se produce *overflow*.

Bits de dispersión: 2	
Sufijo	#Bloque
(00)	3
(01)	2
(10)	1
(11)	0

#Bloque	Bits	Clave R1	Clave R2
0	2	Aconcagua (10100111)	Cervino (01101111)
1	2	Kilimanjaro (10101010)	Mont Blanc (00111110)
2	2	Etna (00110101)	Cho Oyu (01011101)
3	2	Chañi (11110000)	

Inserción de la clave “Vinicunca”:

- Se produce colisión y *overflow* en el bloque 0.
- Se genera el bloque 4.
- Se duplica la cantidad de celdas de la tabla de dispersión y aumentan en uno los bits de dispersión.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	3
(001)	2
(010)	1
(011)	4
(100)	3
(101)	2
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	3	Aconcagua (10100111)	Cervino (01101111)
1	2	Kilimanjaro (10101010)	Mont Blanc (00111110)
2	2	Etna (00110101)	Cho Oyu (01011101)
3	2	Chañi (11110000)	
4	3	Vinicunca (01011011)	

Baja de la clave “Chañi”:

- Se borra la clave “Chañi”.
- Se libera el bloque 3, ya que se puede fusionar con el bloque 1.
- Se reducen en uno los bits del bloque 1.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	1
(001)	2
(010)	1
(011)	4
(100)	1
(101)	2
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	3	Aconcagua (10100111)	Cervino (01101111)
1	1	Kilimanjaro (10101010)	Mont Blanc (00111110)
2	2	Etna (00110101)	Cho Oyu (01011101)
3	2	Chañi (11110000)	
4	3	Vinicunca (01011011)	

Baja de la clave “Cervino”:

- Se borra la clave “Cervino”.
- No se libera el bloque 0, ya que sigue ocupado por la clave “Aconcagua”.

Bits de dispersión: 3	
Sufijo	#Bloque
(000)	1
(001)	2
(010)	1
(011)	4
(100)	1
(101)	2
(110)	1
(111)	0

#Bloque	Bits	Clave R1	Clave R2
0	3	Aconcagua (10100111)	Cervino (01101111)
1	1	Kilimanjaro (10101010)	Mont Blanc (00111110)
2	2	Etna (00110101)	Cho Oyu (01011101)
3	2	Chañi (11110000)	
4	3	Vinicunca (01011011)	